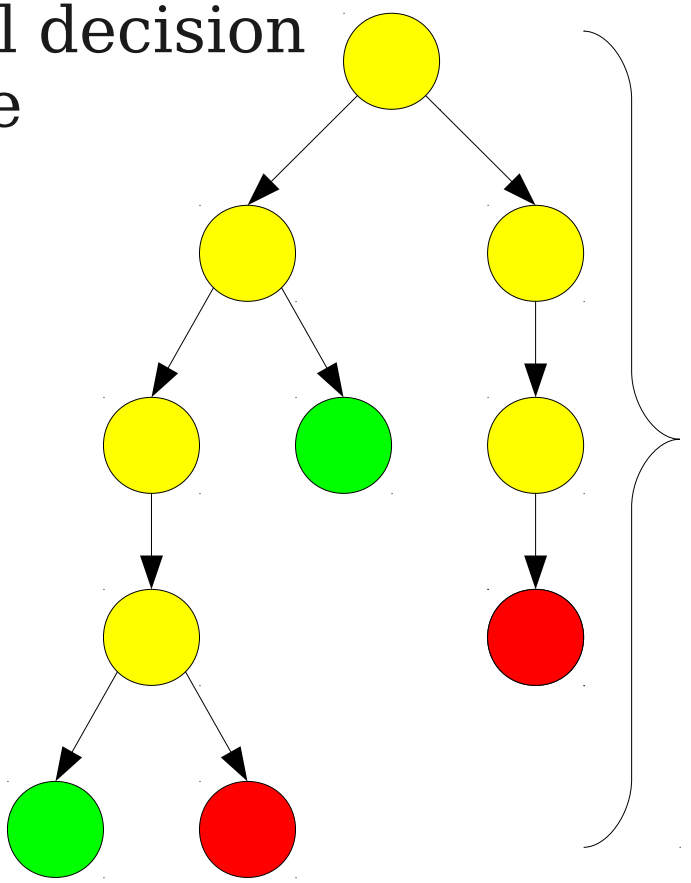


Completeness

NTIME

- The time complexity of a nondeterministic Turing machine is the length of the longest execution path of that NTM on a string of length n.
- The class **NTIME**($f(n)$) consists of all decision problems that can be decided in time O($f(n)$) by a single-tape NTM.



The Complexity Class **NP**

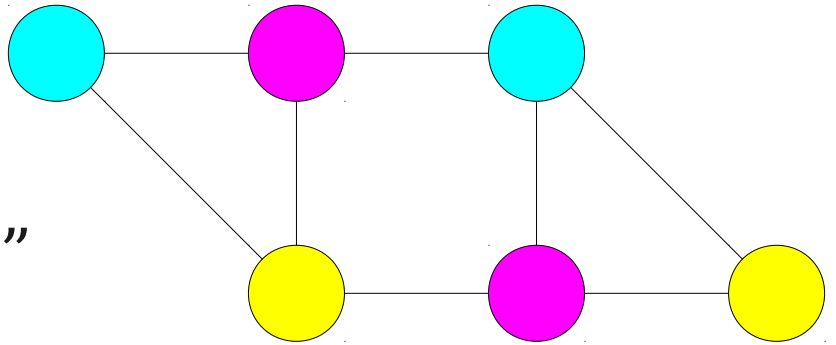
- The complexity class **NP** (**nondeterministic polynomial time**) contains all problems that can be solved in polynomial time by a single-tape NTM.
- Formally:

$$\mathbf{NP} = \bigcup_{k=0}^{\infty} \mathrm{NTIME}(n^k)$$

- Equivalently: A language is in **NP** iff there is a polynomial-time verifier for it.

A Problem in **NP**

- A **graph coloring** is a way of assigning colors to nodes in an undirected graph such that no two nodes joined by an edge have the same color.
  - Applications in compilers, cell phone towers, etc.
- Question: Can graph G be colored with at most k colors?
- M = "On input $\langle G, k, C \rangle$, where C is an alleged coloring:
  - **Deterministically** check whether C is a legal k-coloring of G.
  - If so, accept; if not, reject."



Proving Languages are in **NP**

- **Build a polynomial-time NTM for *L*.**
  - Build an NTM for the language *L*.
  - Prove that it runs in nondeterministic time $O(n^k)$.
- **Build a polynomial-time verifier for *L*.**
  - Build a TM that verifies a string, given a certificate.
  - Prove that it runs in deterministic time $O(n^k)$.
- **Reduce *L* to a language in NP**.
  - Show how a polynomial-time verifier or polynomial-time NTM for some language *L*' can be used to decide *L*.

Proving Languages are in **NP**

Build a polynomial-time NTM for *L*.

Build an NTM for the language *L*.

Prove that it runs in nondeterministic time $O(n^k)$.

Build a polynomial-time verifier for *L*.

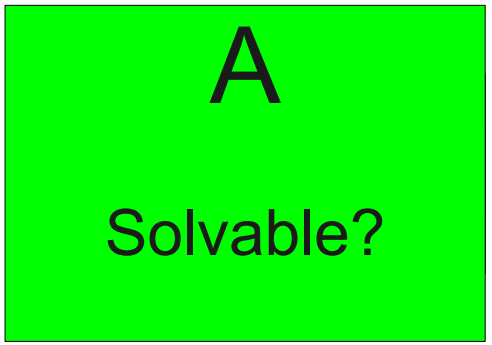
Build a TM that verifies a string, given a certificate.

Prove that it runs in deterministic time $O(n^k)$.

- **Reduce *L* to a language in NP.**
  - Show how a polynomial-time verifier or polynomial-time NTM for some language *L*' can be used to decide *L*.

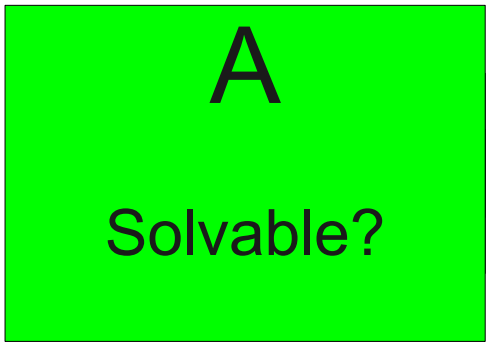
Polynomial-Time Reductions

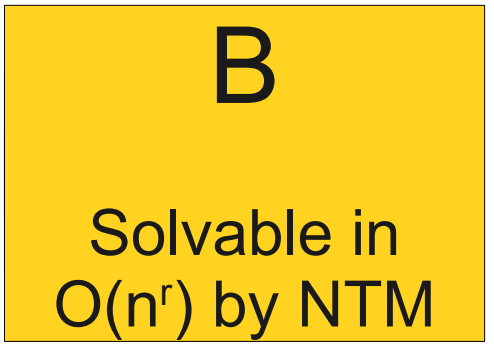
- Suppose that we know that $B \in \mathbf{NP}$.
- Suppose that $A \leq_P B$.



Polynomial-Time Reductions

- Suppose that we know that $B \in \mathbf{NP}$.
- Suppose that $A \leq_P B$.

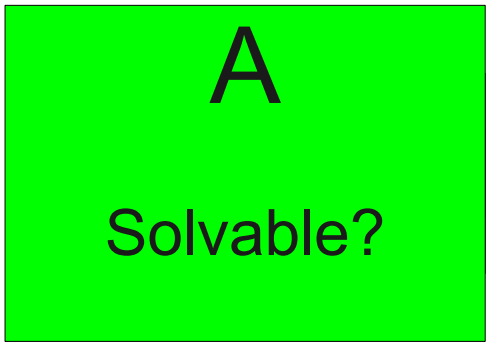


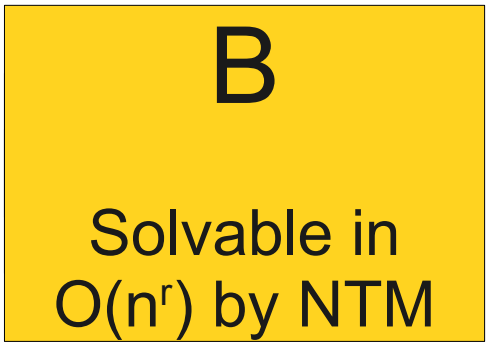


Polynomial-Time Reductions

- Suppose that we know that $B \in \mathbf{NP}$.
- Suppose that $A \leq_P B$.

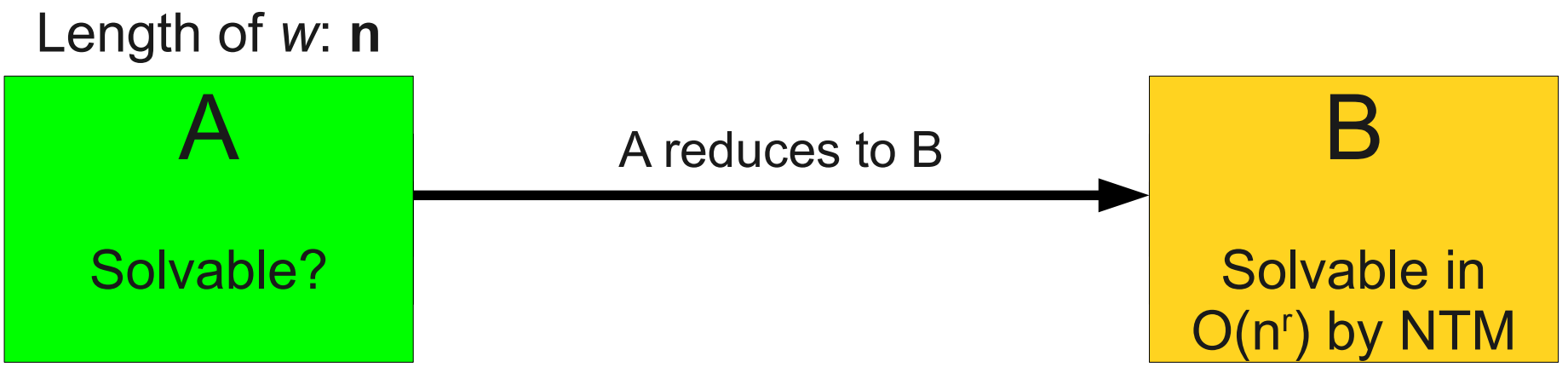
Length of *w*: **n**





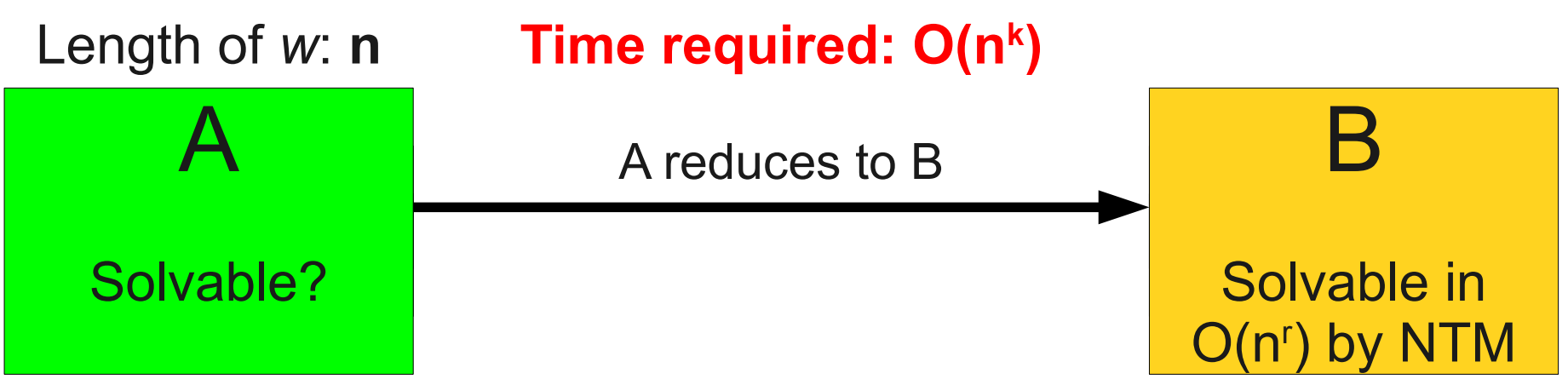
Polynomial-Time Reductions

- Suppose that we know that $B \in \mathbf{NP}$.
- Suppose that $A \leq_P B$.



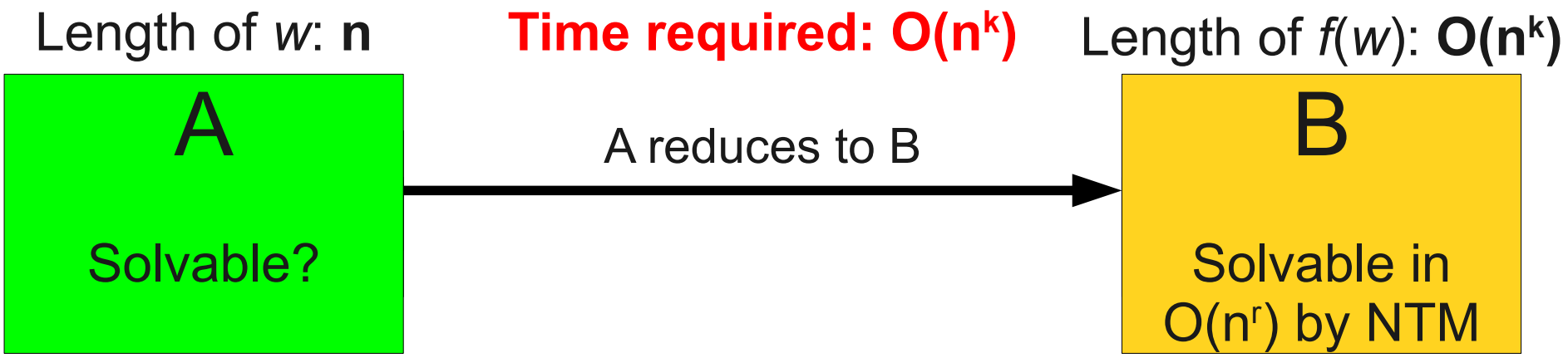
Polynomial-Time Reductions

- Suppose that we know that $B \in \mathbf{NP}$.
- Suppose that $A \leq_P B$.



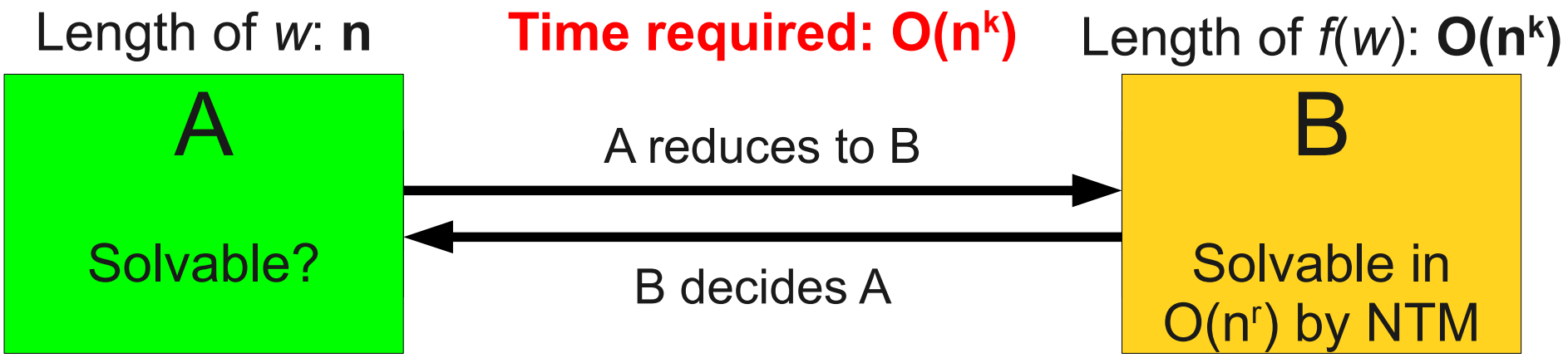
Polynomial-Time Reductions

- Suppose that we know that $B \in \mathbf{NP}$.
- Suppose that $A \leq_P B$.



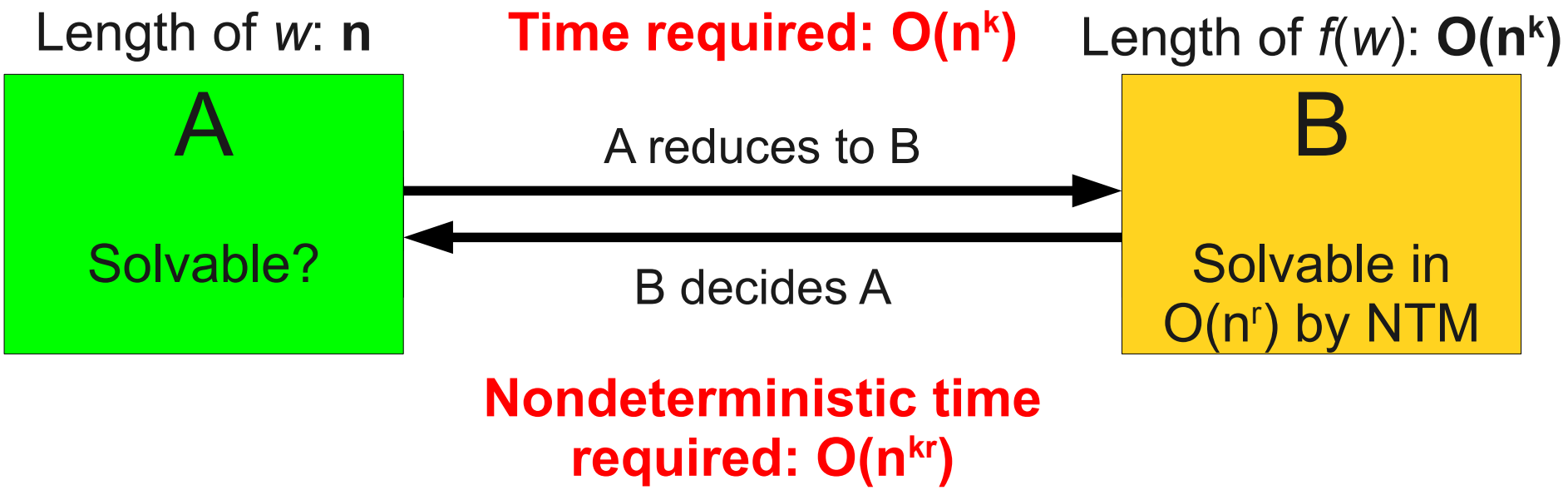
Polynomial-Time Reductions

- Suppose that we know that $B \in \mathbf{NP}$.
- Suppose that $A \leq_P B$.



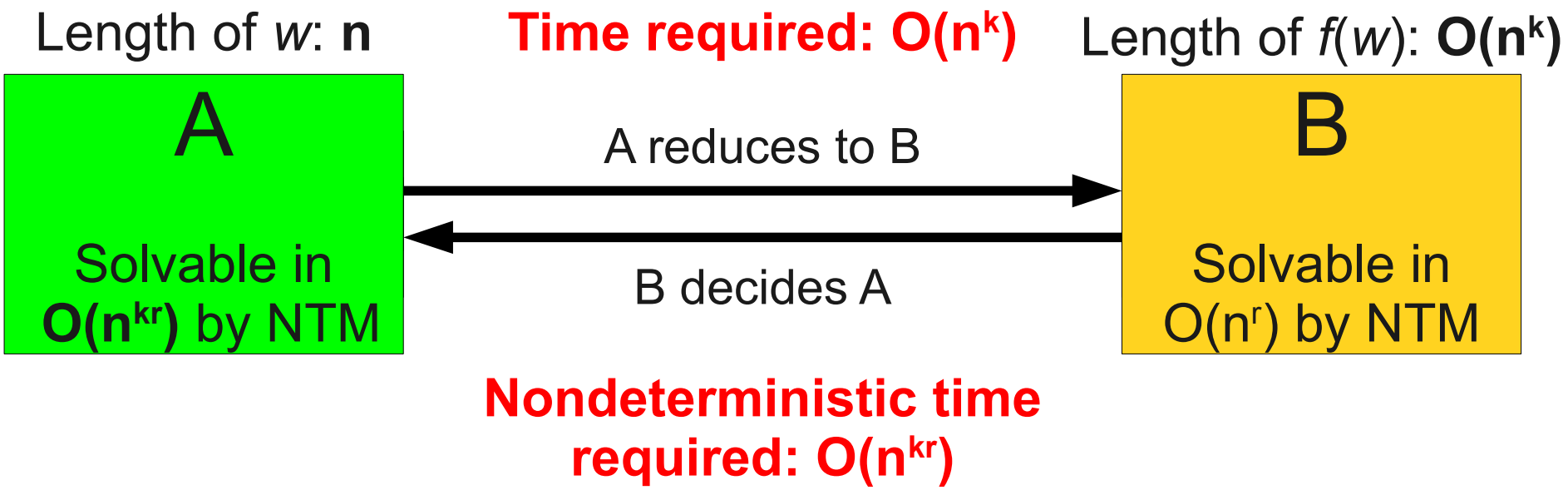
Polynomial-Time Reductions

- Suppose that we know that $B \in \mathbf{NP}$.
- Suppose that $A \leq_P B$.



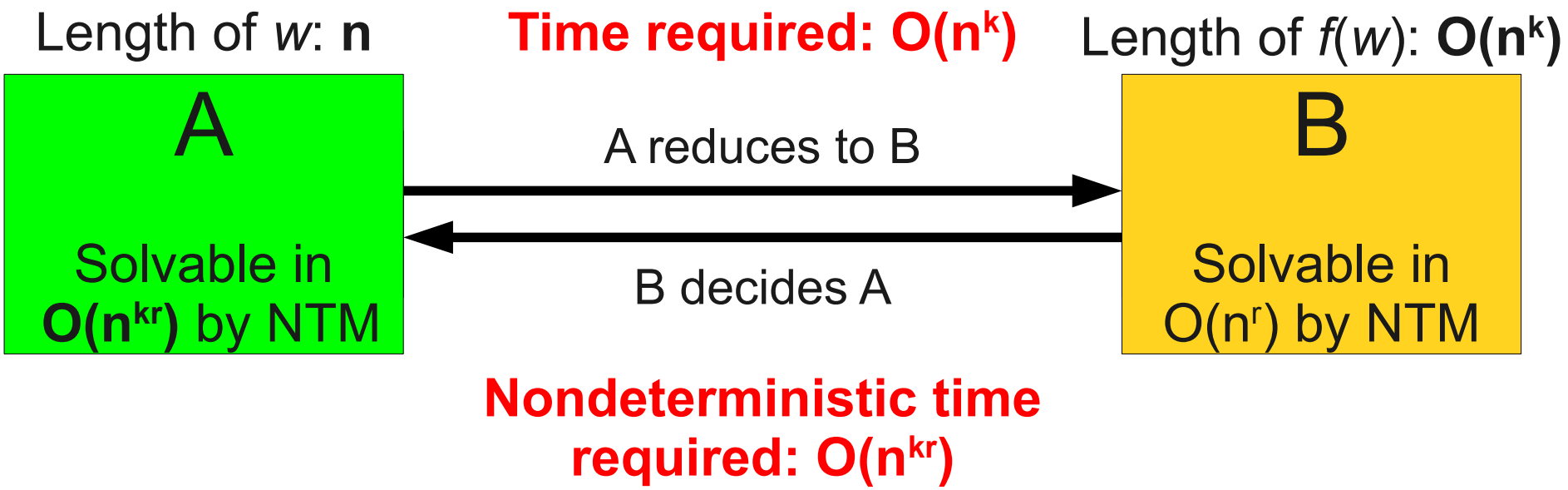
Polynomial-Time Reductions

- Suppose that we know that $B \in \mathbf{NP}$.
- Suppose that $A \leq_P B$.



Polynomial-Time Reductions

- Suppose that we know that $B \in \mathbf{NP}$.
- Suppose that $A \leq_P B$.
- Then $A \in \mathbf{NP}$.



Trust me, these reductions matter.

We'll see why in a few minutes.

The

Most Important Question

in

Theoretical Computer Science

What is the connection between **P** and **NP**?

$$\mathbf{P} = \bigcup_{k=0}^{\infty} \mathrm{TIME}(n^k)$$

$$\mathbf{NP} = \bigcup_{k=0}^{\infty} \mathrm{NTIME}(n^k)$$

$$\mathbf{P} = \bigcup_{k=0}^{\infty} \mathrm{TIME}(n^k)$$

$$\mathbf{NP} = \bigcup_{k=0}^{\infty} \mathrm{NTIME}(n^k)$$

$$\mathrm{TIME}(n^k) \subseteq \mathrm{NTIME}(n^k)$$

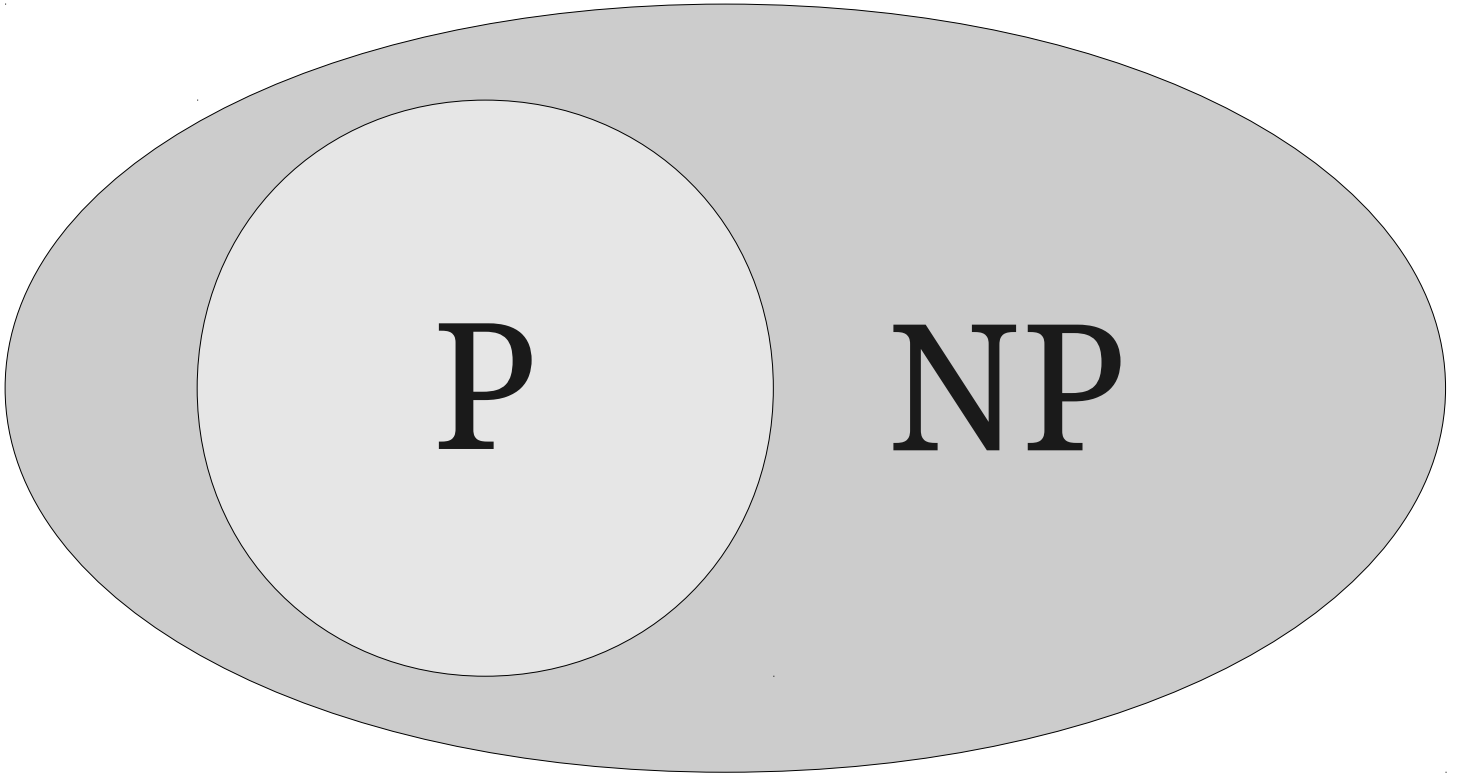
$$\mathbf{P} = \bigcup_{k=0}^{\infty} \mathrm{TIME}(n^k)$$

$$\mathbf{NP} = \bigcup_{k=0}^{\infty} \mathrm{NTIME}(n^k)$$

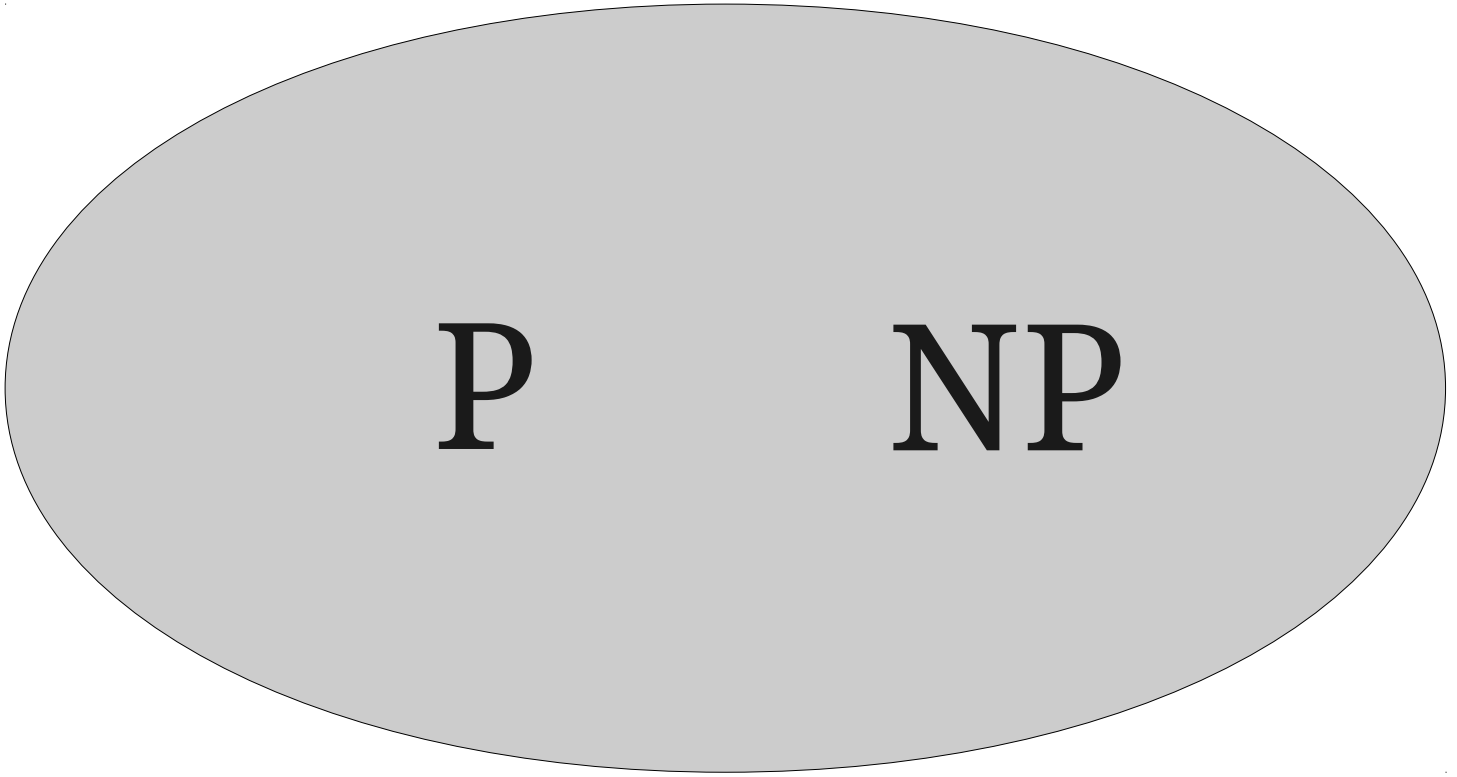
$$\mathrm{TIME}(n^k) \subseteq \mathrm{NTIME}(n^k)$$

$$\mathbf{P} \subseteq \mathbf{NP}$$

Which Picture is Correct?



Which Picture is Correct?



Does **P** = **NP**?

$\mathbf{P \stackrel{?}{=} NP}$

- The question of $\mathbf{P \stackrel{?}{=} NP}$ is the most important question in theoretical computer science.
- With the verifier definition of **NP**, one way of phrasing this question is

If a problem can be **verified** efficiently,
can it be **solved** efficiently?

- An answer either way will give fundamental insights into the nature of computation.

Why This Matters

- The following problems are known to be efficiently verifiable, but have no known efficient solutions:
  - Determining whether an electrical grid can be built to link up some number of houses for some price (Steiner tree problem).
  - Determining whether a simple DNA strand exists that multiple gene sequences could be a part of (shortest common supersequence).
  - Determining the best way to assign hardware resources in a compiler (optimal register allocation).
  - Determining the best way to distribute tasks to multiple workers to minimize completion time (job scheduling).
  - **And many more**.
- If **P = NP**, **all** of these problems have efficient solutions.
- If **P ≠ NP**, **none** of these problems have efficient solutions.

Why This Matters

- If **P = NP**:
  - A huge number of seemingly difficult problems could be solved efficiently.
  - Our capacity to solve many problems will scale well with the size of the problems we want to solve.
- If **P ≠ NP**:
  - Enormous computational power would be required to solve many seemingly easy tasks.
  - Our capacity to solve problems will fail to keep up with our curiosity.

What We Know

- Resolving $\mathbf{P} \stackrel{?}{=} \mathbf{NP}$ has proven ***extremely difficult.***
- In the past 35 years:
  - Not a single correct proof either way has been found.
  - Many types of proofs have been shown to be insufficiently powerful to determine whether **P = NP**.
  - It is commonly believed that **P ≠ NP**, but no one knows for sure.
- Interesting read: Interviews with leading thinkers about $\mathbf{P} \stackrel{?}{=} \mathbf{NP}$:
  - http://web.ing.puc.cl/~jabaier/iic2212/poll-1.pdf

The Million-Dollar Question

The Clay Mathematics Institute has offered a **$1,000,000 prize** to anyone who proves or disproves **P = NP**.

The Million-Dollar Question

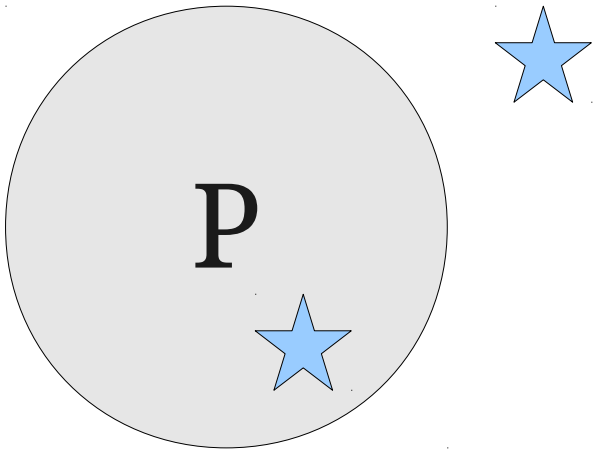


The Clay Mathematics Institute has offered a **$1,000,000 prize** to anyone who proves or disproves **P = NP**.

NP-Completeness

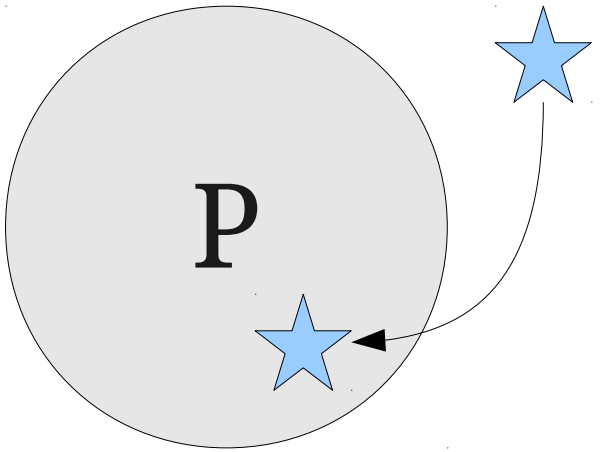
Polynomial-Time Reductions

- If $L_1 \leq_P L_2$ and $L_2 \in \mathbf{P}$, then $L_1 \in \mathbf{P}$.



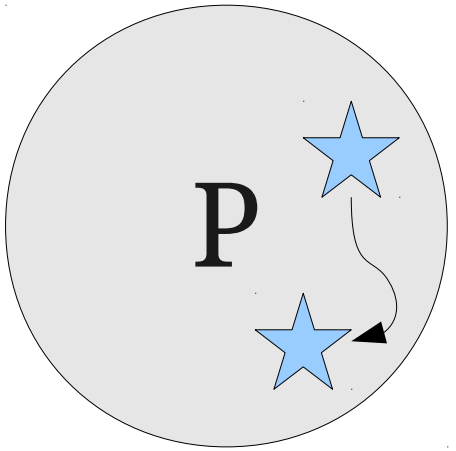
Polynomial-Time Reductions

- If $L_1 \leq_P L_2$ and $L_2 \in \mathbf{P}$, then $L_1 \in \mathbf{P}$.



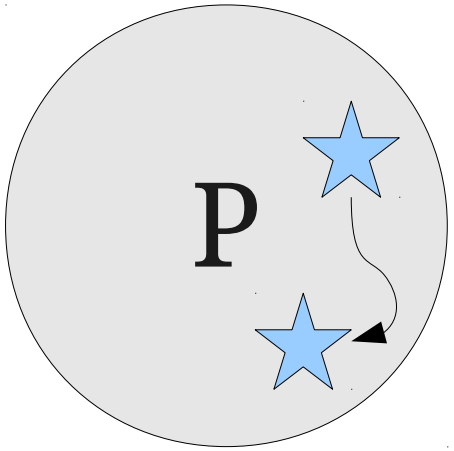
Polynomial-Time Reductions

- If $L_1 \leq_P L_2$ and $L_2 \in \mathbf{P}$, then $L_1 \in \mathbf{P}$.



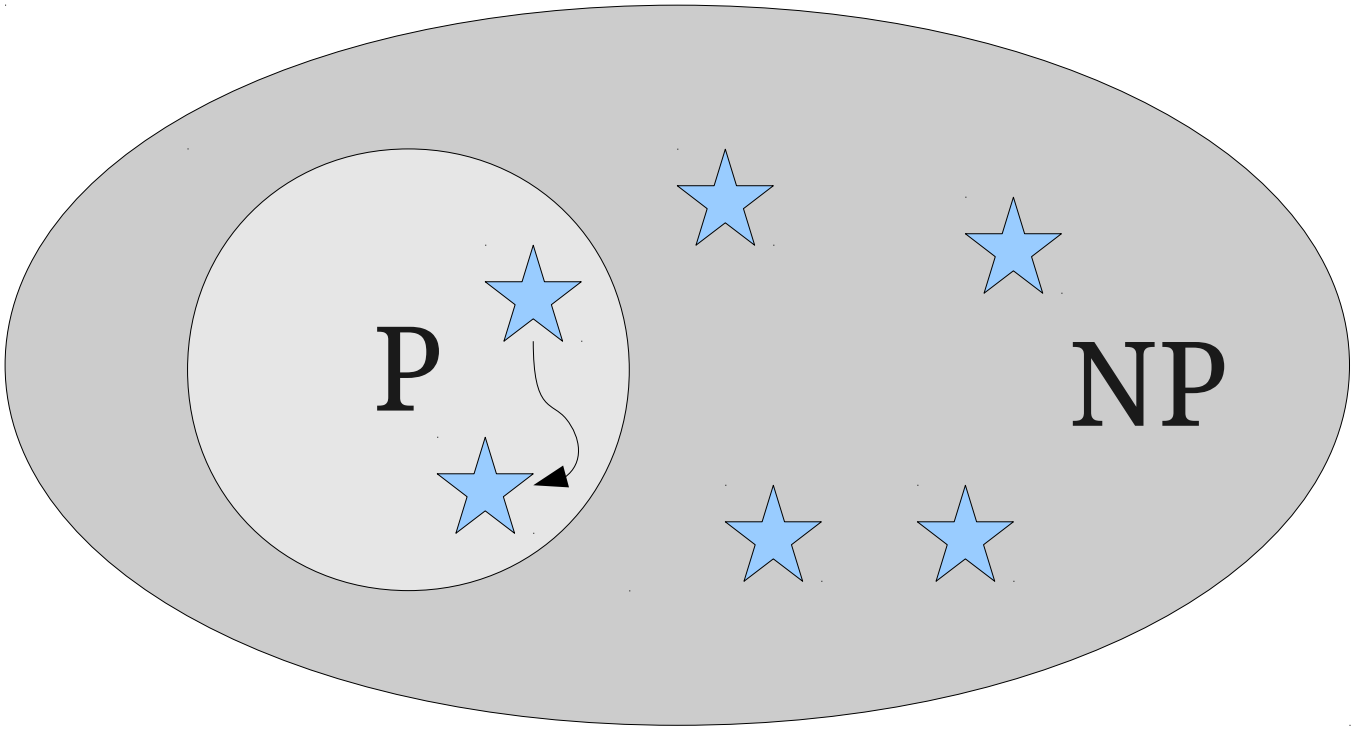
Polynomial-Time Reductions

- If $L_1 \leq_P L_2$ and $L_2 \in \mathbf{P}$, then $L_1 \in \mathbf{P}$.
- If $L_1 \leq_P L_2$ and $L_2 \in \mathbf{NP}$, then $L_1 \in \mathbf{NP}$.



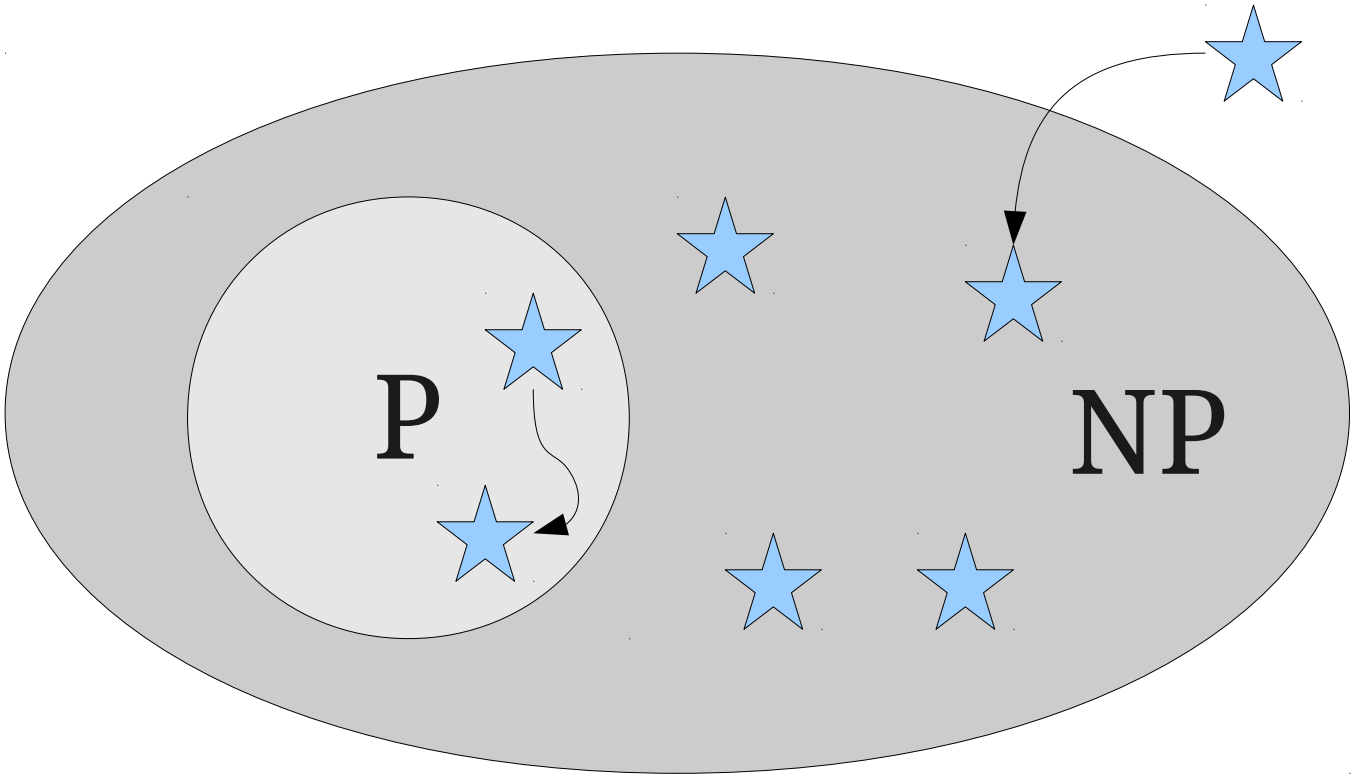
Polynomial-Time Reductions

- If $L_1 \leq_P L_2$ and $L_2 \in \mathbf{P}$, then $L_1 \in \mathbf{P}$.
- If $L_1 \leq_P L_2$ and $L_2 \in \mathbf{NP}$, then $L_1 \in \mathbf{NP}$.



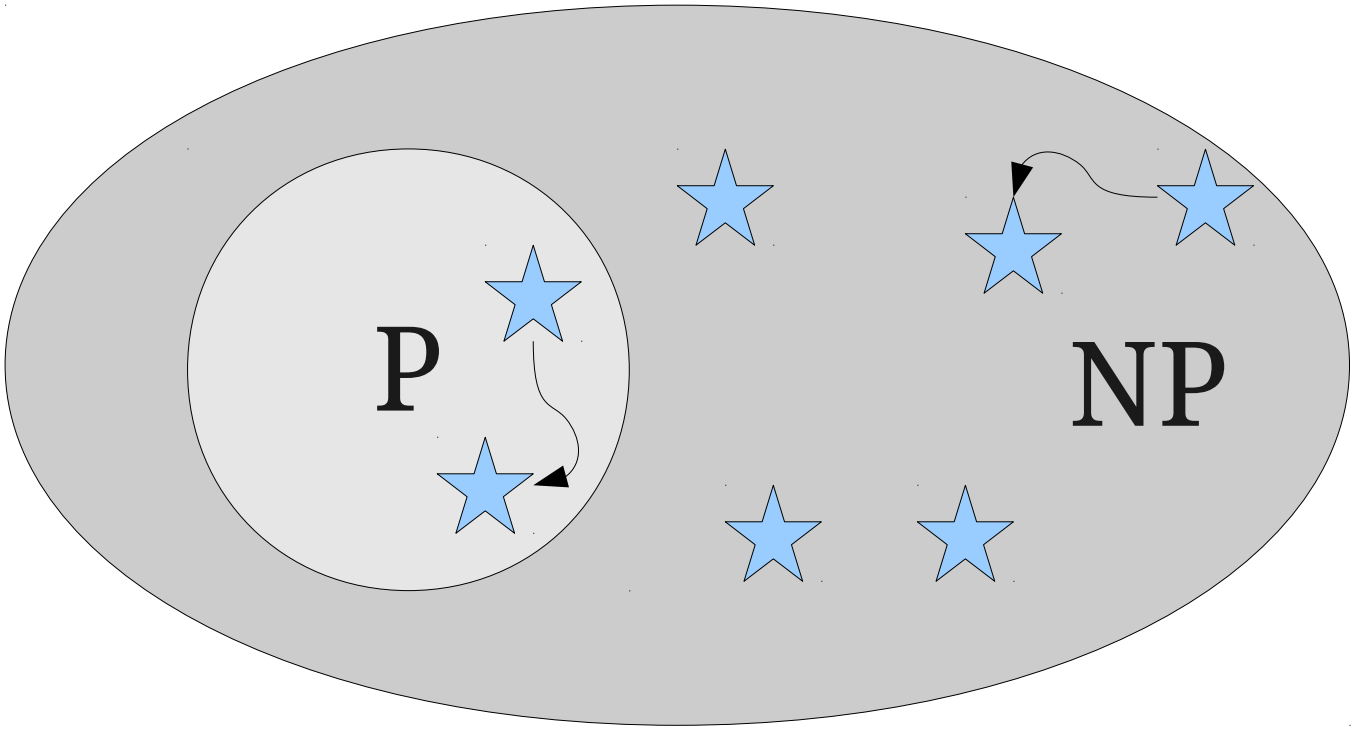
Polynomial-Time Reductions

- If $L_1 \leq_P L_2$ and $L_2 \in \mathbf{P}$, then $L_1 \in \mathbf{P}$.
- If $L_1 \leq_P L_2$ and $L_2 \in \mathbf{NP}$, then $L_1 \in \mathbf{NP}$.



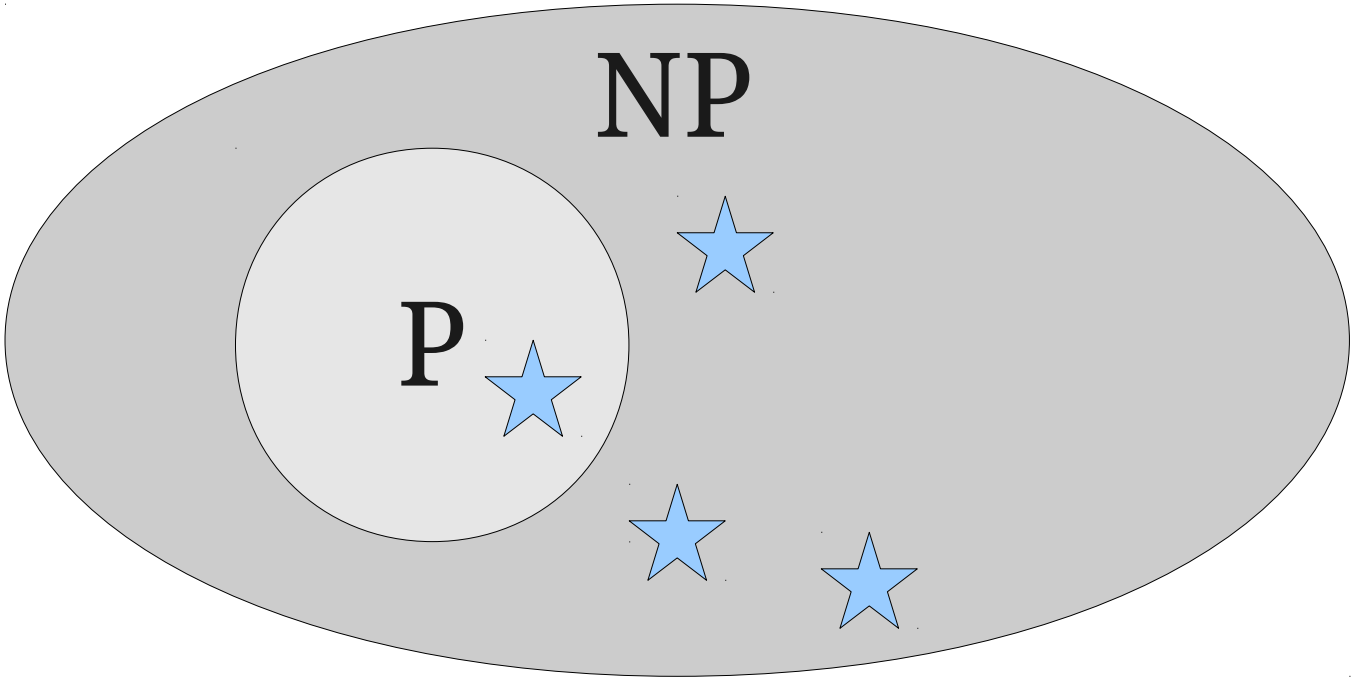
Polynomial-Time Reductions

- If $L_1 \leq_P L_2$ and $L_2 \in \mathbf{P}$, then $L_1 \in \mathbf{P}$.
- If $L_1 \leq_P L_2$ and $L_2 \in \mathbf{NP}$, then $L_1 \in \mathbf{NP}$.



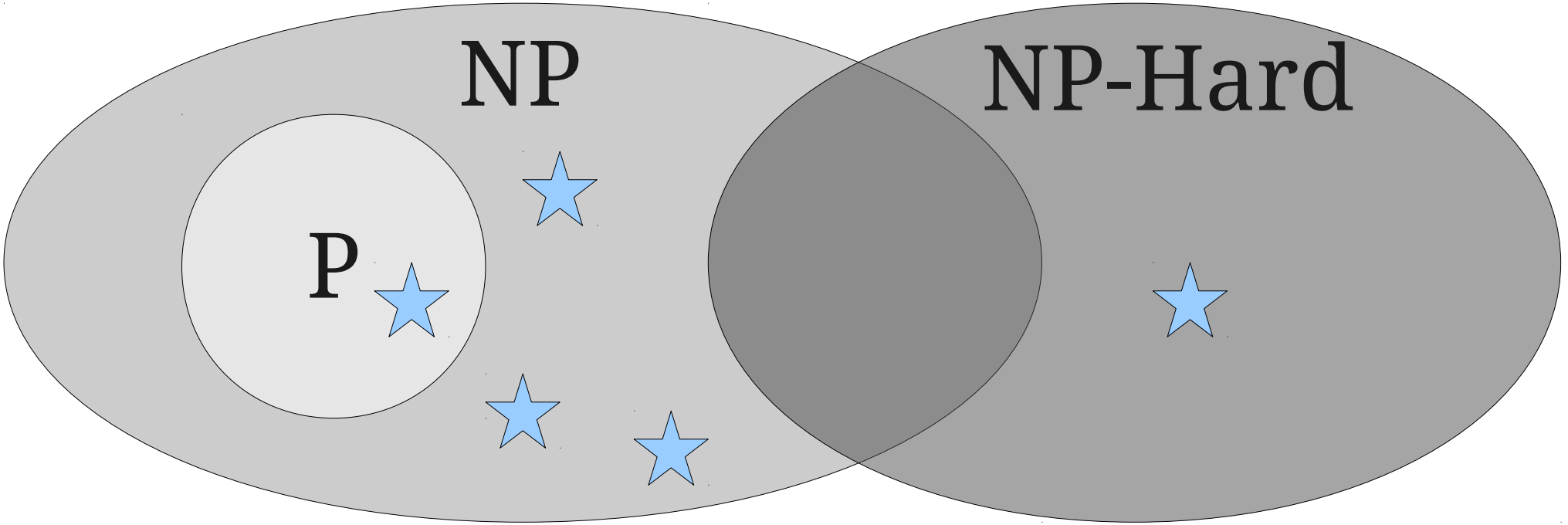
NP-Hardness

- A language L is called **NP-hard** iff for *every* $L' \in \mathbf{NP}$, we have $L' \leq_P L$.



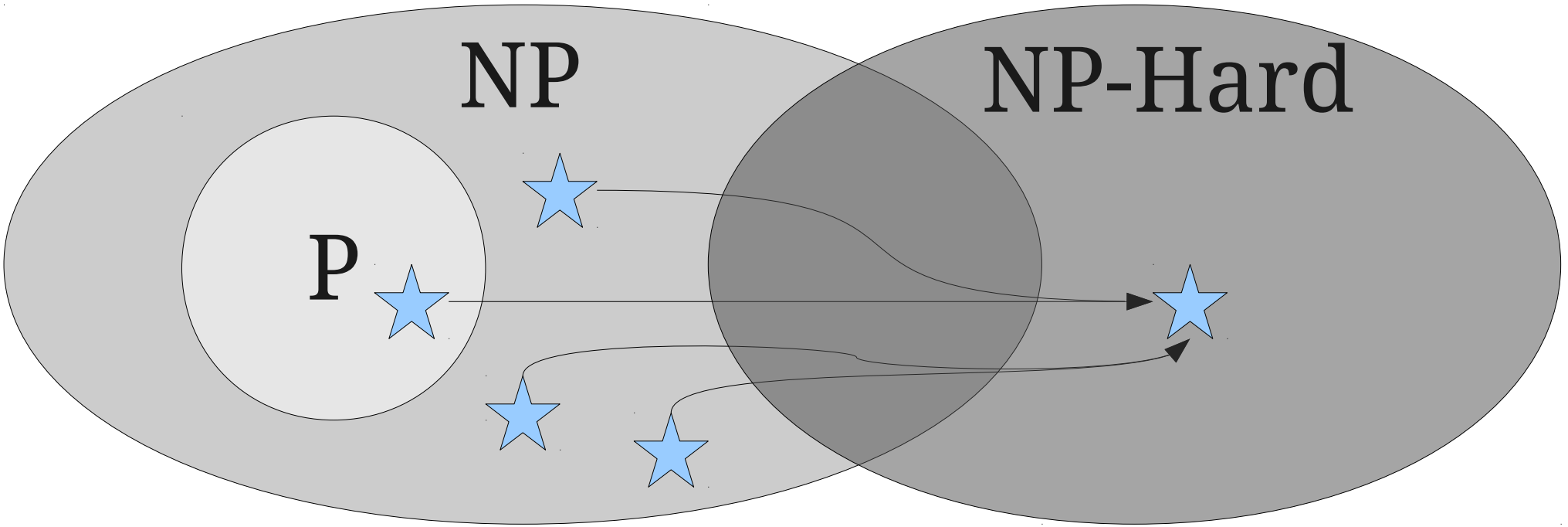
NP-Hardness

- A language L is called **NP-hard** iff for *every* $L' \in$ **NP**, we have $L' \leq_P L$.



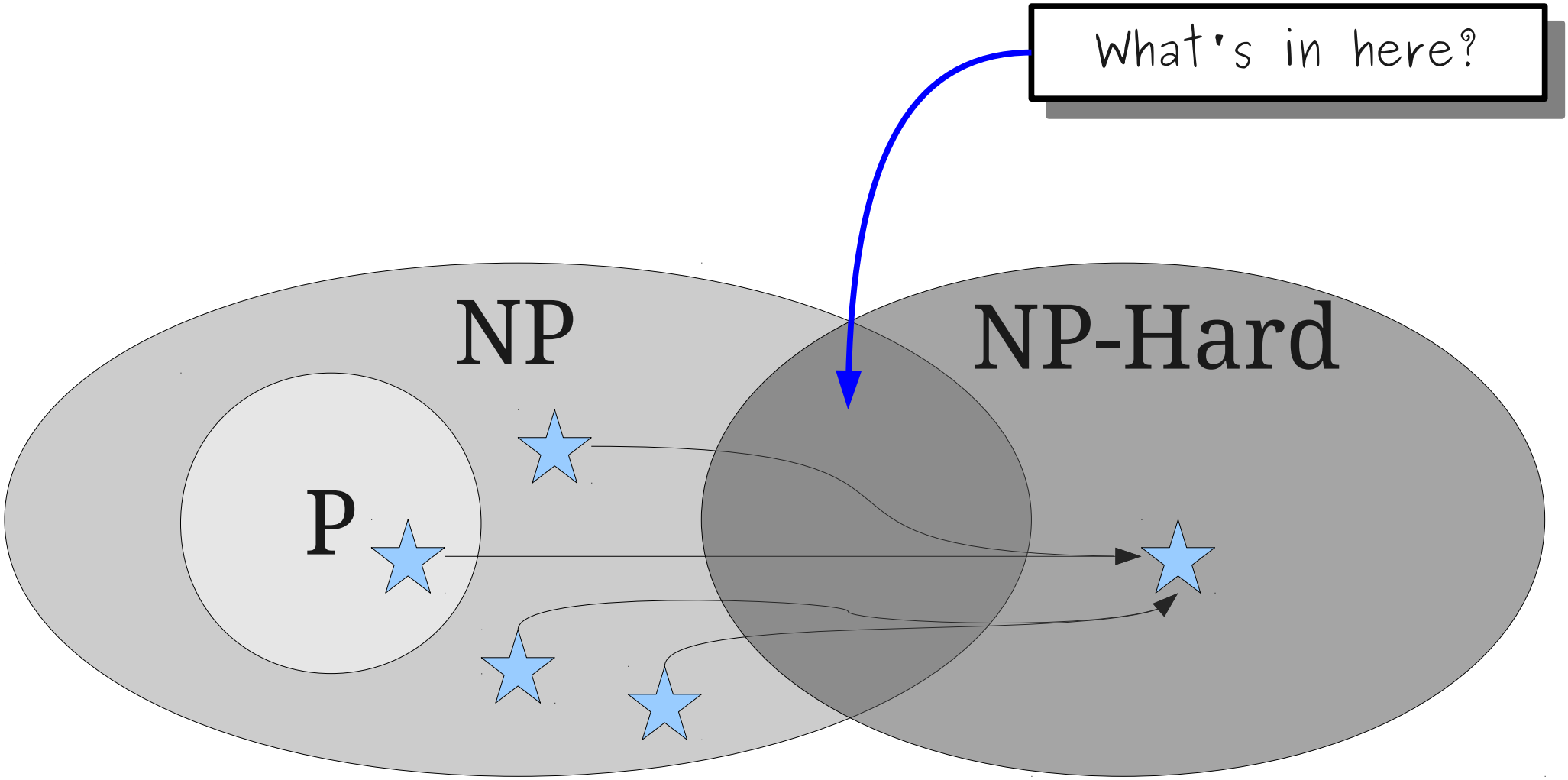
NP-Hardness

- A language L is called **NP-hard** iff for *every* $L' \in$ **NP**, we have $L' \leq_P L$.



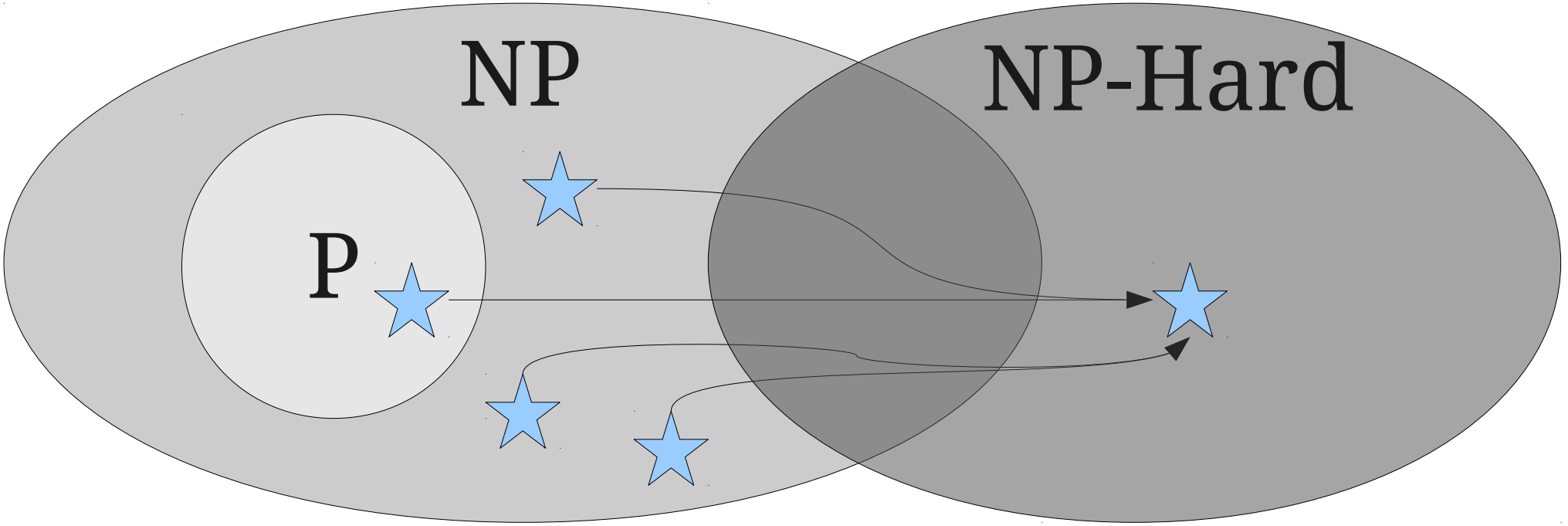
NP-Hardness

- A language L is called **NP-hard** iff for *every* $L' \in$ **NP**, we have $L' \leq_P L$.



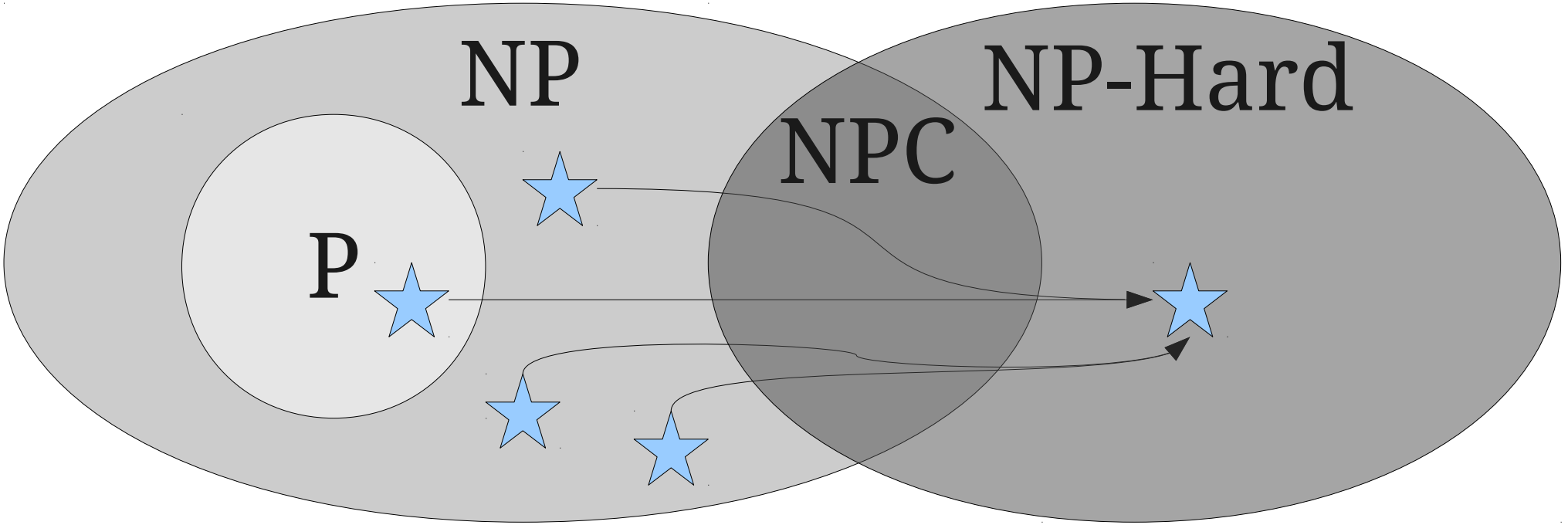
NP-Hardness

- A language L is called **NP-hard** iff for *every* $L' \in \mathbf{NP}$, we have $L' \leq_P L$.
- A language in L is called **NP-complete** iff L is **NP**-hard and $L \in \mathbf{NP}$.
- The class **NPC** is the set of **NP**-complete problems.



NP-Hardness

- A language L is called **NP-hard** iff for *every* $L' \in$ **NP**, we have $L' \leq_P L$.
- A language in L is called **NP-complete** iff L is **NP**-hard and $L \in$ **NP**.
- The class **NPC** is the set of **NP**-complete problems.



The Tantalizing Truth

Theorem: If ***any*** **NP**-complete language is in **P**, then **P** = **NP**.

The Tantalizing Truth

Theorem: If ***any*** **NP**-complete language is in **P**, then **P** = **NP**.

Proof: If $L \in$ **NPC** and $L \in$ **P**, we know for any $L' \in$ **NP** that $L' \leq_P L$, because L is **NP**-complete.

The Tantalizing Truth

Theorem: If ***any*** **NP**-complete language is in **P**, then **P** = **NP**.

Proof: If $L \in$ **NPC** and $L \in$ **P**, we know for any $L' \in$ **NP** that $L' \leq_P L$, because L is **NP**-complete. Since $L' \leq_P L$ and $L \in$ **P**, this means that $L' \in$ **P** as well.

The Tantalizing Truth

Theorem: If ***any*** **NP**-complete language is in **P**, then **P** = **NP**.

Proof: If $L \in$ **NPC** and $L \in$ **P**, we know for any $L' \in$ **NP** that $L' \leq_P L$, because L is **NP**-complete. Since $L' \leq_P L$ and $L \in$ **P**, this means that $L' \in$ **P** as well. Since our choice of L' was arbitrary, any language $L' \in$ **NP** satisfies $L' \in$ **P**, so **NP** $\subseteq$ **P**.

The Tantalizing Truth

Theorem: If ***any*** **NP**-complete language is in **P**, then **P** = **NP**.

Proof: If $L \in$ **NPC** and $L \in$ **P**, we know for any $L' \in$ **NP** that $L' \leq_P L$, because L is **NP**-complete. Since $L' \leq_P L$ and $L \in$ **P**, this means that $L' \in$ **P** as well. Since our choice of L' was arbitrary, any language $L' \in$ **NP** satisfies $L' \in$ **P**, so **NP** $\subseteq$ **P**. Since **P** $\subseteq$ **NP**, this means **P** = **NP**.

The Tantalizing Truth

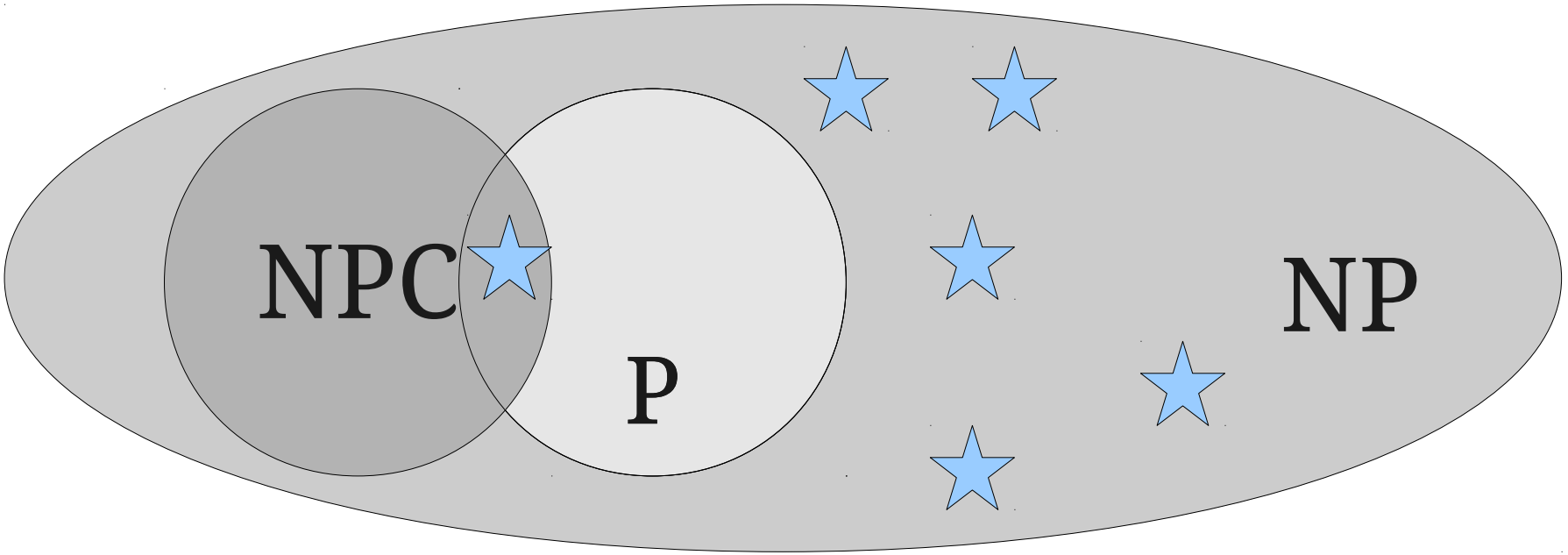
Theorem: If ***any*** **NP**-complete language is in **P**, then **P** = **NP**.

Proof: If $L \in$ **NPC** and $L \in$ **P**, we know for any $L' \in$ **NP** that $L' \leq_P L$, because L is **NP**-complete. Since $L' \leq_P L$ and $L \in$ **P**, this means that $L' \in$ **P** as well. Since our choice of L' was arbitrary, any language $L' \in$ **NP** satisfies $L' \in$ **P**, so **NP** $\subseteq$ **P**. Since **P** $\subseteq$ **NP**, this means **P** = **NP**. ■

The Tantalizing Truth

Theorem: If ***any*** **NP**-complete language is in **P**, then **P** = **NP**.

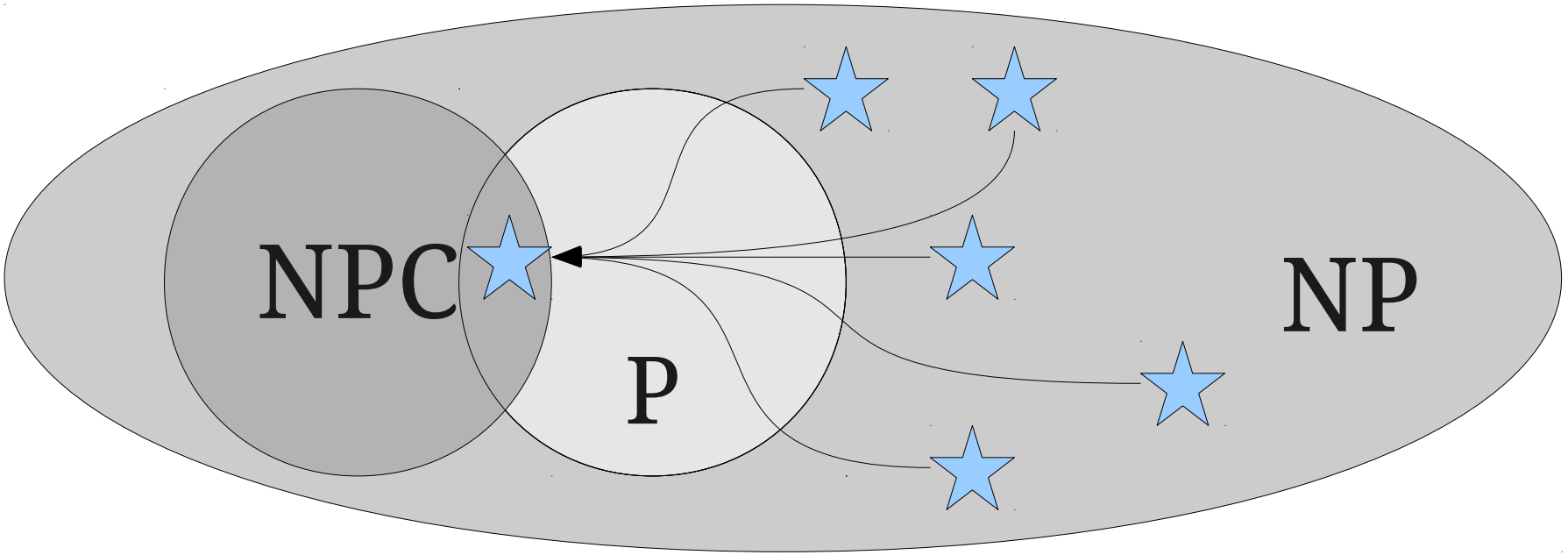
Proof: If $L \in$ **NPC** and $L \in$ **P**, we know for any $L' \in$ **NP** that $L' \leq_P L$, because L is **NP**-complete. Since $L' \leq_P L$ and $L \in$ **P**, this means that $L' \in$ **P** as well. Since our choice of L' was arbitrary, any language $L' \in$ **NP** satisfies $L' \in$ **P**, so **NP** $\subseteq$ **P**. Since **P** $\subseteq$ **NP**, this means **P** = **NP**. ■



The Tantalizing Truth

Theorem: If ***any*** **NP**-complete language is in **P**, then **P** = **NP**.

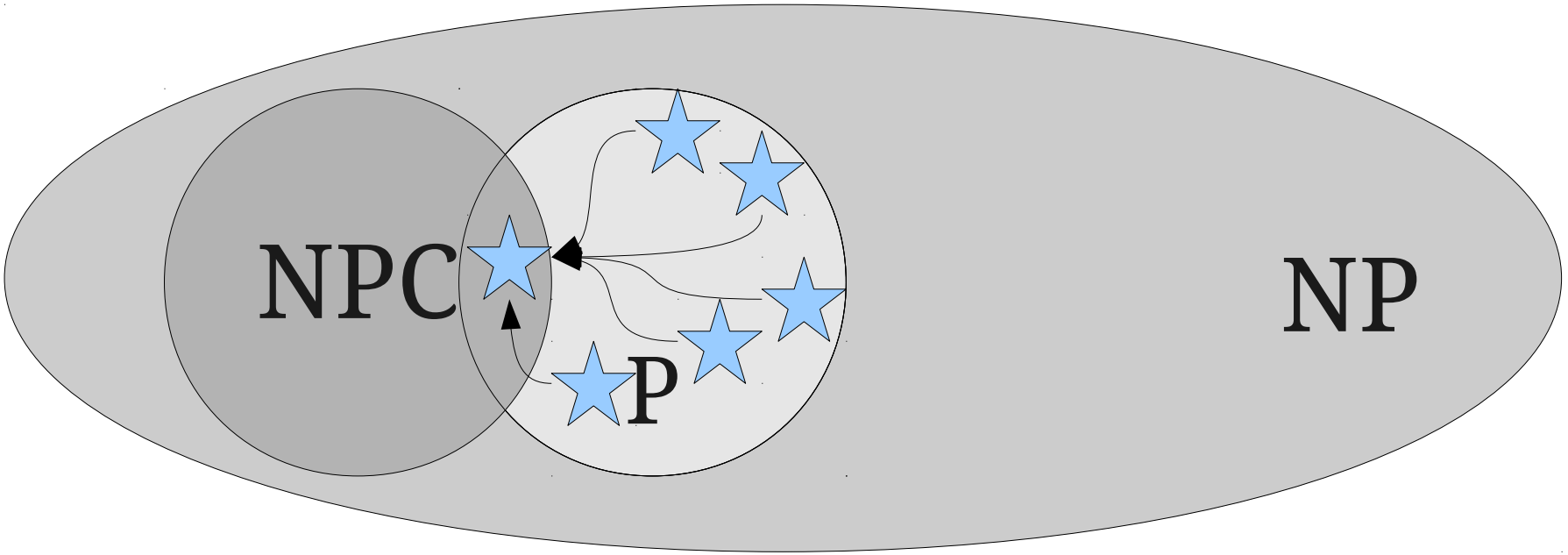
Proof: If $L \in$ **NPC** and $L \in$ **P**, we know for any $L' \in$ **NP** that $L' \leq_P L$, because L is **NP**-complete. Since $L' \leq_P L$ and $L \in$ **P**, this means that $L' \in$ **P** as well. Since our choice of L' was arbitrary, any language $L' \in$ **NP** satisfies $L' \in$ **P**, so **NP** $\subseteq$ **P**. Since **P** $\subseteq$ **NP**, this means **P** = **NP**. ■



The Tantalizing Truth

Theorem: If ***any*** **NP**-complete language is in **P**, then **P** = **NP**.

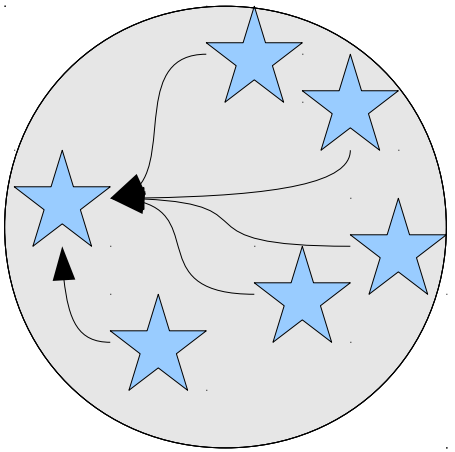
Proof: If $L \in$ **NPC** and $L \in$ **P**, we know for any $L' \in$ **NP** that $L' \leq_P L$, because L is **NP**-complete. Since $L' \leq_P L$ and $L \in$ **P**, this means that $L' \in$ **P** as well. Since our choice of L' was arbitrary, any language $L' \in$ **NP** satisfies $L' \in$ **P**, so **NP** $\subseteq$ **P**. Since **P** $\subseteq$ **NP**, this means **P** = **NP**. ■



The Tantalizing Truth

Theorem: If ***any*** **NP**-complete language is in **P**, then **P** = **NP**.

Proof: If $L \in$ **NPC** and $L \in$ **P**, we know for any $L' \in$ **NP** that $L' \leq_P L$, because L is **NP**-complete. Since $L' \leq_P L$ and $L \in$ **P**, this means that $L' \in$ **P** as well. Since our choice of L' was arbitrary, any language $L' \in$ **NP** satisfies $L' \in$ **P**, so **NP** $\subseteq$ **P**. Since **P** $\subseteq$ **NP**, this means **P** = **NP**. ■

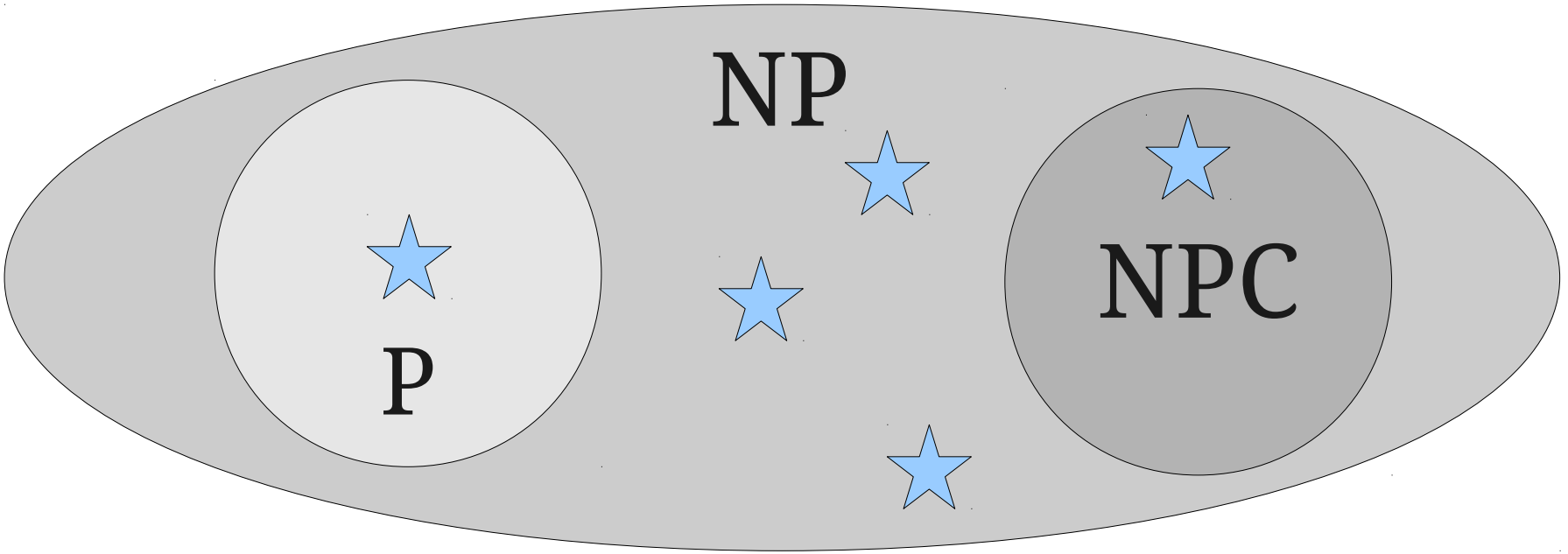


P = NP

The Tantalizing Truth

Theorem: If ***any*** **NP**-complete language is not in **P**, then **P ≠ NP**.

Proof: If $L \in$ **NPC**, then $L \in$ **NP**. Thus if $L \notin$ **P**, then $L \in$ **NP** – **P**.
This means that **NP** – **P** ≠ Ø, so **P ≠ NP**. ■



A Feel for **NP**-Completeness

- If a problem is **NP**-complete, then under the (commonly-held) assumption that **P ≠ NP**, there cannot be an efficient algorithm for it.
- In a sense, **NP**-complete problems are the hardest problems in **NP**.
- All known **NP**-complete problems are enormously hard to solve:
  - All known algorithms for **NP**-complete problems run in worst-case exponential time.
  - Most algorithms for **NP**-complete problems are infeasible for reasonably-sized inputs.

What Problems are **NP**-Complete?

- **NP**-complete problems give a promising approach for resolving $\mathbf{P} \stackrel{?}{=} \mathbf{NP}$:
  - If any **NPC** problem is in **P**, then **P** = **NP**.
  - If any **NPC** problem is not in **P**, then **P** ≠ **NP**.
- However, we haven't shown that any problems are **NP**-complete in the first place!
- How do we even know they exist?

Satisfiability

- A propositional logic formula φ is called **satisfiable** if there is some assignment to its variables that makes it evaluate to true.
- An assignment of true and false to the variables of φ that makes it evaluate to true is called a **satisfying assignment**.
- Similar terms:
  - φ is **tautological** if it is always true.
  - φ is **satisfiable** if it *can* be made true.
  - φ is **unsatisfiable** if it is always false.

SAT

- The **boolean satisfiability problem** (**SAT**) is the following:

  Given a propositional logic formula φ, is φ satisfiable?

- Formally:

  SAT = { ⟨φ⟩ | φ is a satisfiable PL formula }

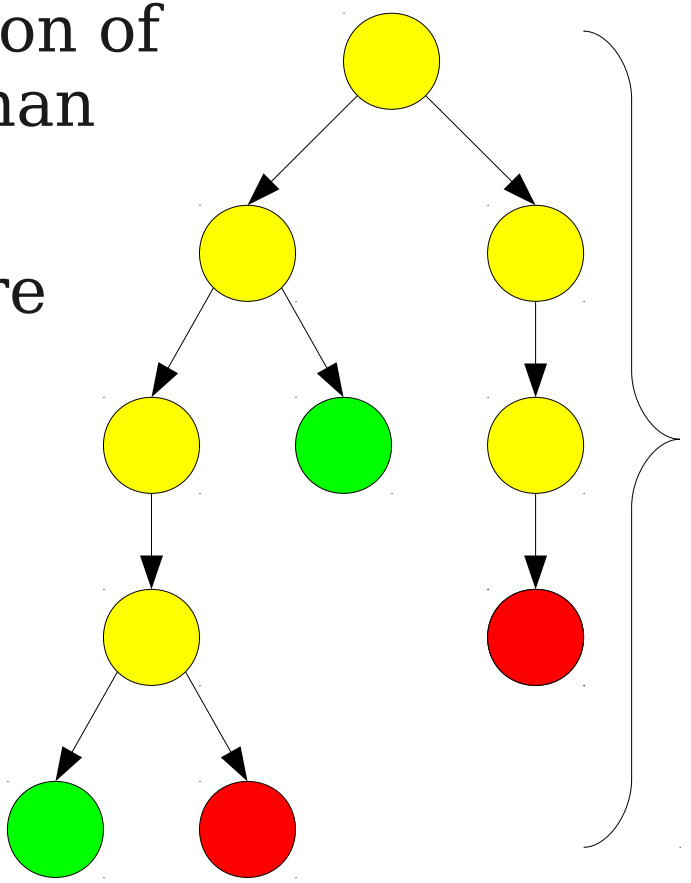
Theorem (Cook-Levin): SAT is **NP**-complete.

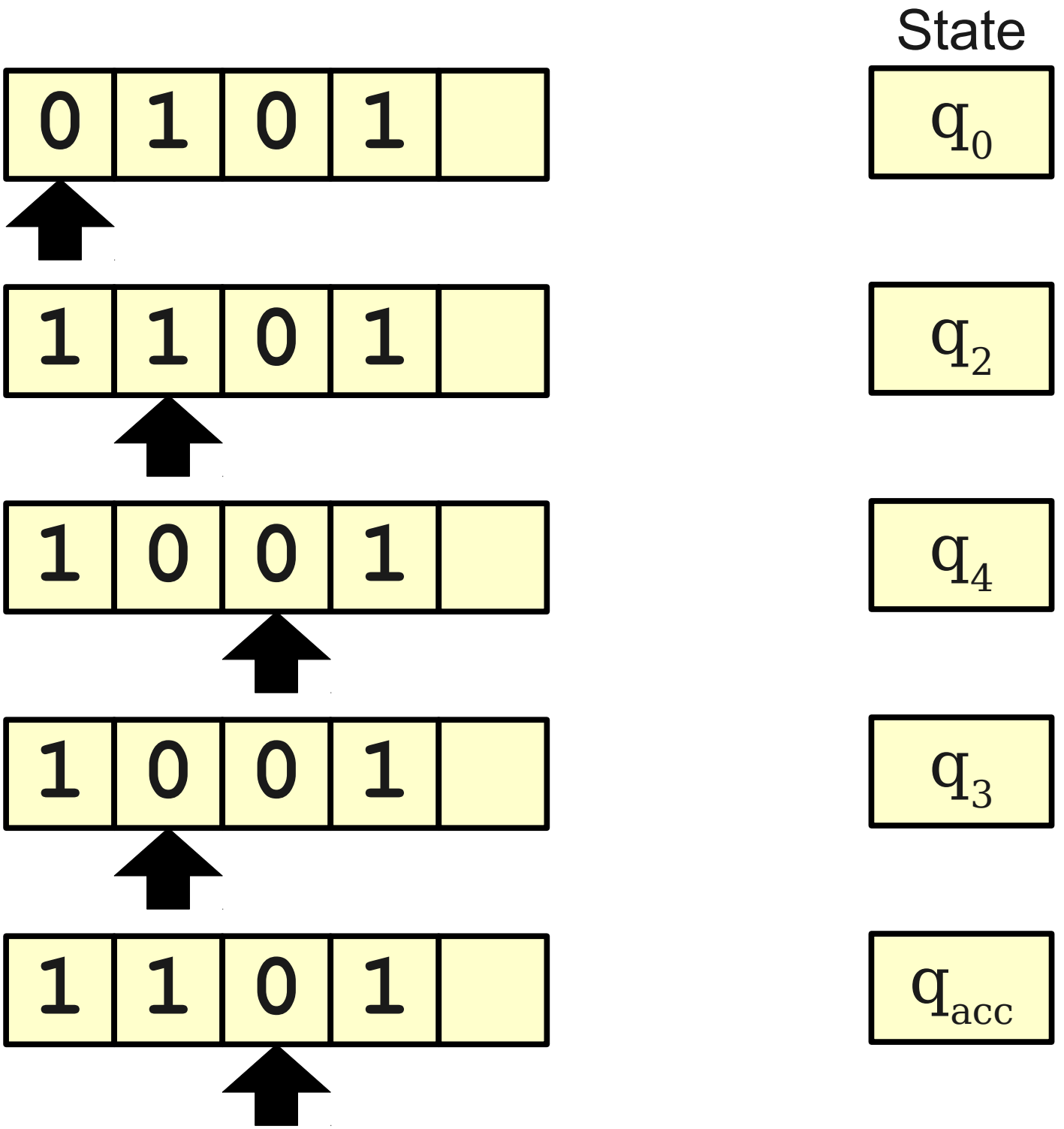
Sketch of the Proof

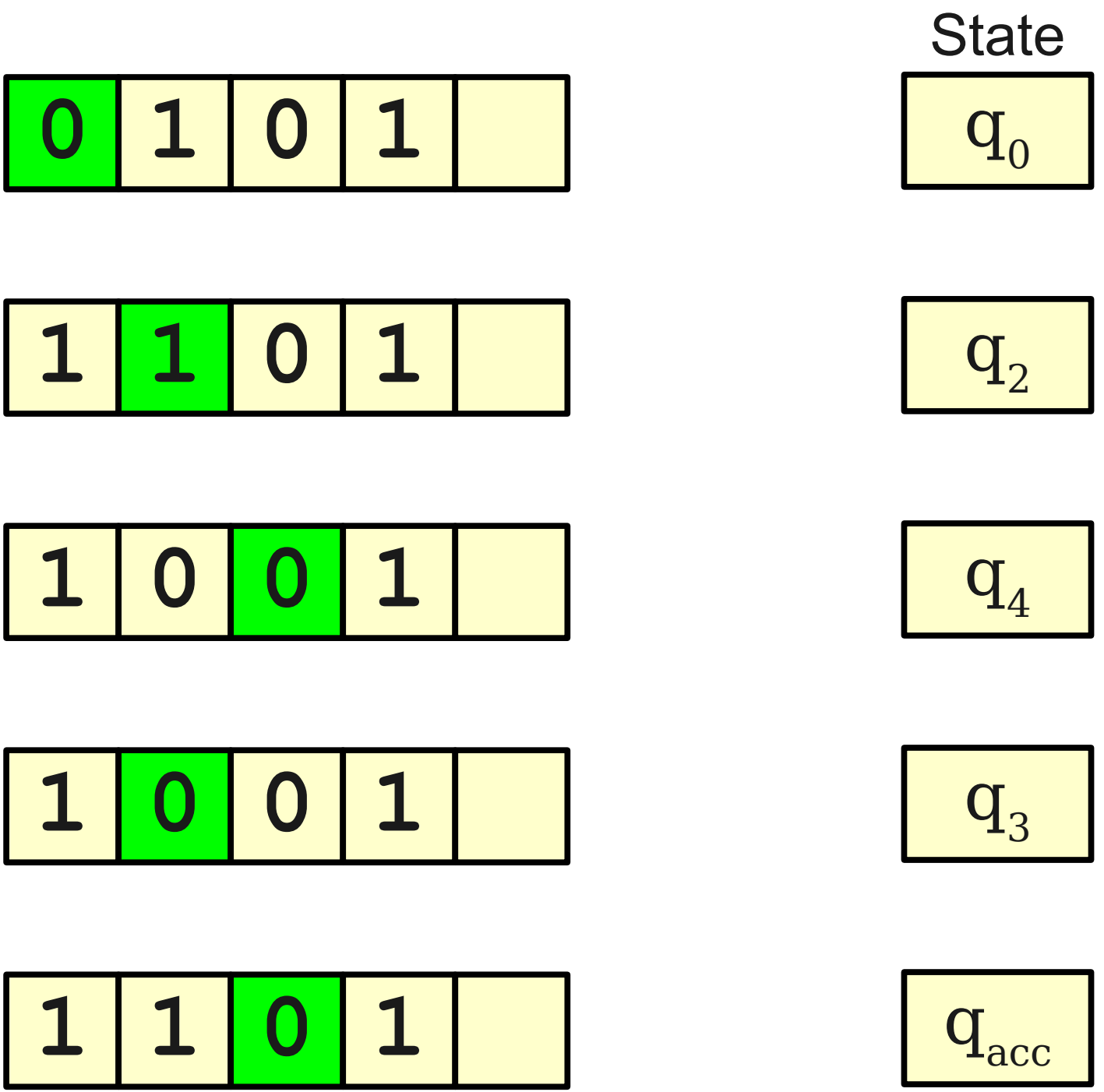
- We need to show that every single language in **NP** has a polynomial-time reduction to SAT.
- To do so, we will use the fact that every language in **NP** has a polynomial-time NTM.
- We can build a SAT formula that encodes the rules for how that NTM operates.
- If there is some set of choices where the NTM accepts, our formula will be satisfiable.
- If there are no choices we can make where the NTM accepts, our formula will be unsatisfiable.

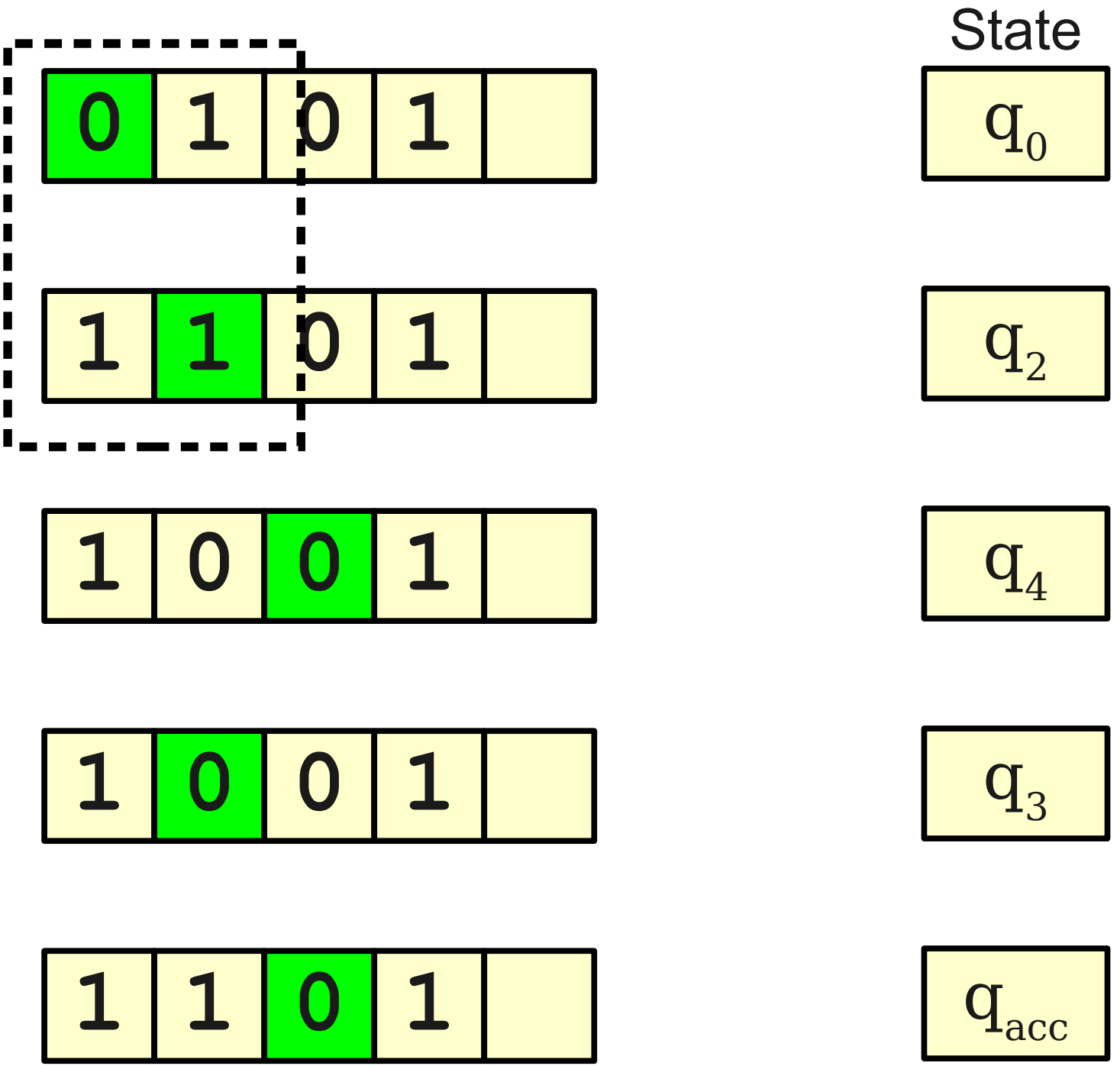
Polynomial-Time NTMs

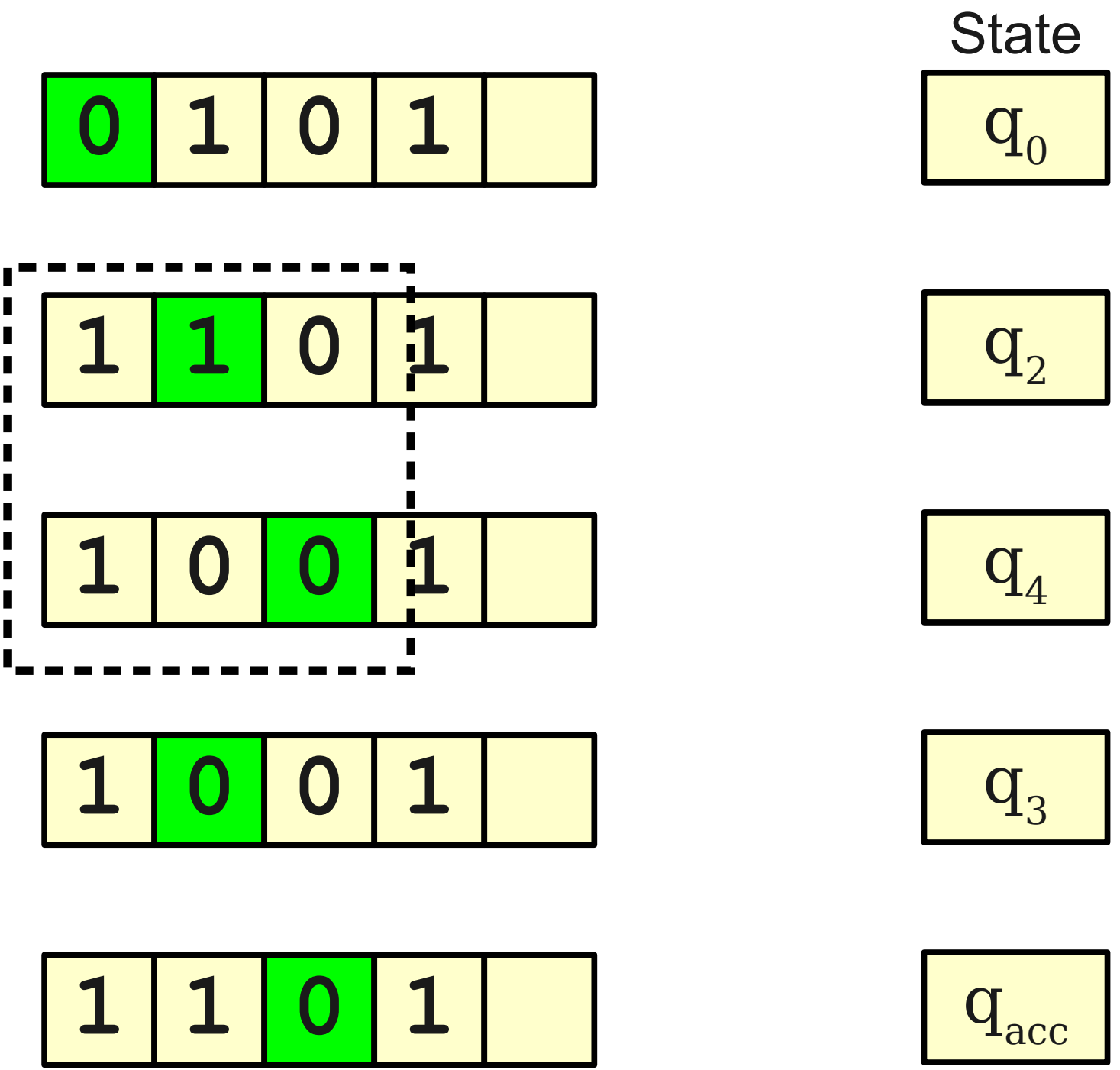
- Recall: The runtime of an NTM on a string w is the height of its computation tree on w.
- If an NTM runs in polynomial time, there is some polynomial $p(n)$ such that no execution of the NTM on a string w takes more than $p(|w|)$ time on any branch.
- This means the NTM never uses more than $p(|w|)$ tape on any branch of its computation on w.

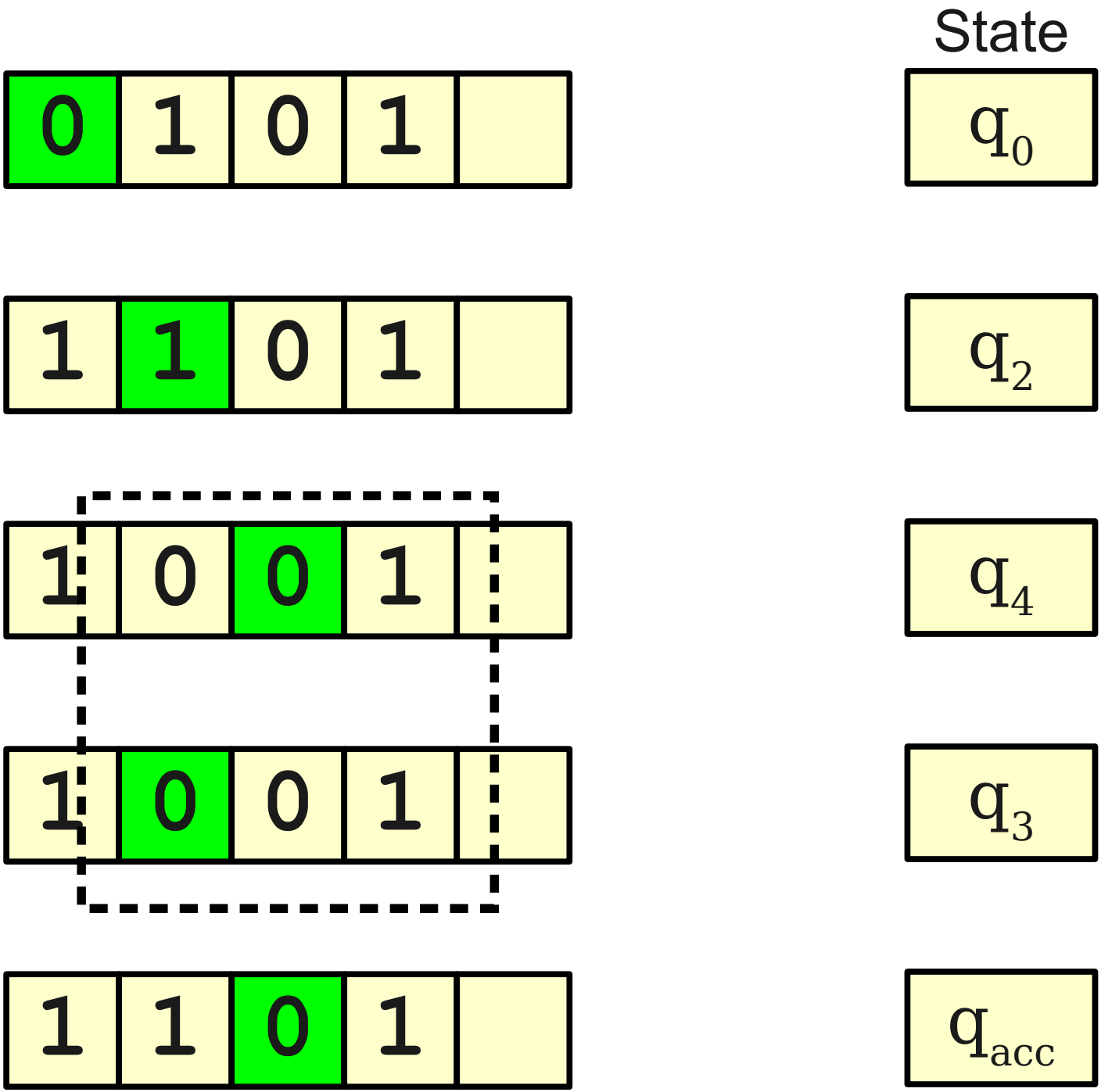


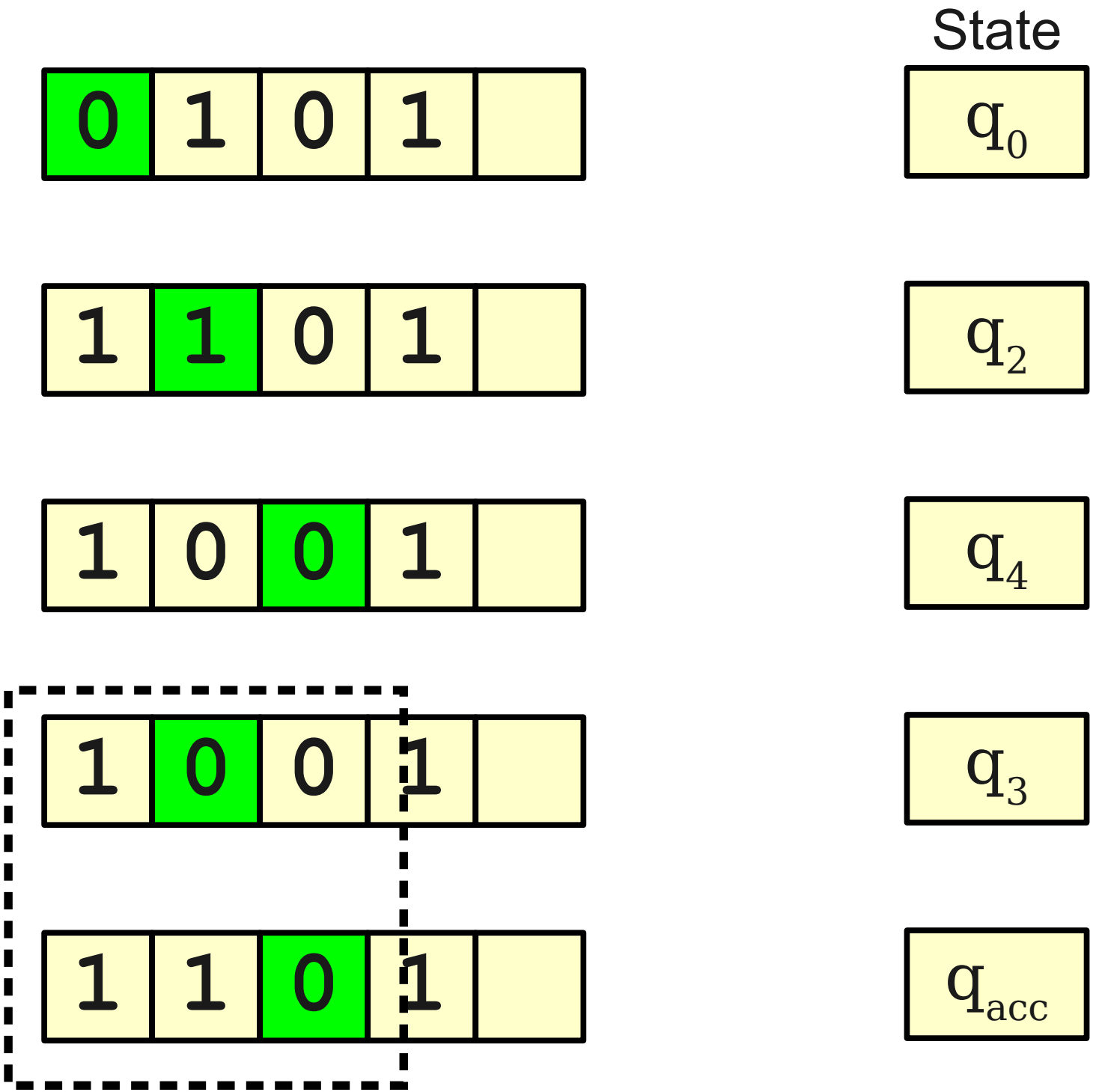

State
0 1 0 1
q_0
1 1 0 1
q_2
1 0 0 1
q_4
1 0 0 1
q_3
1 1 0 1
q_{acc}

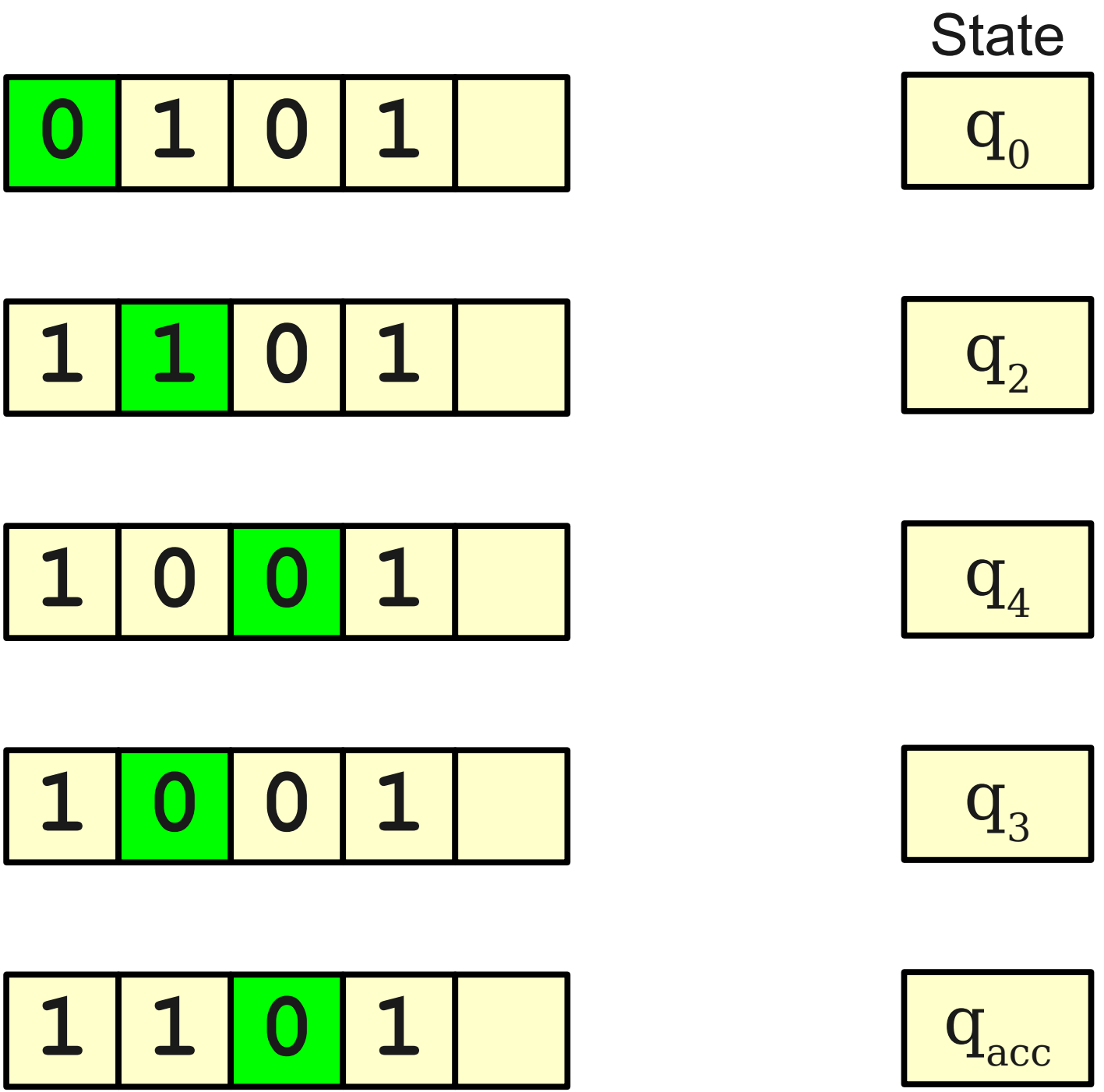

State
0 1 0 1
q_0
1 1 0 1
q_2
1 0 0 1
q_4
1 0 0 1
q_3
1 1 0 1
q_{acc}


State
0 1 0 1
q_0
1 1 0 1
q_2
1 0 0 1
q_4
1 0 0 1
q_3
1 1 0 1
q_{acc}


State
0 1 0 1
q_0
1 1 0 1
q_2
1 0 0 1
q_4
1 0 0 1
q_3
1 1 0 1
q_{acc}


State
0 1 0 1
q_0
1 1 0 1
q_2
1 0 0 1
q_4
1 0 0 1
q_3
1 1 0 1
q_{acc}


State
0 1 0 1
q_0
1 1 0 1
q_2
1 0 0 1
q_4
1 0 0 1
q_3
1 1 0 1
q_{acc}


State
0 1 0 1
q_0
1 1 0 1
q_2
1 0 0 1
q_4
1 0 0 1
q_3
1 1 0 1
q_{acc}

Proving Cook-Levin

- Build a PL formula that encodes the following idea:
  - Machine M begins with w written on its tape, followed by blanks.
  - Each step of the computation legally follows from the previous step.
  - The machine ends in an accepting state.
- This formula is satisfiable iff there is some series of choices M can make such that M accepts w.
- This formula has size polynomial in $|w|$.
- **See Sipser for Details.**

A Simpler **NP**-Complete Problem

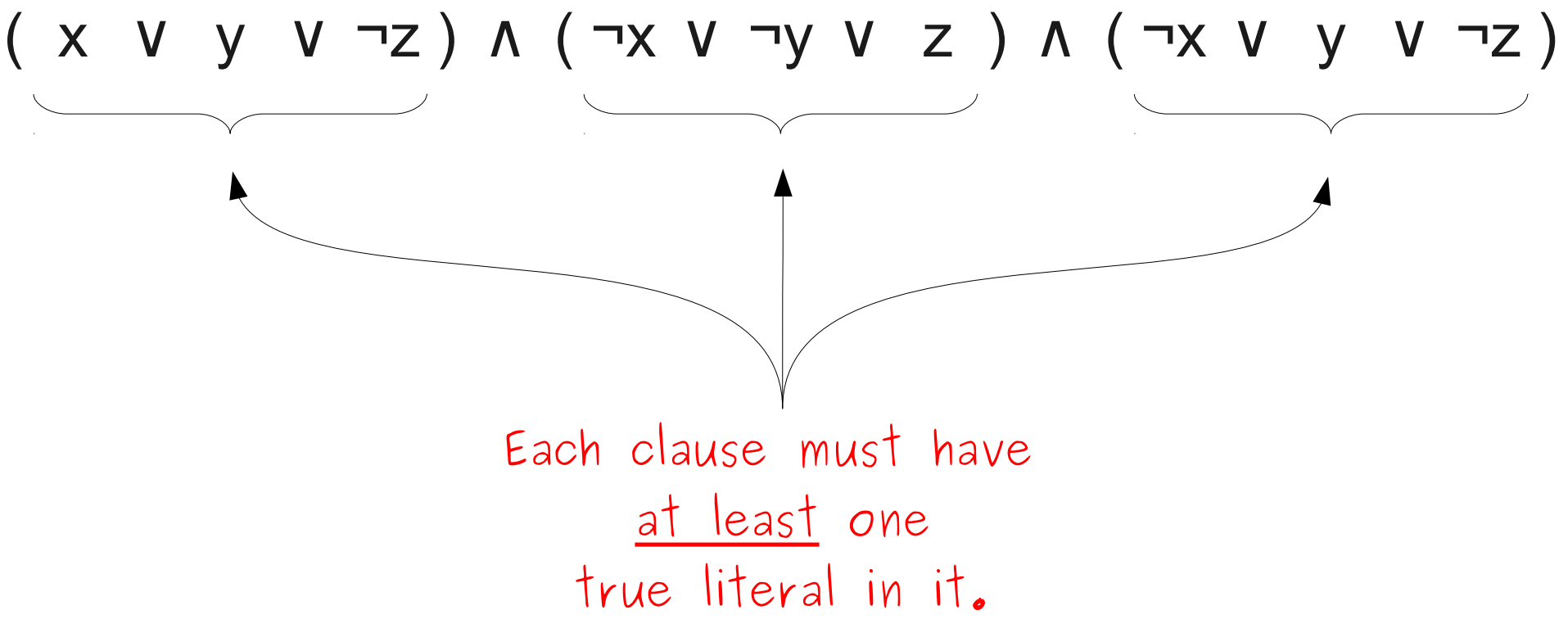
Literals and Clauses

- A **literal** in propositional logic is a variable or its negation:
  - x
  - $\neg y$
  - But not $x \wedge y$.
- A **clause** is a many-way OR (*disjunction*) of literals.
  - $\neg x \vee y \vee \neg z$
  - x
  - But not $x \vee \neg(y \vee z)$

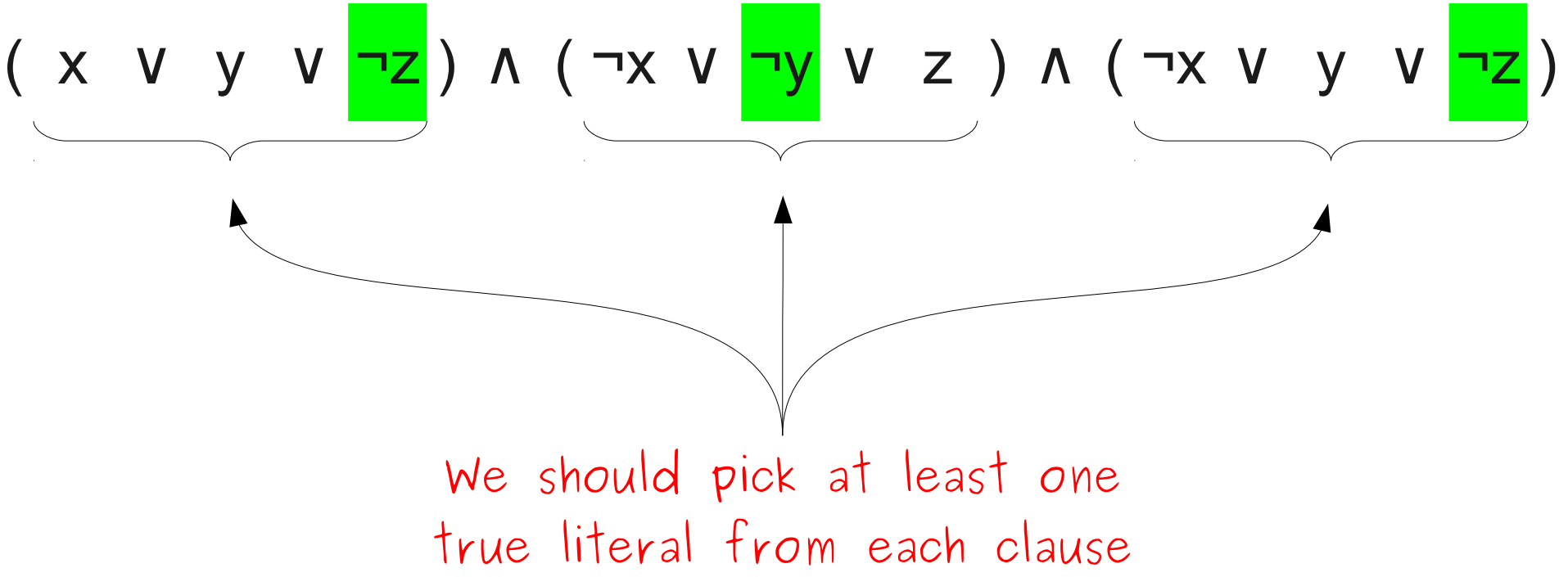
Conjunctive Normal Form

- A propositional logic formula φ is in **conjunctive normal form** (**CNF**) if it is the many-way AND (*conjunction*) of clauses.
  - $(x \vee y \vee z) \wedge (\neg x \vee \neg y) \wedge (x \vee y \vee z \vee \neg w)$
  - $x \vee z$
  - But not $(x \vee (y \wedge z)) \vee (x \vee y)$
- Only legal operators are $\neg$, $\vee$, $\wedge$.
- No nesting allowed.

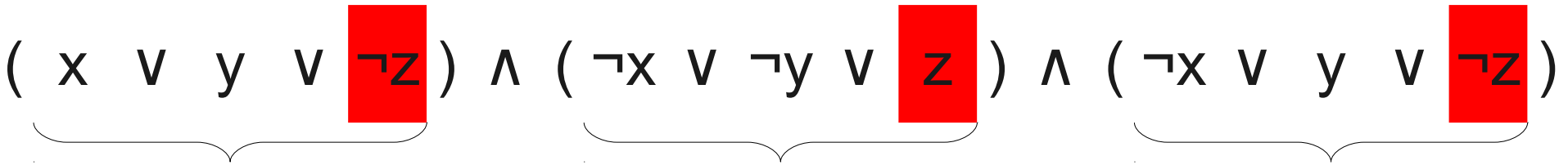
The Structure of CNF



The Structure of CNF



The Structure of CNF



… subject to the constraint that
we never choose a literal
and its negation

3-CNF

- A propositional formula is in **3-CNF** if
  - It is in CNF, and
  - Every clause has *exactly* three literals.
- For example:
  - $(x \vee y \vee z) \wedge (\neg x \vee \neg y \vee z)$
  - $(x \vee x \vee x) \wedge (y \vee \neg y \vee \neg x) \wedge (x \vee y \vee \neg y)$
  - But not $(x \vee y \vee z \vee w) \wedge (x \vee y)$
- The language **3SAT** is defined as follows:

3SAT = { ⟨φ⟩ | φ is a satisfiable 3-CNF formula }

Theorem (Cook-Levin): 3SAT is **NP**-Complete

Using the Cook-Levin Theorem

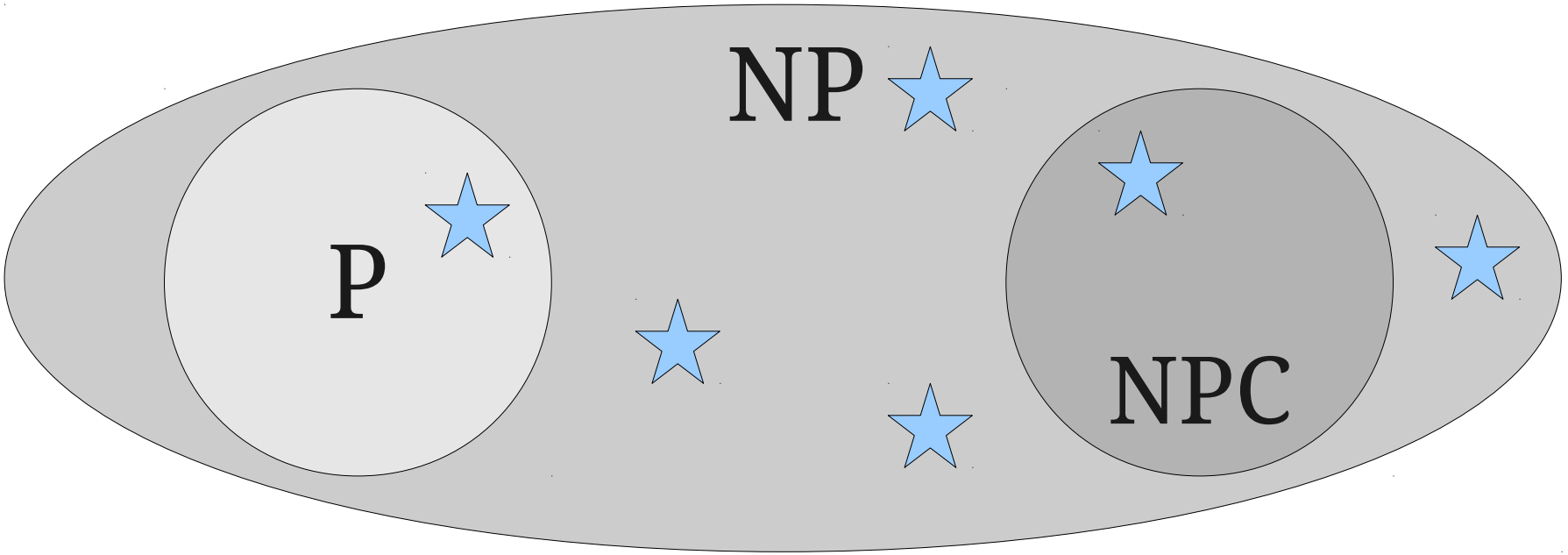
- When discussing decidability, we used the fact that $A_{TM} \notin \mathbf{R}$ as a starting point for finding other undecidable languages.
  - **Idea:** Reduce A_{TM} to some other language.
- When discussing **NP**-completeness, we will use the fact that 3SAT $\in$ **NPC** as a starting point for finding other **NPC** languages.
  - **Idea**: Reduce 3SAT to some other language.

NP-Completeness

- **Theorem**: If $L \in \mathbf{NPC}$, $L \leq_P L'$, and $L' \in \mathbf{NP}$, then $L' \in \mathbf{NPC}$.
- **Proof**: Consider any language $X \in \mathbf{NP}$. Since $L \in \mathbf{NPC}$, we know that $X \leq_P L$. Since $L \leq_P L'$, we have $X \leq_P L'$. Since our choice of X was arbitrary, this means L' is **NP**-hard. Since L' is **NP**-hard and $L' \in \mathbf{NP}$, we have $L' \in \mathbf{NPC}$. ■

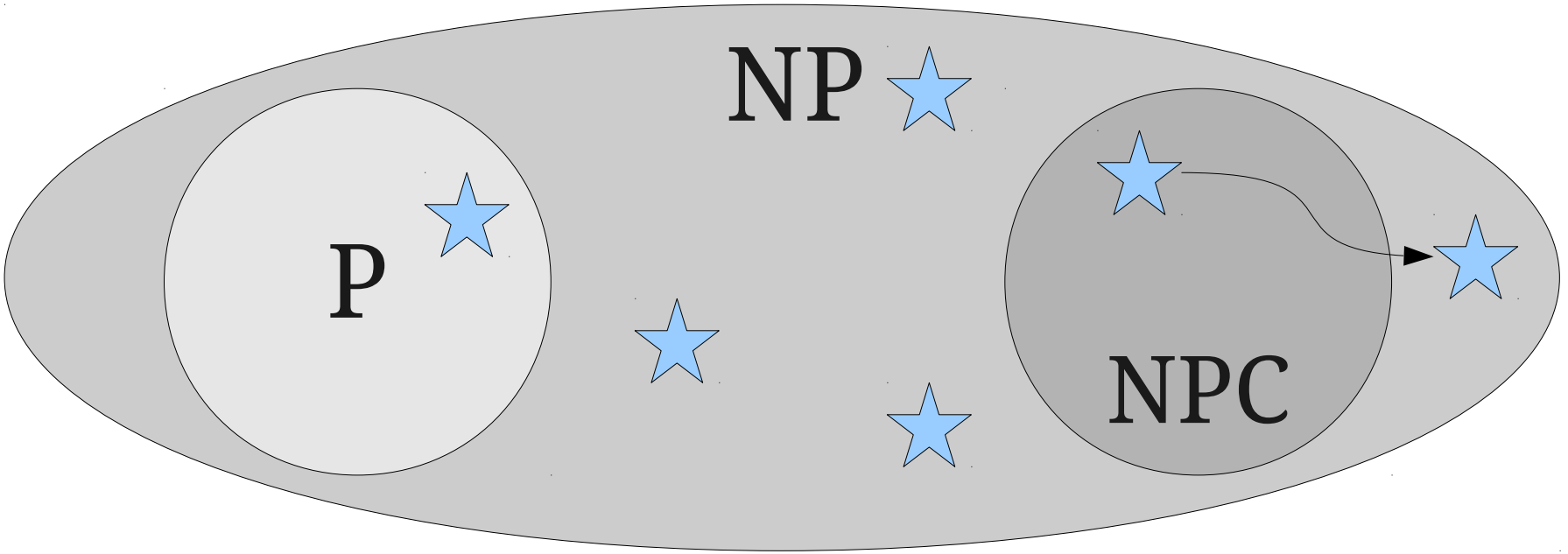
NP-Completeness

- **Theorem**: If $L \in$ **NPC**, $L \leq_P L'$, and $L' \in$ **NP**, then $L' \in$ **NPC**.
- **Proof**: Consider any language $X \in$ **NP**. Since $L \in$ **NPC**, we know that $X \leq_P L$. Since $L \leq_P L'$, we have $X \leq_P L'$. Since our choice of X was arbitrary, this means L' is **NP**-hard. Since L' is **NP**-hard and $L' \in$ **NP**, we have $L' \in$ **NPC**. ■



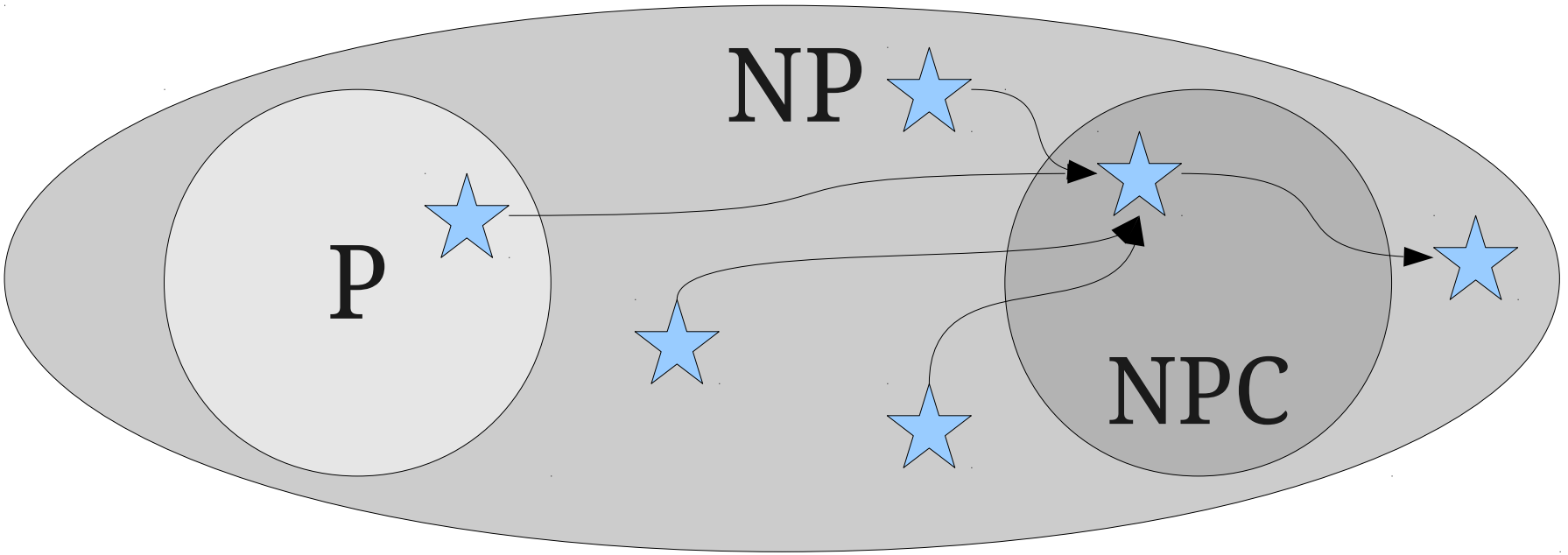
NP-Completeness

- **Theorem**: If $L \in$ **NPC**, $L \leq_P L'$, and $L' \in$ **NP**, then $L' \in$ **NPC**.
- **Proof**: Consider any language $X \in$ **NP**. Since $L \in$ **NPC**, we know that $X \leq_P L$. Since $L \leq_P L'$, we have $X \leq_P L'$. Since our choice of X was arbitrary, this means L' is **NP**-hard. Since L' is **NP**-hard and $L' \in$ **NP**, we have $L' \in$ **NPC**. ■



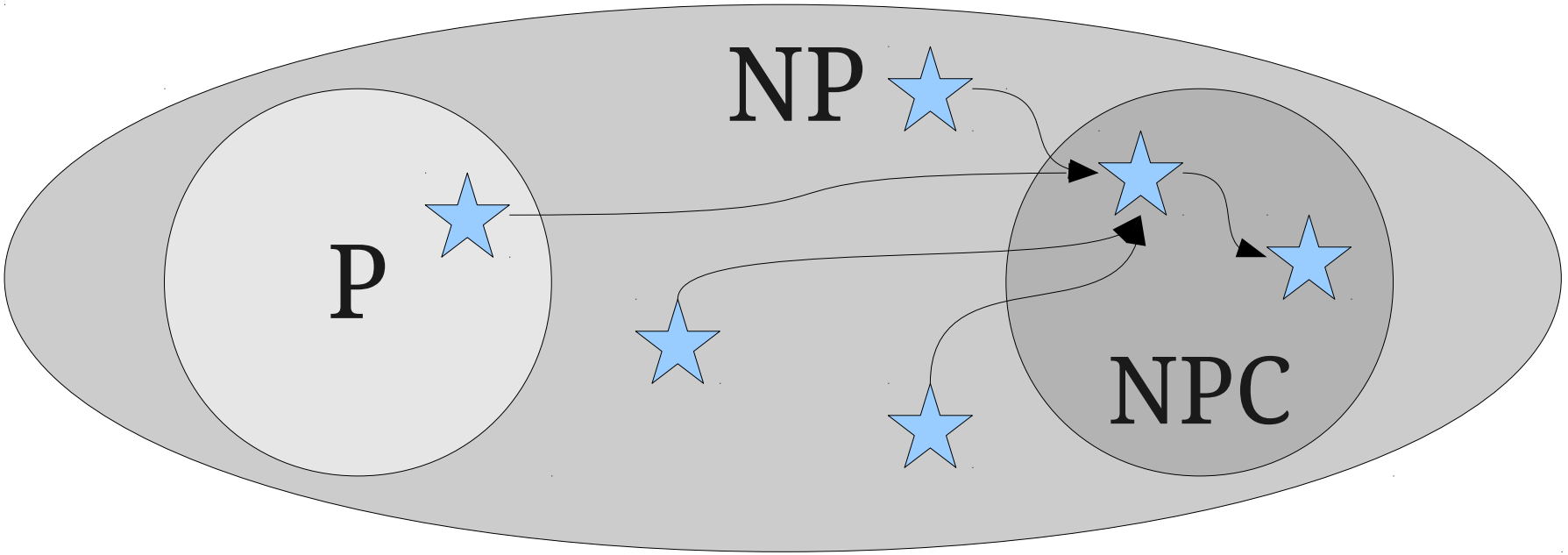
NP-Completeness

- **Theorem**: If $L \in$ **NPC**, $L \leq_P L'$, and $L' \in$ **NP**, then $L' \in$ **NPC**.
- **Proof**: Consider any language $X \in$ **NP**. Since $L \in$ **NPC**, we know that $X \leq_P L$. Since $L \leq_P L'$, we have $X \leq_P L'$. Since our choice of X was arbitrary, this means L' is **NP**-hard. Since L' is **NP**-hard and $L' \in$ **NP**, we have $L' \in$ **NPC**. ■



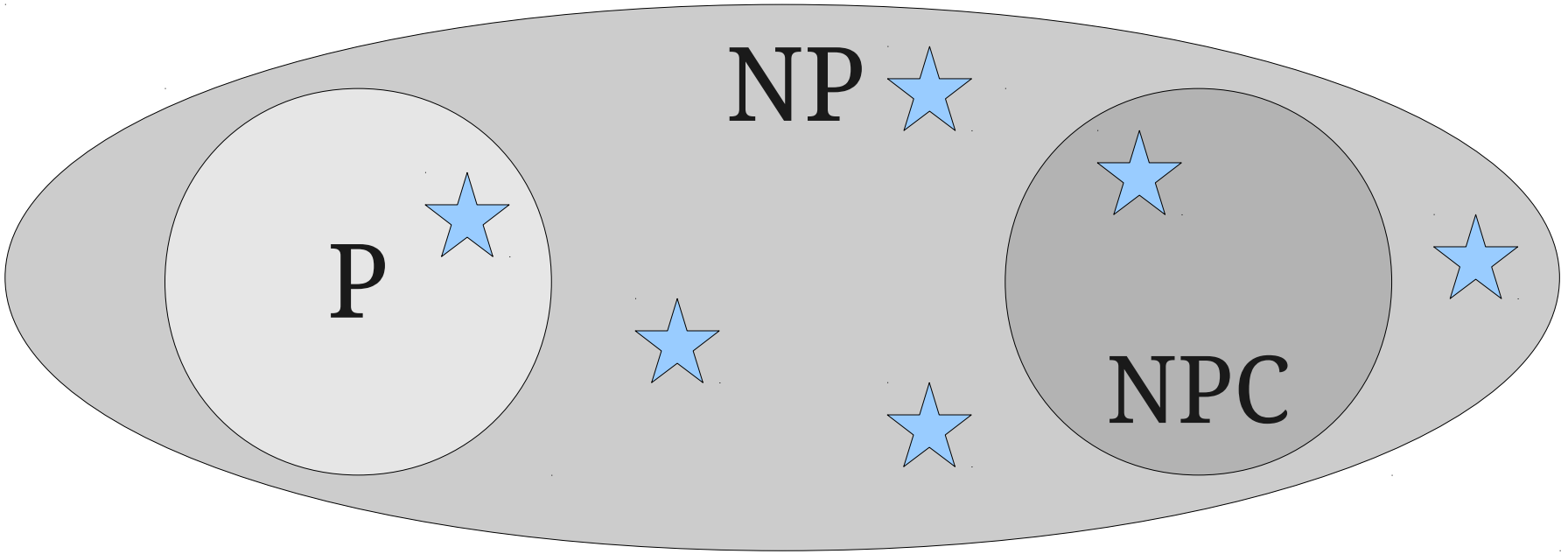
NP-Completeness

- **Theorem**: If $L \in$ **NPC**, $L \leq_P L'$, and $L' \in$ **NP**, then $L' \in$ **NPC**.
- **Proof**: Consider any language $X \in$ **NP**. Since $L \in$ **NPC**, we know that $X \leq_P L$. Since $L \leq_P L'$, we have $X \leq_P L'$. Since our choice of X was arbitrary, this means L' is **NP**-hard. Since L' is **NP**-hard and $L' \in$ **NP**, we have $L' \in$ **NPC**. ■



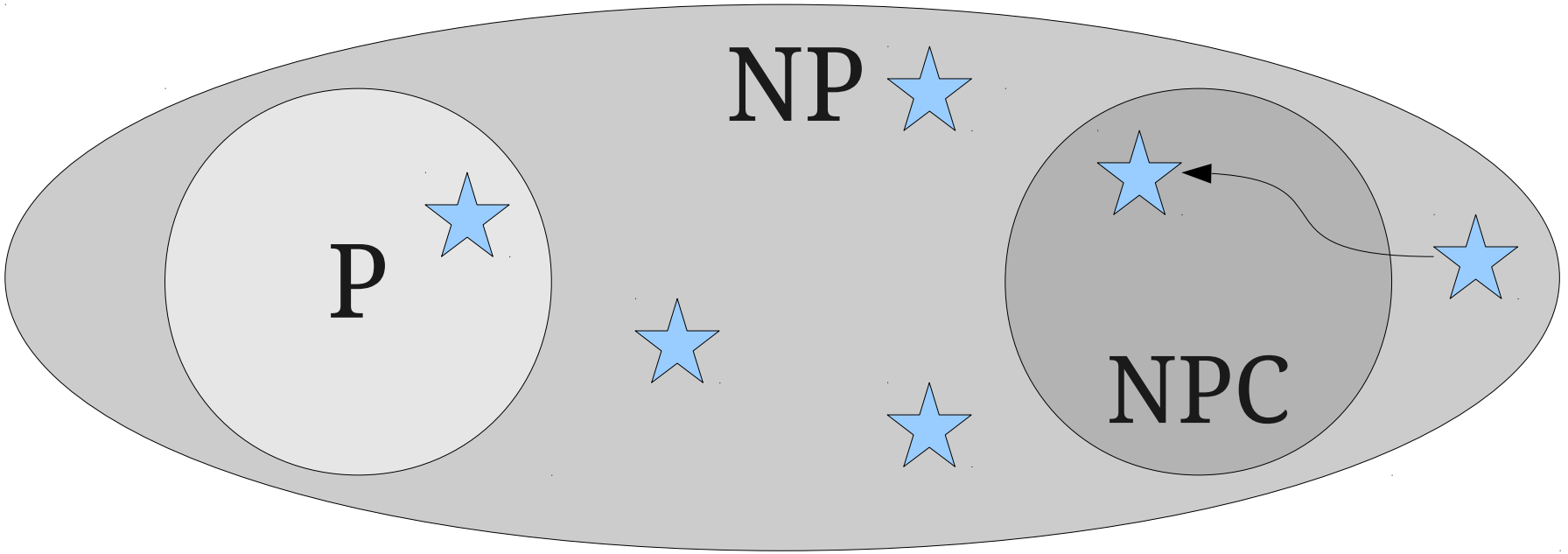
Be Careful!

- To prove that some language L is **NP**-complete, show that $L \in$ **NP**, then reduce some known **NP**-complete problem to L.
- **Do not** reduce L to a known **NP**-complete problem.
  - We already knew you could do this; *every* **NP** problems is reducible to any **NP**-complete problem!



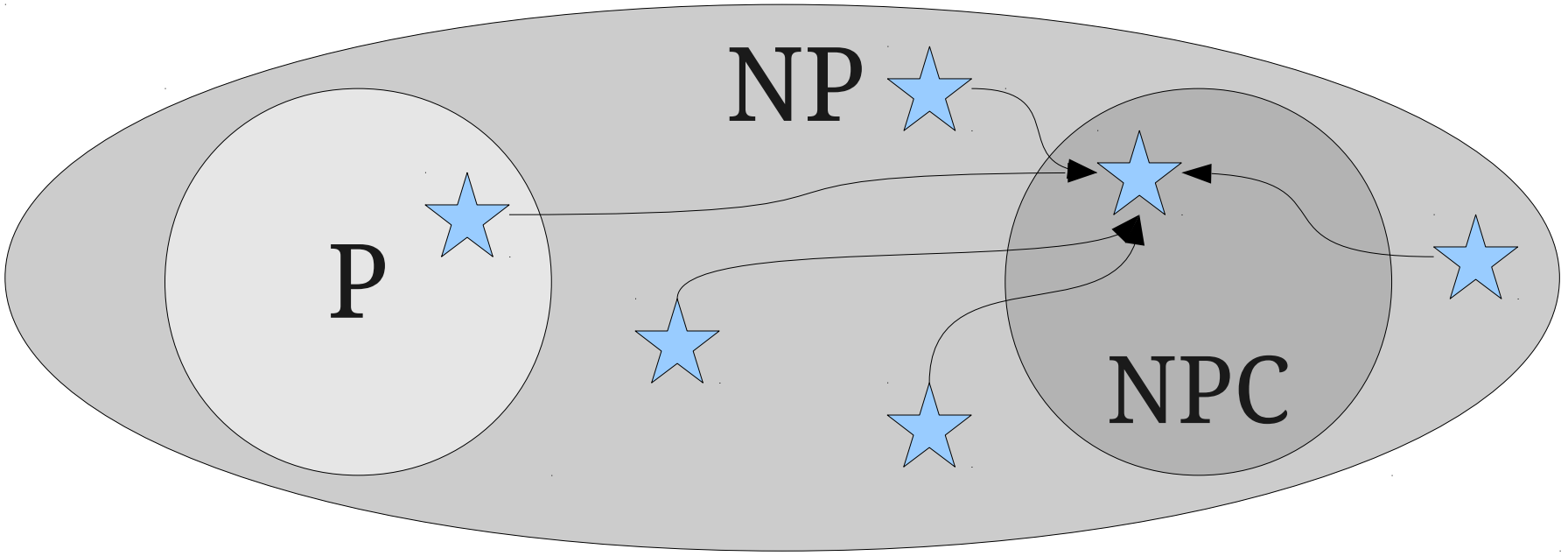
Be Careful!

- To prove that some language L is **NP**-complete, show that $L \in$ **NP**, then reduce some known **NP**-complete problem to L.
- **Do not** reduce L to a known **NP**-complete problem.
  - We already knew you could do this; *every* **NP** problems is reducible to any **NP**-complete problem!



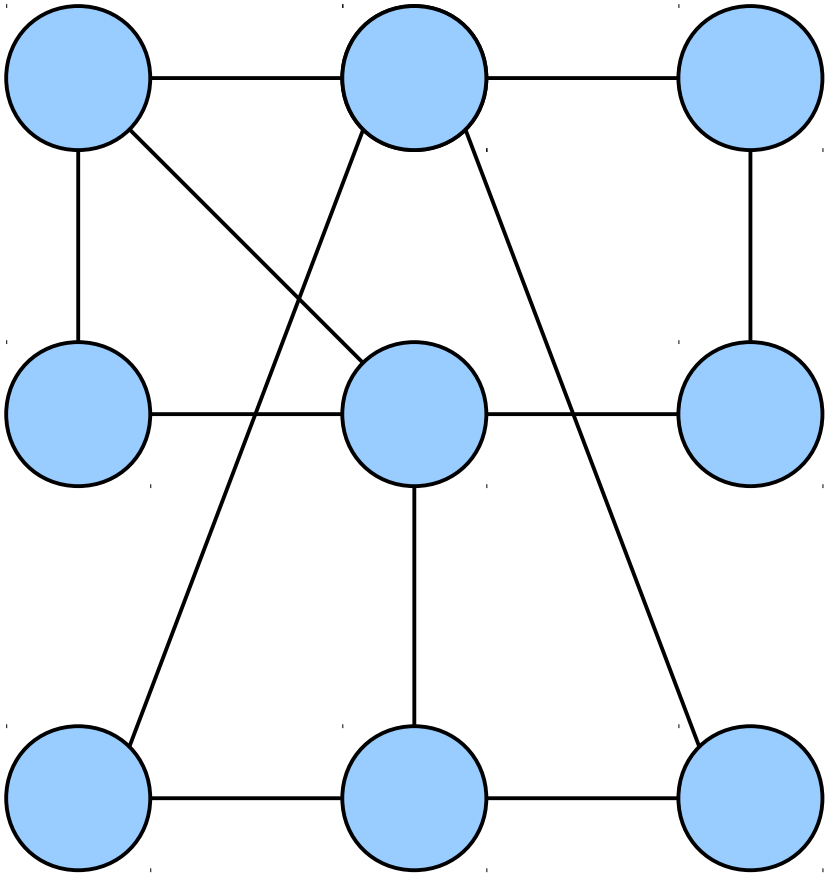
Be Careful!

- To prove that some language L is **NP**-complete, show that $L \in$ **NP**, then reduce some known **NP**-complete problem to L.
- **Do not** reduce L to a known **NP**-complete problem.
  - We already knew you could do this; *every* **NP** problems is reducible to any **NP**-complete problem!

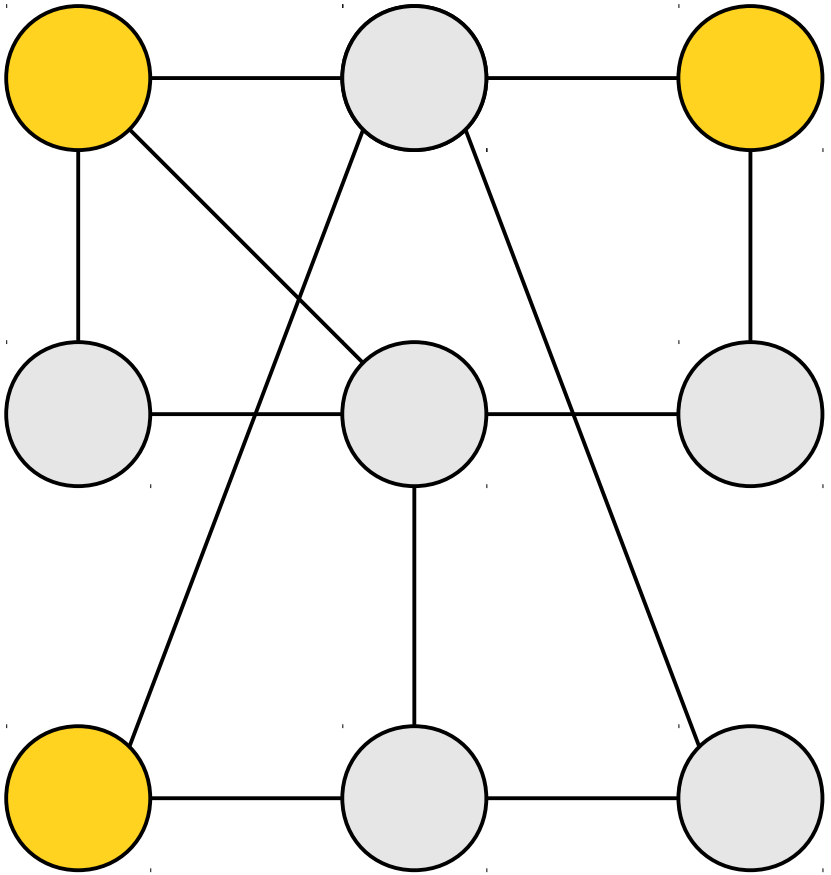


So what other problems are **NP**-complete?

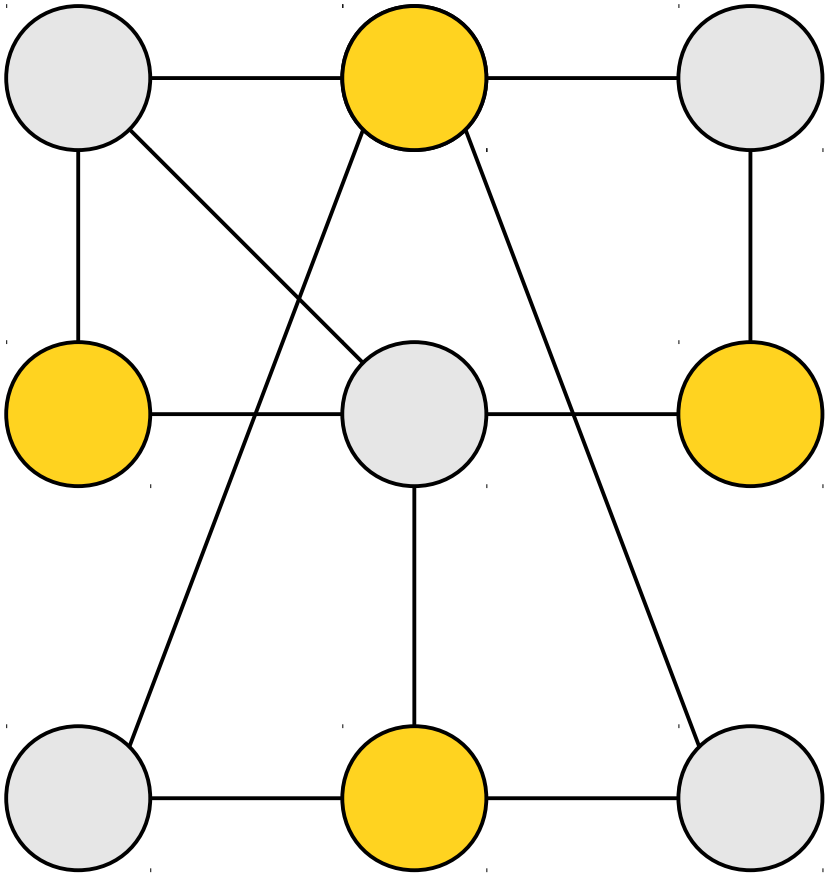
An **independent set** in an undirected graph
is a set of vertices that have no edges between them



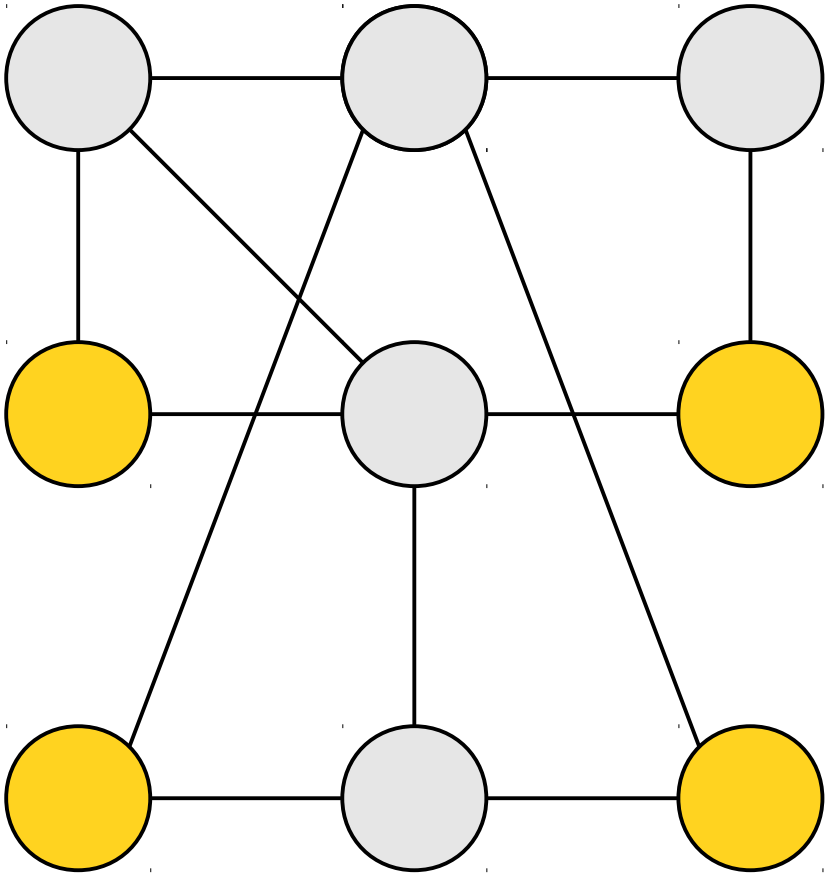
An **independent set** in an undirected graph
is a set of vertices that have no edges between them



An **independent set** in an undirected graph
is a set of vertices that have no edges between them



An **independent set** in an undirected graph
is a set of vertices that have no edges between them



An **independent set** in an undirected graph
is a set of vertices that have no edges between them

The Independent Set Problem

- Given an undirected graph G and a natural number n, the **independent set problem** is

  Does G contain an independent set of size at least n?

- As a formal language:

  INDSET = { $\langle G, n \rangle$ | G is an undirected graph with an independent set of size at least n }

INDSET ∈ **NP**

- The independent set problem is in **NP**.
- Here is a polynomial-time verifier that checks whether S is an n-element independent set:
  - V = "On input $\langle G, n, S \rangle$:
    - If $|S| < n$, reject.
    - For each edge in G, if both endpoints are in S, reject.
    - Otherwise, accept."

INDSET ∈ **NPC**

- The *INDSET* problem is **NP**-complete.
- To prove this, we will find a polynomial-time reduction from 3SAT to *INDSET*.
- Goal: Given a 3CNF formula φ, construct a graph G and number n such that φ is satisfiable iff G has an independent set of size n.
- How can we accomplish this?

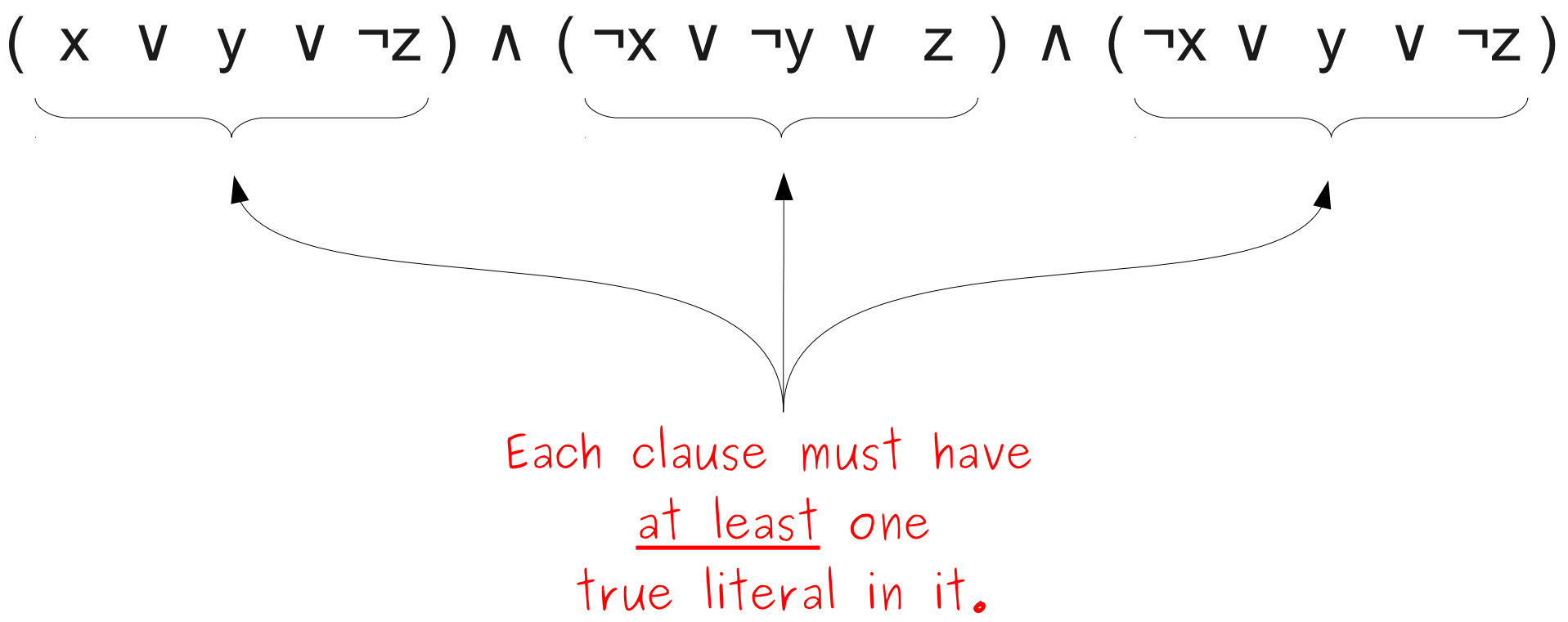
The Structure of 3CNF

$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$

The Structure of 3CNF

$$(\underbrace{x \vee y \vee \neg z}) \wedge (\underbrace{\neg x \vee \neg y \vee z}) \wedge (\underbrace{\neg x \vee y \vee \neg z})$$

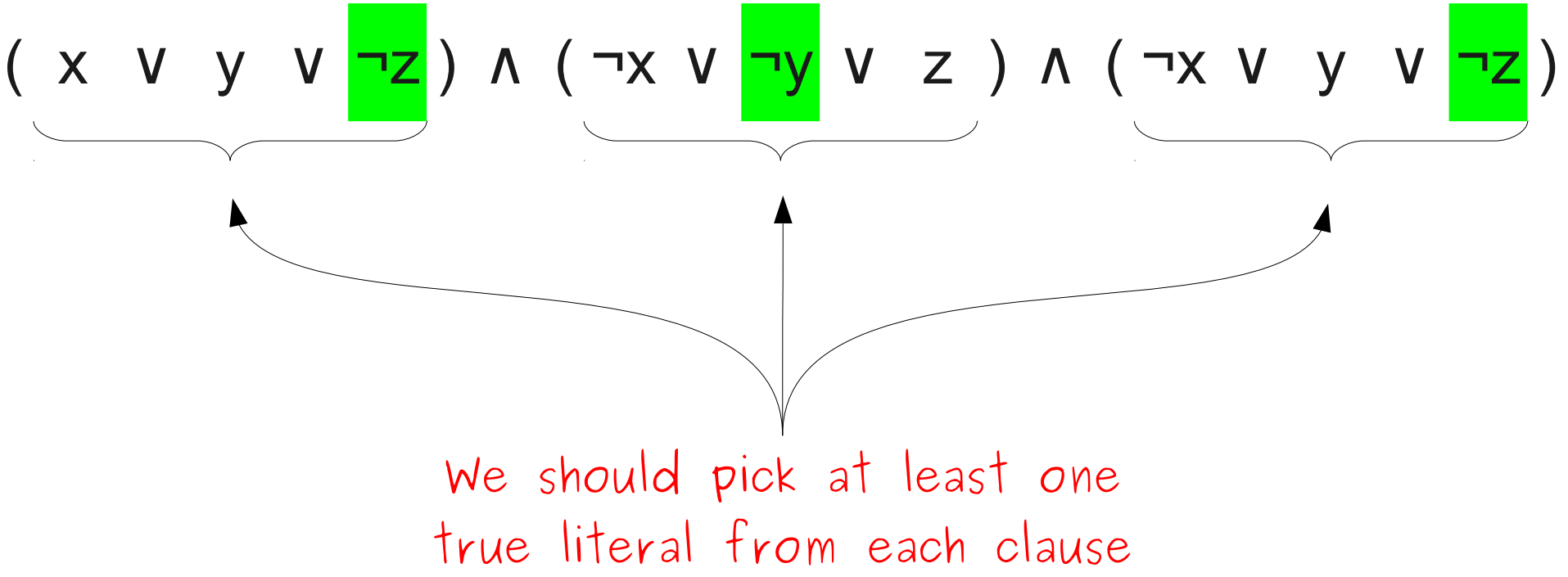
The Structure of 3CNF



The Structure of 3CNF

$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$

The Structure of 3CNF



The Structure of 3CNF

$$(\underbrace{x \vee y \vee \neg z}) \wedge (\underbrace{\neg x \vee \neg y \vee z}) \wedge (\underbrace{\neg x \vee y \vee \neg z})$$

The Structure of 3CNF

$$(\underbrace{x \vee y \vee \neg z}) \wedge (\underbrace{\neg x \vee \neg y \vee z}) \wedge (\underbrace{\neg x \vee y \vee \neg z})$$

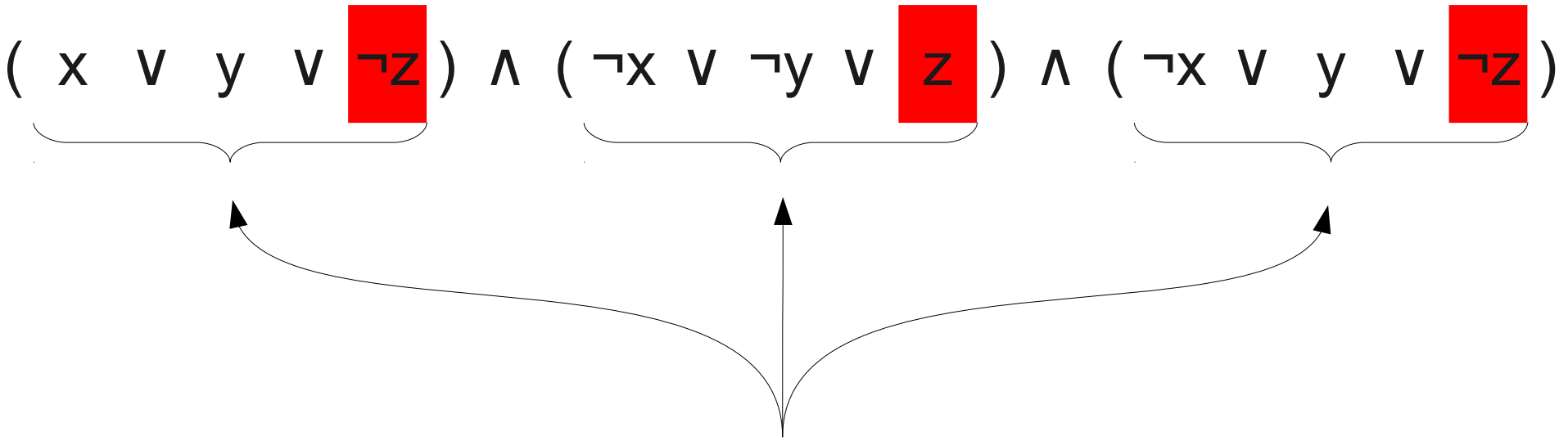
The Structure of 3CNF

$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$

The Structure of 3CNF

$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$

The Structure of 3CNF



… subject to the constraint that
we never choose a literal
and its negation

From 3SAT to INDSET

- To convert a 3SAT instance φ to an *INDSET* instance, we need a graph G and number n such that an independent set of size at least n in G
  - gives us a way to choose which literal in each clause of φ should be true,
  - doesn't simultaneously choose a literal and its negation, and
  - has size polynomially large in the length of the formula φ.

From 3SAT to INDSET

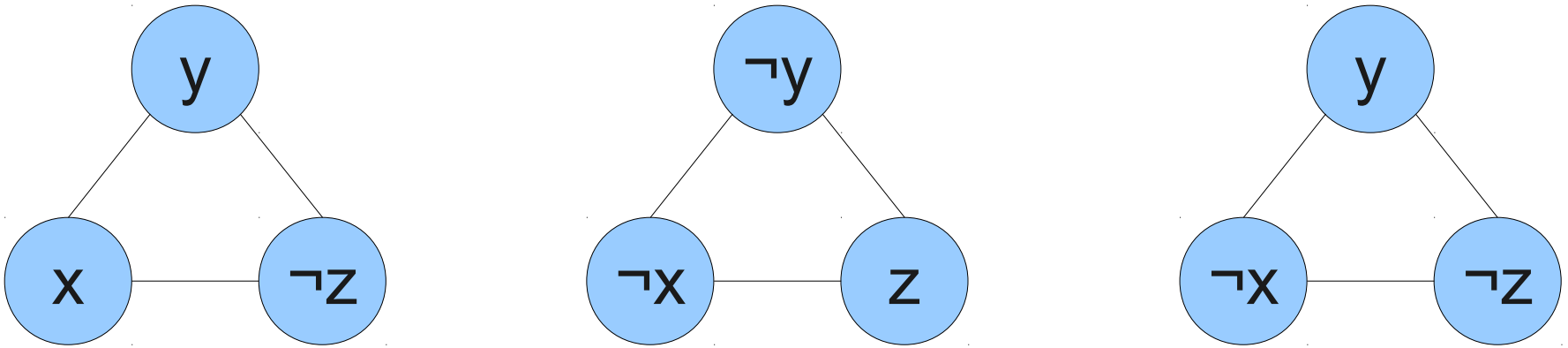
$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$

From 3SAT to INDSET

$$(\underbrace{x \vee y \vee \neg z}) \wedge (\underbrace{\neg x \vee \neg y \vee z}) \wedge (\underbrace{\neg x \vee y \vee \neg z})$$

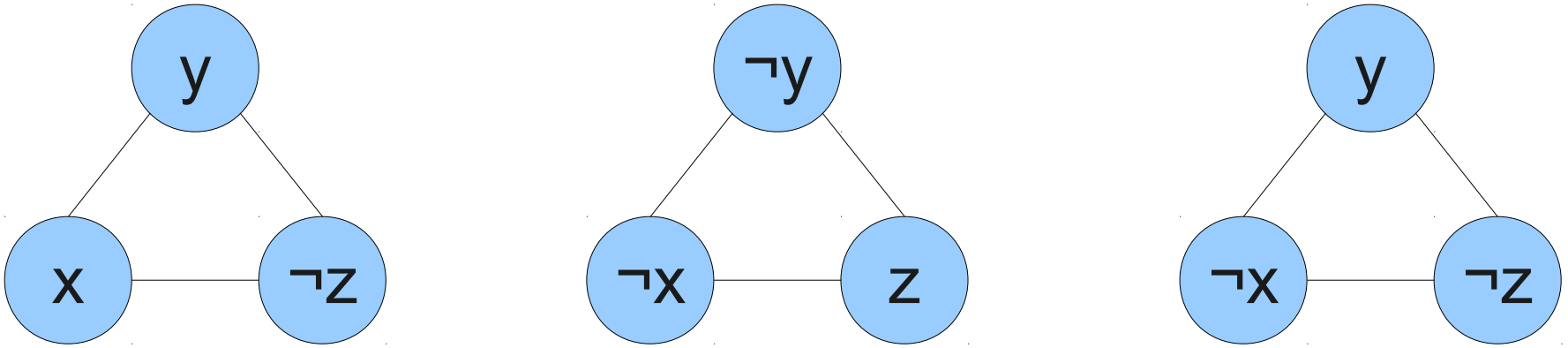
From 3SAT to INDSET

$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$



From 3SAT to INDSET

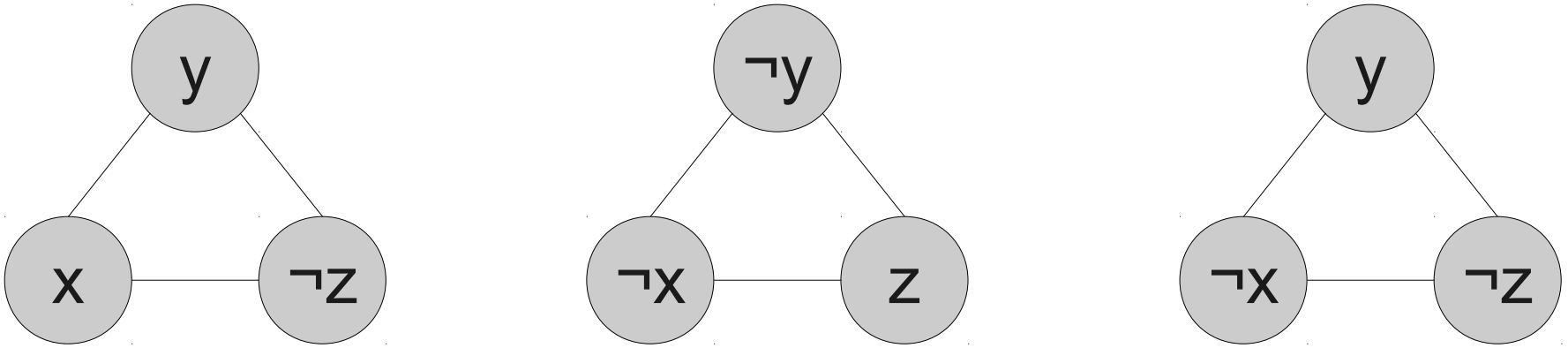
$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$



Any independent set in this graph chooses exactly one literal from each clause to be true.

From 3SAT to INDSET

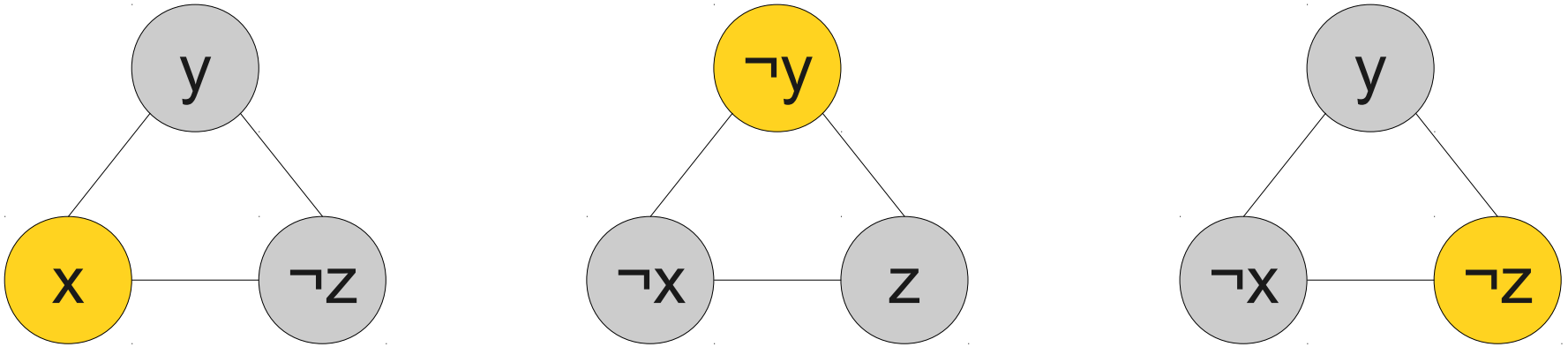
$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$



Any independent set in this graph chooses exactly one literal from each clause to be true.

From 3SAT to INDSET

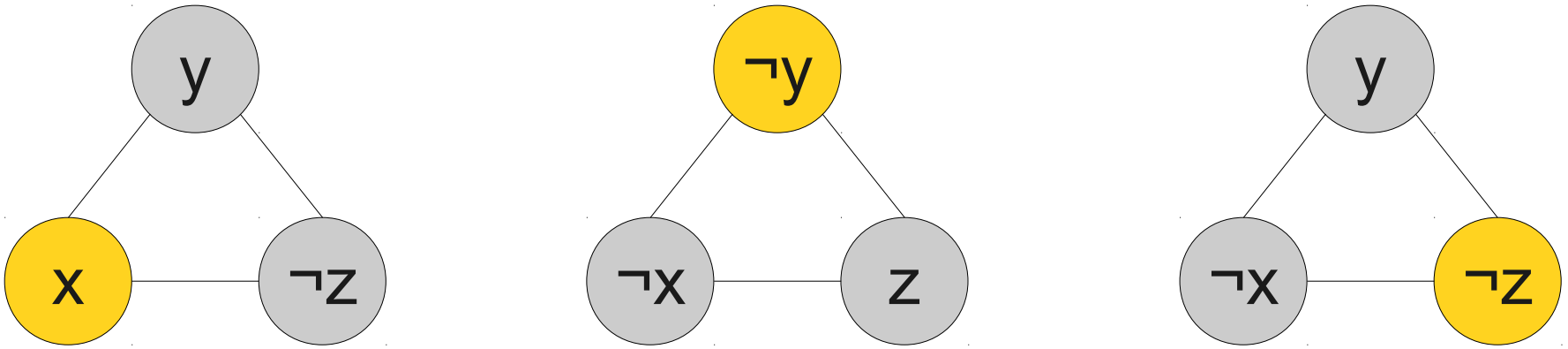
$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$



Any independent set in this graph chooses exactly one literal from each clause to be true.

From 3SAT to INDSET

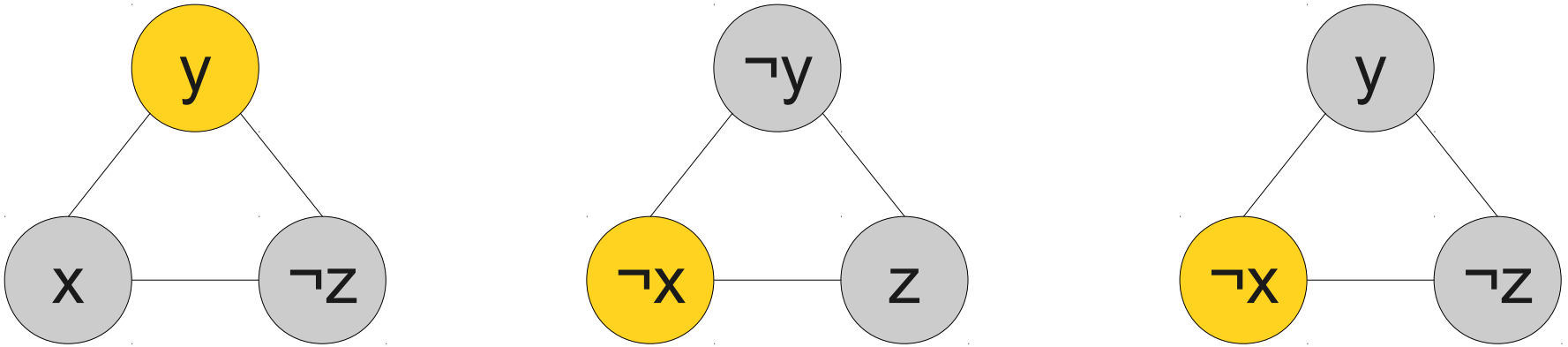
$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$



Any independent set in this graph chooses exactly one literal from each clause to be true.

From 3SAT to INDSET

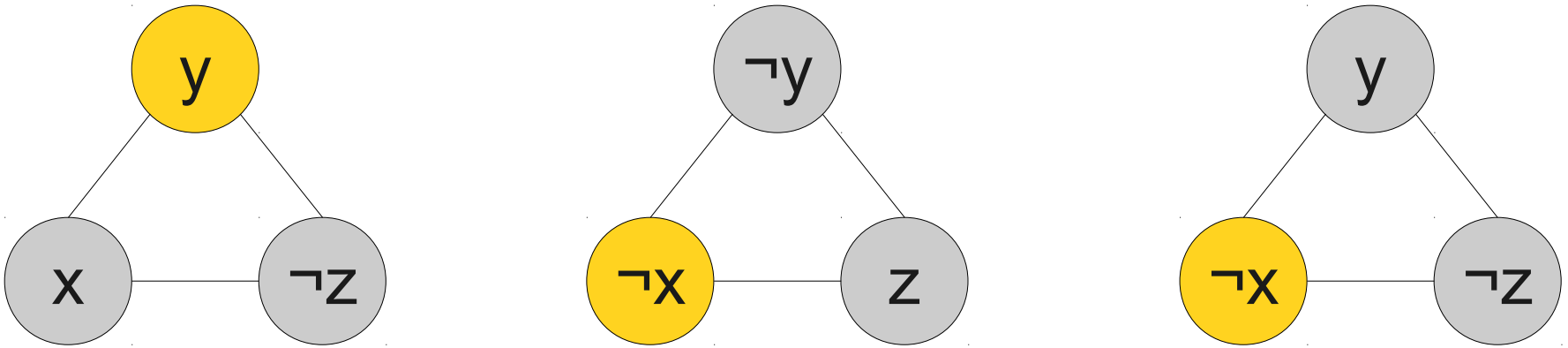
$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$



Any independent set in this graph chooses exactly one literal from each clause to be true.

From 3SAT to INDSET

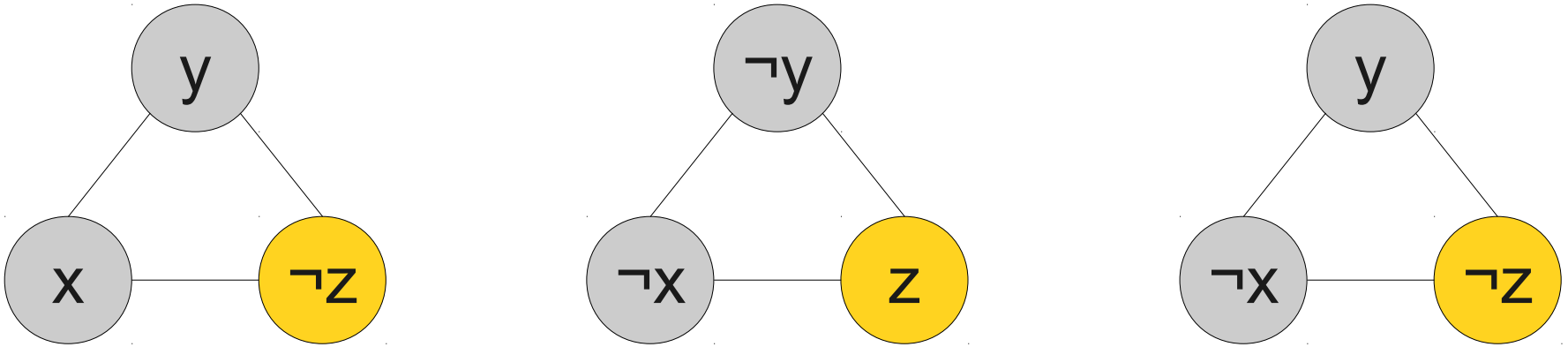
$$(\ x \ \vee \ y \ \vee \ \neg z\ ) \wedge (\ \neg x \vee \neg y \vee \ z\ ) \wedge (\ \neg x \vee \ y \ \vee \neg z\ )$$



Any independent set in this graph chooses exactly one literal from each clause to be true.

From 3SAT to INDSET

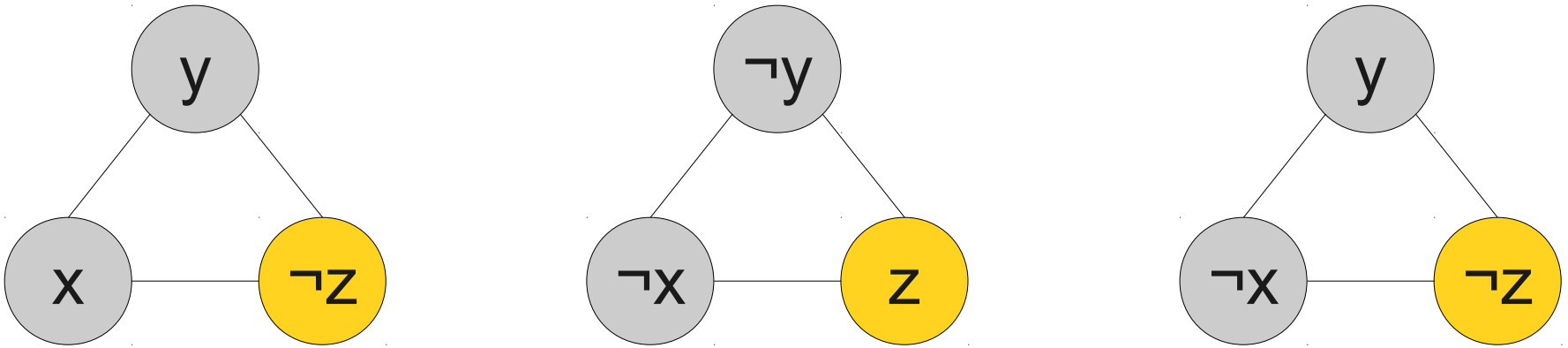
$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$



Any independent set in this graph chooses exactly one literal from each clause to be true.

From 3SAT to INDSET

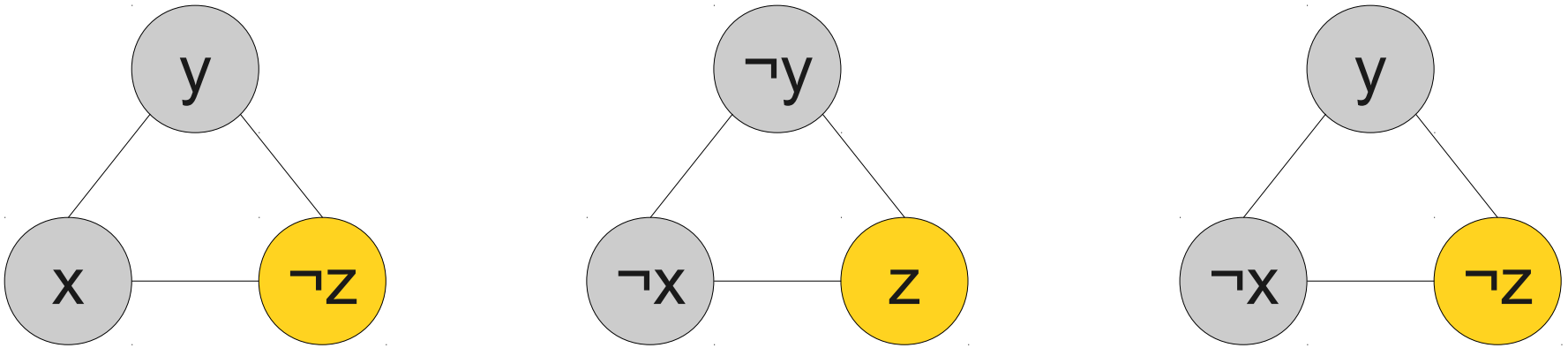
$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$



Any independent set in this graph chooses exactly one literal from each clause to be true.

From 3SAT to INDSET

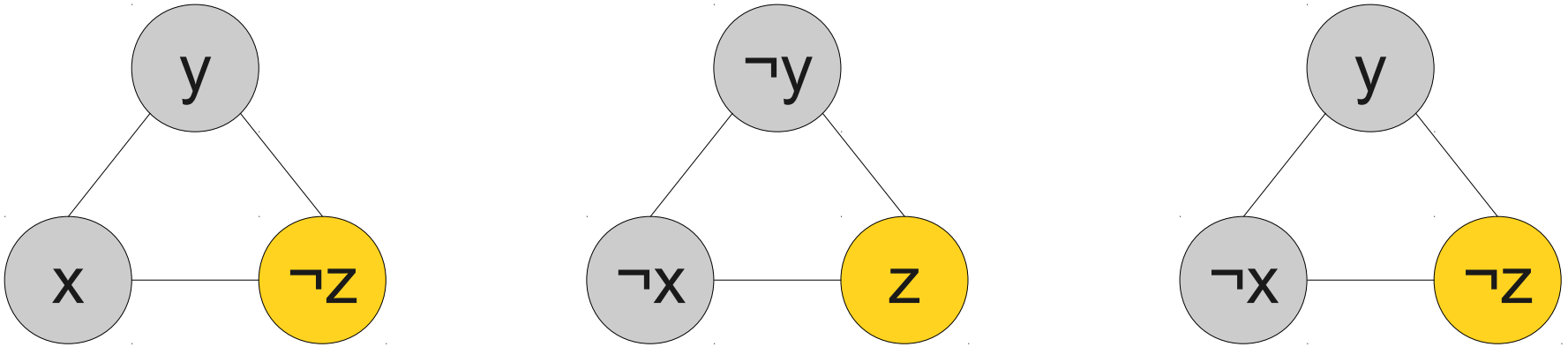
$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$



Any independent set in this graph chooses exactly one literal from each clause to be true.

From 3SAT to INDSET

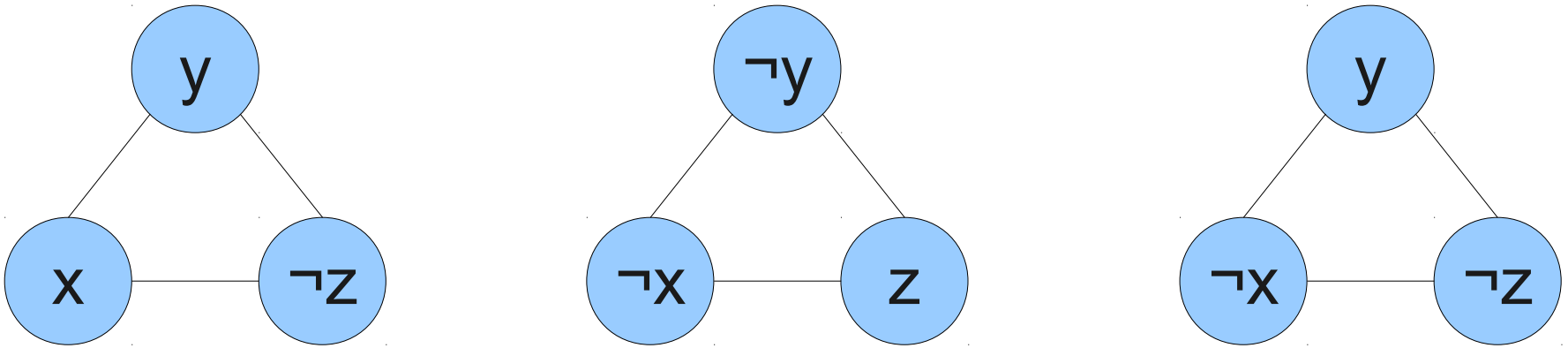
$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$



We need a way to ensure we never pick a literal and its negation.

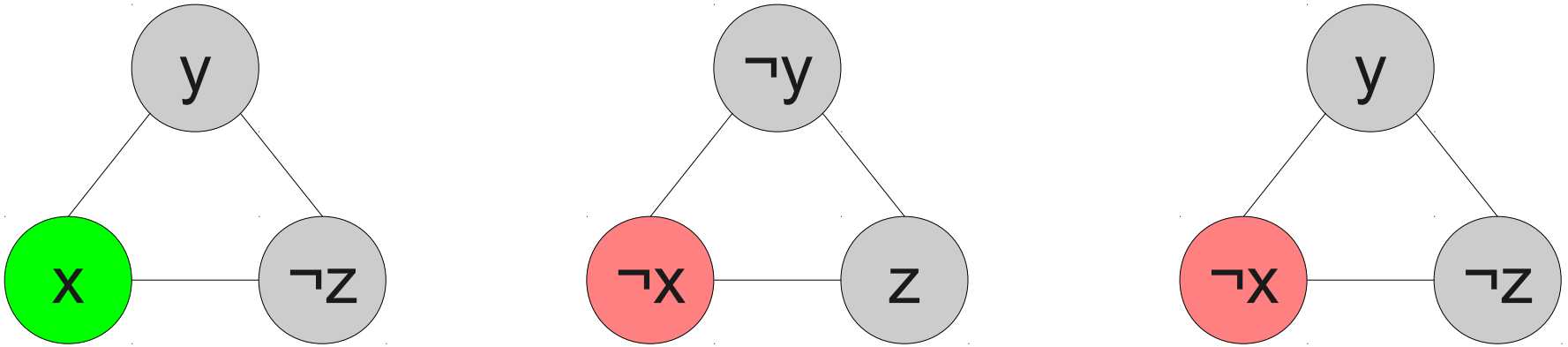
From 3SAT to INDSET

$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$

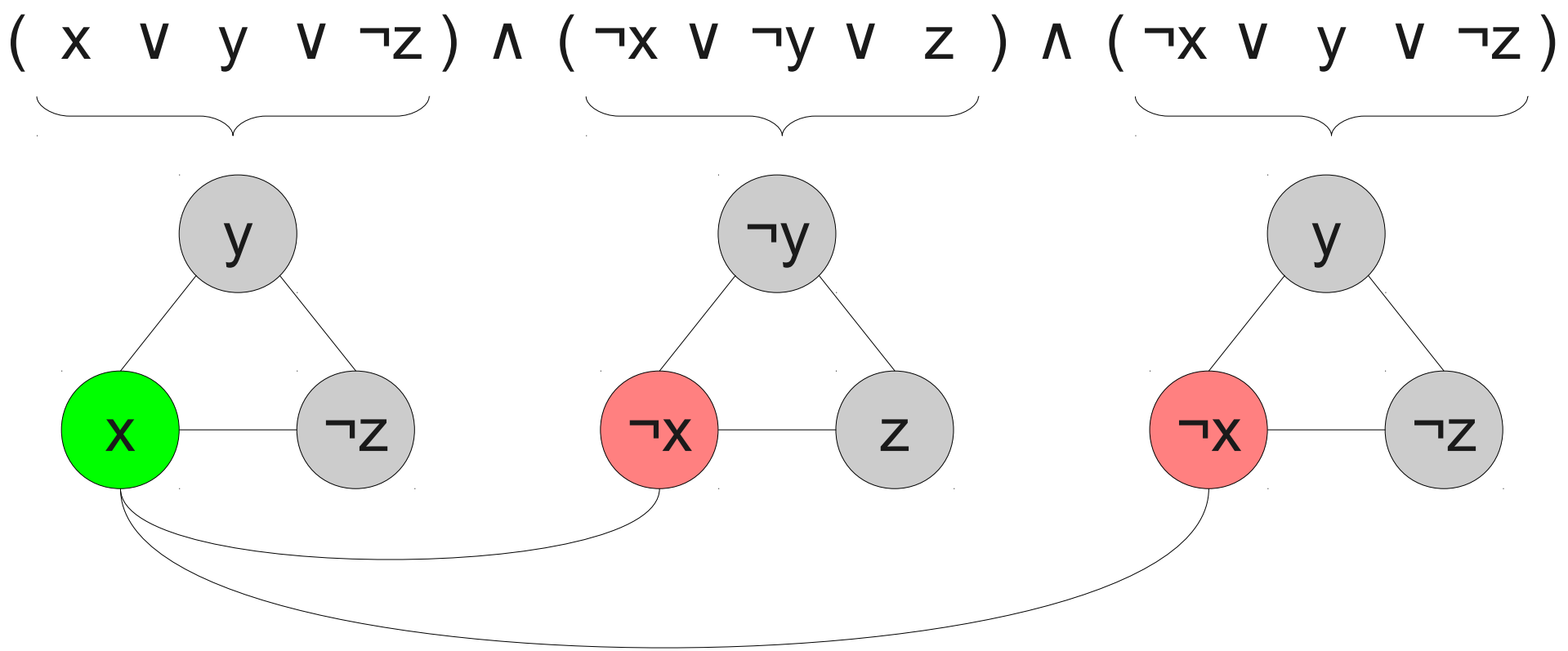


From 3SAT to INDSET

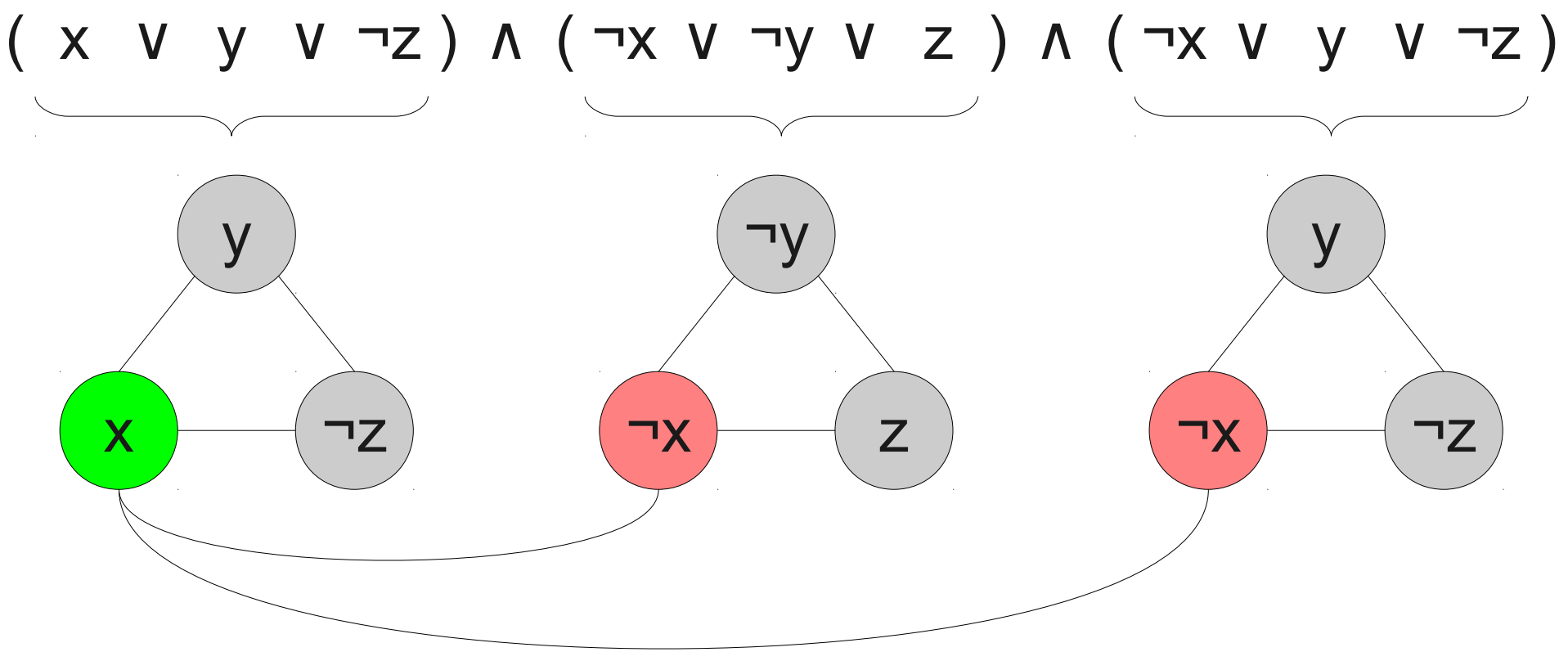
$$(\ x \ \vee \ y \ \vee \neg z\ ) \wedge (\ \neg x \vee \neg y \vee \ z\ ) \wedge (\ \neg x \vee \ y \ \vee \neg z\ )$$



From 3SAT to INDSET

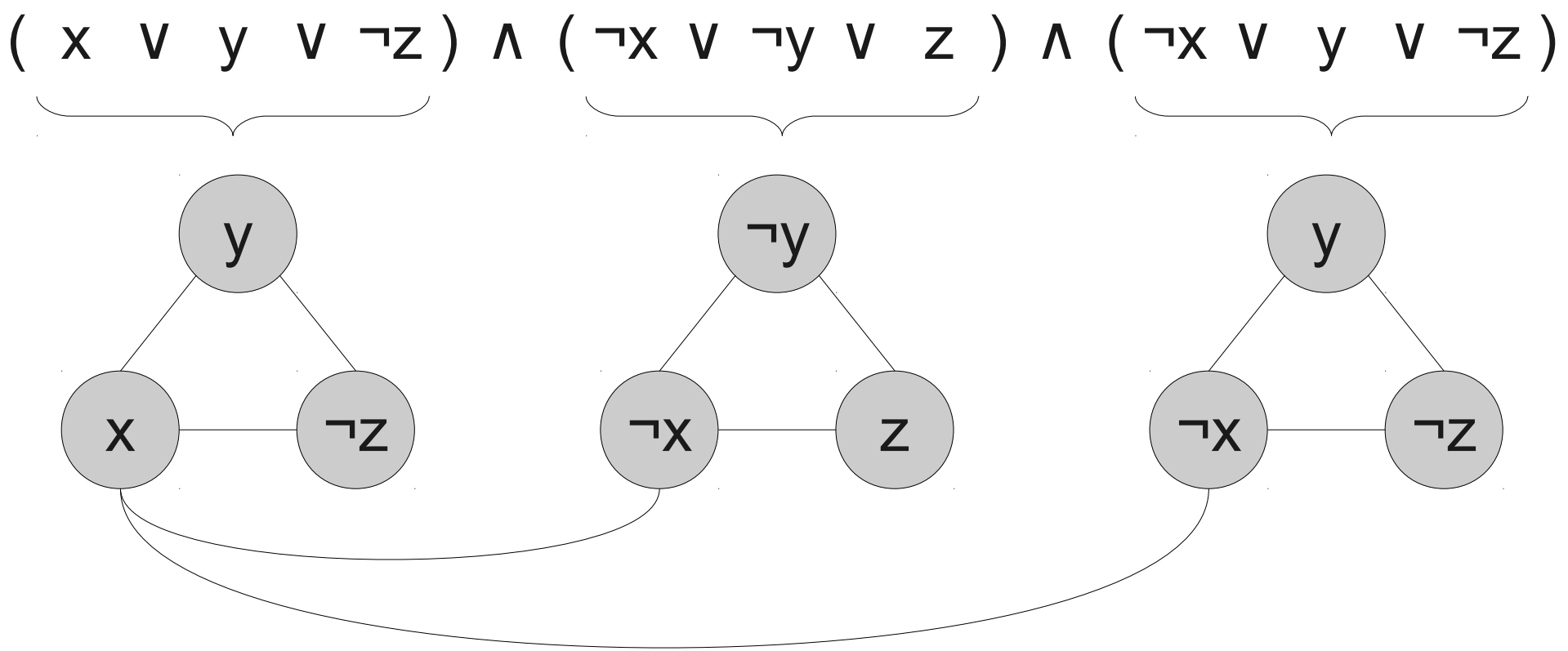


From 3SAT to INDSET

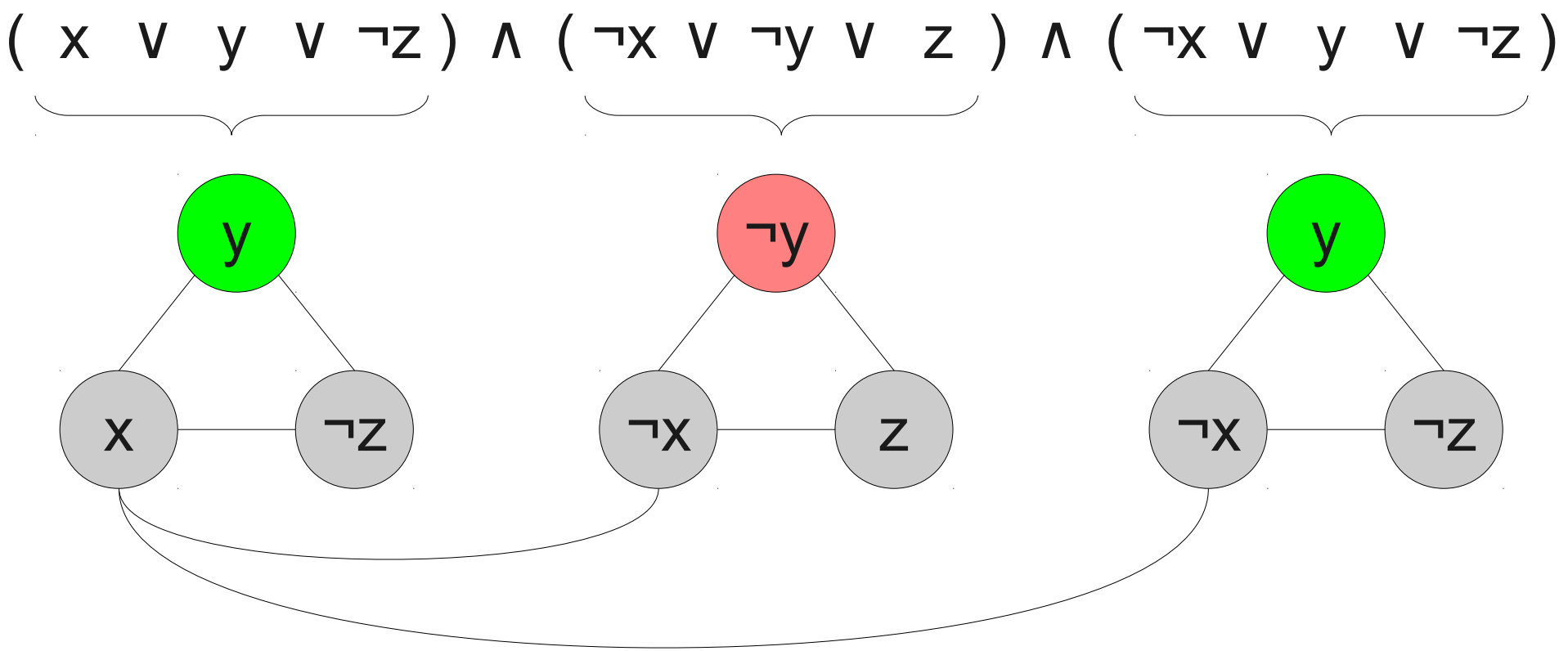


No independent set in this graph can choose two nodes labeled **x** and **¬x**.

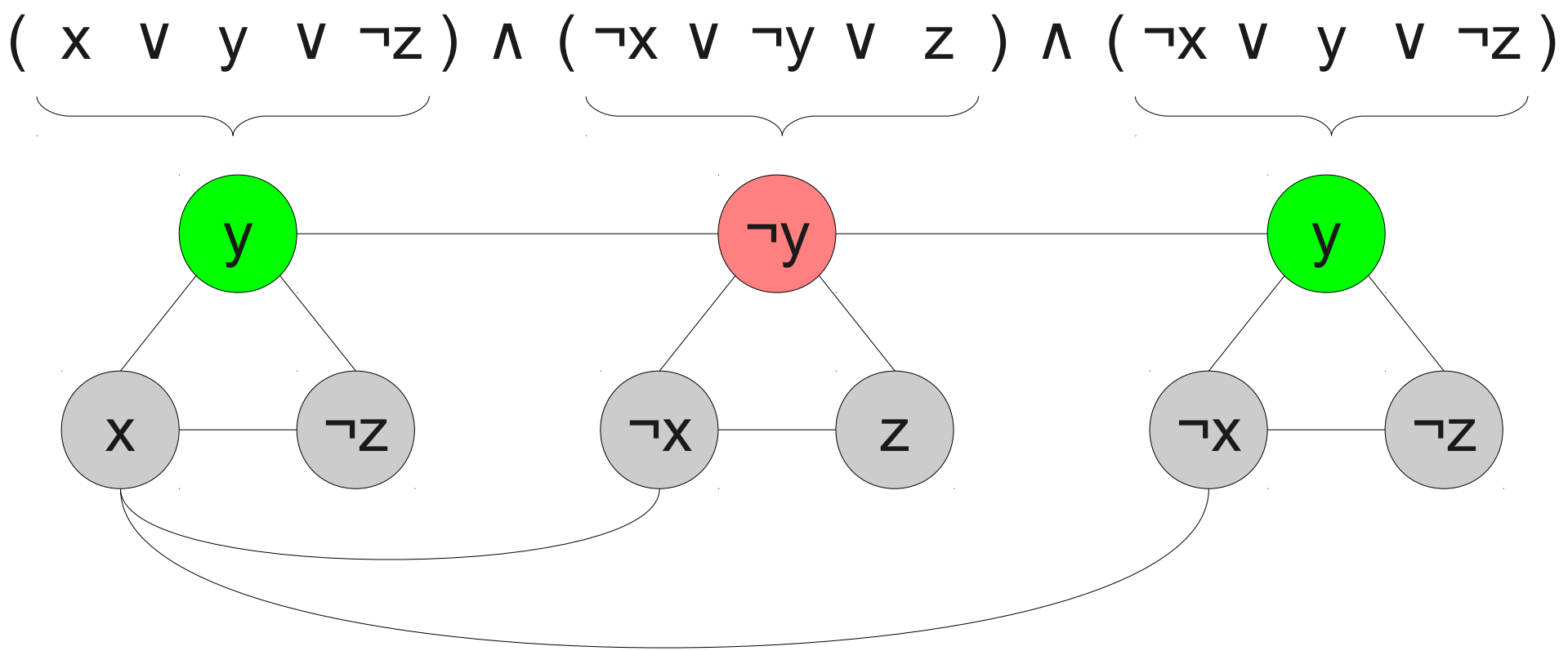
From 3SAT to INDSET



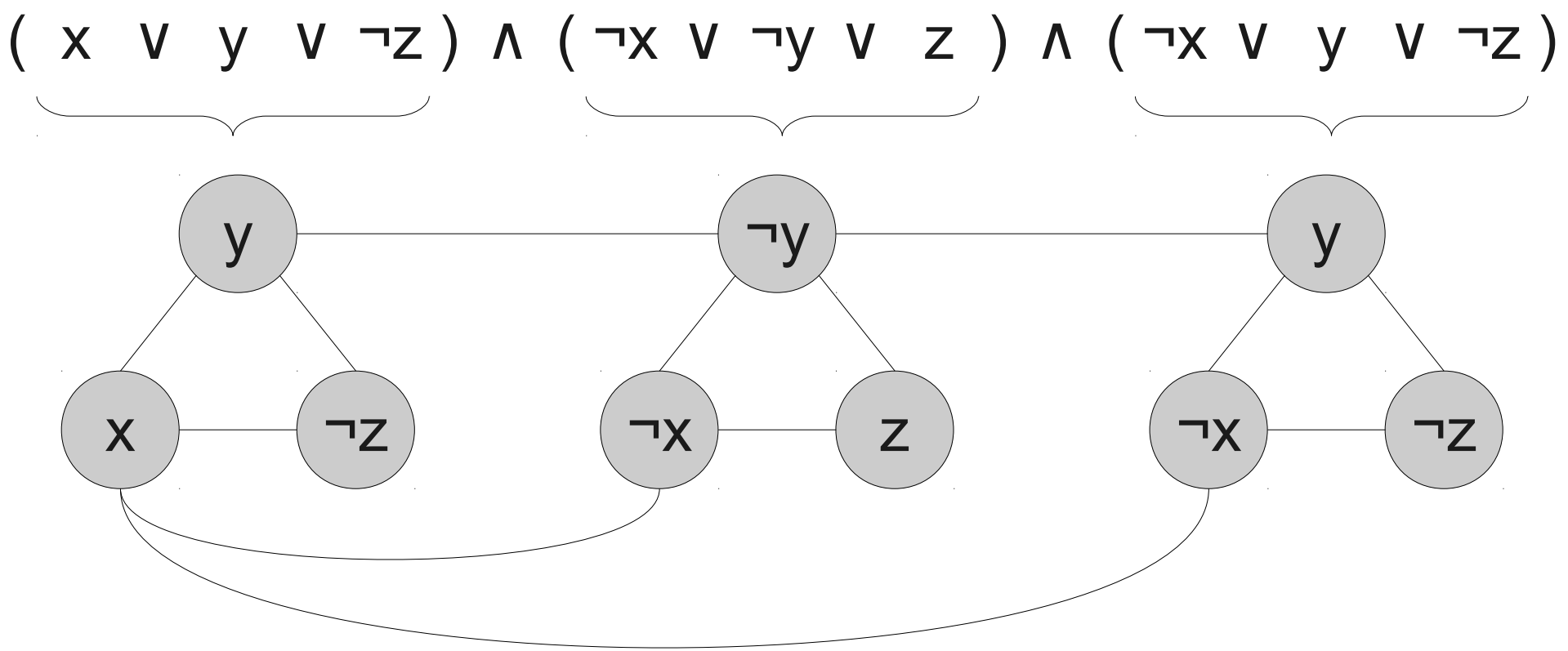
From 3SAT to INDSET



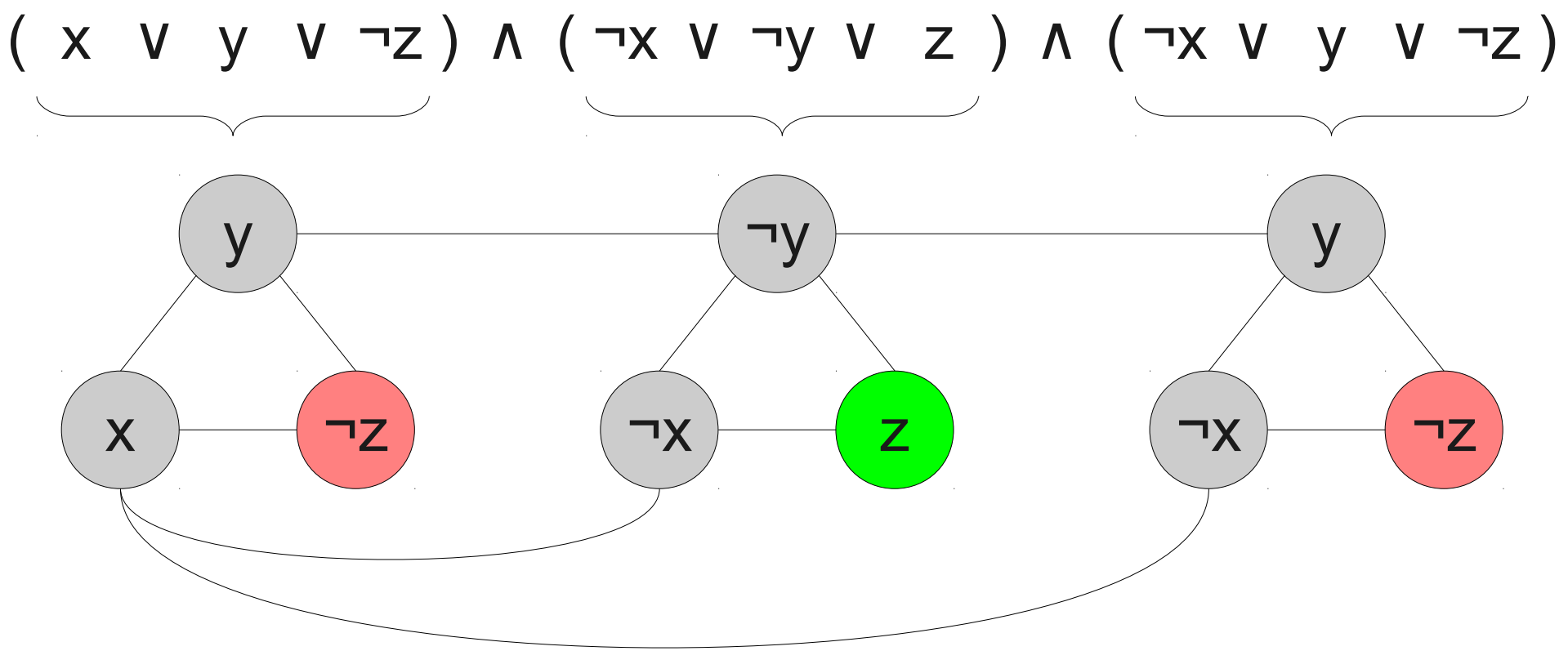
From 3SAT to INDSET



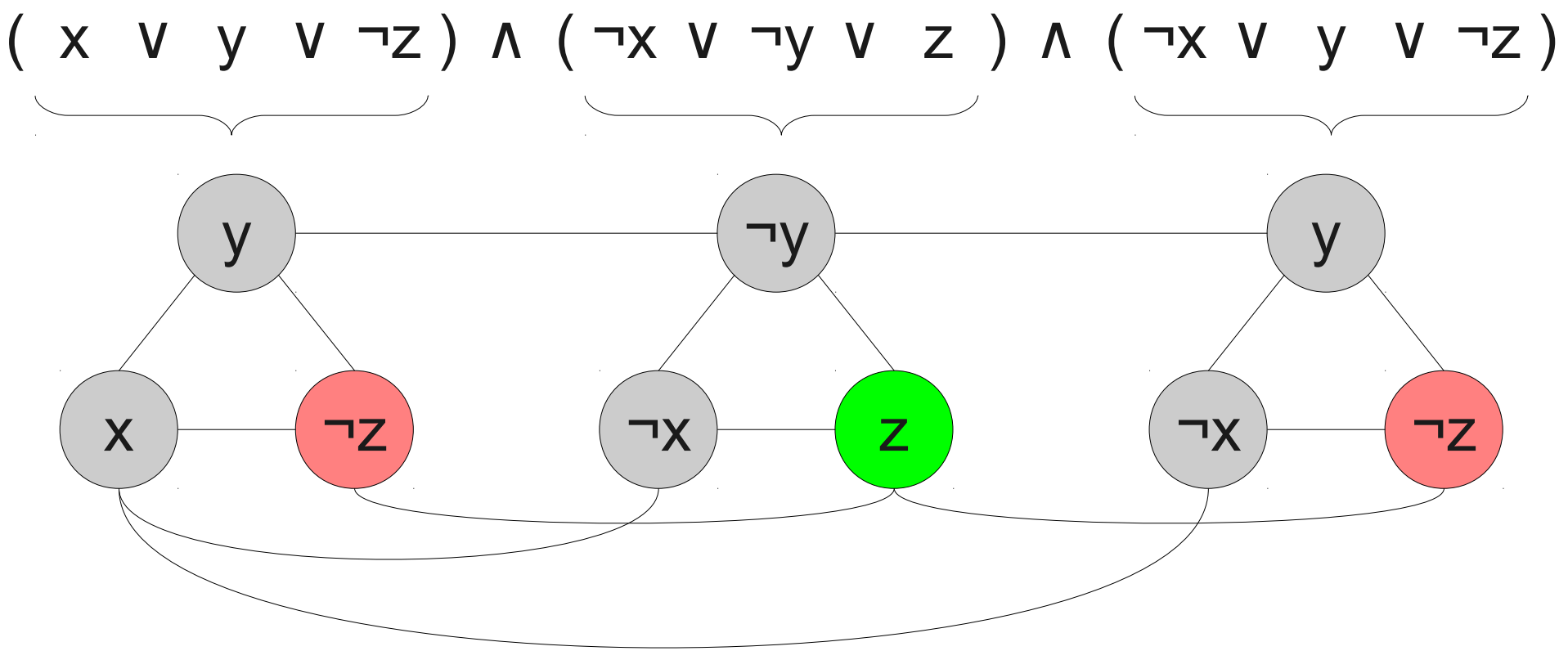
From 3SAT to INDSET



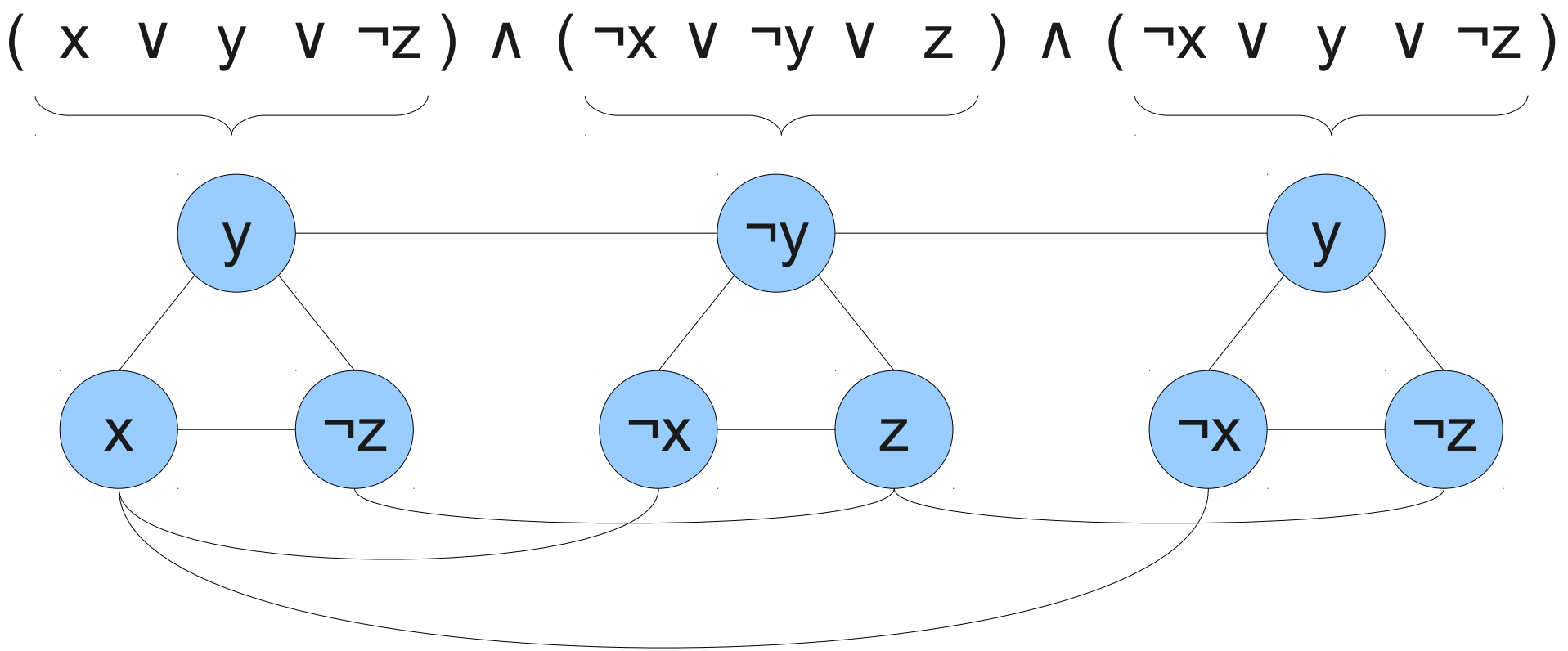
From 3SAT to INDSET



From 3SAT to INDSET

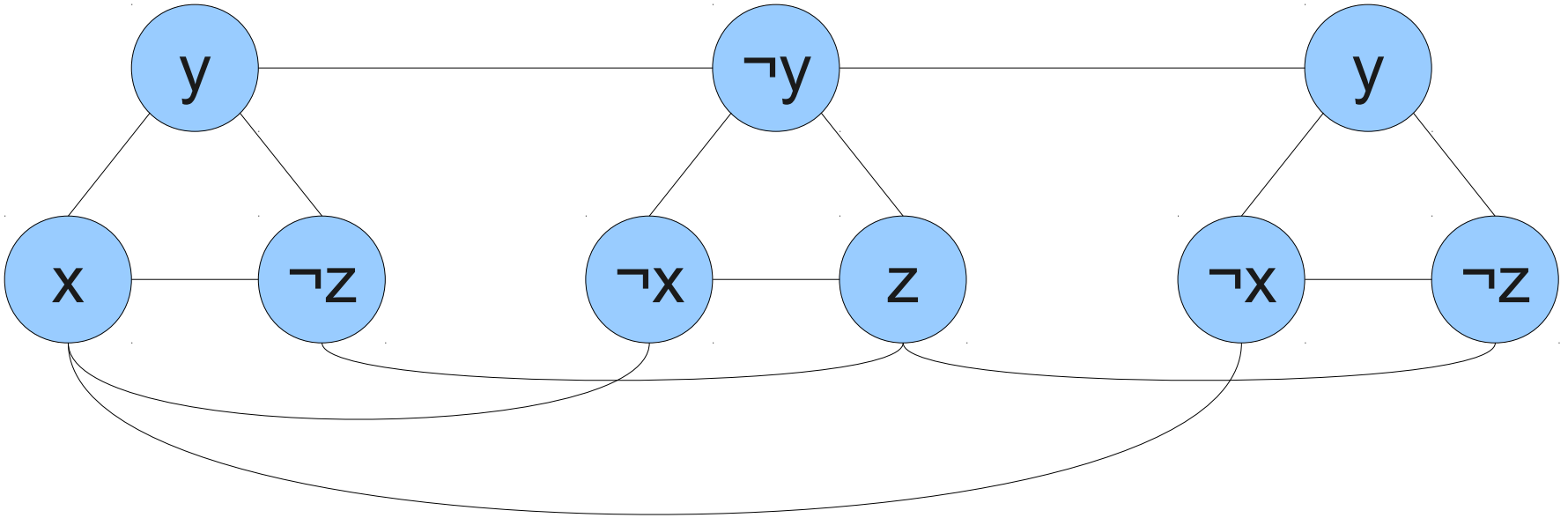


From 3SAT to INDSET



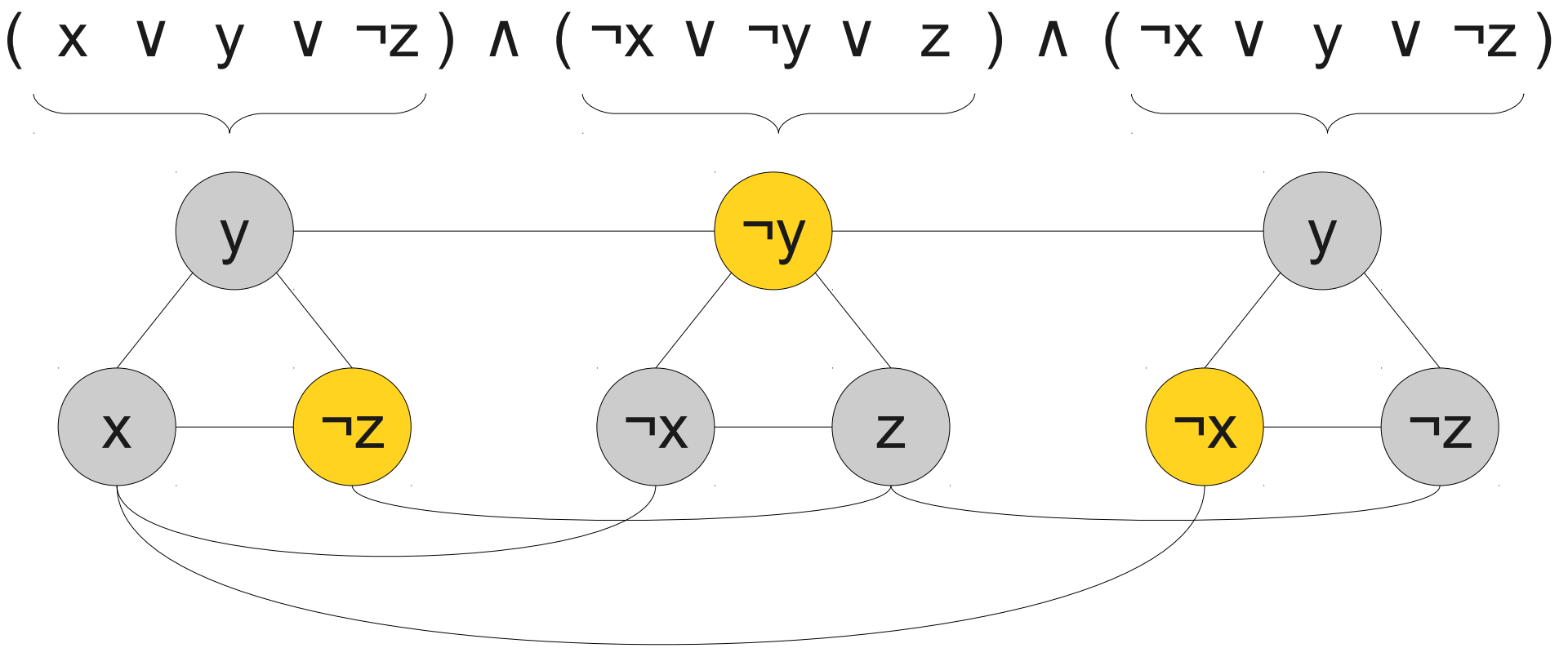
From 3SAT to INDSET

$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$



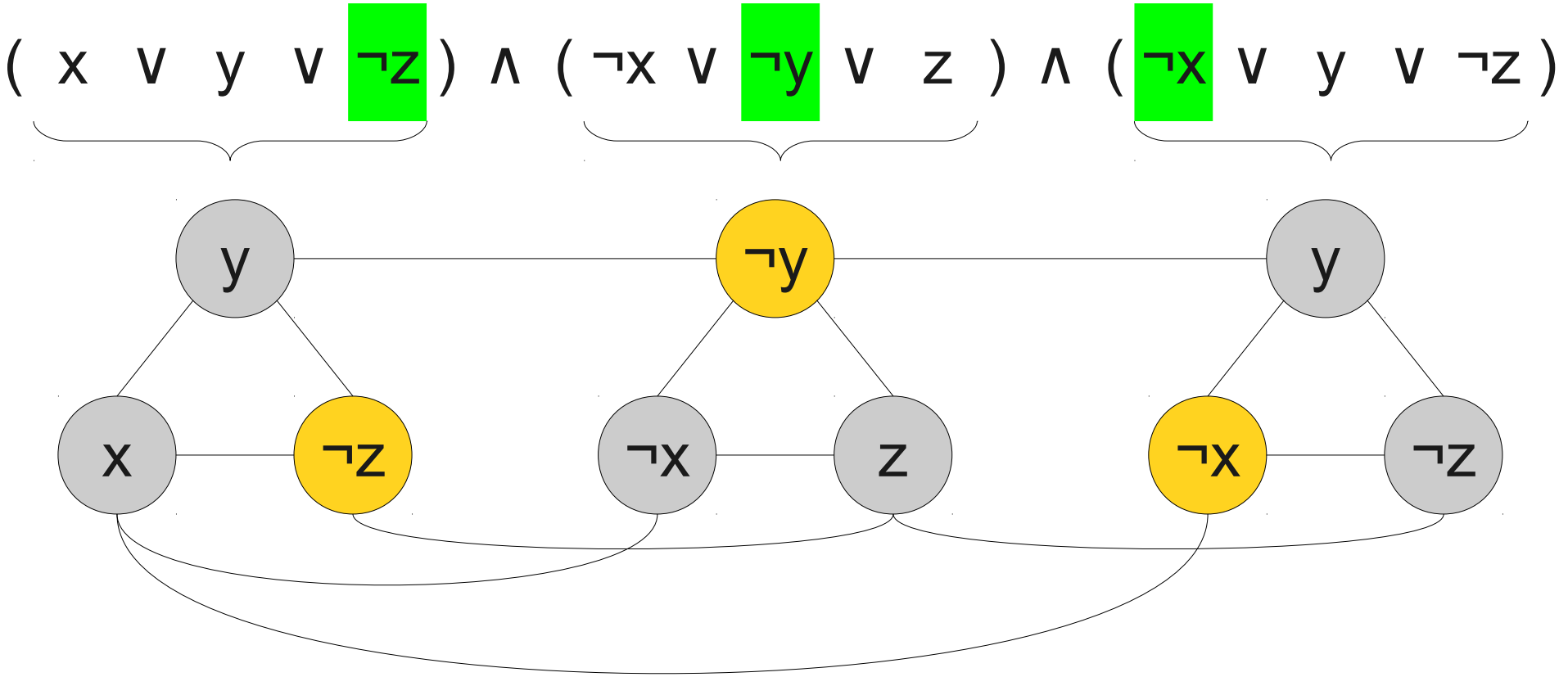
If this graph has an independent set of size three, the original formula is satisfiable.

From 3SAT to INDSET



If this graph has an independent set of size three, the original formula is satisfiable.

From 3SAT to INDSET

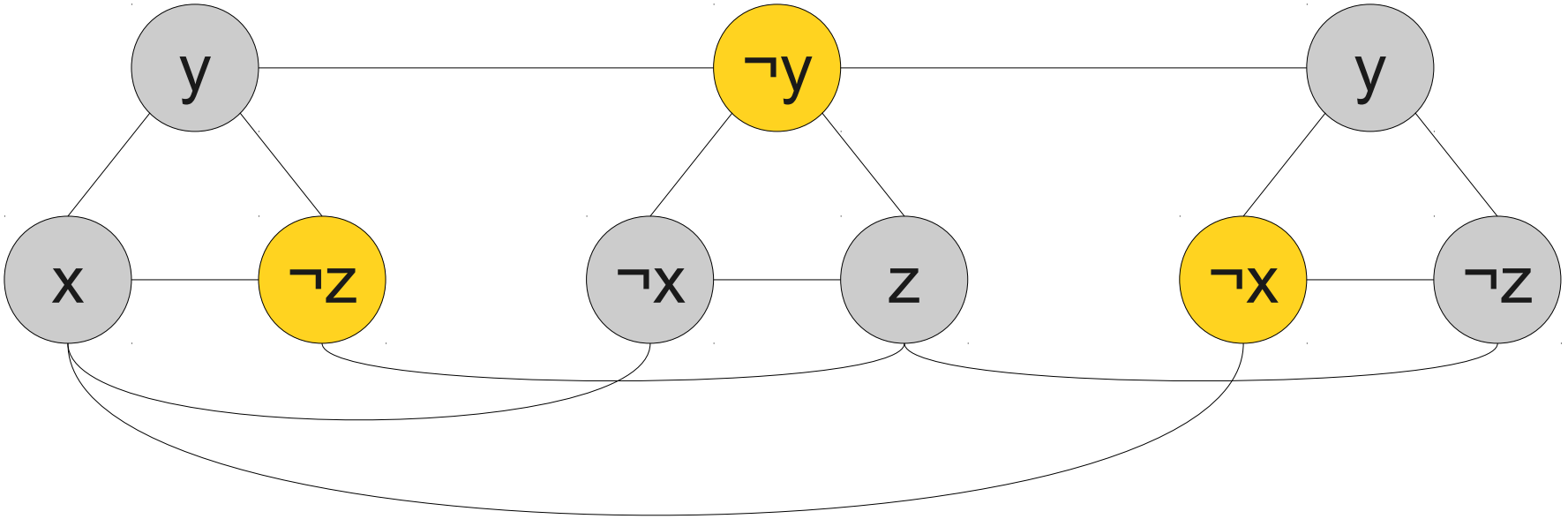


If this graph has an independent set of size three, the original formula is satisfiable.

From 3SAT to INDSET

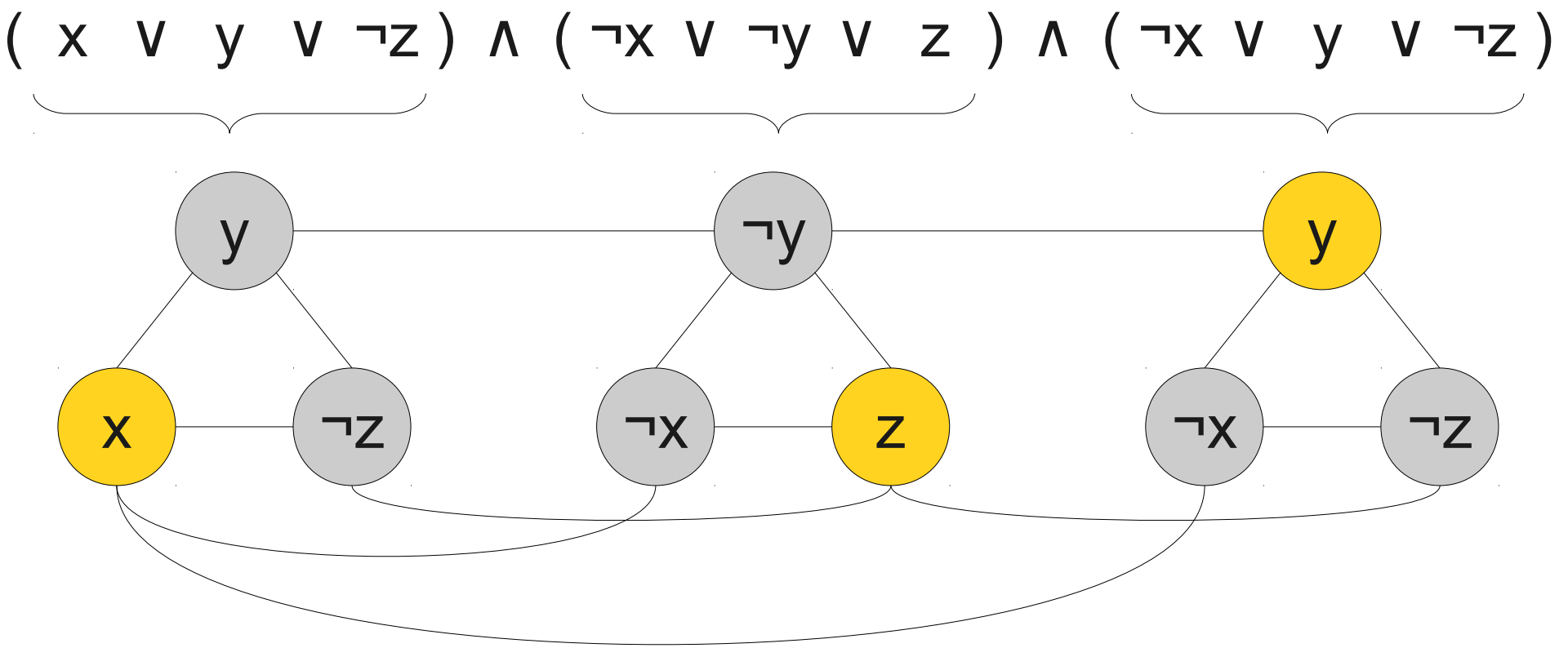
x = false, y = false, z = false.

(x ∨ y ∨ ¬z) ∧ (¬x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ ¬z)



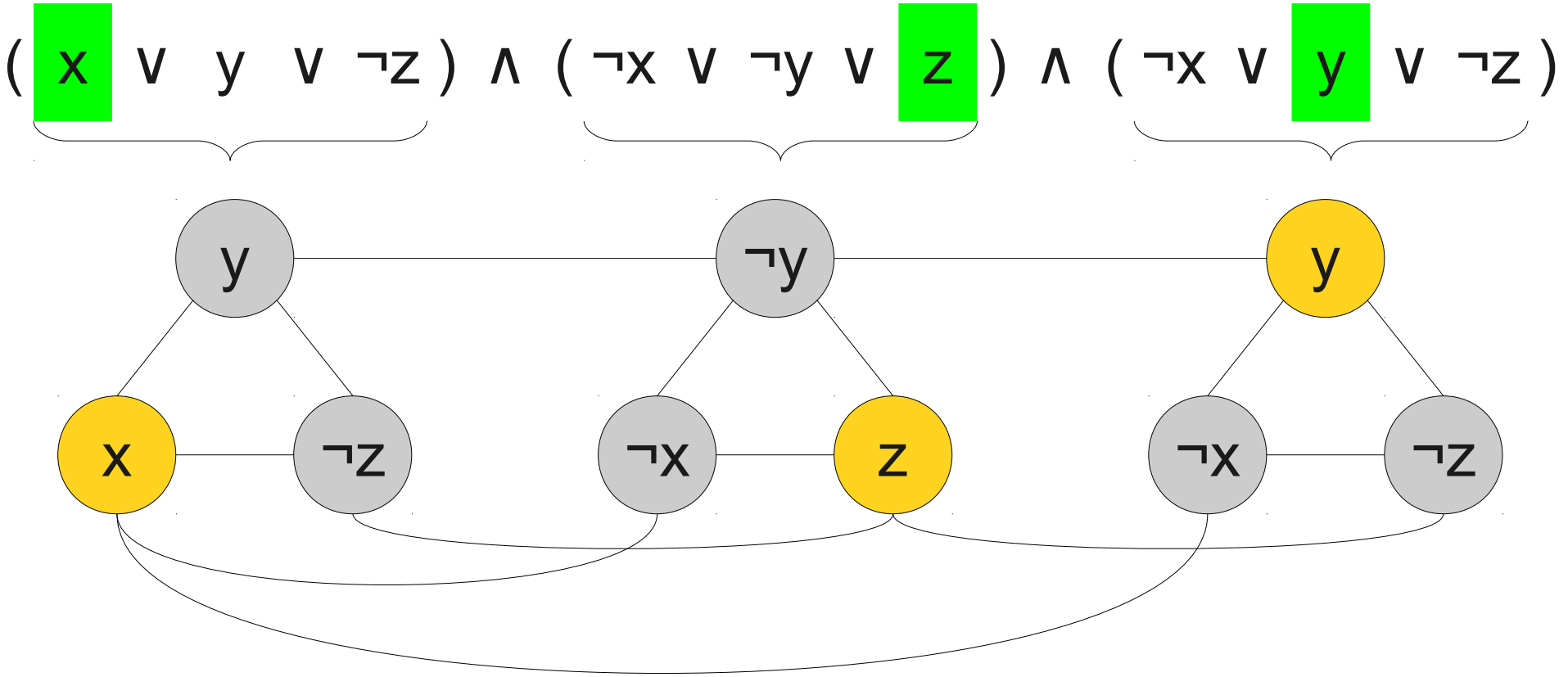
If this graph has an independent set of size three, the original formula is satisfiable.

From 3SAT to INDSET



If this graph has an independent set of size three, the original formula is satisfiable.

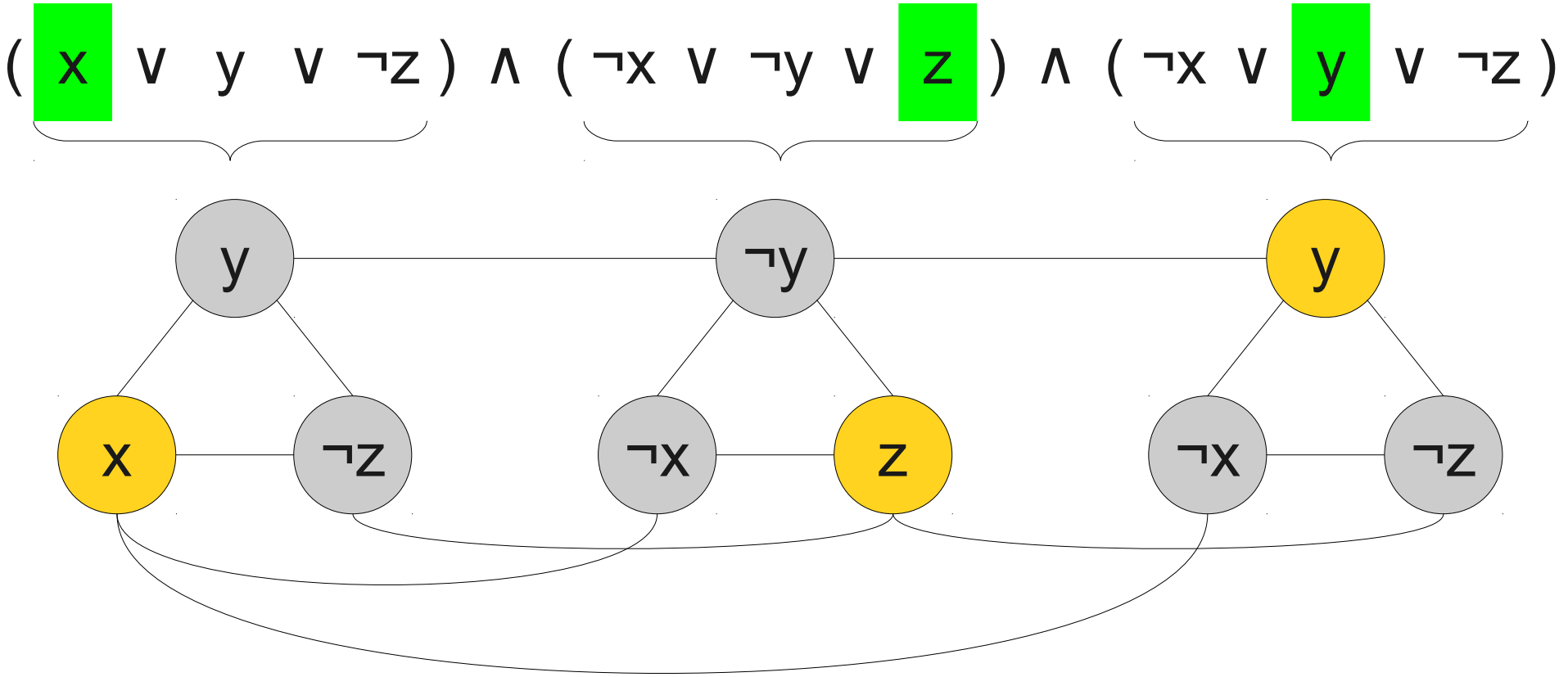
From 3SAT to INDSET



If this graph has an independent set of size three, the original formula is satisfiable.

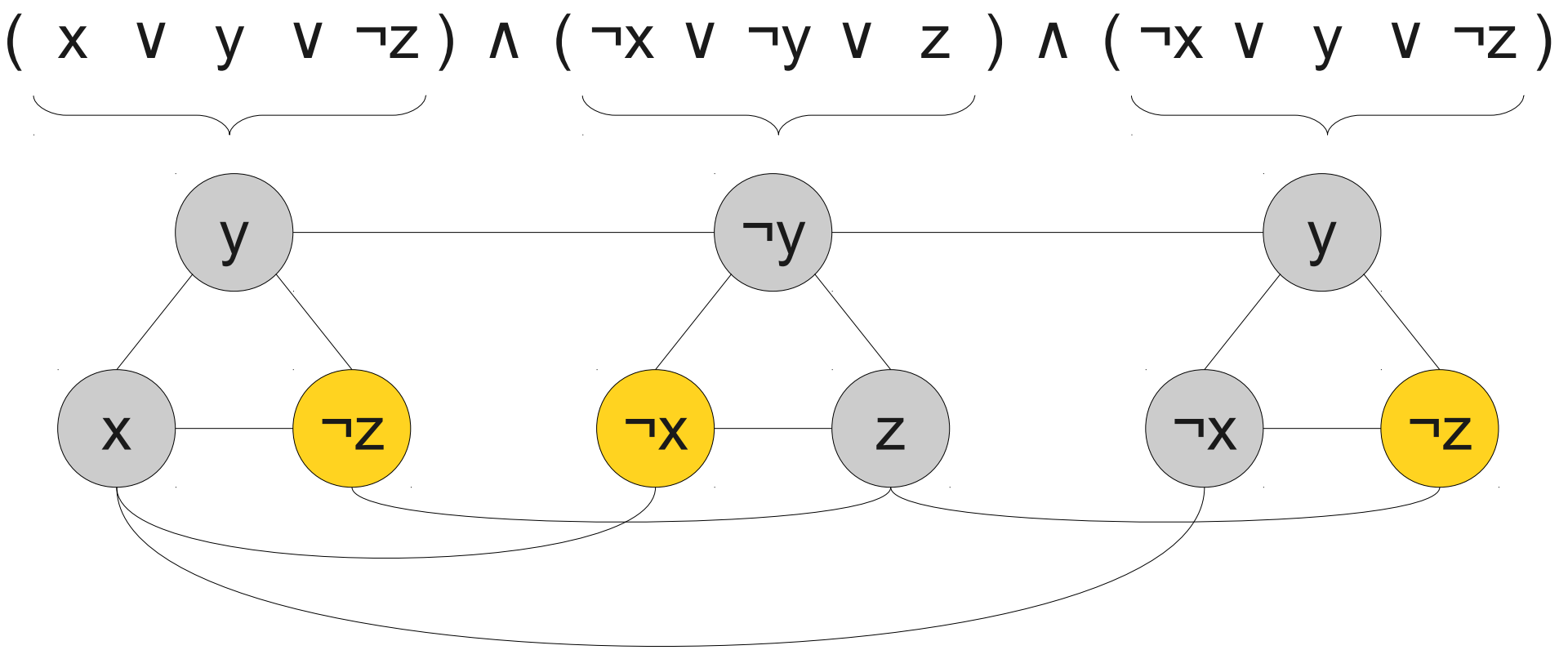
From 3SAT to INDSET

x = true, y = true, z = true.



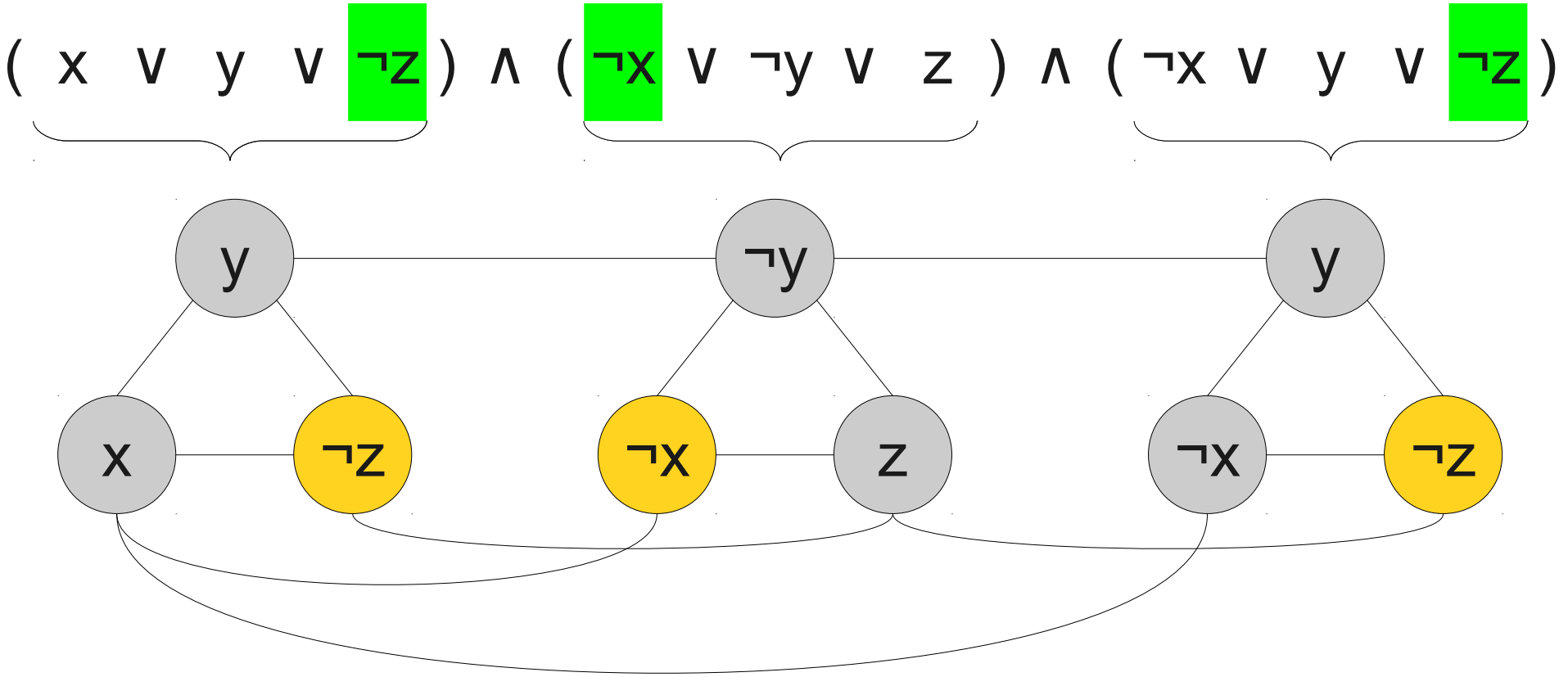
If this graph has an independent set of size three, the original formula is satisfiable.

From 3SAT to INDSET



If this graph has an independent set of size three, the original formula is satisfiable.

From 3SAT to INDSET

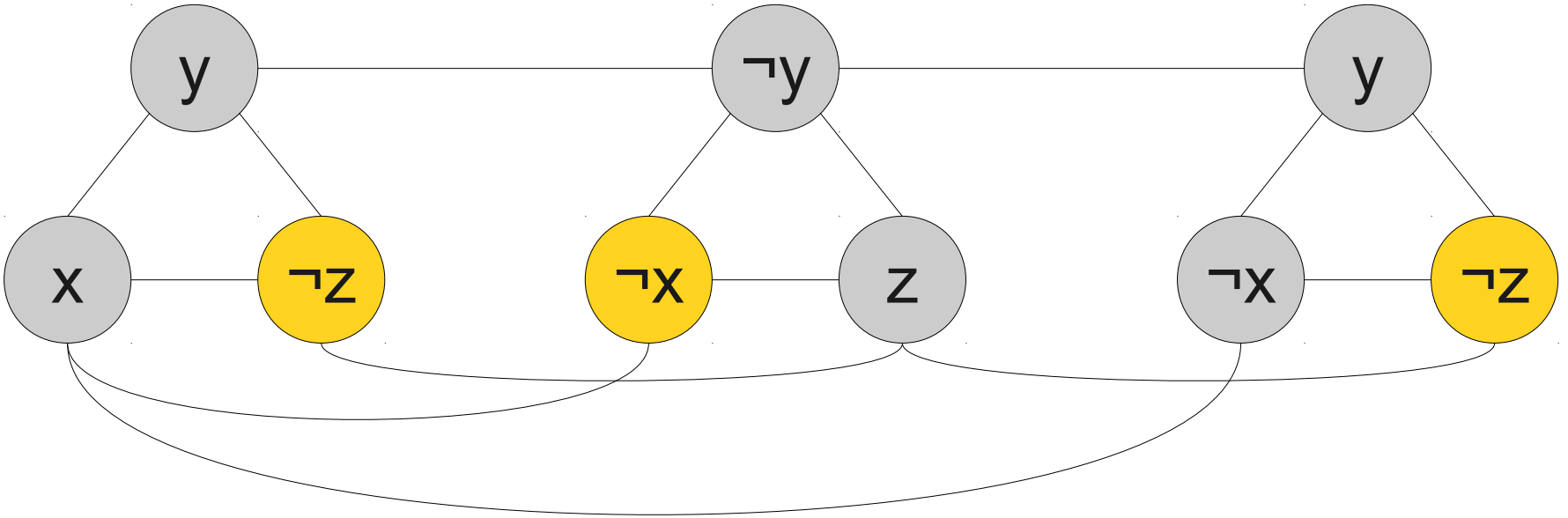


If this graph has an independent set of size three, the original formula is satisfiable.

From 3SAT to INDSET

x = false, y = ??, z = false.

(x ∨ y ∨ ¬z) ∧ (¬x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ ¬z)

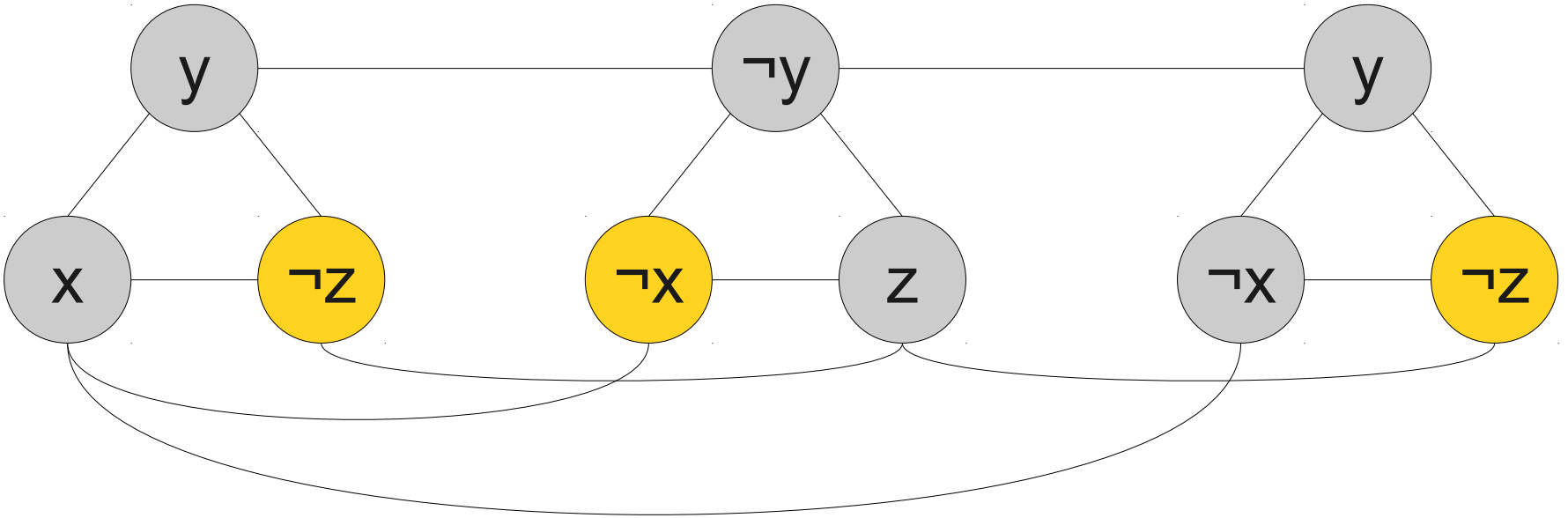


If this graph has an independent set of size three, the original formula is satisfiable.

From 3SAT to INDSET

x = false, y = true, z = false.

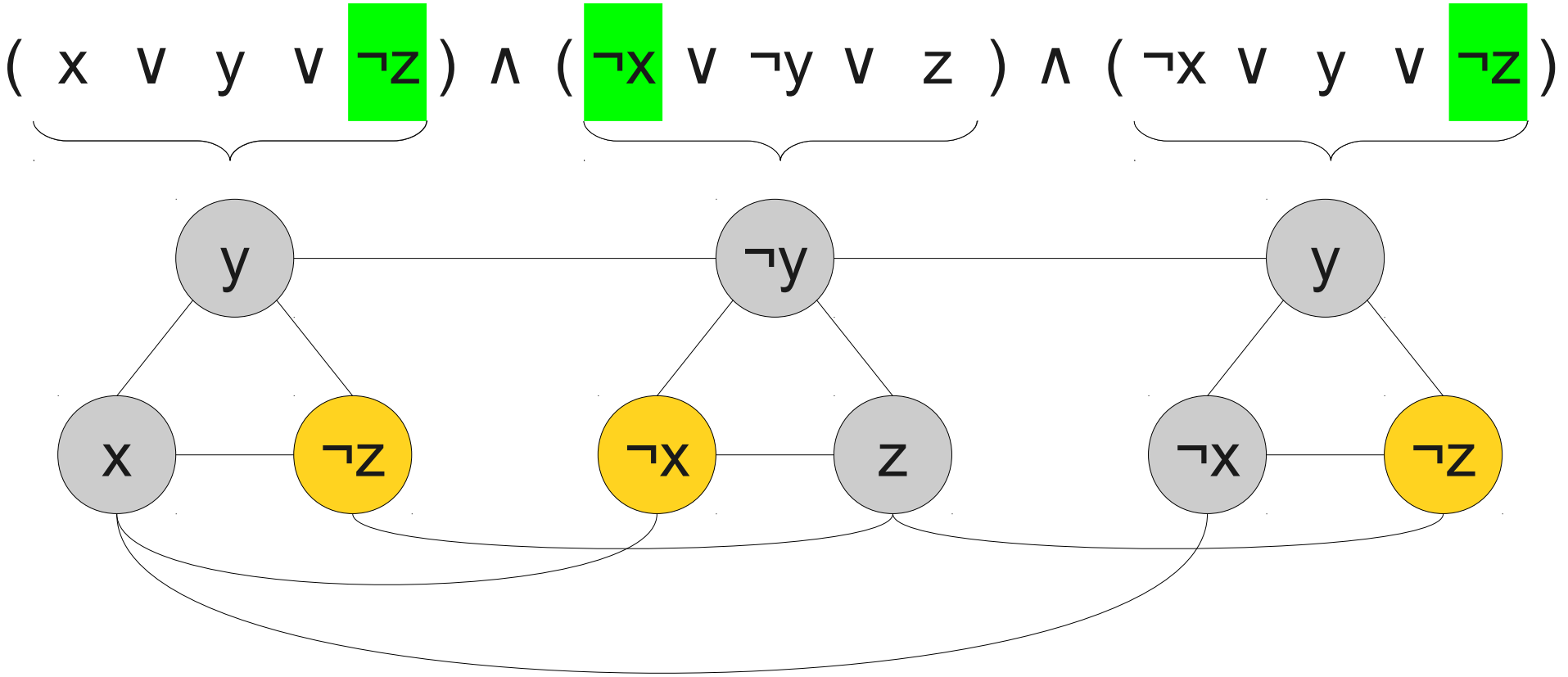
(x ∨ y ∨ ¬z) ∧ (¬x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ ¬z)



If this graph has an independent set of size three, the original formula is satisfiable.

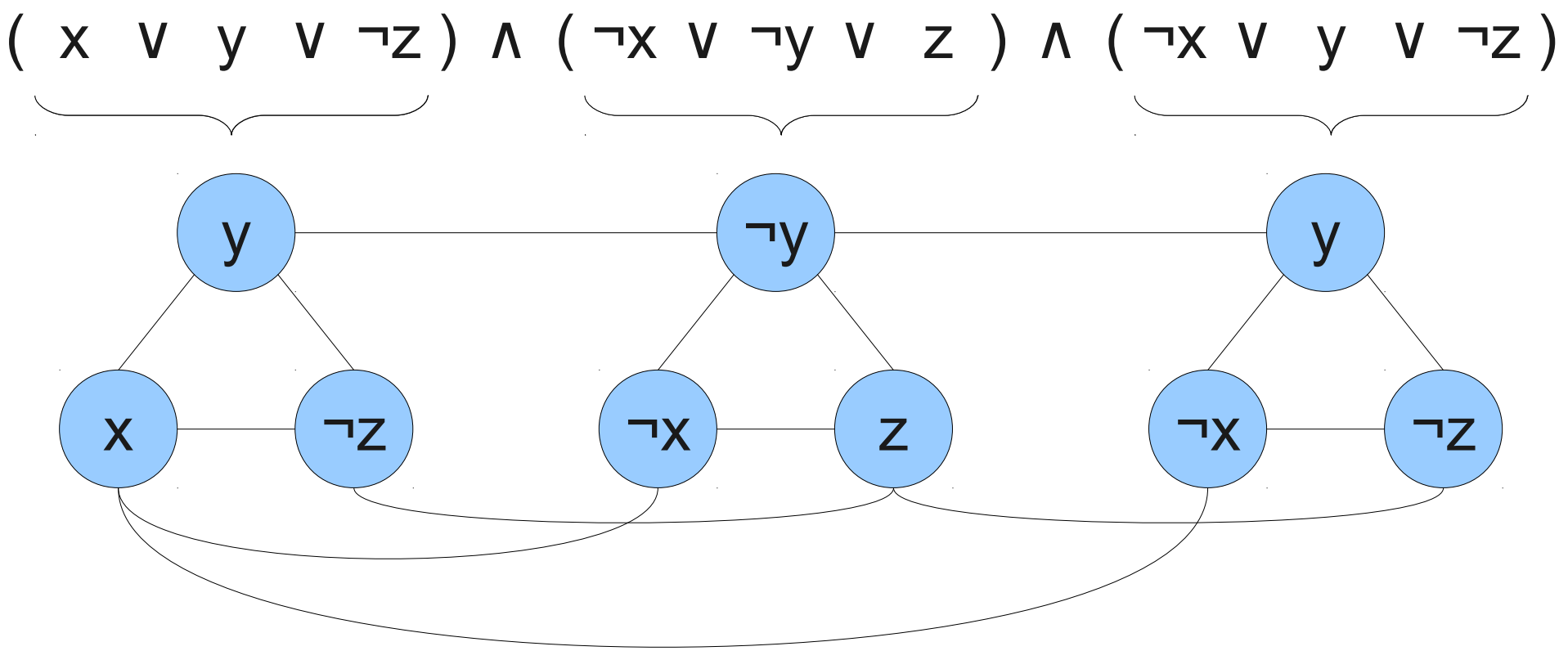
From 3SAT to INDSET

x = false, y = false, z = false.



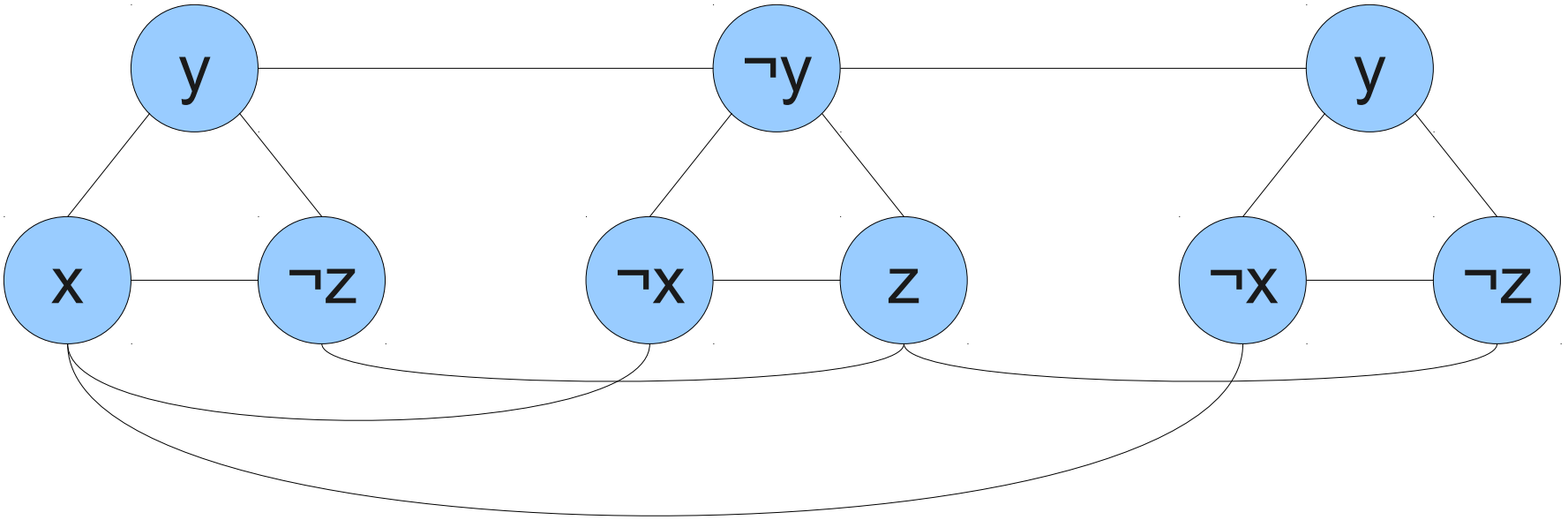
If this graph has an independent set of size three, the original formula is satisfiable.

From 3SAT to INDSET



From 3SAT to INDSET

$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$

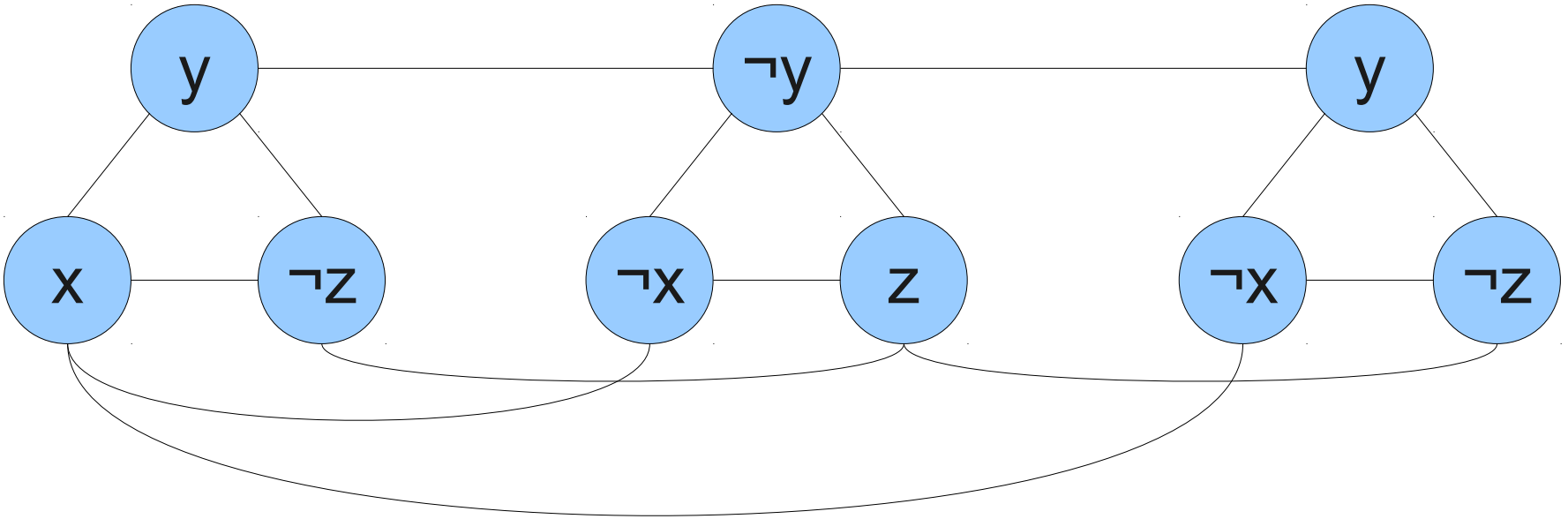


If the original formula is satisfiable,
this graph has an independent set of size three.

From 3SAT to INDSET

x = false, y = true, z = false.

$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$

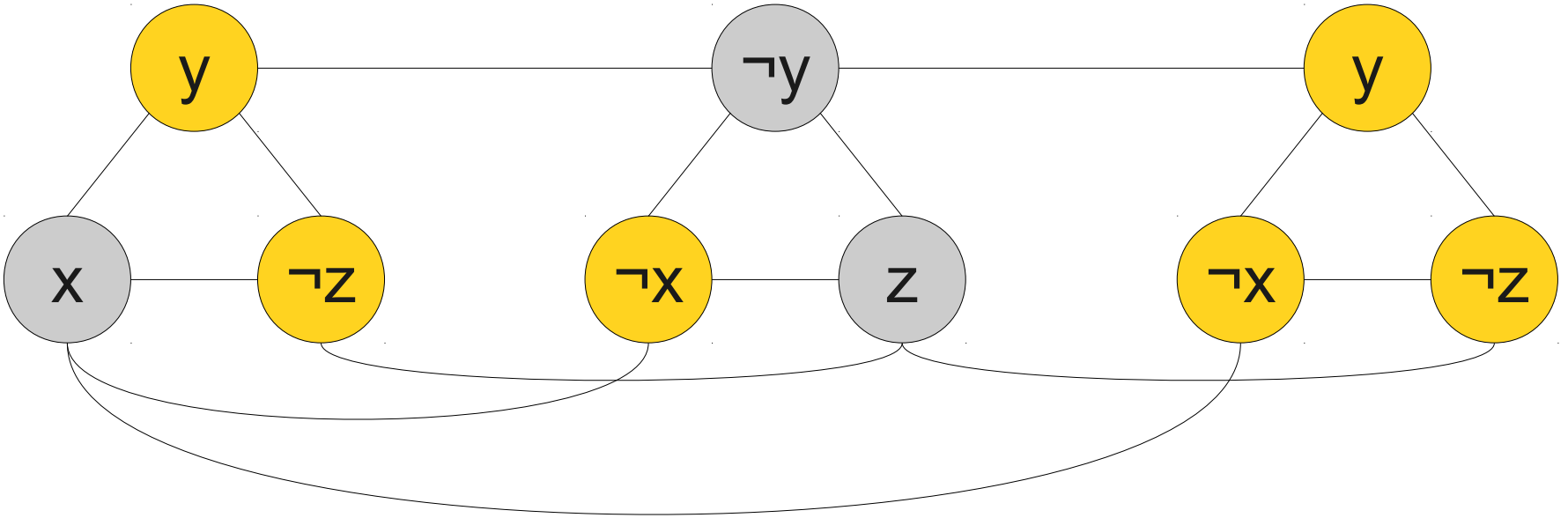


If the original formula is satisfiable,
this graph has an independent set of size three.

From 3SAT to INDSET

x = false, y = true, z = false.

(x ∨ y ∨ ¬z) ∧ (¬x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ ¬z)

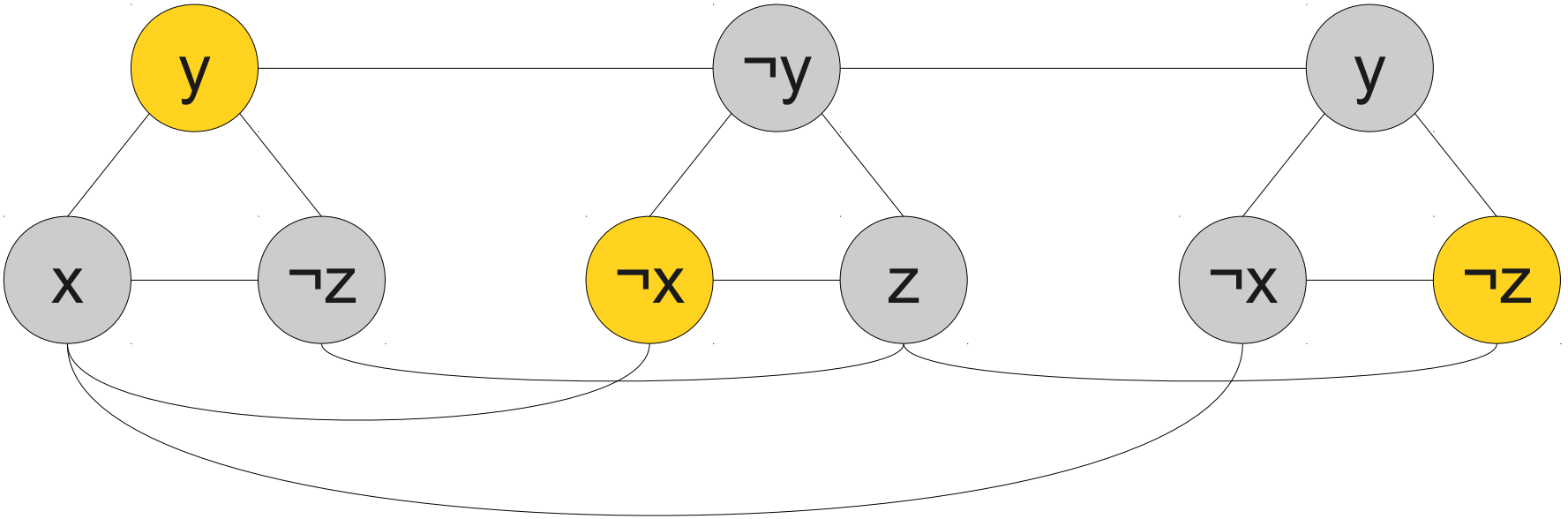


If the original formula is satisfiable,
this graph has an independent set of size three.

From 3SAT to INDSET

x = false, y = true, z = false.

(x ∨ y ∨ ¬z) ∧ (¬x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ ¬z)



If the original formula is satisfiable,
this graph has an independent set of size three.

From 3SAT to INDSET

- Let $\varphi = C_1 \wedge C_2 \wedge \ldots \wedge C_n$ be a 3-CNF formula.
- Construct the graph G as follows:
  - For each clause $C_i = x_1 \vee x_2 \vee x_3$, where x_1, x_2, and x_3 are literals, add three new nodes into G with edges connecting them.
  - For each pair of nodes v_i and $\neg v_i$, where v_i is some variable, add an edge connecting v_i and $\neg v_i$. (Note that there are multiple copies of these nodes)
- **Claim One:** This reduction can be computed in polynomial time.
- **Claim**: G has an independent set of size n iff φ is satisfiable.

Lemma: This reduction can be computed in polynomial time.

Lemma: This reduction can be computed in polynomial time.

Proof: Suppose that the original 3-CNF formula φ has n clauses, each of which has three literals.

Lemma: This reduction can be computed in polynomial time.

Proof: Suppose that the original 3-CNF formula φ has n clauses, each of which has three literals. Then we construct $3n$ nodes in our graph.

Lemma: This reduction can be computed in polynomial time.

Proof: Suppose that the original 3-CNF formula φ has n clauses, each of which has three literals. Then we construct $3n$ nodes in our graph. Each clause contributes 3 edges, so there are $O(n)$ edges added from clauses.

Lemma: This reduction can be computed in polynomial time.

Proof: Suppose that the original 3-CNF formula φ has n clauses, each of which has three literals. Then we construct $3n$ nodes in our graph. Each clause contributes 3 edges, so there are $O(n)$ edges added from clauses. For each pair of nodes representing opposite literals, we introduce one edge.

Lemma: This reduction can be computed in polynomial time.

Proof: Suppose that the original 3-CNF formula φ has n clauses, each of which has three literals. Then we construct $3n$ nodes in our graph. Each clause contributes 3 edges, so there are $O(n)$ edges added from clauses. For each pair of nodes representing opposite literals, we introduce one edge. Since there are $O(n^2)$ pairs of literals, this introduces at most $O(n^2)$ new edges.

Lemma: This reduction can be computed in polynomial time.

Proof: Suppose that the original 3-CNF formula φ has n clauses, each of which has three literals. Then we construct $3n$ nodes in our graph. Each clause contributes 3 edges, so there are $O(n)$ edges added from clauses. For each pair of nodes representing opposite literals, we introduce one edge. Since there are $O(n^2)$ pairs of literals, this introduces at most $O(n^2)$ new edges. This gives a graph with $O(n)$ nodes and $O(n^2)$ edges.

Lemma: This reduction can be computed in polynomial time.

Proof: Suppose that the original 3-CNF formula φ has n clauses, each of which has three literals. Then we construct $3n$ nodes in our graph. Each clause contributes 3 edges, so there are $O(n)$ edges added from clauses. For each pair of nodes representing opposite literals, we introduce one edge. Since there are $O(n^2)$ pairs of literals, this introduces at most $O(n^2)$ new edges. This gives a graph with $O(n)$ nodes and $O(n^2)$ edges. Each node and edge can be constructed in polynomial time, so overall this reduction can be computed in polynomial time, as required.

Lemma: This reduction can be computed in polynomial time.

Proof: Suppose that the original 3-CNF formula φ has n clauses, each of which has three literals. Then we construct $3n$ nodes in our graph. Each clause contributes 3 edges, so there are $O(n)$ edges added from clauses. For each pair of nodes representing opposite literals, we introduce one edge. Since there are $O(n^2)$ pairs of literals, this introduces at most $O(n^2)$ new edges. This gives a graph with $O(n)$ nodes and $O(n^2)$ edges. Each node and edge can be constructed in polynomial time, so overall this reduction can be computed in polynomial time, as required. ■

Lemma: If the graph G has an independent set of size n (where n is the number of clauses in φ), then φ is satisfiable.

Proof: Suppose G has an independent set of size n, call if S. No two nodes in S can correspond to v and $\neg v$ for any variable v, because there is an edge between all nodes with this property. Thus for each variable v, either there is a node in S with label v, or there is a node in S with label $\neg v$, or no node in S has either label. In the first case, set v to true; in the second case, set v to false; in the third case, choose a value for v arbitrarily. We claim that this gives a satisfying assignment for φ.

To see this, we show that each clause C in φ is satisfied. By construction, no two nodes in S can come from nodes added by C, because each has an edge to the other. Since there are n nodes in S and n clauses in φ, for any clause in φ some node corresponding to a literal from that clause is in S. If that node has the form x, then C contains x, and since we set x to true, C is satisfied. If that node has the form $\neg x$, then C contains $\neg x$, and since we set x to false, C is satisfied. Thus all clauses in φ are satisfied, so φ is satisfied by this assignment. ■

Lemma: If φ is satisfiable and has n clauses, then G has an independent set of size n.

Proof: Suppose that φ is satisfiable and consider any satisfying assignment for it. Thus under that assignment, for each clause C, there is some literal that evaluates to true. For each clause C, choose some literal that evaluates to true and add the corresponding node in G to a set S. Then S has size n, since it contains one node per clause.

We claim moreover that S is an independent set in G. To see this, note that there are two types of edges in G: edges between nodes representing literals in the same clause, and edges between variables and their negations. No two nodes joined by edges within a clause are in S, because we explicitly picked one node per clause. Moreover, no two nodes joined by edges between opposite literals are in S, because in a satisfying assignment both of the two could not be true. Thus no nodes in S are joined by edges, so S is an independent set. ■

Putting it All Together

Theorem: INDSET is **NP**-complete.

Putting it All Together

Theorem: INDSET is **NP**-complete.

Proof: We know that INDSET ∈ **NP**, because we constructed a polynomial-time verifier for it.

Putting it All Together

Theorem: INDSET is **NP**-complete.

Proof: We know that INDSET $\in$ **NP**, because we constructed a polynomial-time verifier for it. So all we need to show is that every problem in **NP** is polynomial-time reducible to INDSET.

Putting it All Together

Theorem: INDSET is **NP**-complete.

Proof: We know that INDSET $\in$ **NP**, because we constructed a polynomial-time verifier for it. So all we need to show is that every problem in **NP** is polynomial-time reducible to INDSET.

To do this, we use the polynomial-time reduction from 3SAT to INDSET that we just gave.

Putting it All Together

Theorem: INDSET is **NP**-complete.

Proof: We know that INDSET ∈ **NP**, because we constructed a polynomial-time verifier for it. So all we need to show is that every problem in **NP** is polynomial-time reducible to INDSET.

To do this, we use the polynomial-time reduction from 3SAT to INDSET that we just gave. As we proved, φ ∈ 3SAT iff ⟨G, n⟩ ∈ INDSET, and this reduction can be computed in polynomial time.

Putting it All Together

Theorem: INDSET is **NP**-complete.

Proof: We know that INDSET ∈ **NP**, because we constructed a polynomial-time verifier for it. So all we need to show is that every problem in **NP** is polynomial-time reducible to INDSET.

To do this, we use the polynomial-time reduction from 3SAT to INDSET that we just gave. As we proved, φ ∈ 3SAT iff ⟨G, n⟩ ∈ INDSET, and this reduction can be computed in polynomial time. Thus 3SAT is polynomial-time reducible to INDSET, so INDSET is **NP**-complete.

Putting it All Together

Theorem: INDSET is **NP**-complete.

Proof: We know that INDSET ∈ **NP**, because we constructed a polynomial-time verifier for it. So all we need to show is that every problem in **NP** is polynomial-time reducible to INDSET.

To do this, we use the polynomial-time reduction from 3SAT to INDSET that we just gave. As we proved, φ ∈ 3SAT iff ⟨G, n⟩ ∈ INDSET, and this reduction can be computed in polynomial time. Thus 3SAT is polynomial-time reducible to INDSET, so INDSET is **NP**-complete. ■

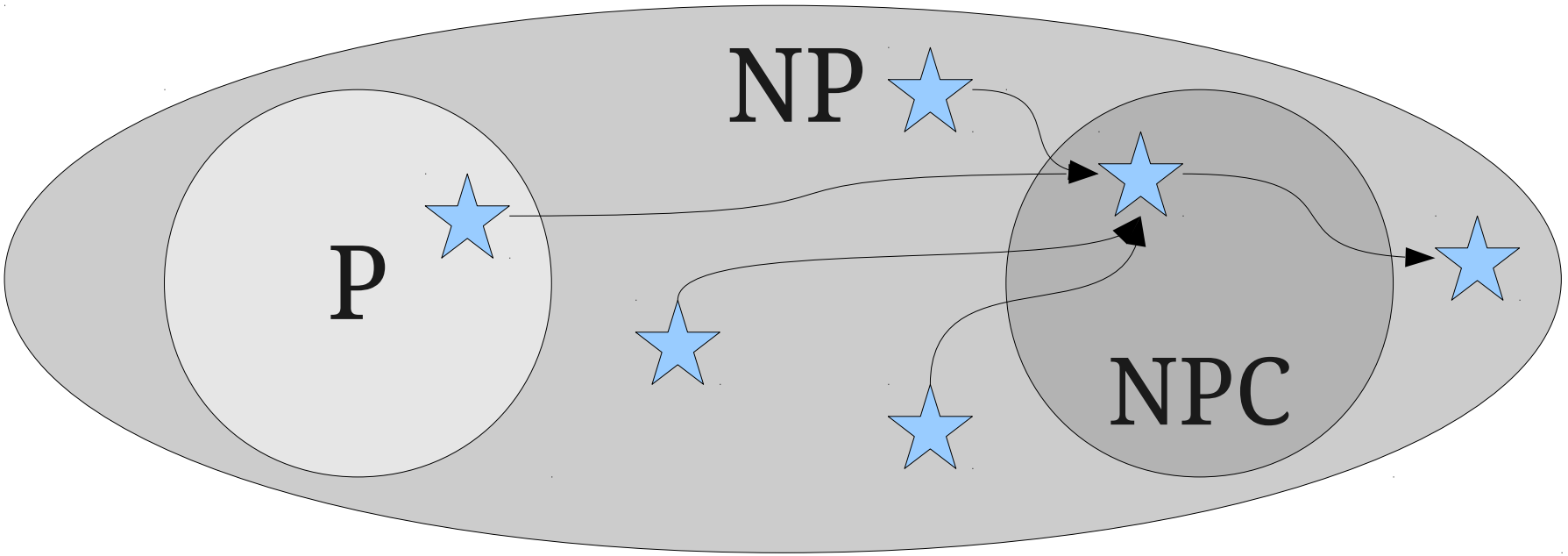
More



Completeness

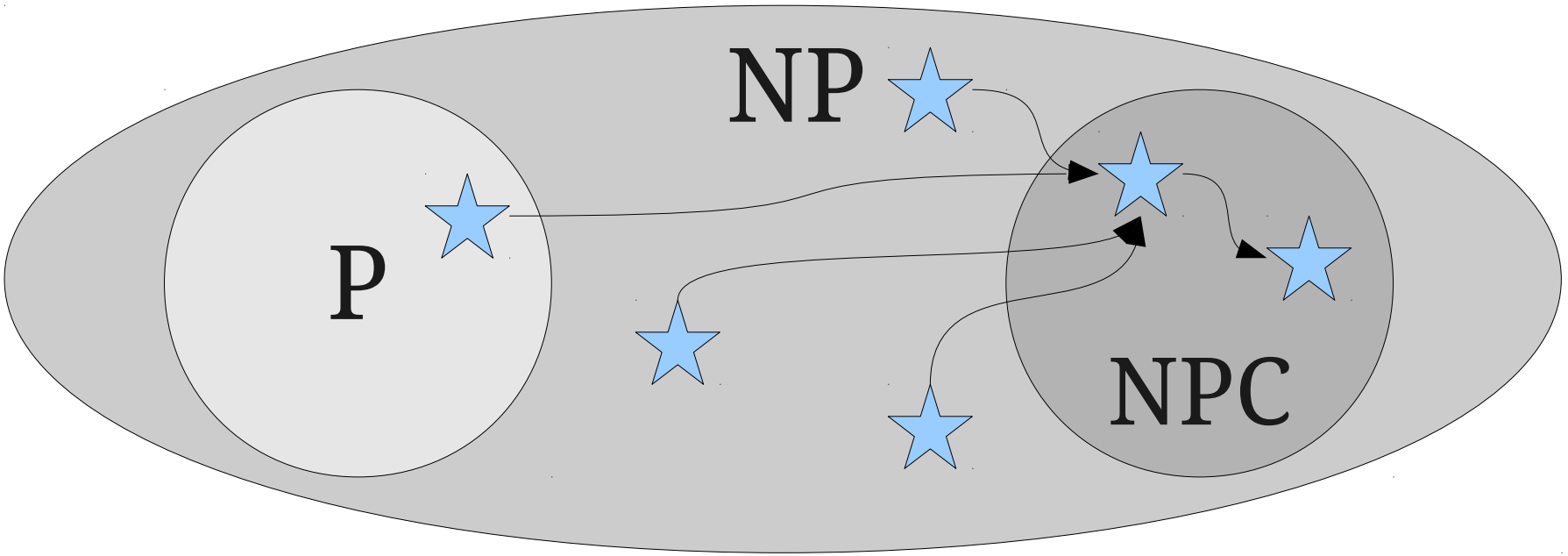
NP-Completeness

Theorem: If $L \in$ **NPC**, $L \leq_P L'$, and $L' \in$ **NP**, then $L' \in$ **NPC**.



NP-Completeness

Theorem: If $L \in$ **NPC**, $L \leq_p L'$, and $L' \in$ **NP**, then $L' \in$ **NPC**.



Structuring **NP**-Completeness Reductions

The Shape of a Reduction

- Polynomial-time reductions work by solving one problem with a solver for a different problem.
- Most problems in **NP** have different pieces that must be solved simultaneously.
- For example, in 3SAT:
  - Each clause must be made true,
  - but no literal and its complement may be picked.

Reductions and Gadgets

- Many reductions used to show **NP**-completeness work by using **gadgets**.
- Each piece of the original problem is translated into a “gadget” that handles some particular detail of the problem.
- These gadgets are then connected together to solve the overall problem.

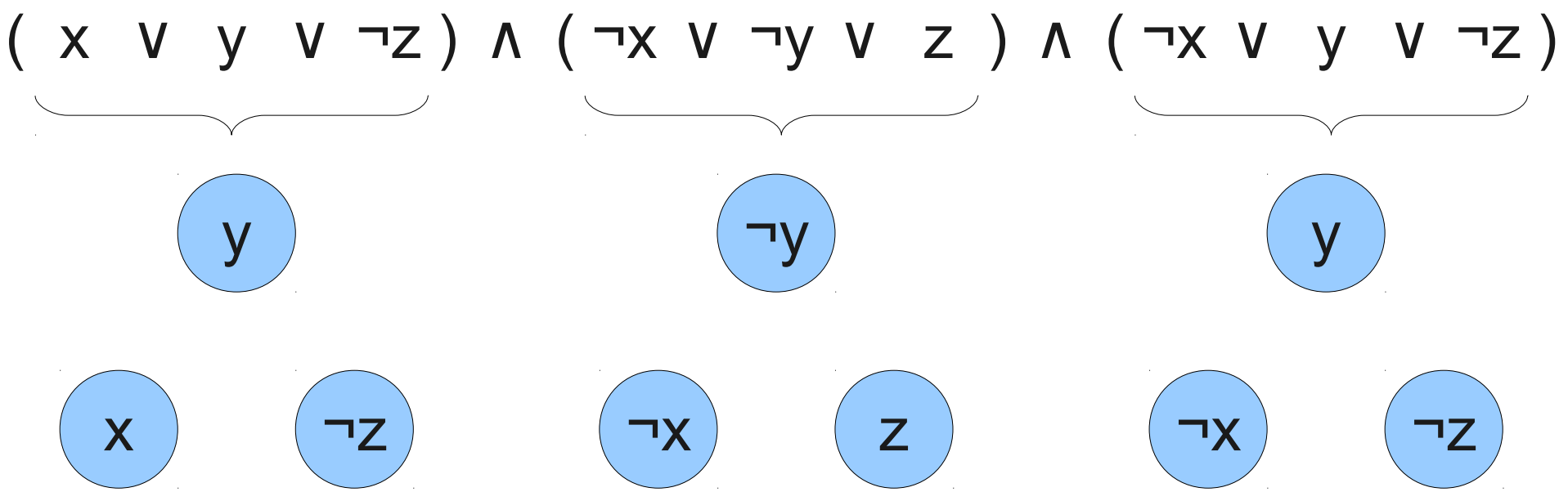
Gadgets in INDSET

$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$

Gadgets in INDSET

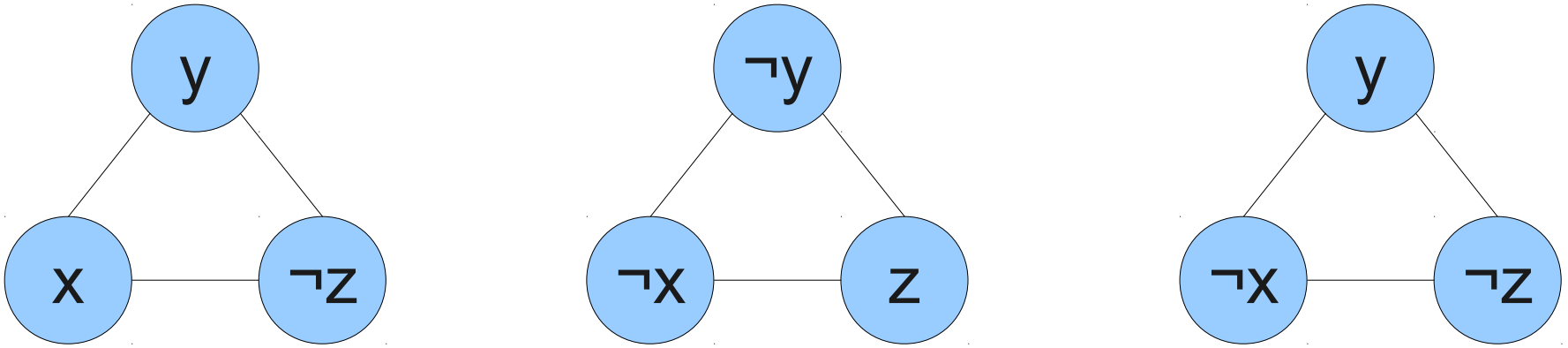
$$(\underbrace{x \vee y \vee \neg z}) \wedge (\underbrace{\neg x \vee \neg y \vee z}) \wedge (\underbrace{\neg x \vee y \vee \neg z})$$

Gadgets in INDSET



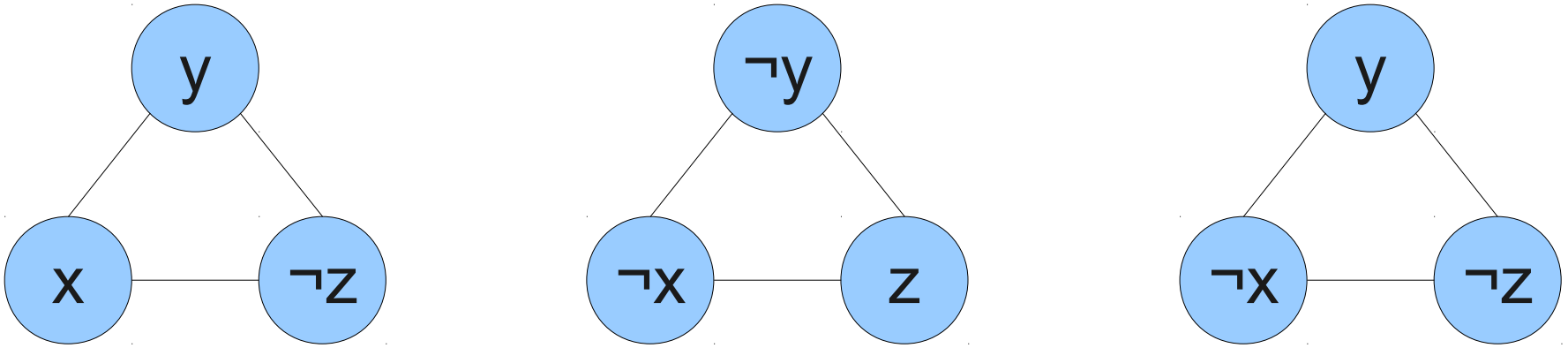
Gadgets in INDSET

$$(\ x \ \vee \ y \ \vee \neg z\ ) \wedge (\ \neg x \vee \neg y \vee \ z\ ) \wedge (\ \neg x \vee \ y \ \vee \neg z\ )$$



Gadgets in INDSET

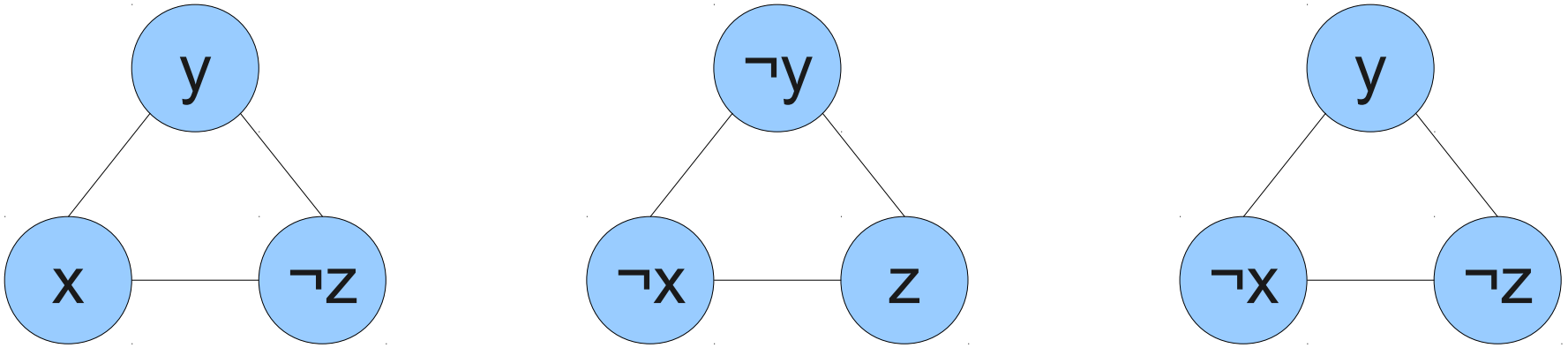
$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$



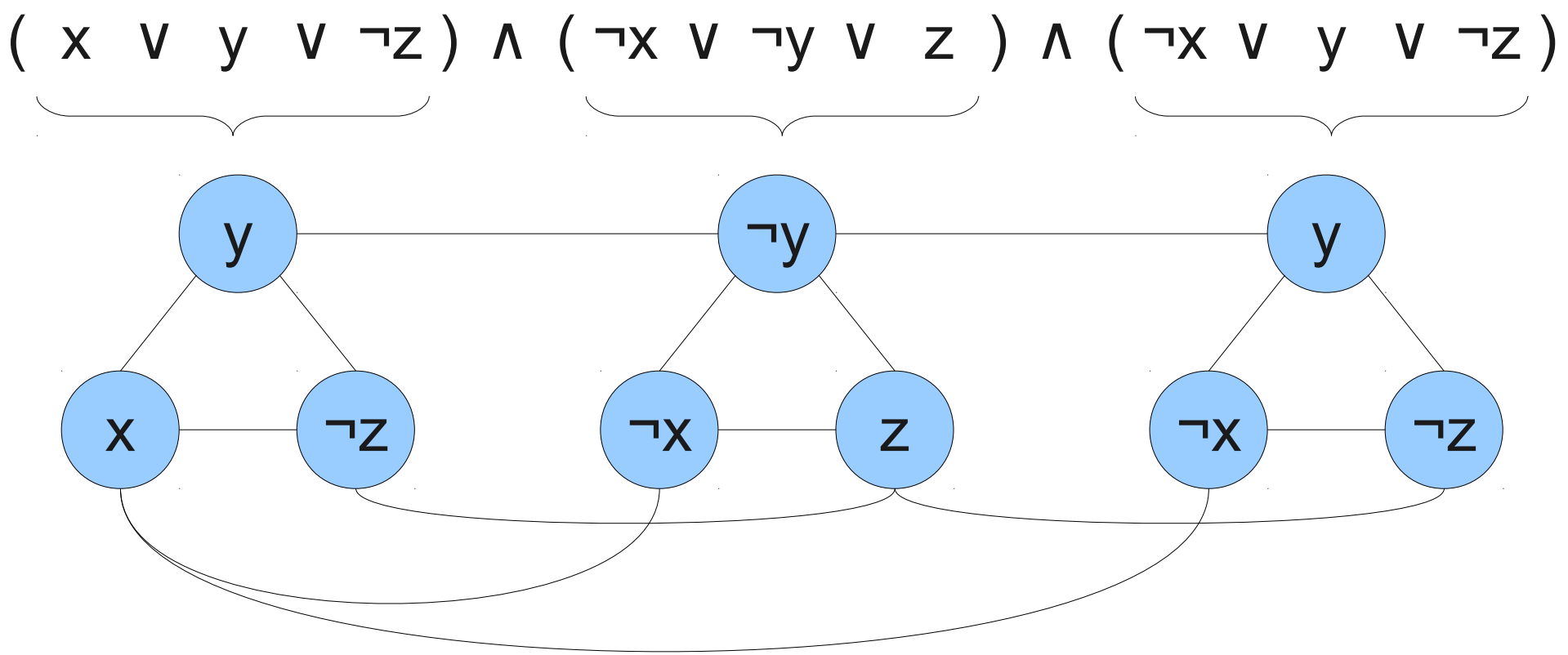
Each of these gadgets is designed to solve one part of the problem: ensuring each clause is satisfied.

Gadgets in INDSET

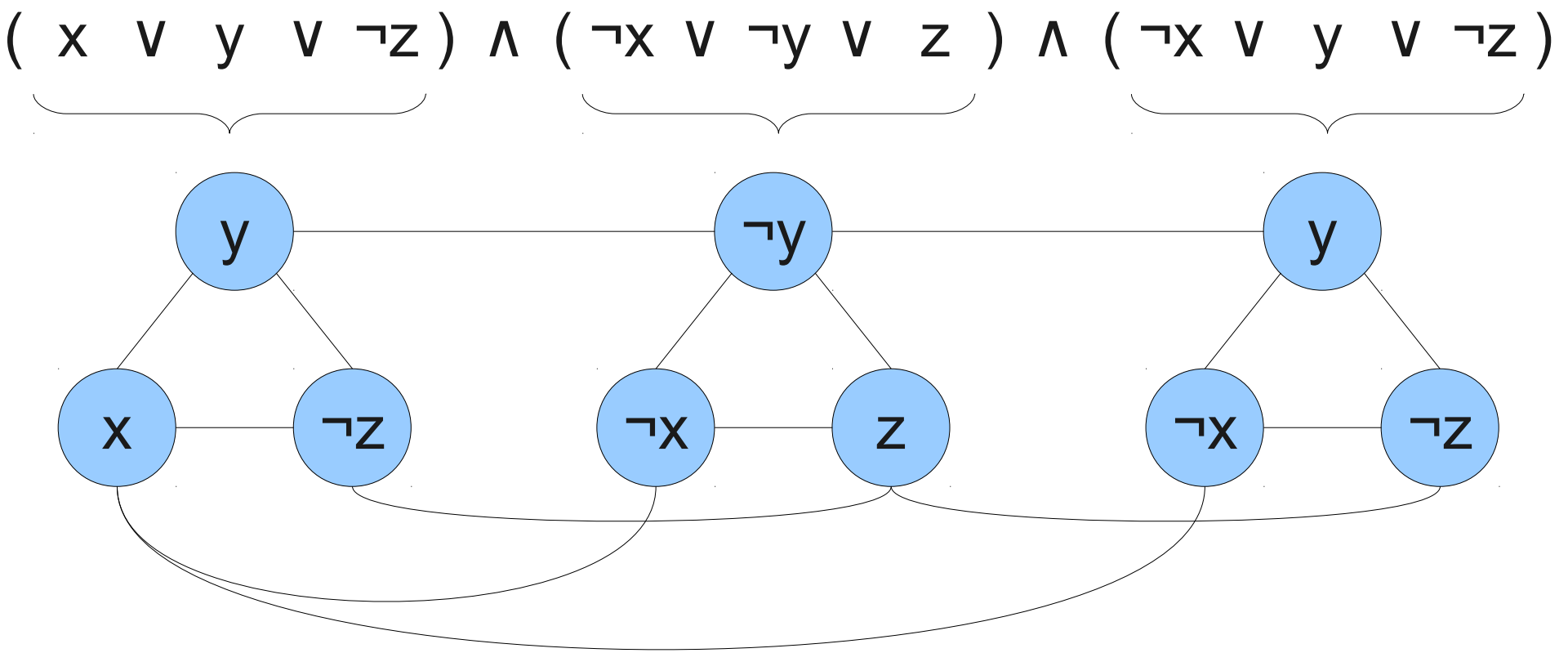
$$(\ x \ \vee \ y \ \vee \neg z\ ) \wedge (\ \neg x \vee \neg y \vee \ z\ ) \wedge (\ \neg x \vee \ y \ \vee \neg z\ )$$



Gadgets in INDSET

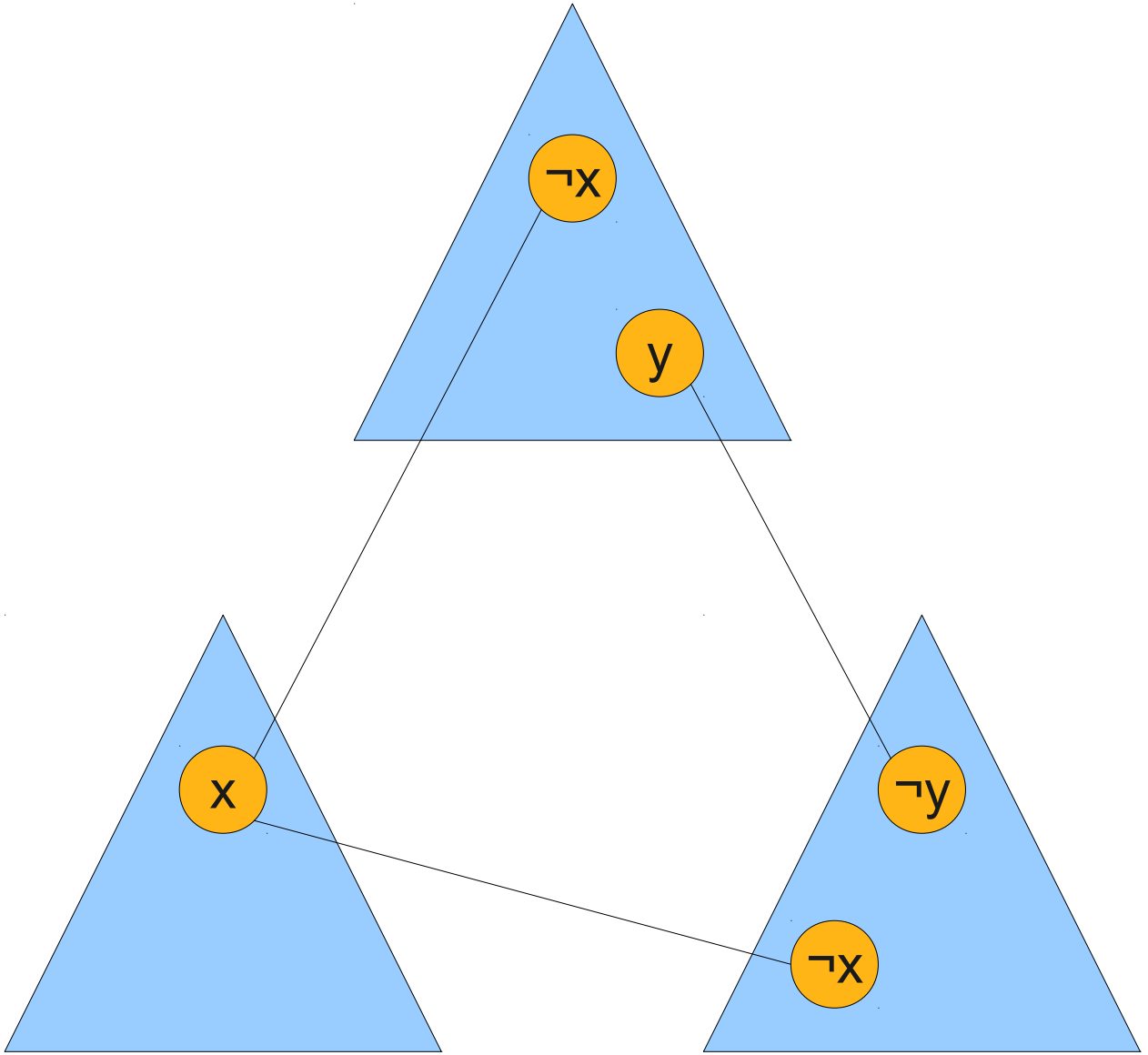


Gadgets in INDSET



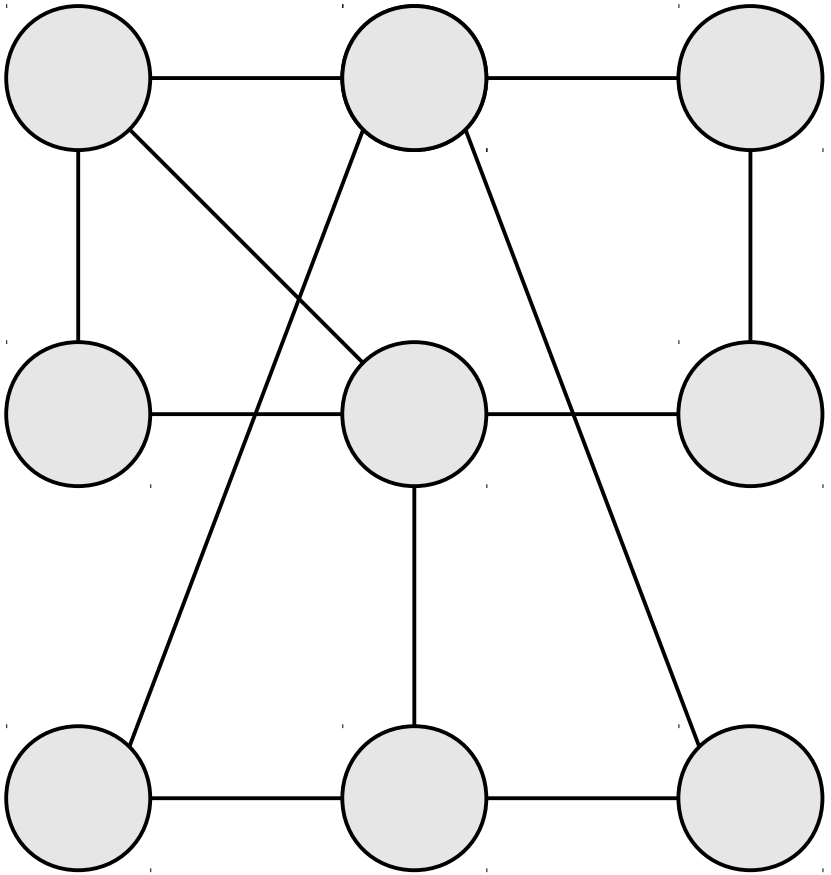
These connections ensure that the solutions to each gadget are linked to one another.

Gadgets in INDSET

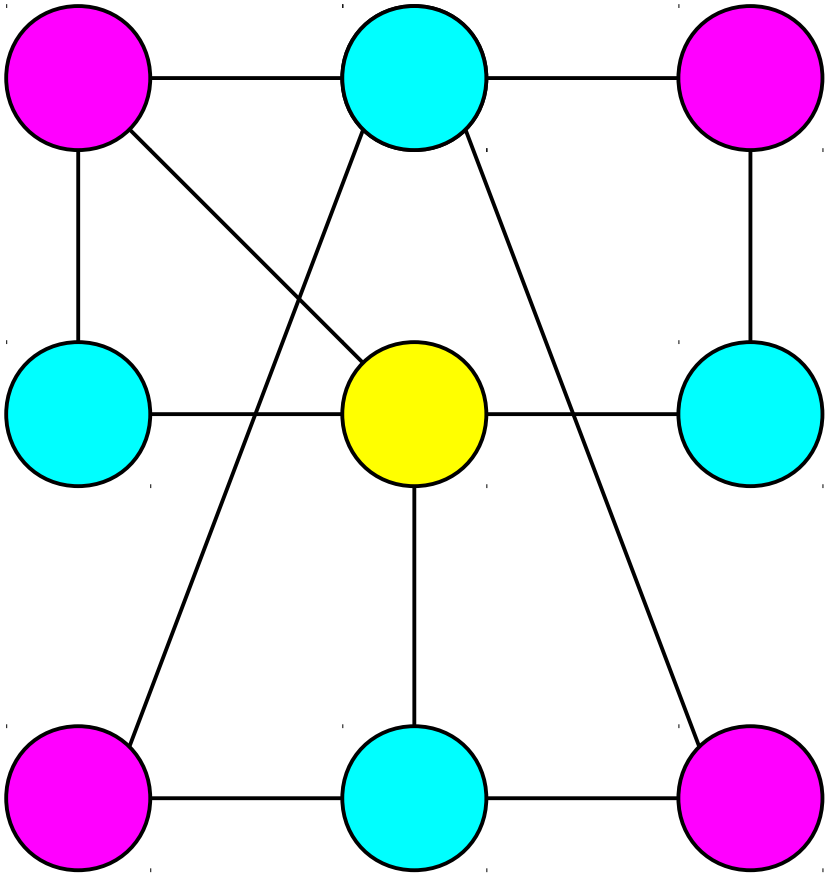


A More Complex Reduction

A **3-coloring** of a graph is a way of coloring its nodes one of three colors such that no two connected nodes have the same color.



A **3-coloring** of a graph is a way of coloring its nodes one of three colors such that no two connected nodes have the same color.



A **3-coloring** of a graph is a way of coloring its nodes one of three colors such that no two connected nodes have the same color.

The 3-Coloring Problem

- The **3-coloring problem** is

  Given an undirected graph *G*, is there a legal 3-coloring of its nodes?

- As a formal language:

  3COLOR = { ⟨*G*⟩ | *G* is an undirected graph with a legal 3-coloring. }

- This problem is known to be **NP**-complete by a reduction from 3SAT.

3COLOR ∈ **NP**

- We can prove that 3COLOR ∈ **NP** by designing a polynomial-time nondeterministic TM for 3COLOR.
- M = “On input ⟨*G*⟩:
  - **Nondeterministically** guess an assignment of colors to the nodes.
  - **Deterministically** check whether it is a 3-coloring.
  - If so, accept; otherwise reject.”

A Note on Terminology

- Although 3COLOR and 3SAT both have “3” in their names, the two are very different problems.
  - 3SAT means “there are three literals in every clause.” However, each literal can take on only one of two different values.
  - 3COLOR means “every node can take on one of three different colors.”
- **Key difference**:
  - In 3SAT variables have two choices of value.
  - In 3COLOR nodes have three choices of value.

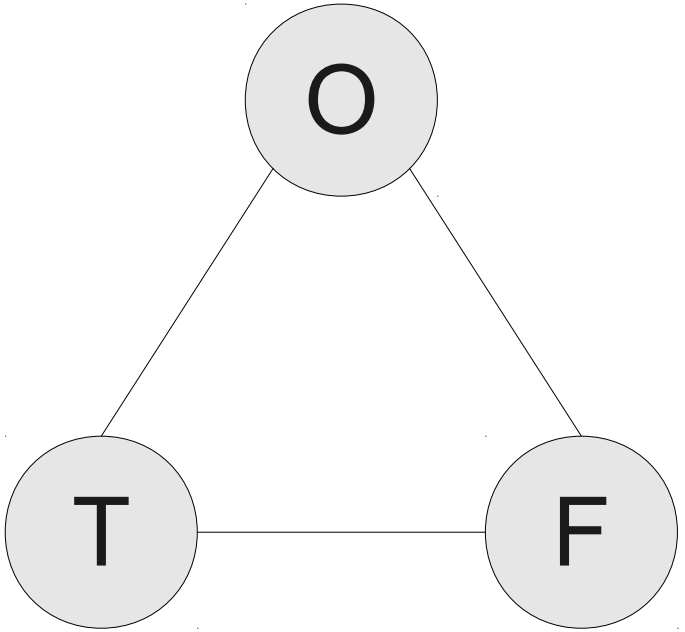
Why Not Two Colors?

- It would seem that 2COLOR (whether a graph has a 2-coloring) would be a better fit.
  - Every variable has one of two values.
  - Every node has one of two values.
- Interestingly, 2COLOR is known to be in **P** and is conjectured not to be **NP**-complete.
  - Though, if you can prove that it is, you've just won $1,000,000!

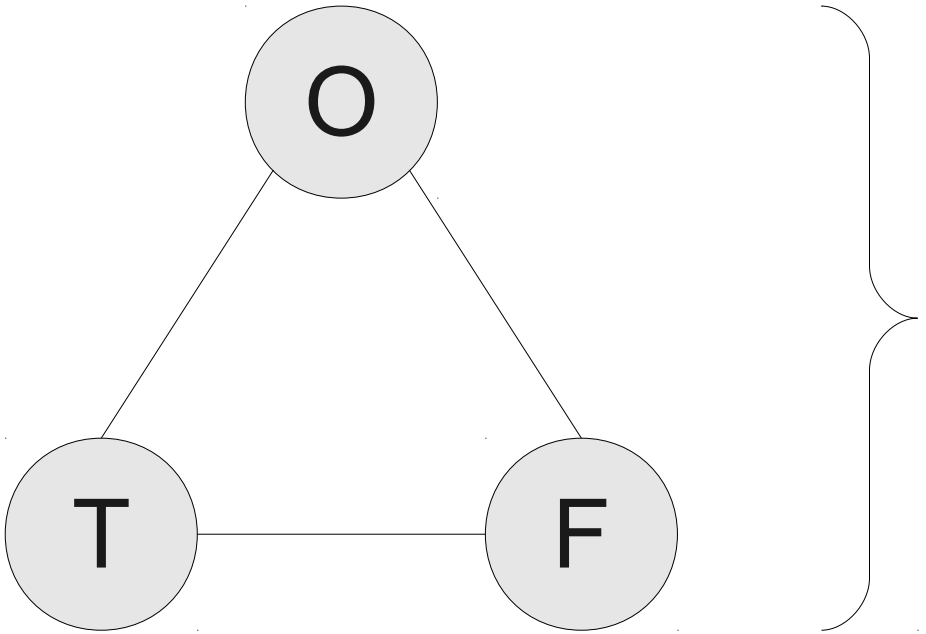
From 3SAT to 3COLOR

- In order to reduce 3SAT to 3COLOR, we need to somehow make a graph that is 3-colorable iff some 3-CNF formula φ is satisfiable.
- **Idea**: Use a collection of gadgets to solve the problem.
  - Build a gadget to assign two of the colors the labels “true” and “false.”
  - Build a gadget to force each variable to be either true or false.
  - Build a series of gadgets to force those variable assignments to satisfy each clause.

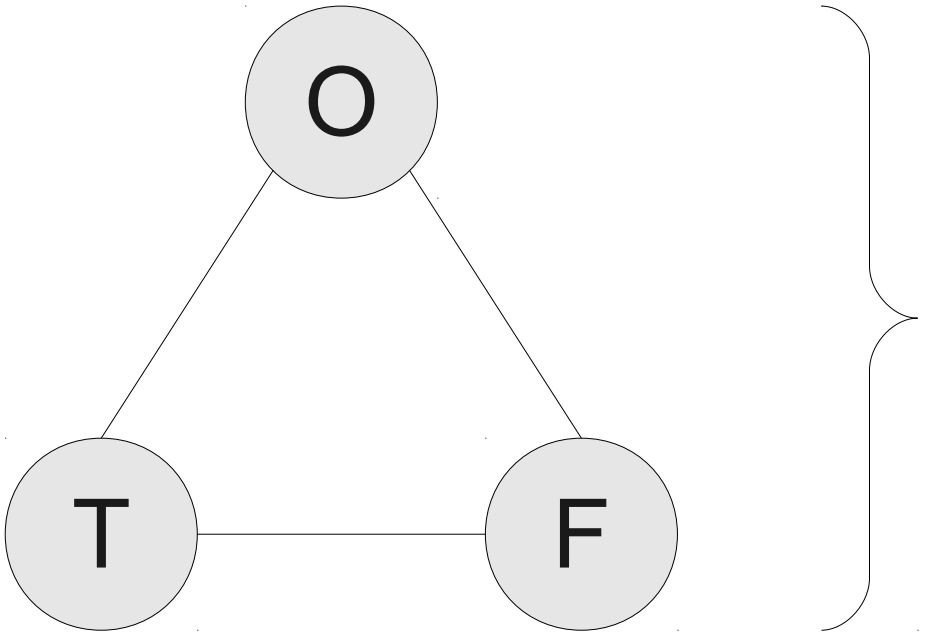
Gadget One: Assigning Meanings



Gadget One: Assigning Meanings

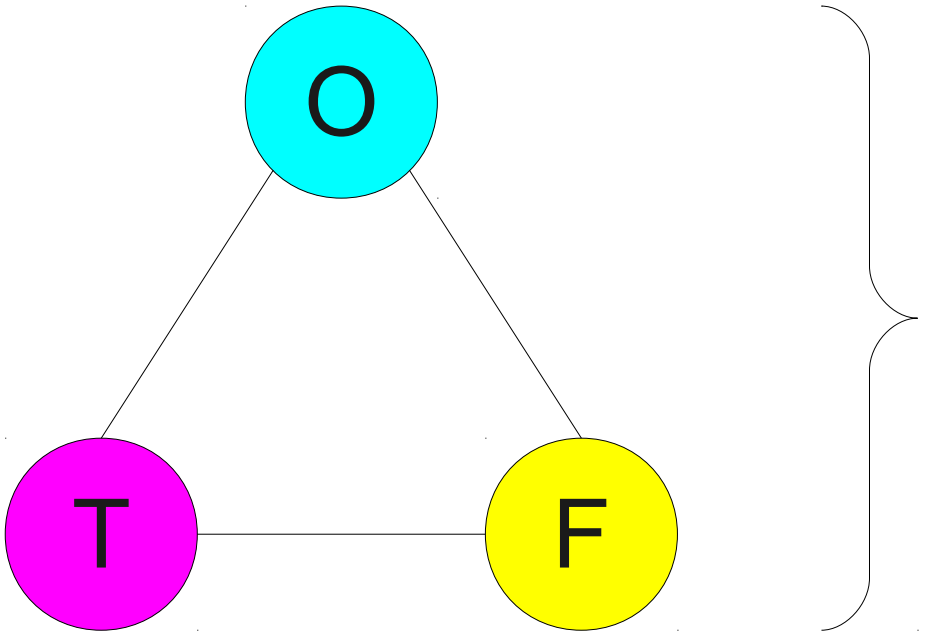


Gadget One: Assigning Meanings



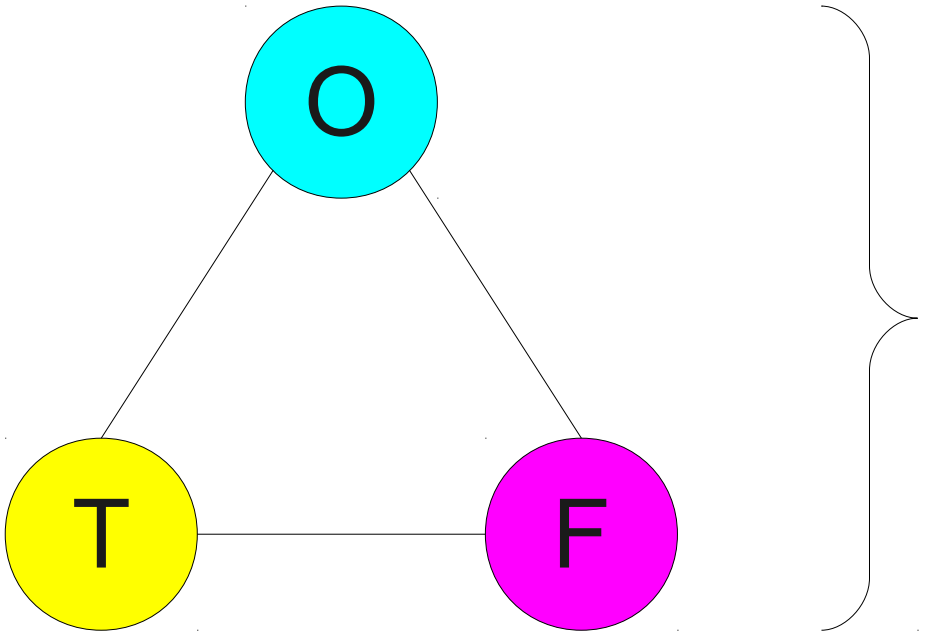
The color assigned to T will be interpreted as "true."
The color assigned to F will be interpreted as "false."
We do not associate any special meaning with O.

Gadget One: Assigning Meanings



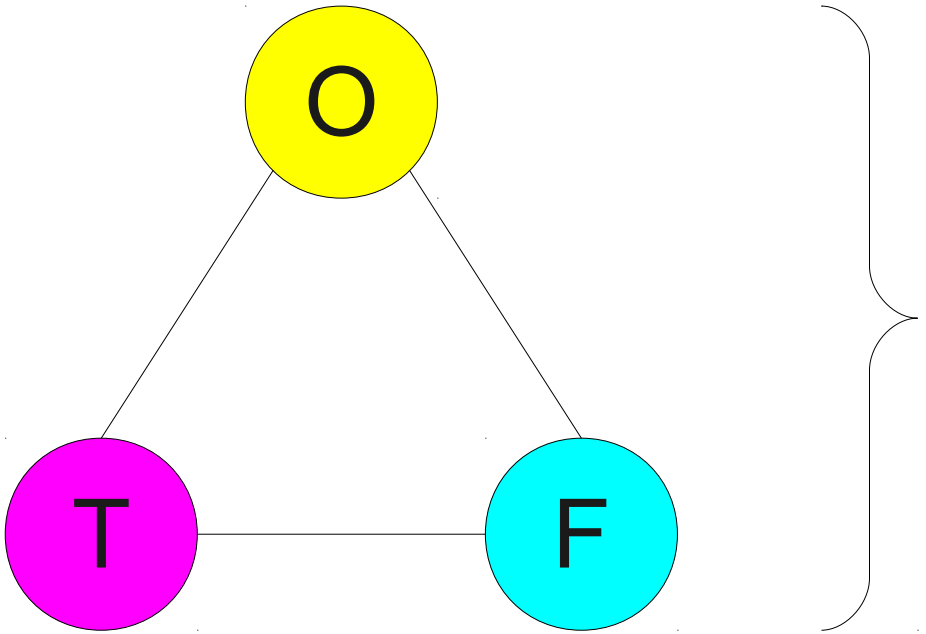
The color assigned to T will be interpreted as "true."
The color assigned to F will be interpreted as "false."
We do not associate any special meaning with O.

Gadget One: Assigning Meanings



The color assigned to T will be interpreted as "true."
The color assigned to F will be interpreted as "false."
We do not associate any special meaning with O.

Gadget One: Assigning Meanings



The color assigned to T will be interpreted as "true."
The color assigned to F will be interpreted as "false."
We do not associate any special meaning with O.

Gadget Two: Forcing a Choice

$$(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z)$$

Gadget Two: Forcing a Choice

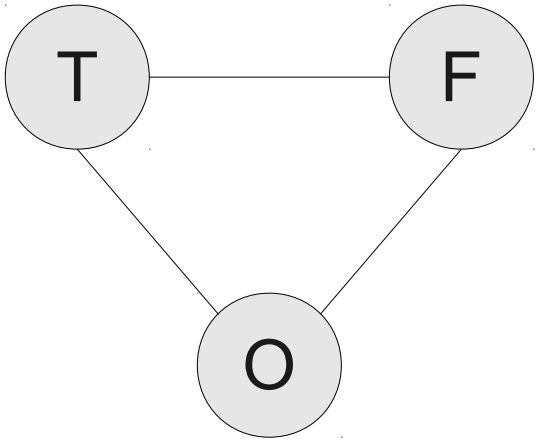
(x ∨ y ∨ ¬z) ∧ (¬x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ ¬z)

Gadget Two: Forcing a Choice

(x ∨ y ∨ ¬z) ∧ (¬x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ ¬z)

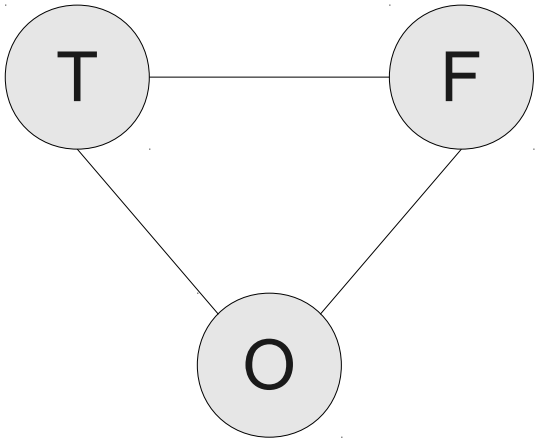
Gadget Two: Forcing a Choice

(x ∨ y ∨ ¬z) ∧ (¬x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ ¬z)



Gadget Two: Forcing a Choice

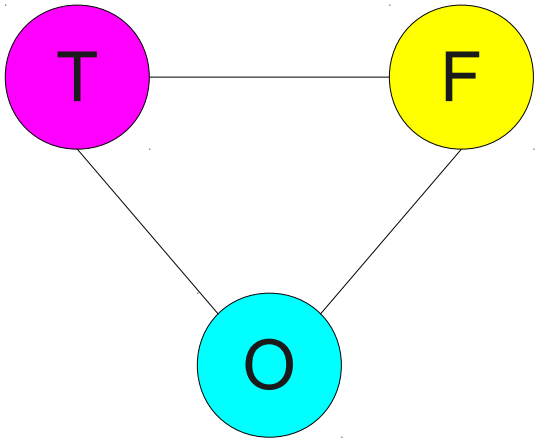
(x ∨ y ∨ ¬z) ∧ (¬x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ ¬z)





Gadget Two: Forcing a Choice

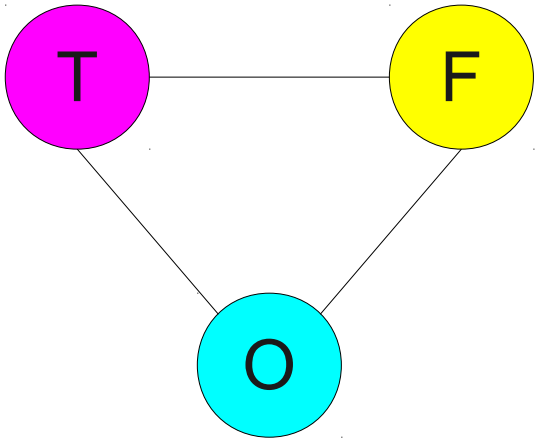
(x ∨ y ∨ ¬z) ∧ (¬x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ ¬z)





Gadget Two: Forcing a Choice

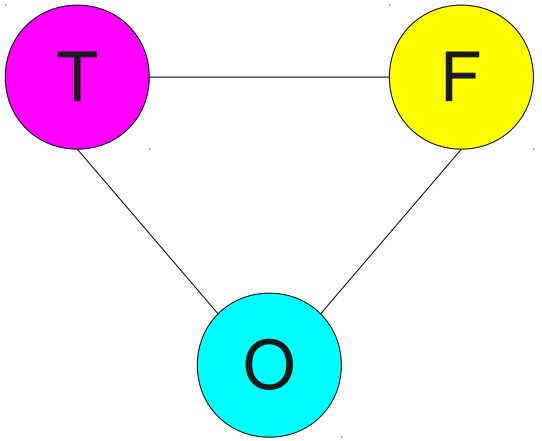
(x ∨ y ∨ ¬z) ∧ (¬x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ ¬z)





Gadget Two: Forcing a Choice

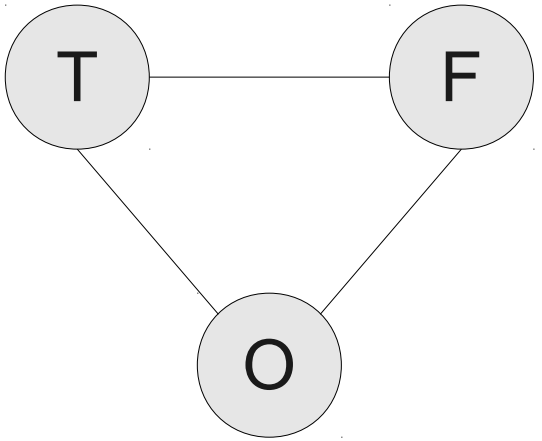
(x ∨ y ∨ ¬z) ∧ (¬x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ ¬z)





Gadget Two: Forcing a Choice

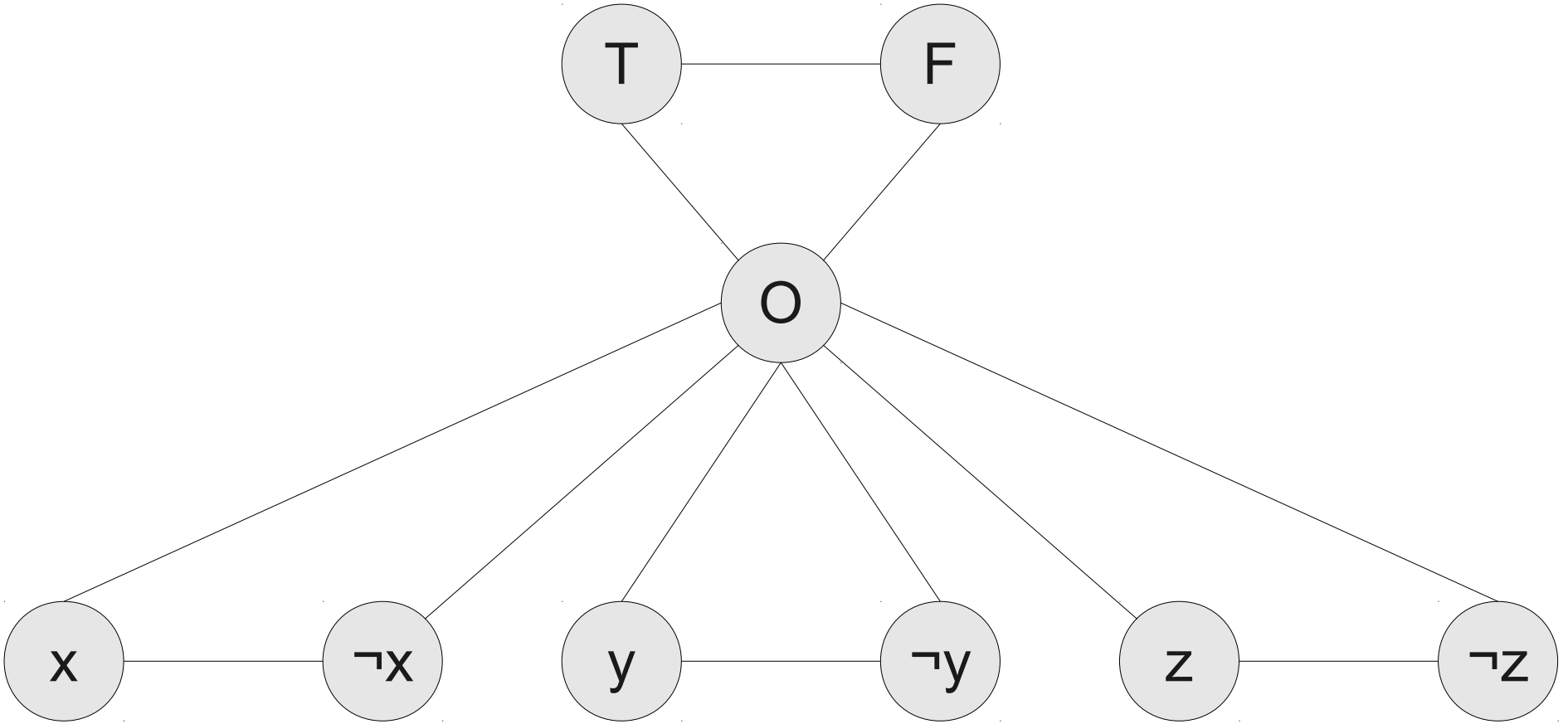
(x ∨ y ∨ ¬z) ∧ (¬x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ ¬z)





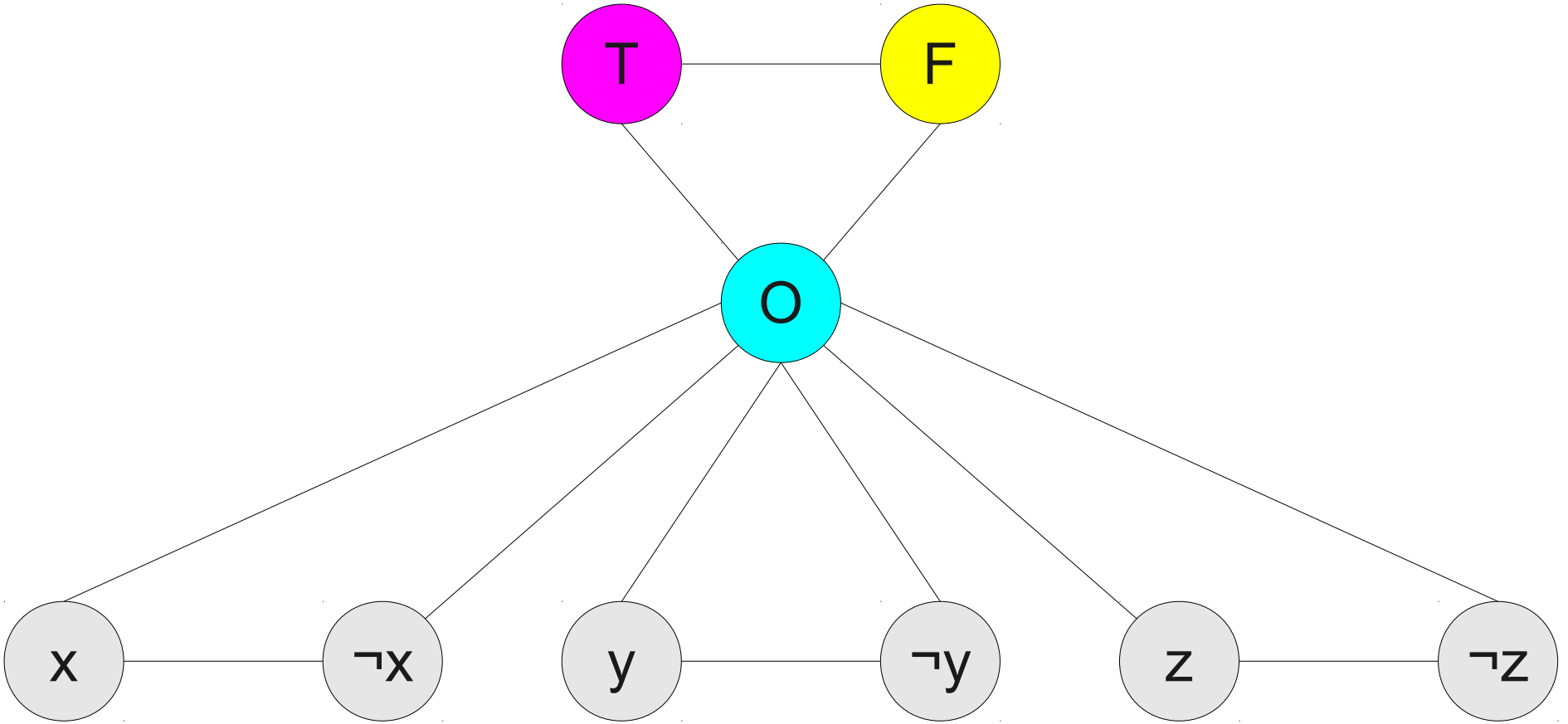
Gadget Two: Forcing a Choice

(x ∨ y ∨ ¬z) ∧ (¬x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ ¬z)



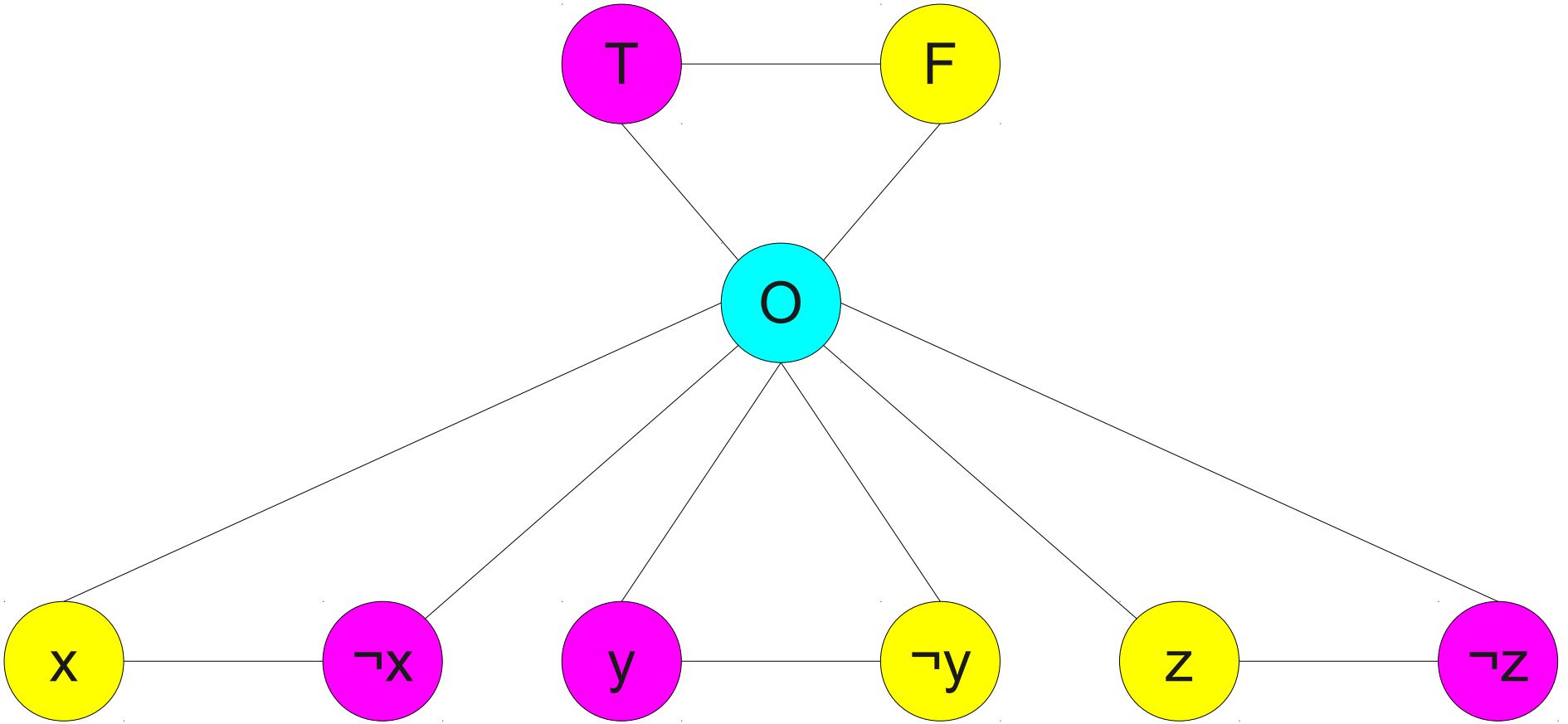
Gadget Two: Forcing a Choice

(x ∨ y ∨ ¬z) ∧ (¬x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ ¬z)



Gadget Two: Forcing a Choice

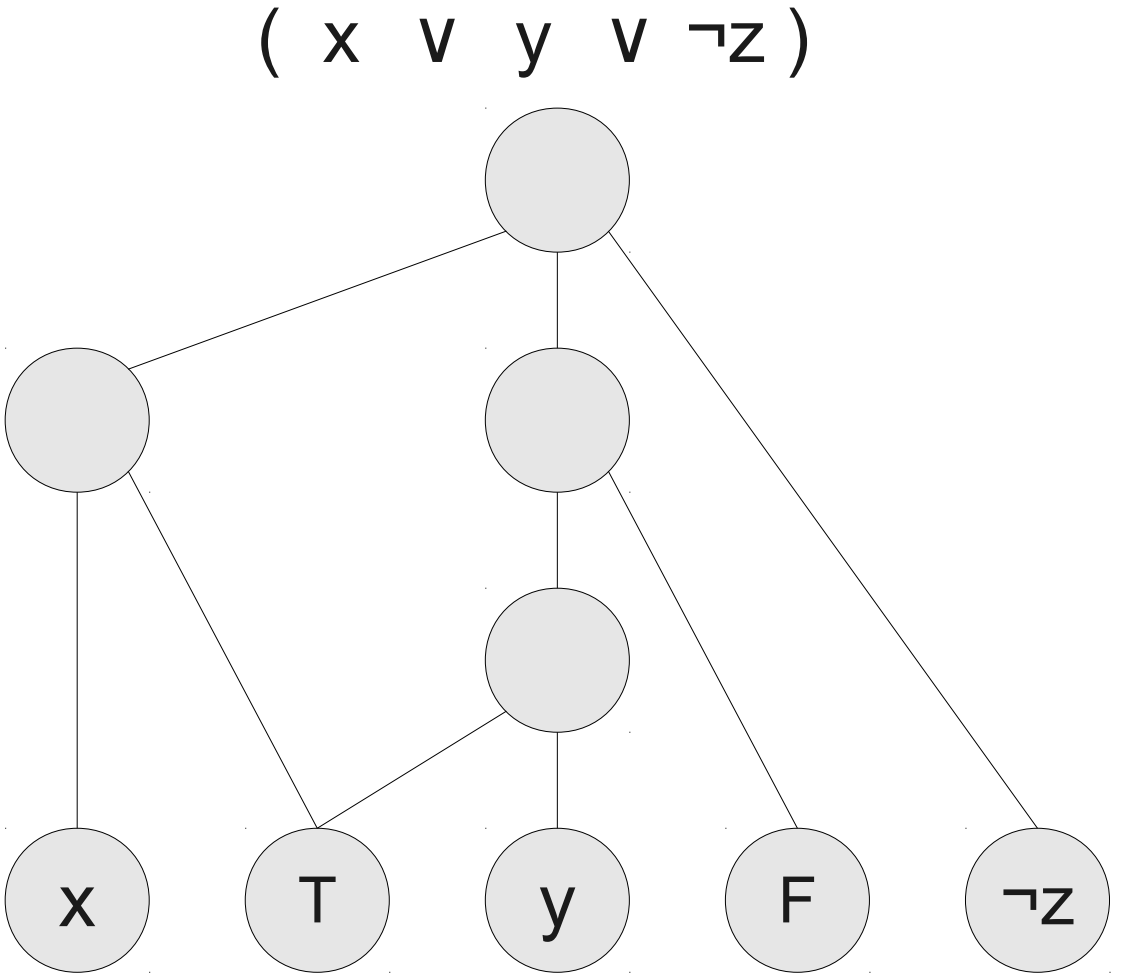
(x ∨ y ∨ ¬z) ∧ (¬x ∨ ¬y ∨ z) ∧ (¬x ∨ y ∨ ¬z)



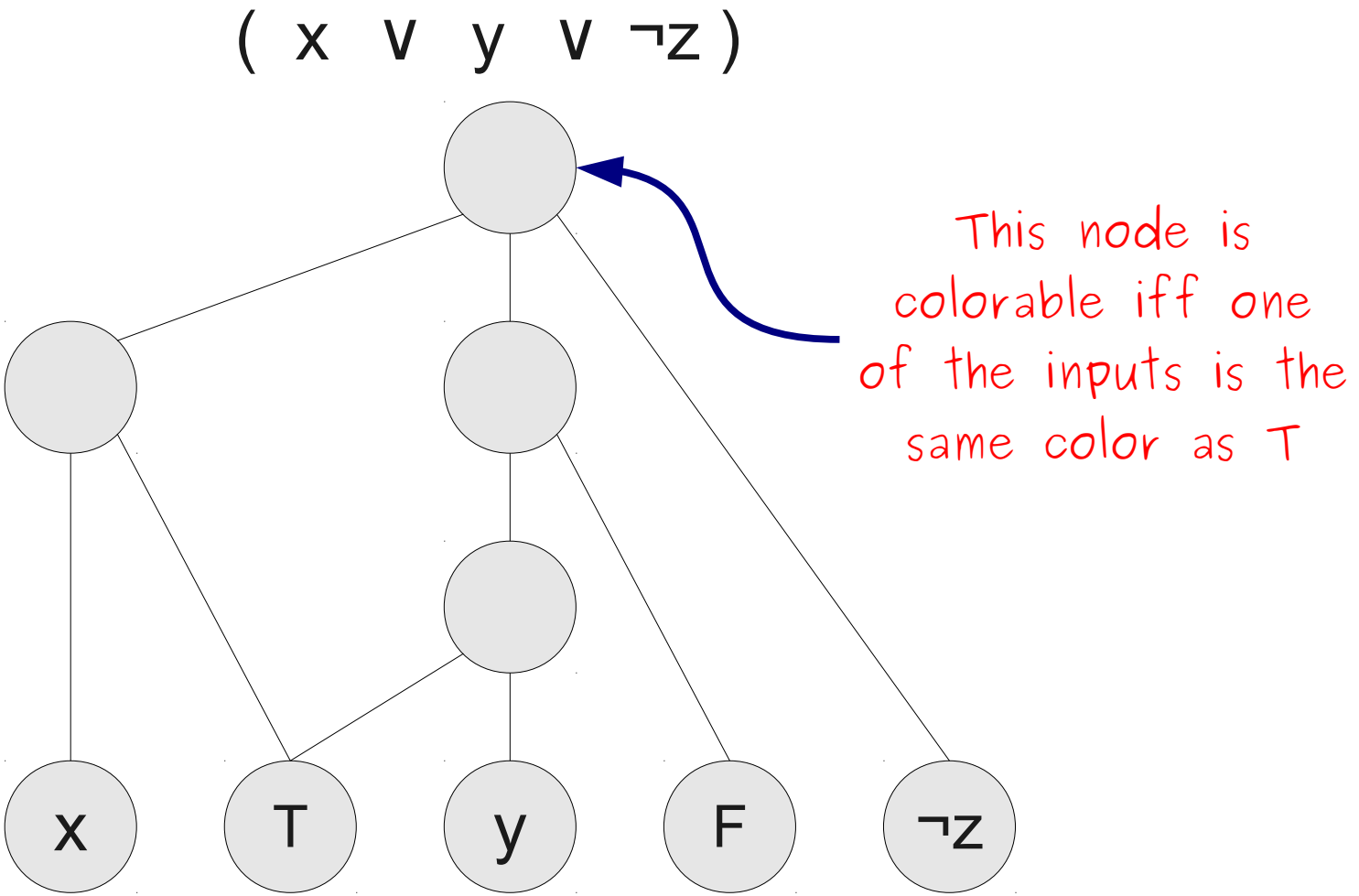
Gadget Three: Clause Satisfiability

$$(x \vee y \vee \neg z)$$

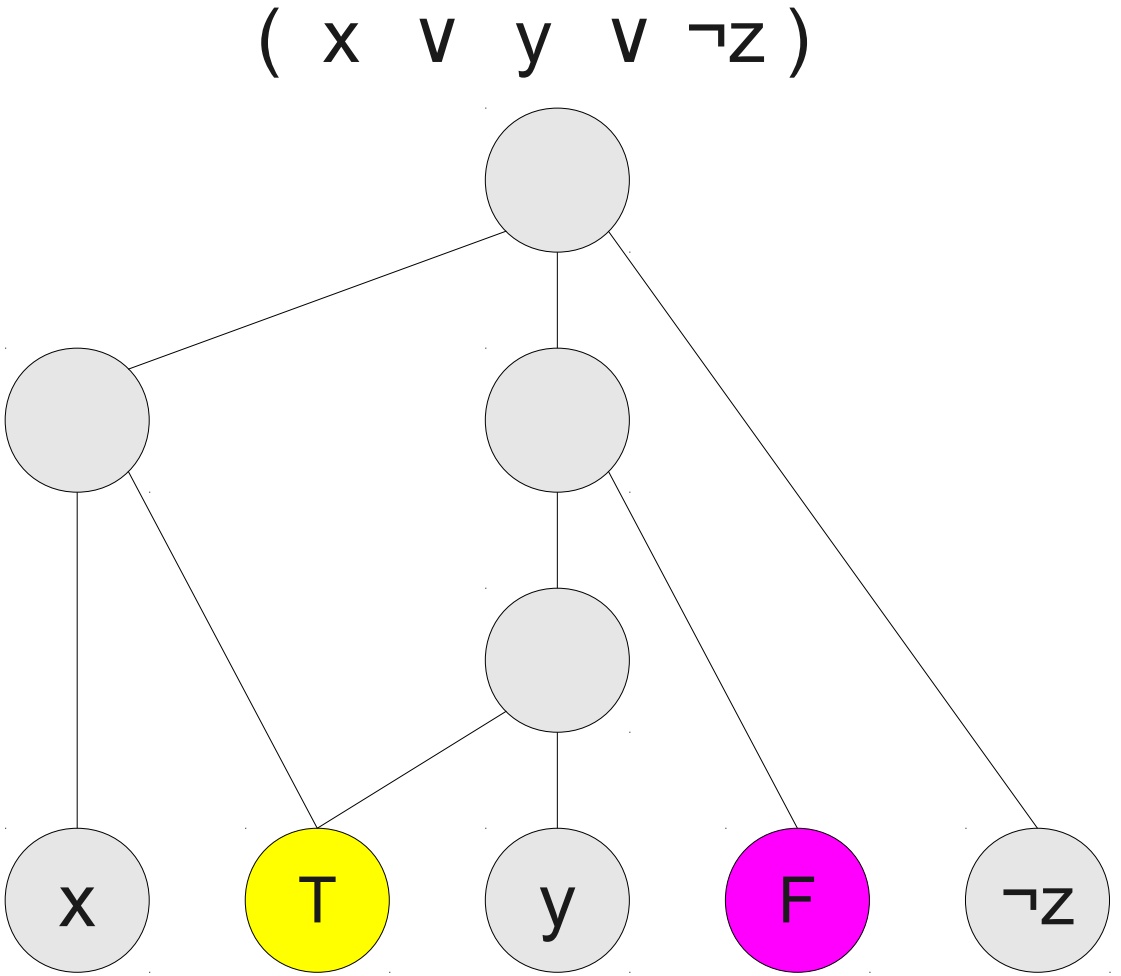
Gadget Three: Clause Satisfiability



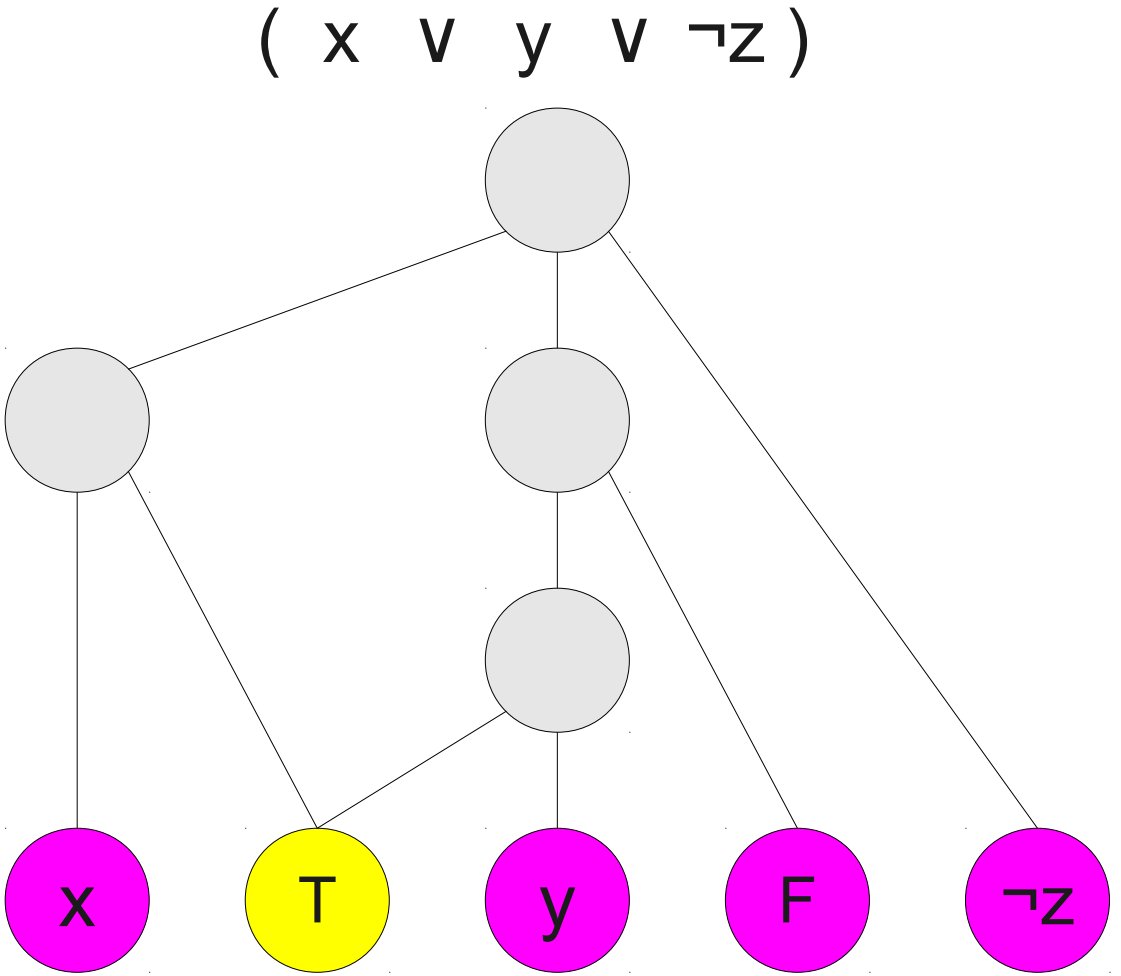
Gadget Three: Clause Satisfiability



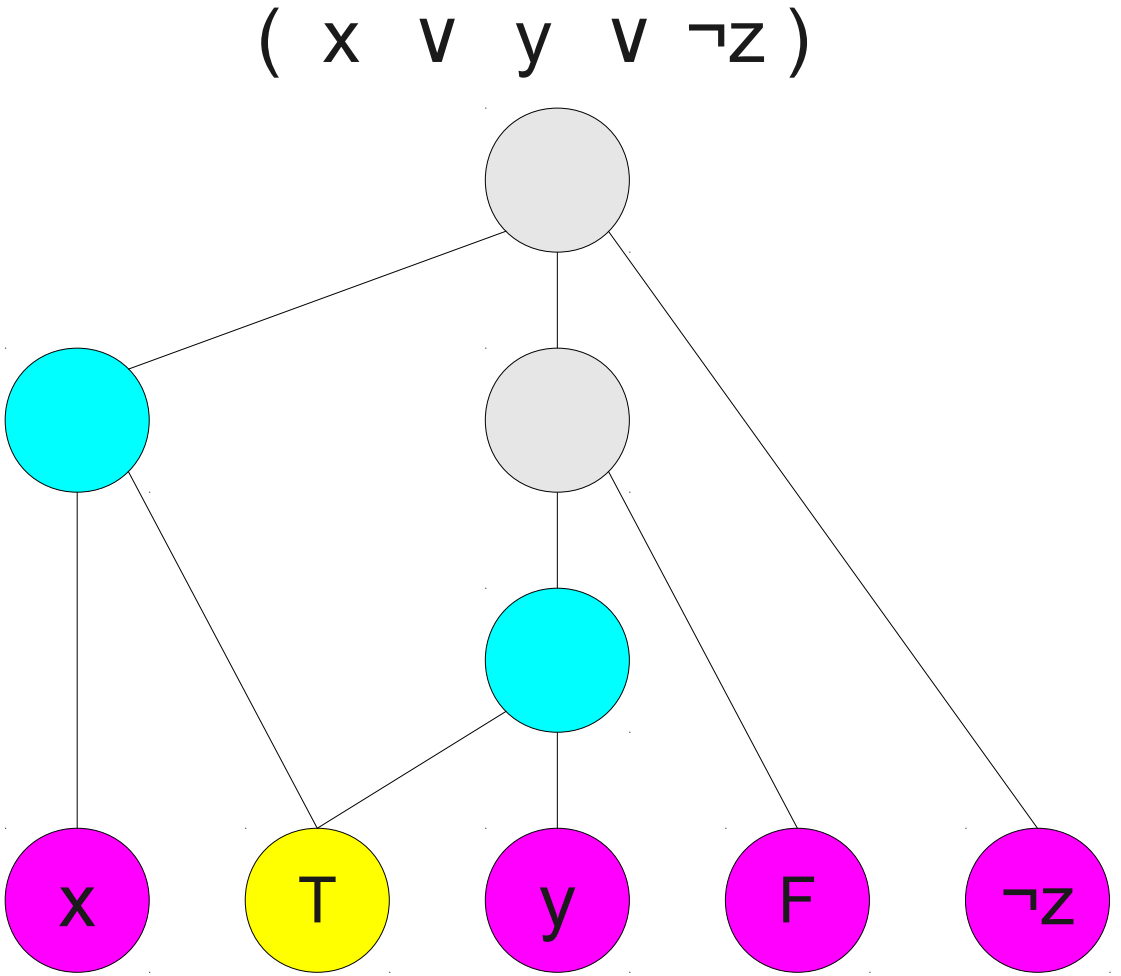
Gadget Three: Clause Satisfiability



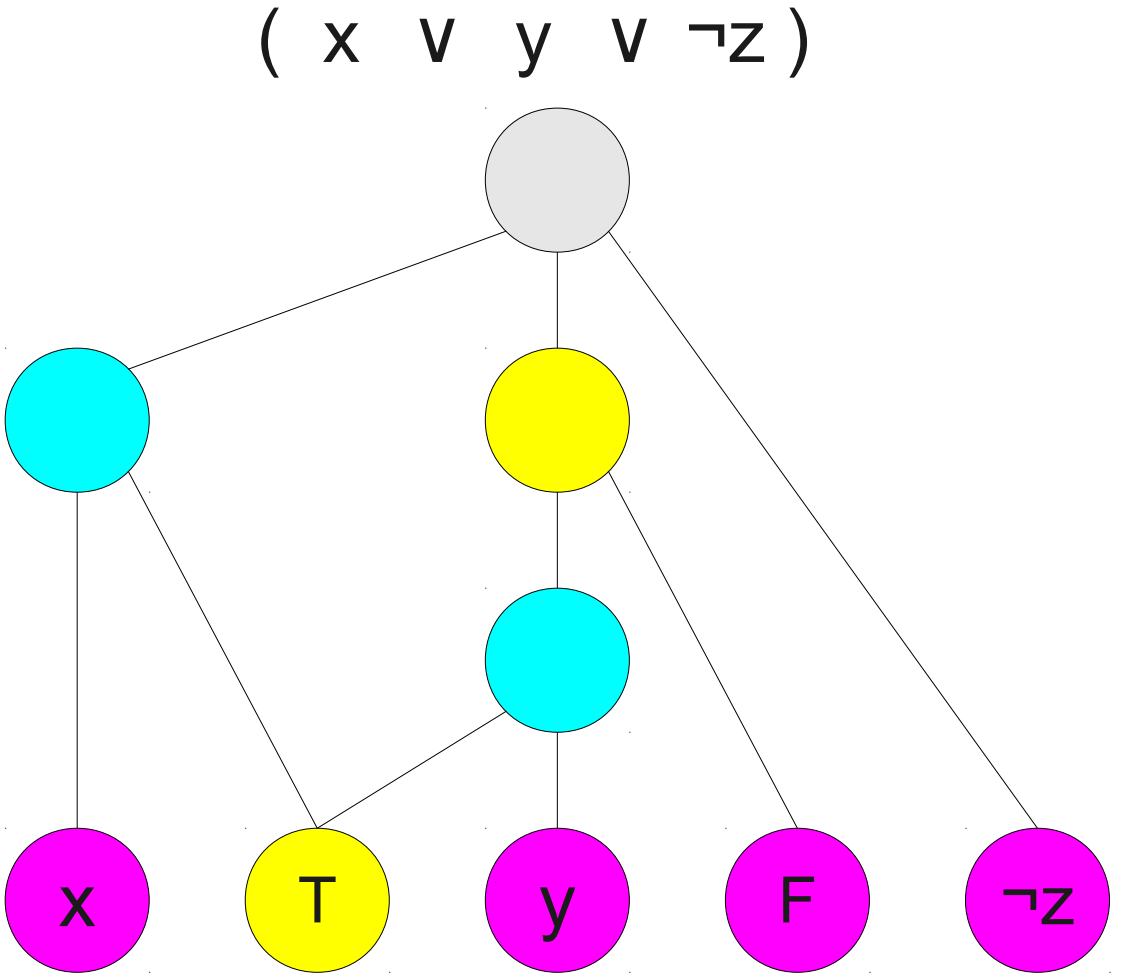
Gadget Three: Clause Satisfiability



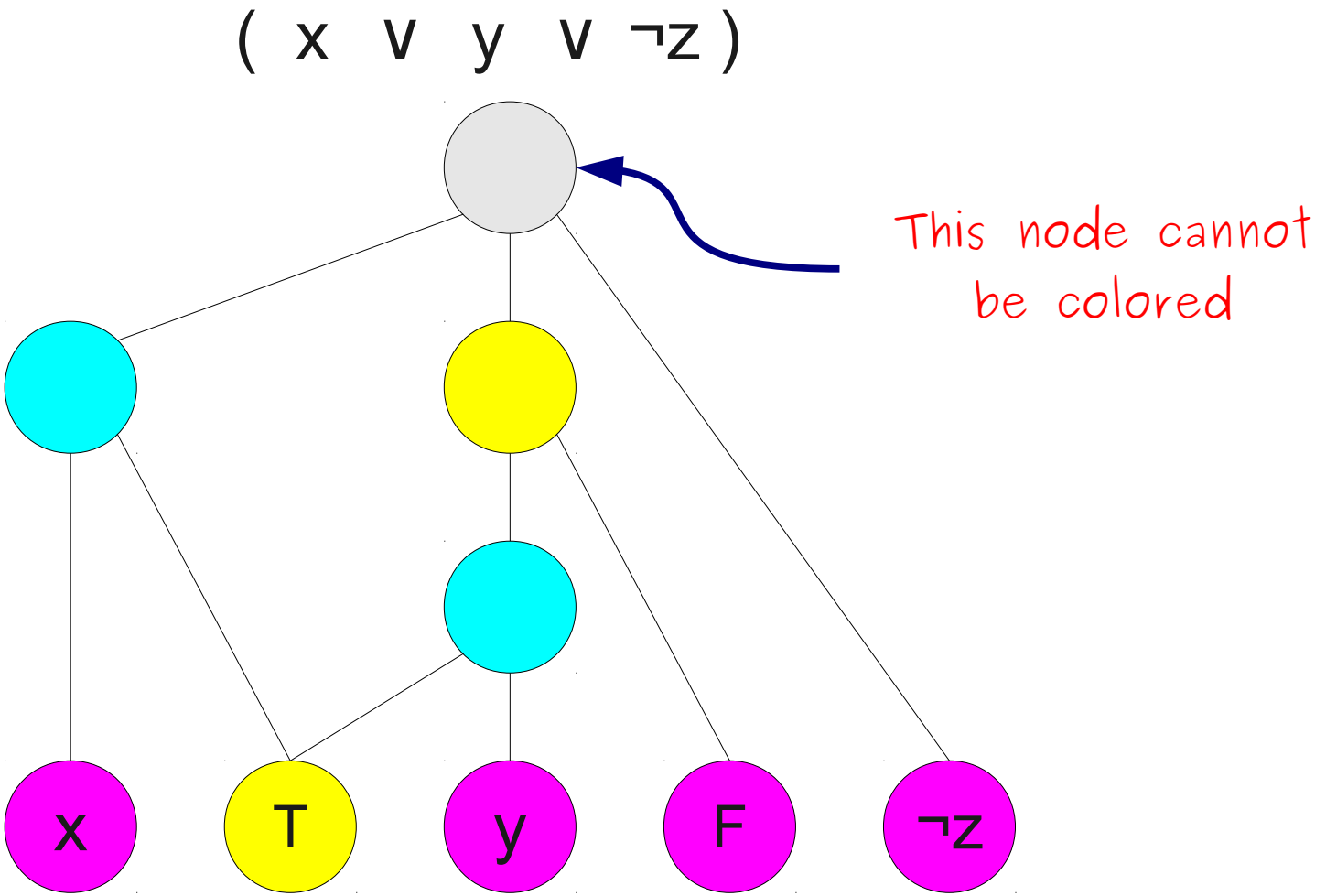
Gadget Three: Clause Satisfiability



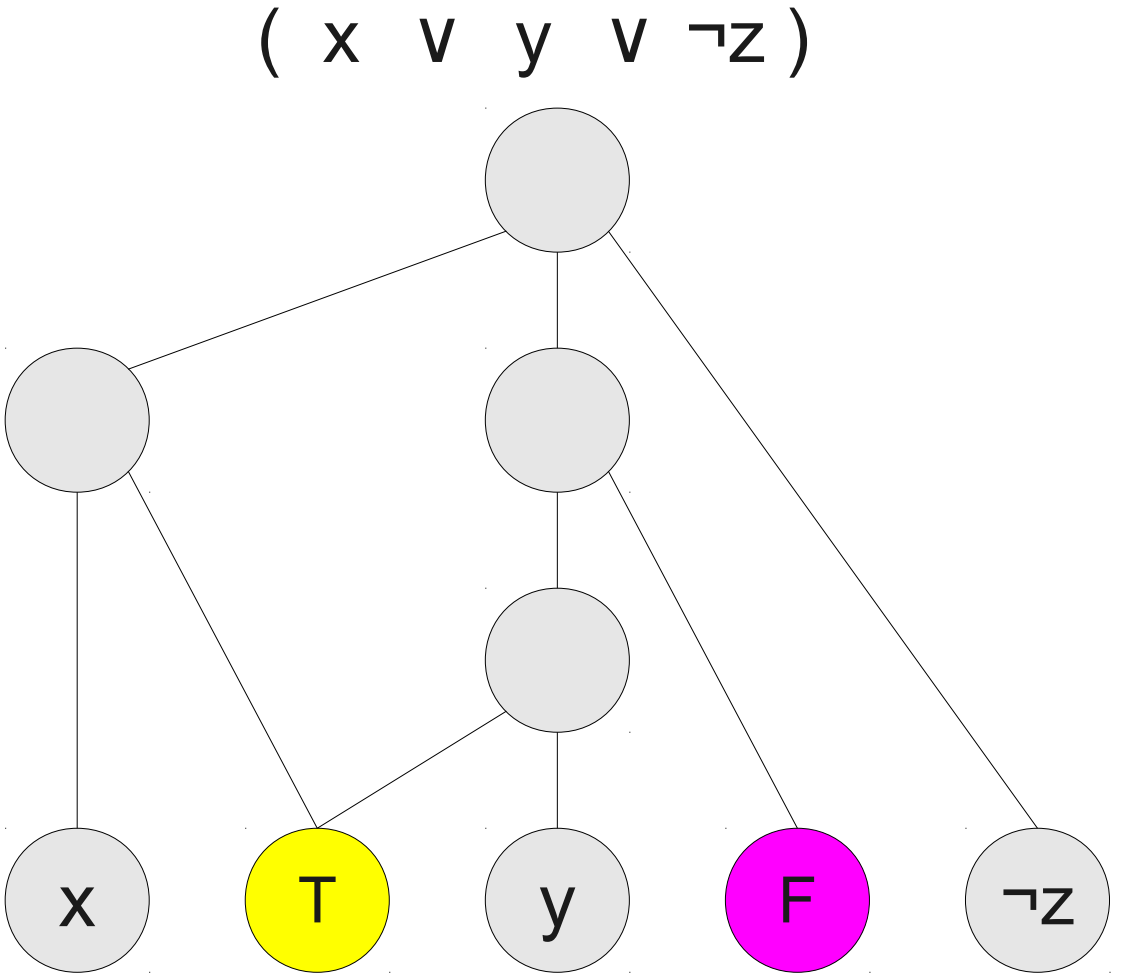
Gadget Three: Clause Satisfiability



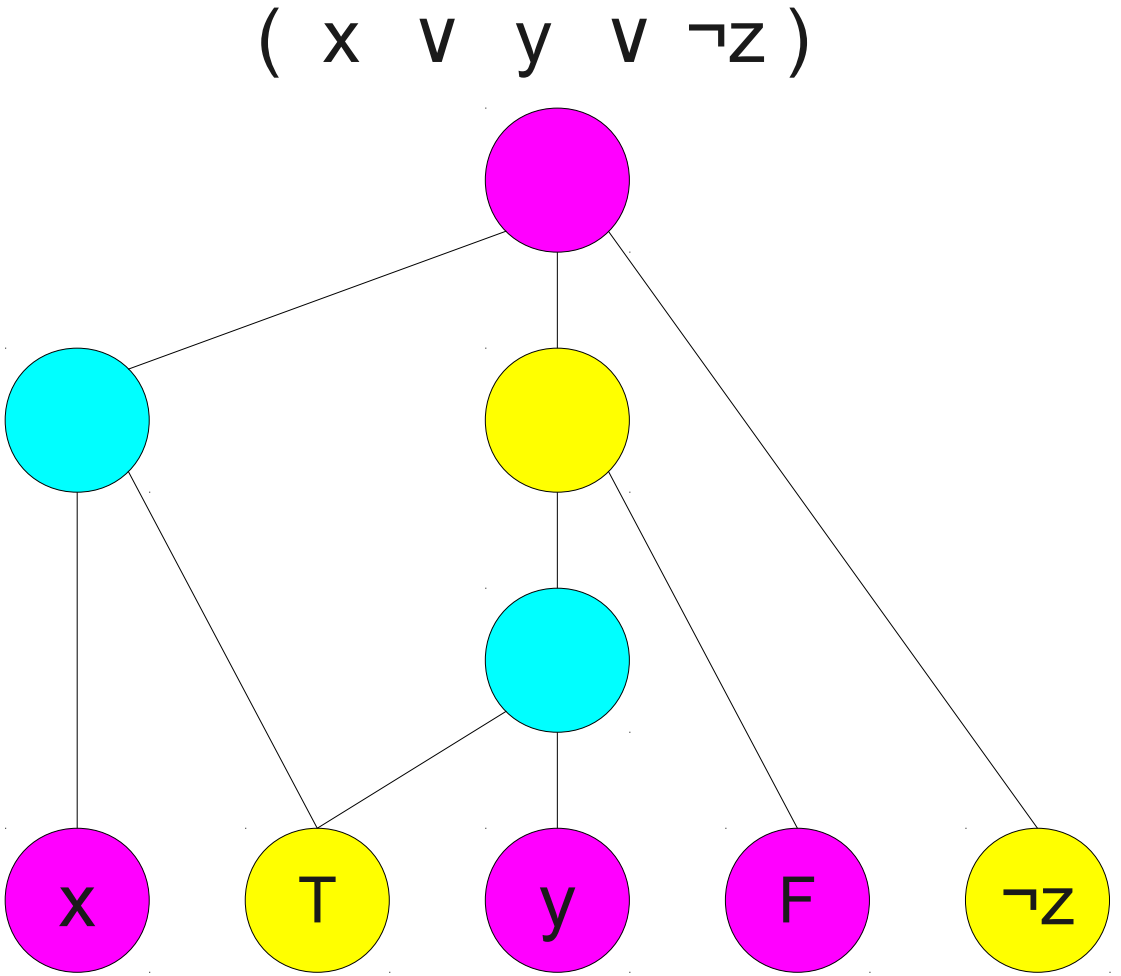
Gadget Three: Clause Satisfiability



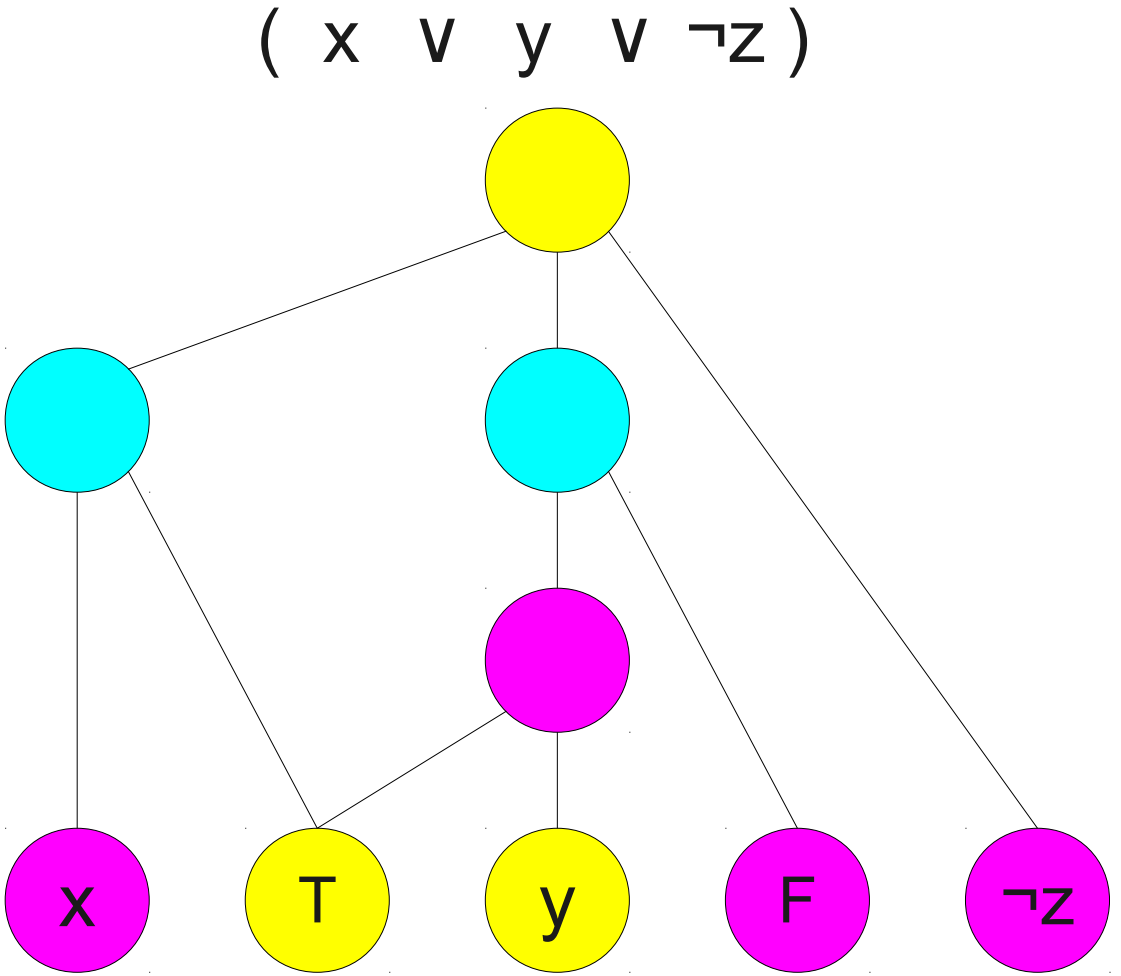
Gadget Three: Clause Satisfiability



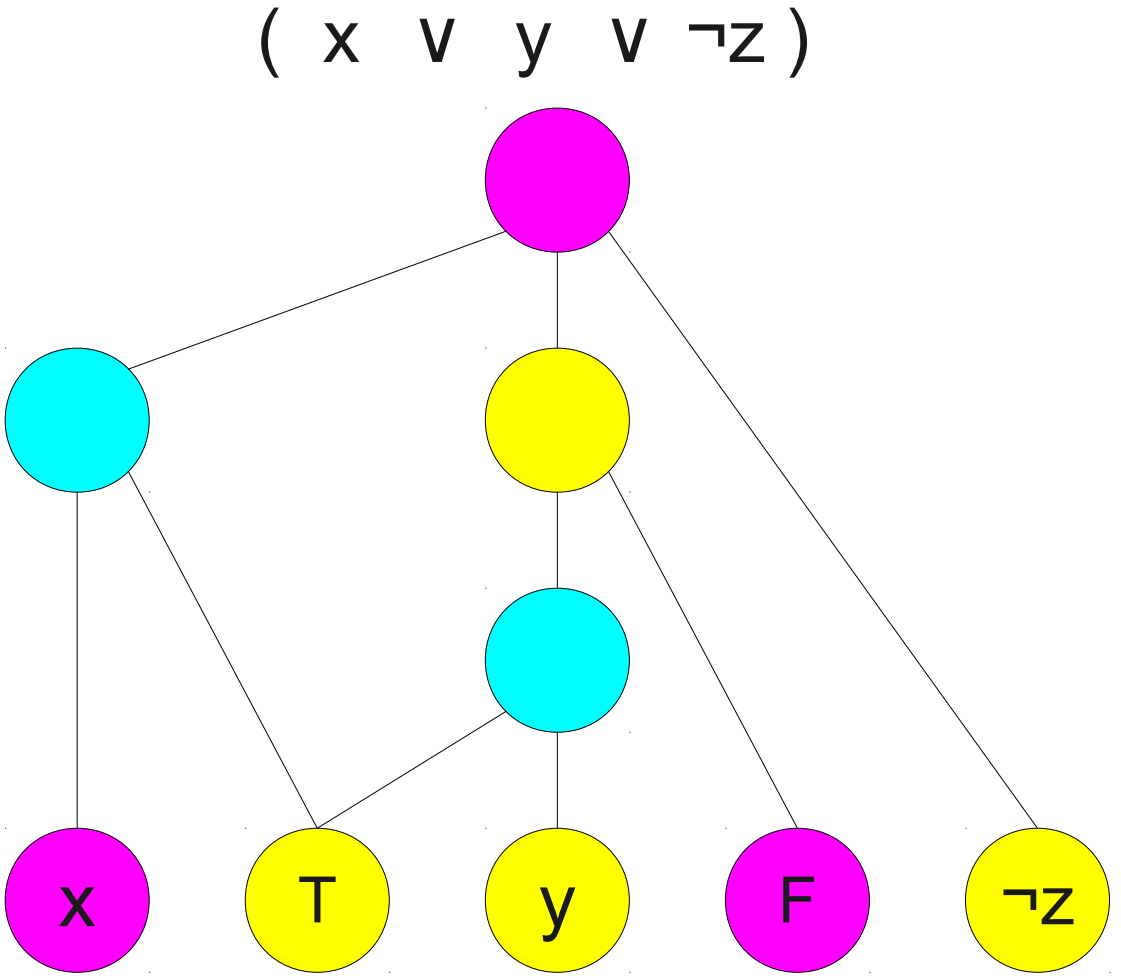
Gadget Three: Clause Satisfiability



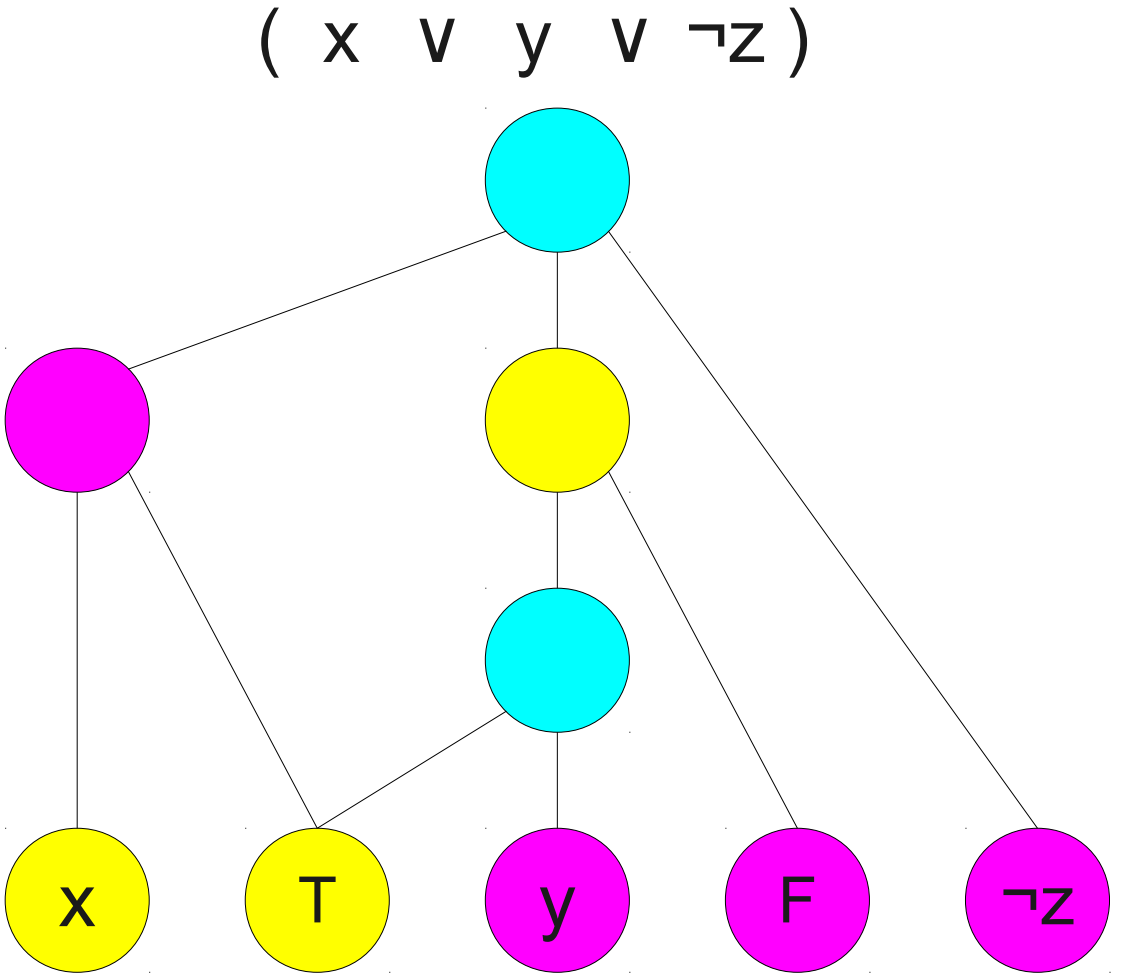
Gadget Three: Clause Satisfiability



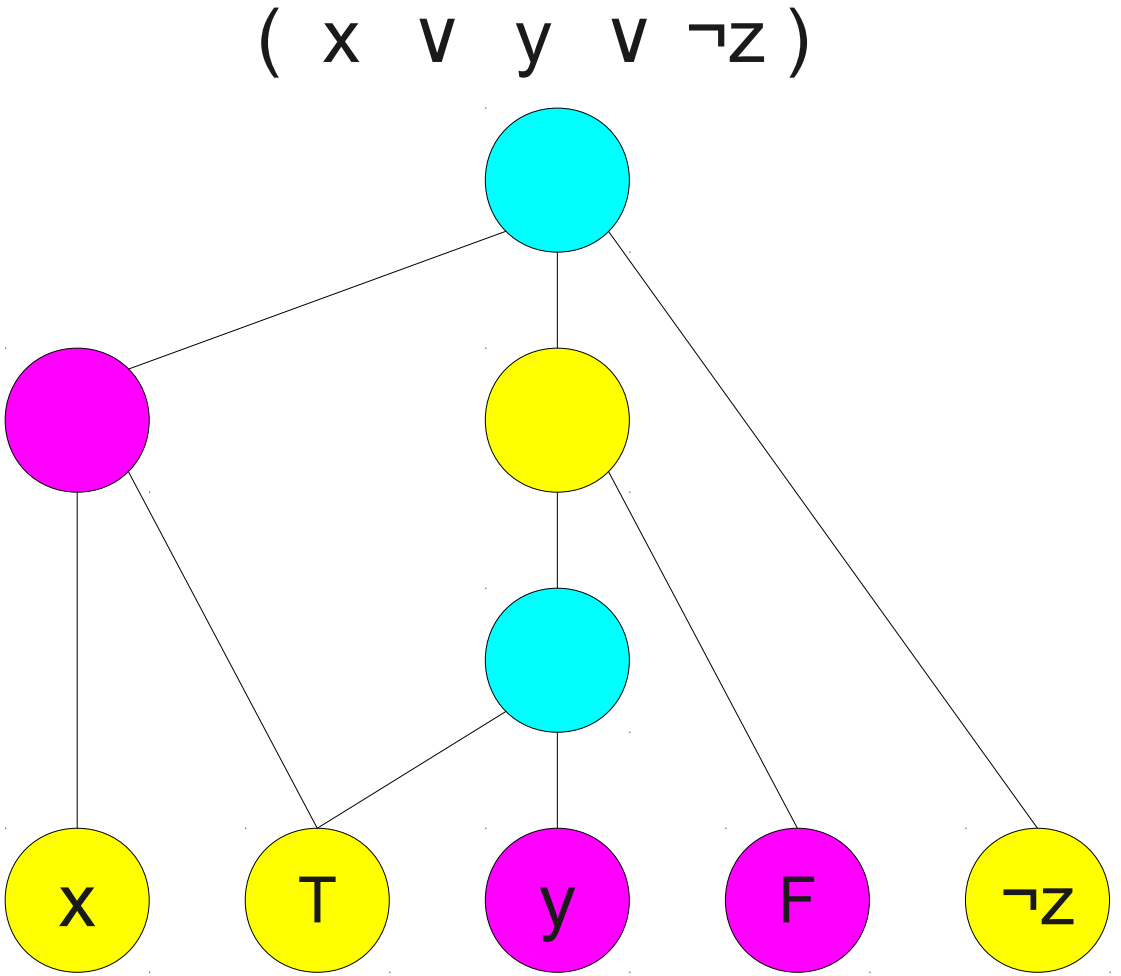
Gadget Three: Clause Satisfiability



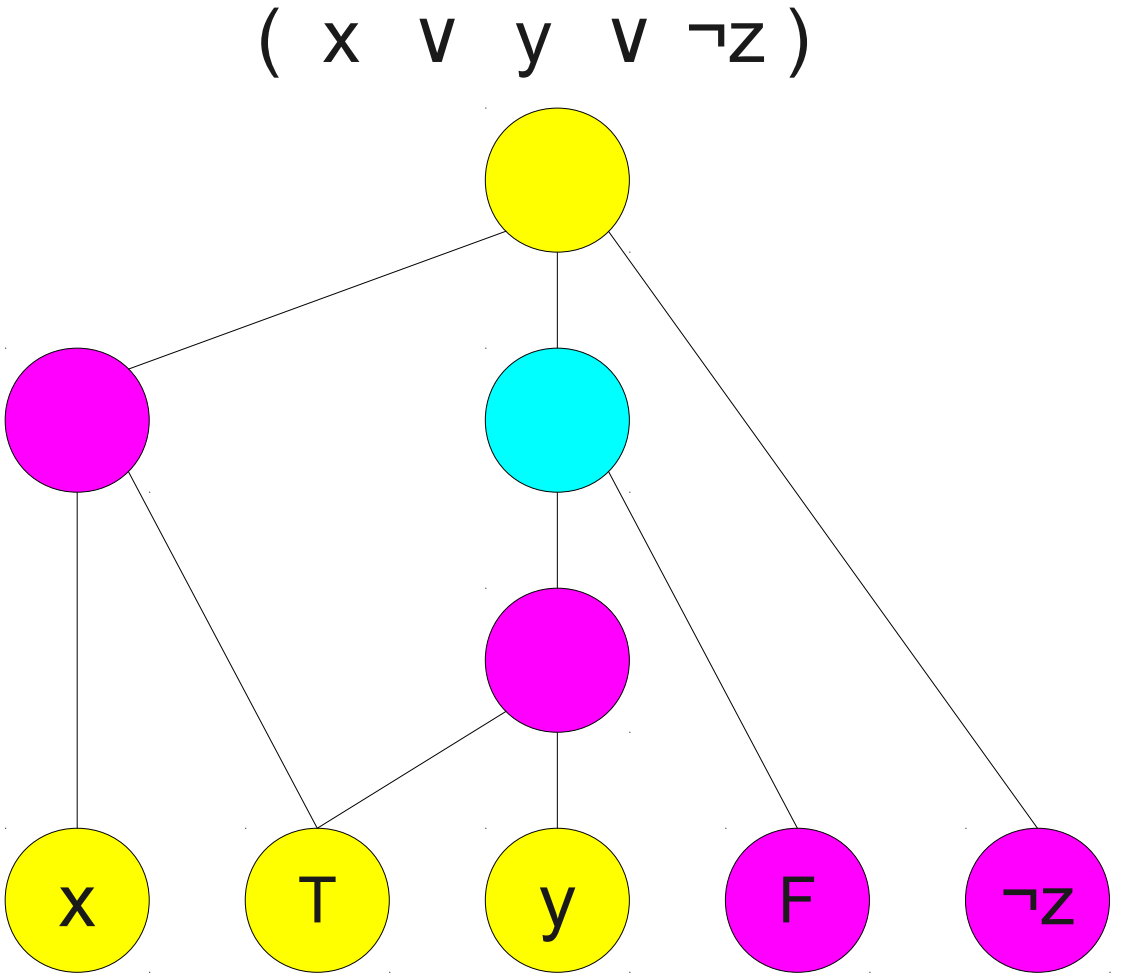
Gadget Three: Clause Satisfiability



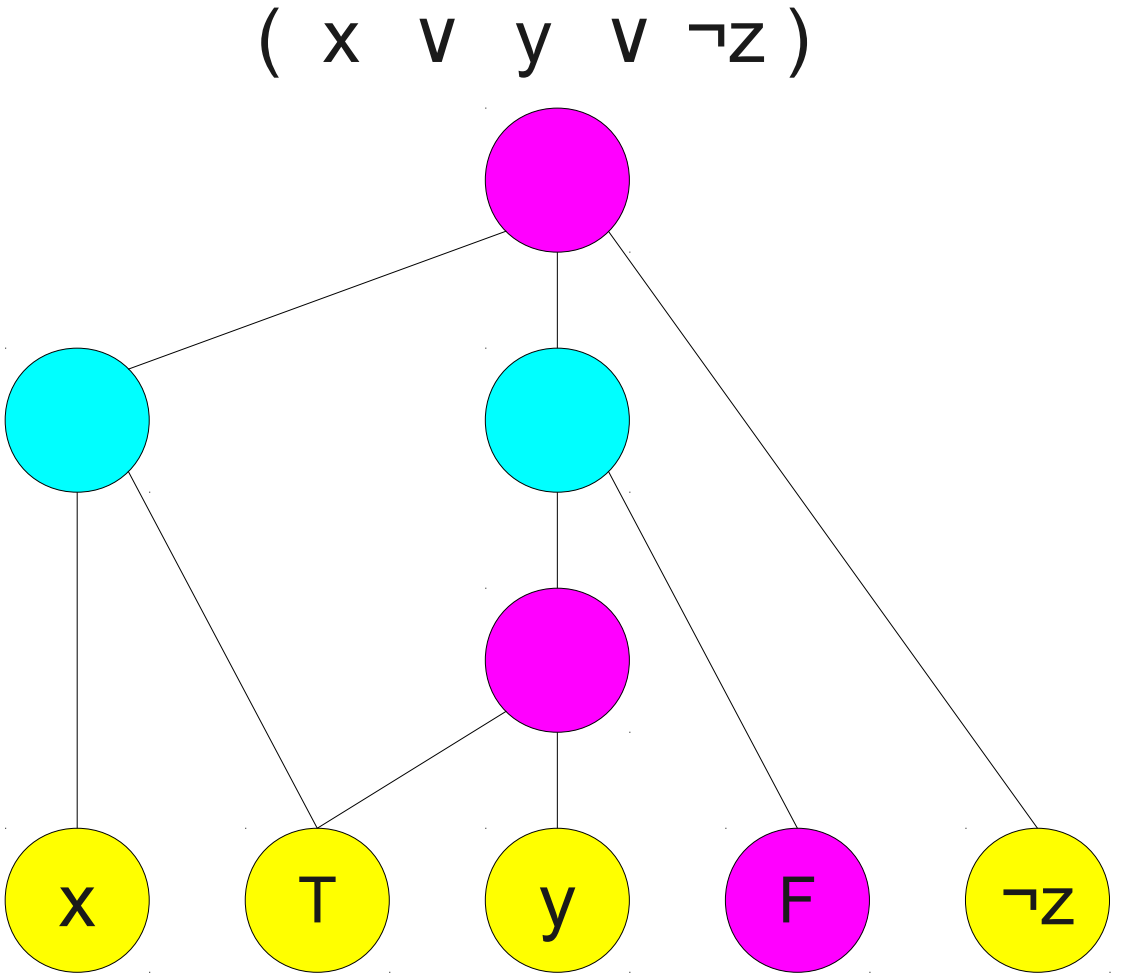
Gadget Three: Clause Satisfiability



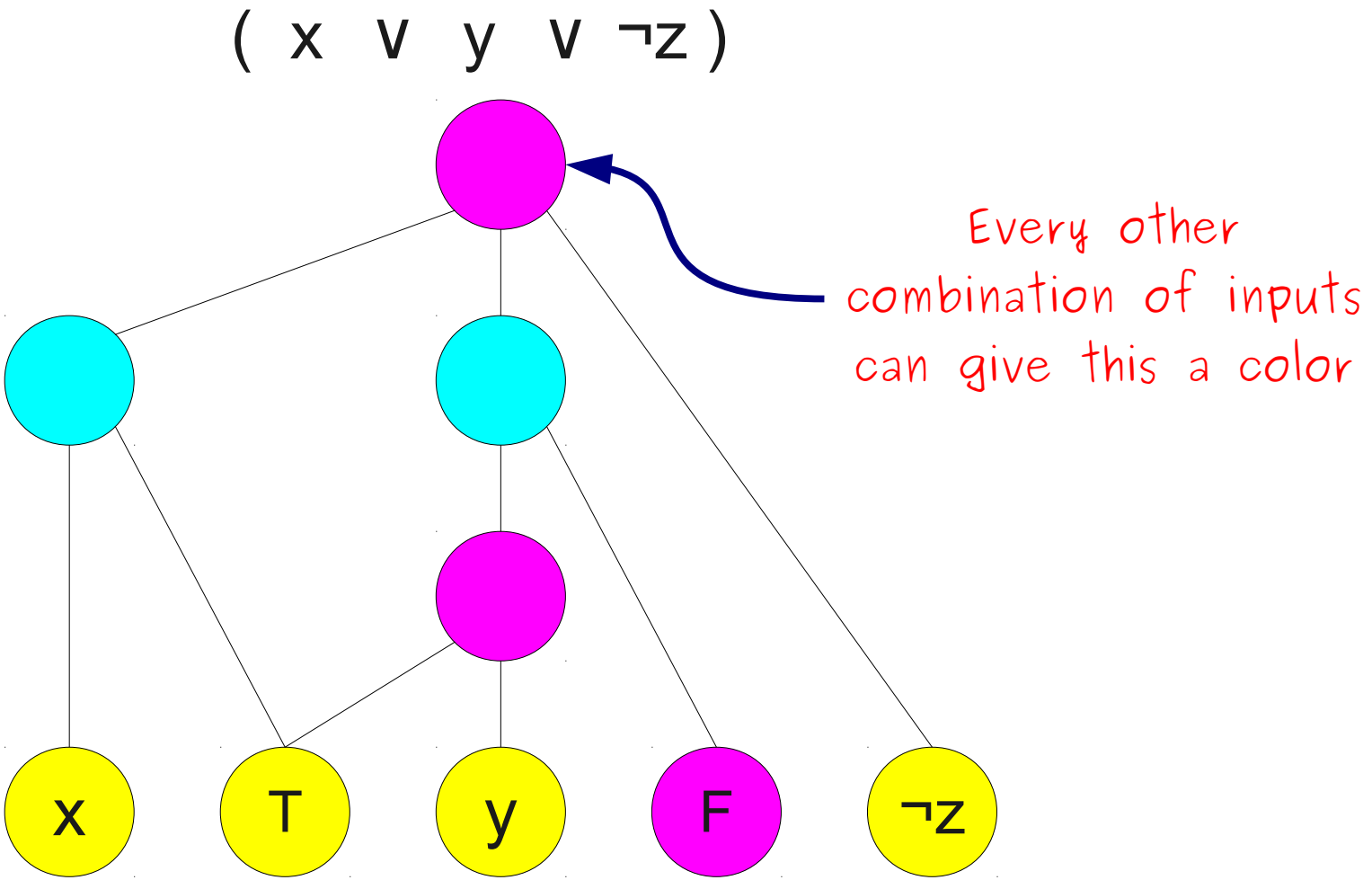
Gadget Three: Clause Satisfiability



Gadget Three: Clause Satisfiability



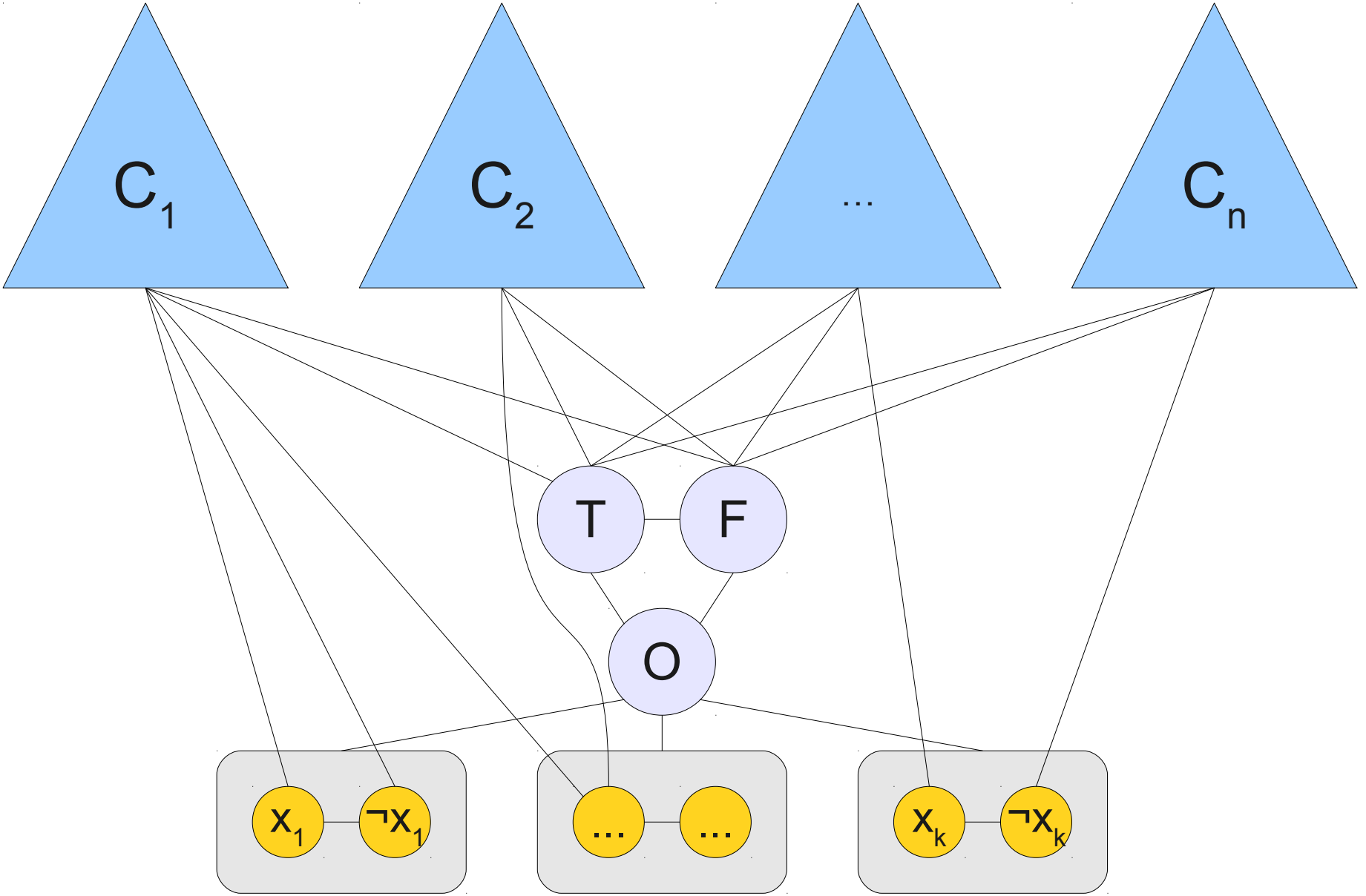
Gadget Three: Clause Satisfiability



Putting It All Together

- Construct the first gadget so we have a consistent definition of true and false.
- For each variable v:
  - Construct nodes v and $\neg v$.
  - Add an edge between v and $\neg v$.
  - Add an edge between v and O and between $\neg v$ and O.
- For each clause C:
  - Construct the earlier gadget from C by adding in the extra nodes and edges.

Putting It All Together



Analyzing the Reduction

- How large is the resulting graph?
- We have O(1) nodes to give meaning to “true” and “false.”
- Each variable gives O(1) nodes for its true and false values.
- Each clause gives O(1) nodes for its colorability gadget.
- Collectively, if there are n clauses, there are O(n) variables.
- Total size of the graph is O(n).

Another **NP**-Complete Problem

$$U = \{1, 2, 3, 4, 5, 6\}$$

$$S = \left\{ \begin{array}{ccc} \{1, 2, 5\}, & \{2, 5\}, & \{1, 3, 6\}, \\ \{2, 3, 4\}, & \{4\}, & \{1, 5, 6\} \end{array} \right\}$$

Let U be a set of elements (the **universe**) and $S \subseteq \wp(U)$. An **exact covering** of U is a collection of sets $I \subseteq S$ such that every element of U belongs to exactly one set in I.

$$U = \{1, 2, 3, 4, 5, 6\}$$

$$S = \{\{1, 2, 5\}, \{2, 5\}, \{1, 3, 6\}, \{2, 3, 4\}, \{4\}, \{1, 5, 6\}\}$$

Let U be a set of elements (the **universe**) and $S \subseteq \wp(U)$. An **exact covering** of U is a collection of sets $I \subseteq S$ such that every element of U belongs to exactly one set in I.

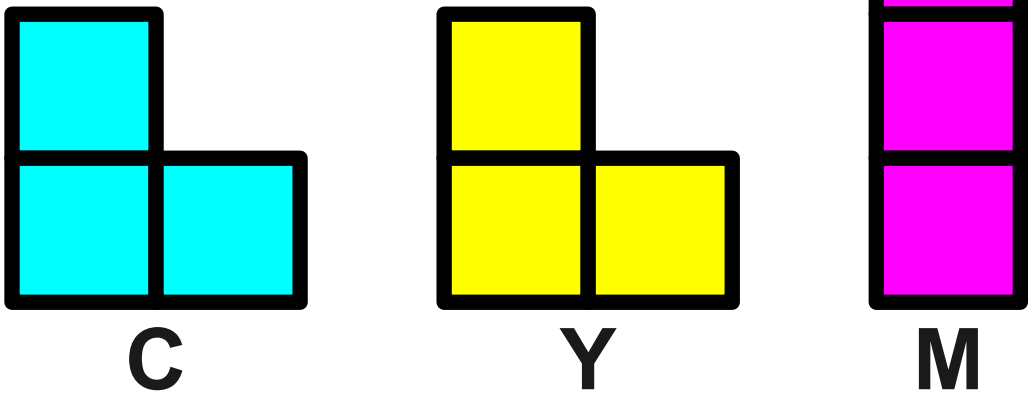
$$U = \{1, 2, 3, 4, 5, 6\}$$

$$S = \{\{1, 2, 5\}, \{2, 5\}, \{1, 3, 6\}, \{2, 3, 4\}, \{4\}, \{1, 5, 6\}\}$$

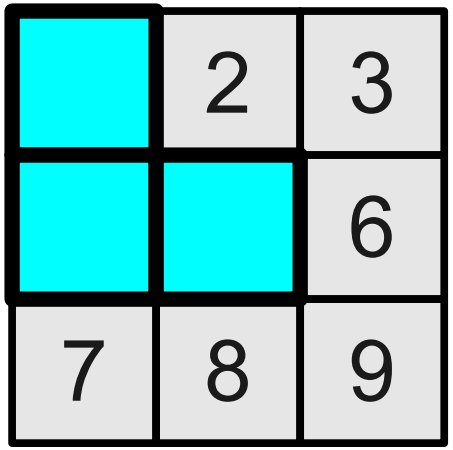
Let U be a set of elements (the **universe**) and $S \subseteq \wp(U)$. An **exact covering** of U is a collection of sets $I \subseteq S$ such that every element of U belongs to exactly one set in I.

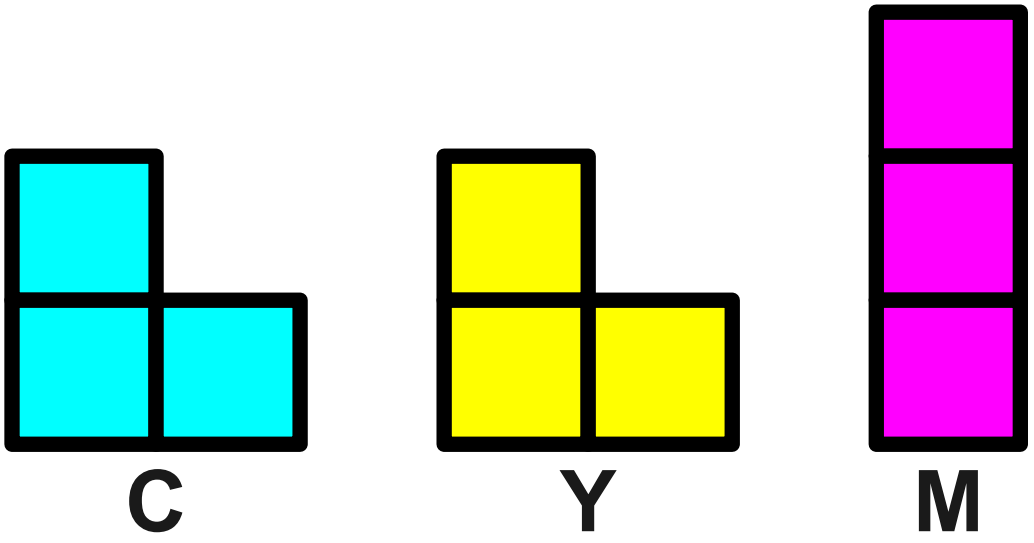
Applications of Exact Covering

1	2	3
4	5	6
7	8	9

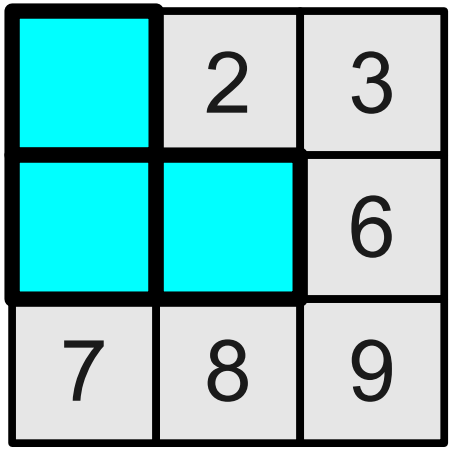


Applications of Exact Covering

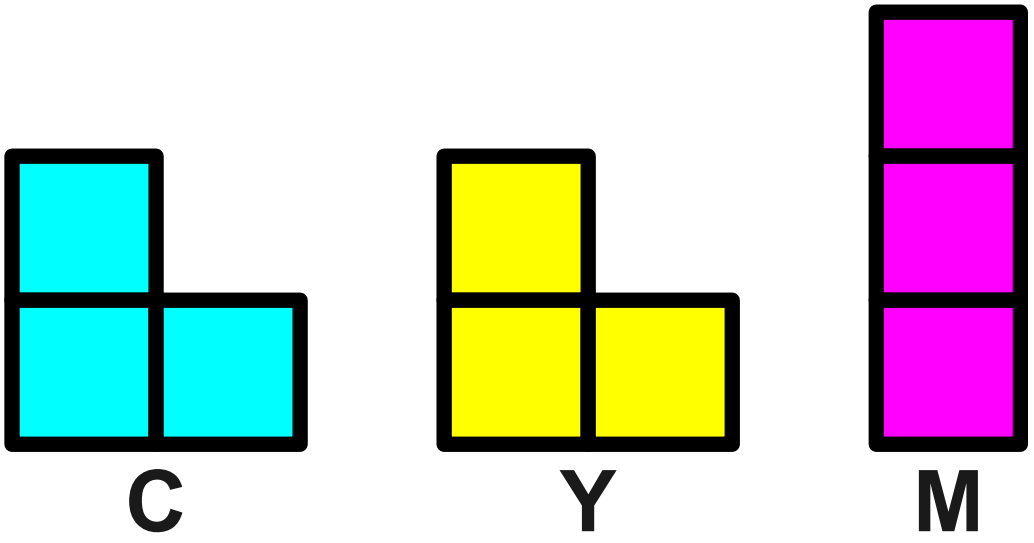




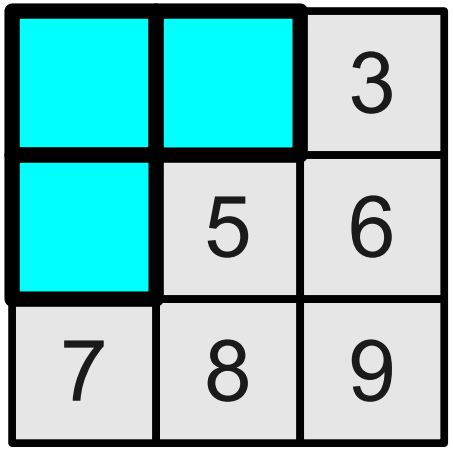
Applications of Exact Covering



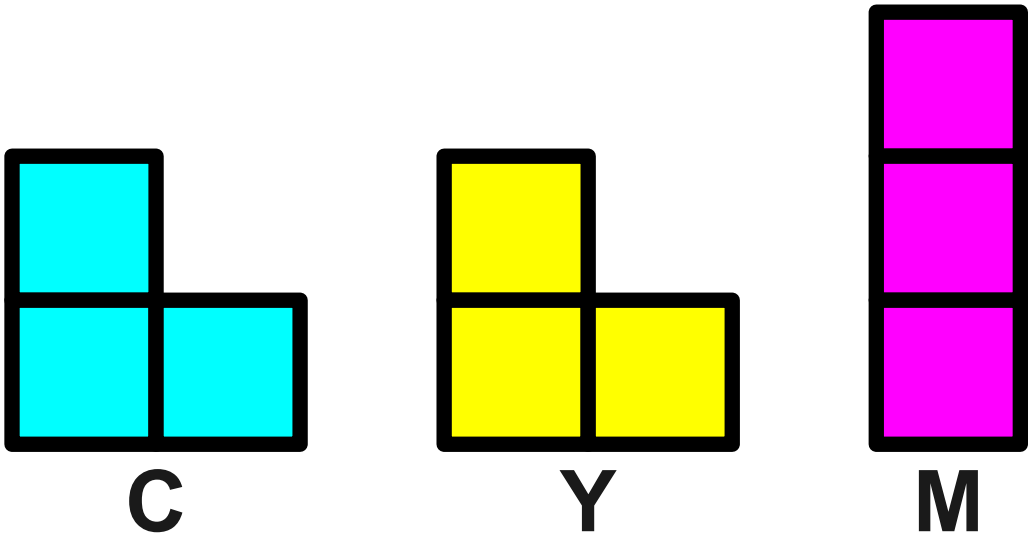
{ C, 1, 4, 5 }



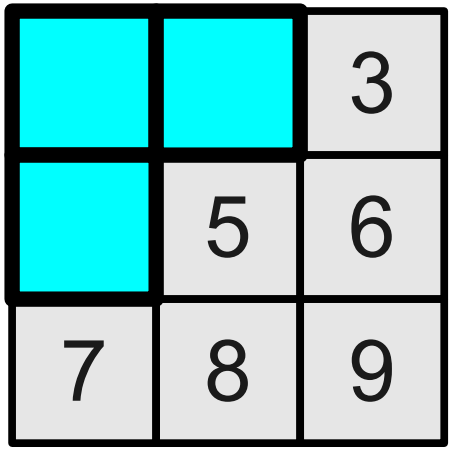
Applications of Exact Covering



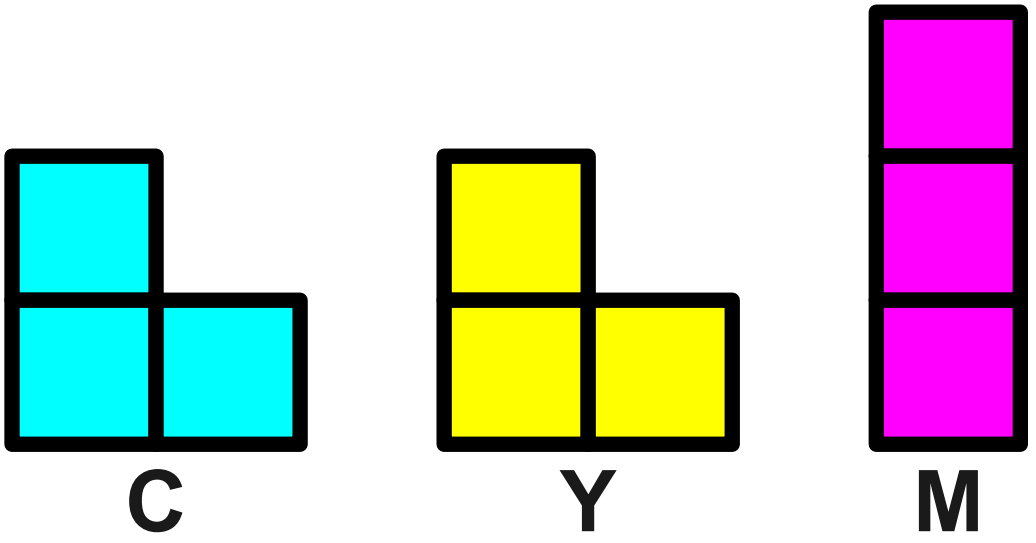
{ C, 1, 4, 5 }



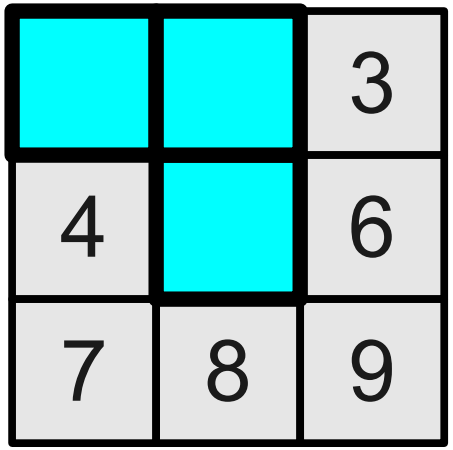
Applications of Exact Covering



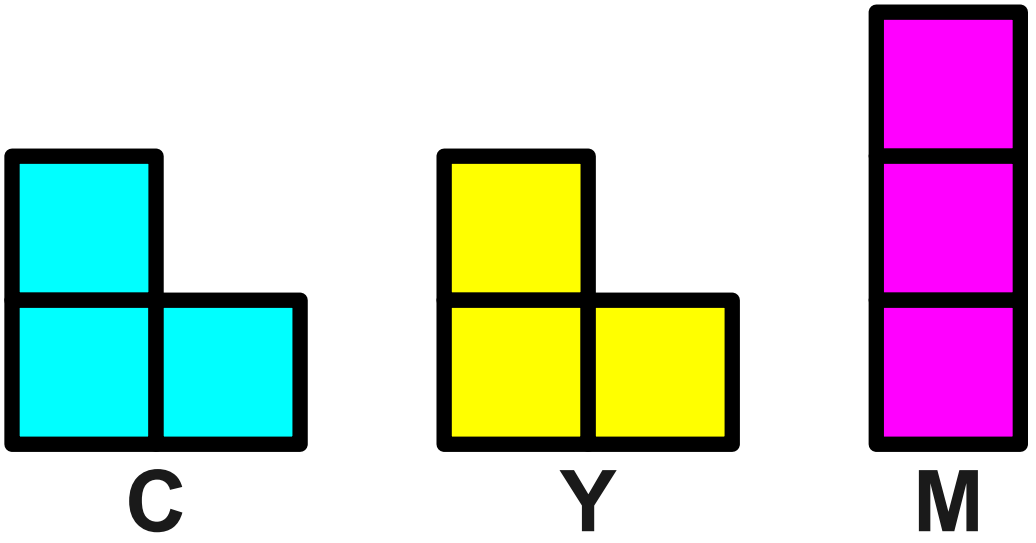
{ C, 1, 4, 5 }
{ C, 1, 2, 4 }



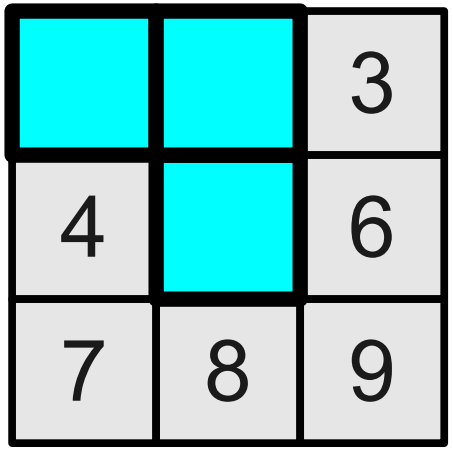
Applications of Exact Covering



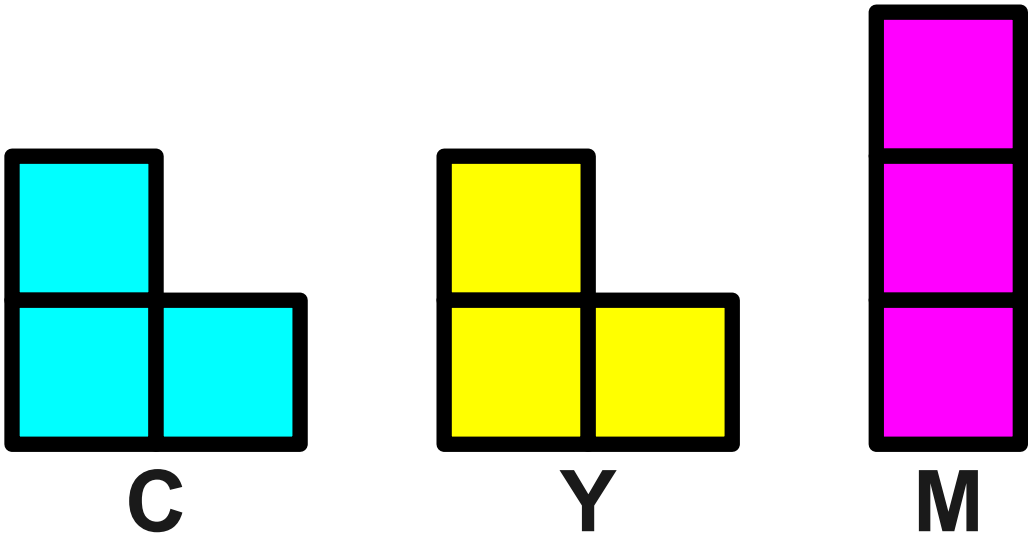
{ C, 1, 4, 5 }
{ C, 1, 2, 4 }



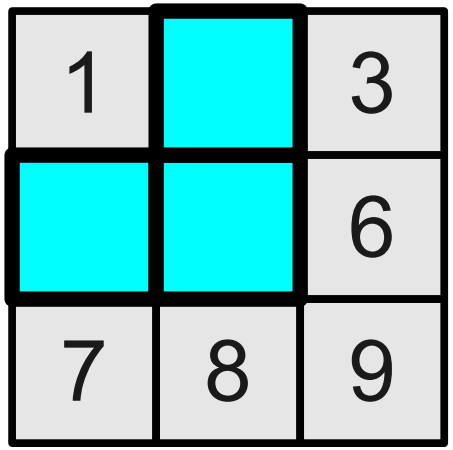
Applications of Exact Covering



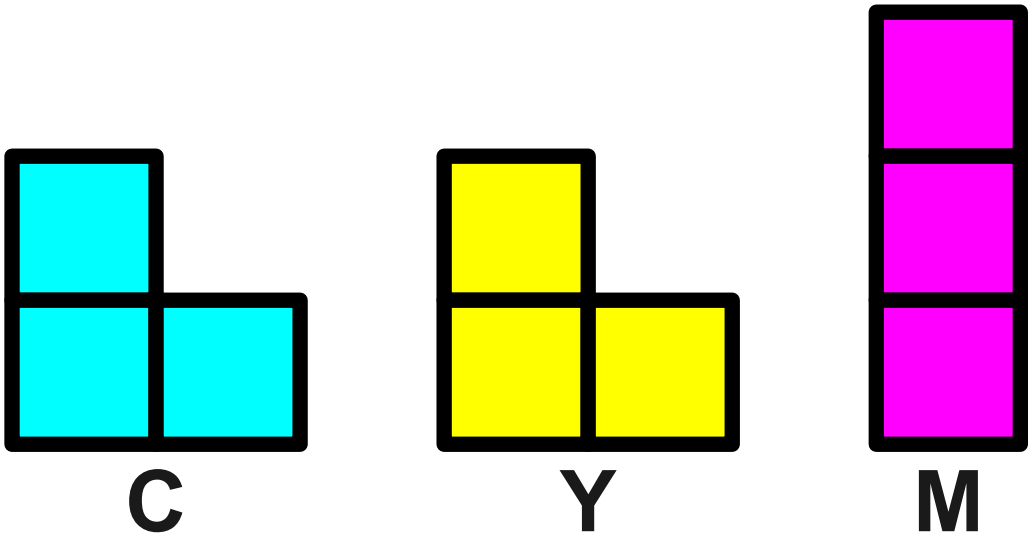
{ C, 1, 4, 5 }
{ C, 1, 2, 4 }
{ C, 1, 2, 5 }



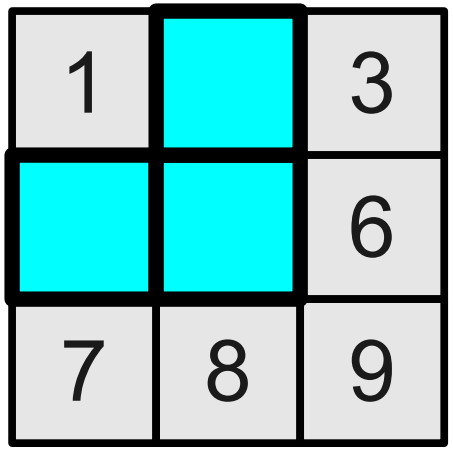
Applications of Exact Covering



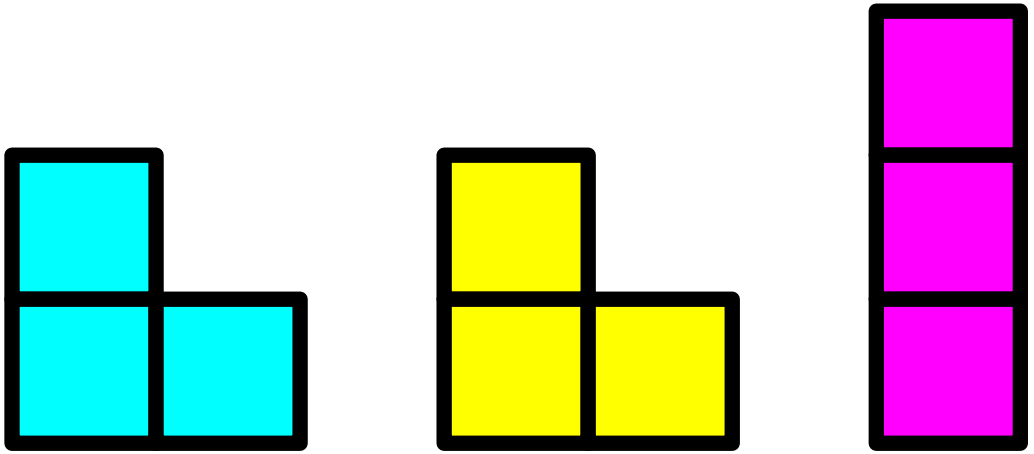
{ C, 1, 4, 5 }
{ C, 1, 2, 4 }
{ C, 1, 2, 5 }



Applications of Exact Covering

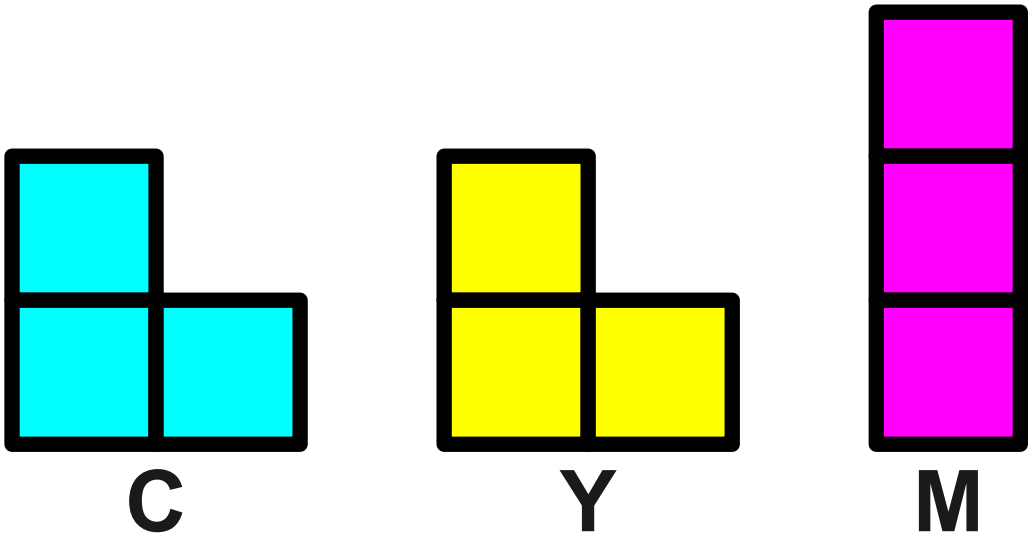


{ C, 1, 4, 5 }
{ C, 1, 2, 4 }
{ C, 1, 2, 5 }
{ C, 2, 4, 5 }



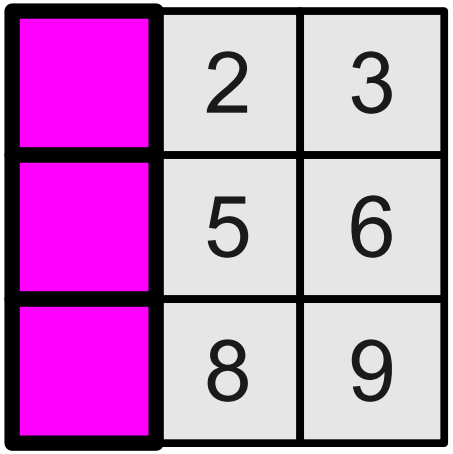
Applications of Exact Covering

1	2	3
4	5	6
7	8	9

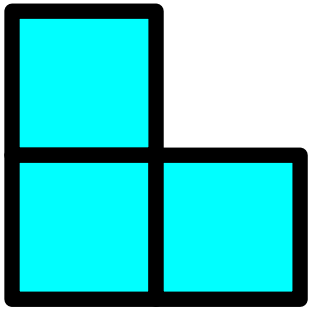


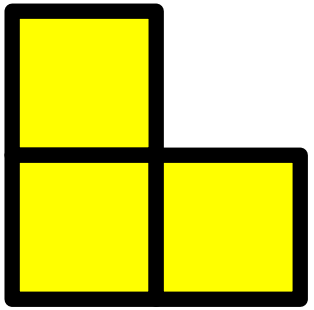
{ C, 1, 4, 5 }
{ C, 1, 2, 4 }
{ C, 1, 2, 5 }
{ C, 2, 4, 5 }

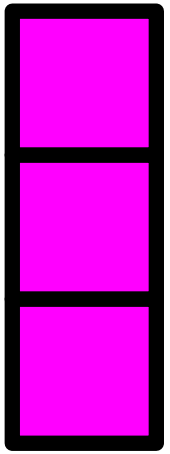
Applications of Exact Covering



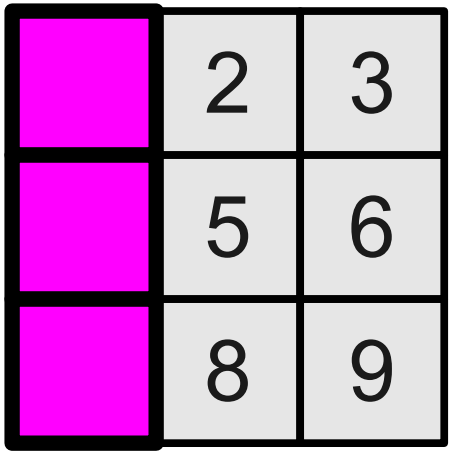
{ C, 1, 4, 5 }
{ C, 1, 2, 4 }
{ C, 1, 2, 5 }
{ C, 2, 4, 5 }

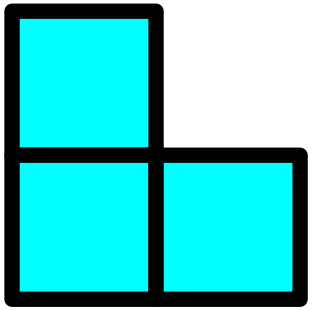




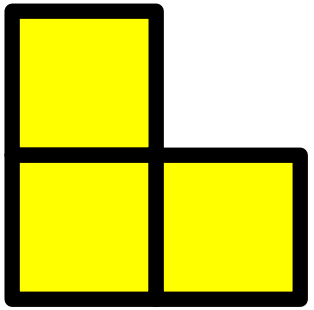


Applications of Exact Covering

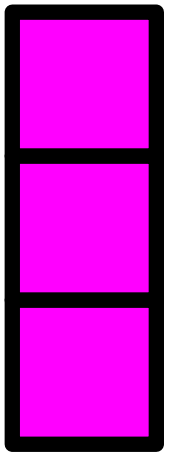




C



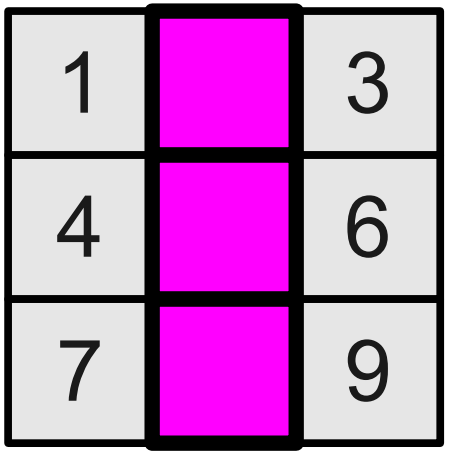
Y

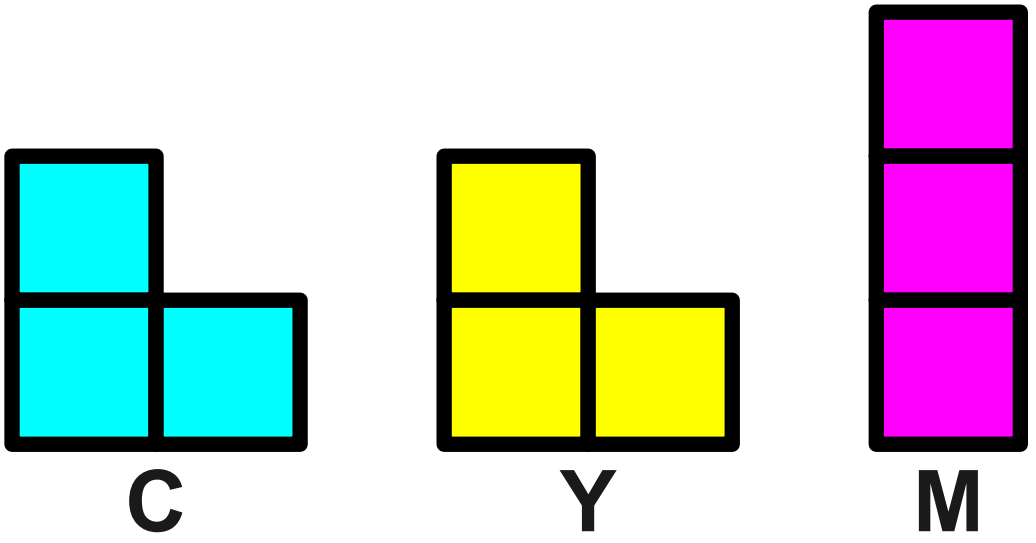


M

{ C, 1, 4, 5 }
{ C, 1, 2, 4 }
{ C, 1, 2, 5 }
{ C, 2, 4, 5 }
{ M, 1, 4, 7 }

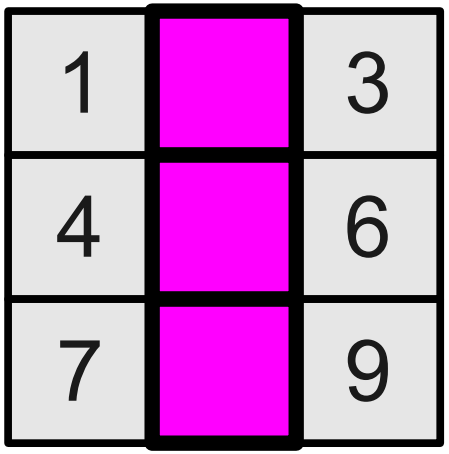
Applications of Exact Covering

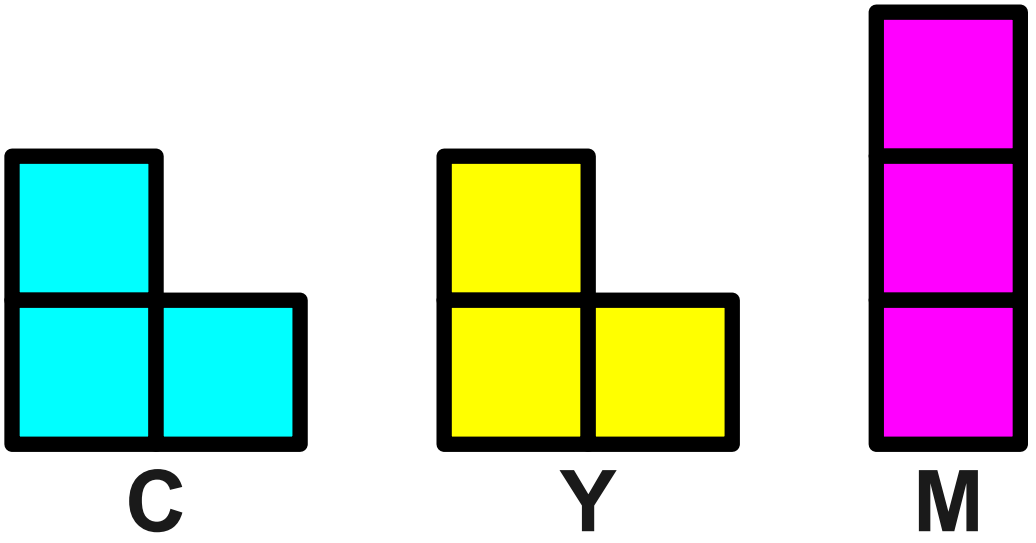




{ C, 1, 4, 5 }
{ C, 1, 2, 4 }
{ C, 1, 2, 5 }
{ C, 2, 4, 5 }
{ M, 1, 4, 7 }

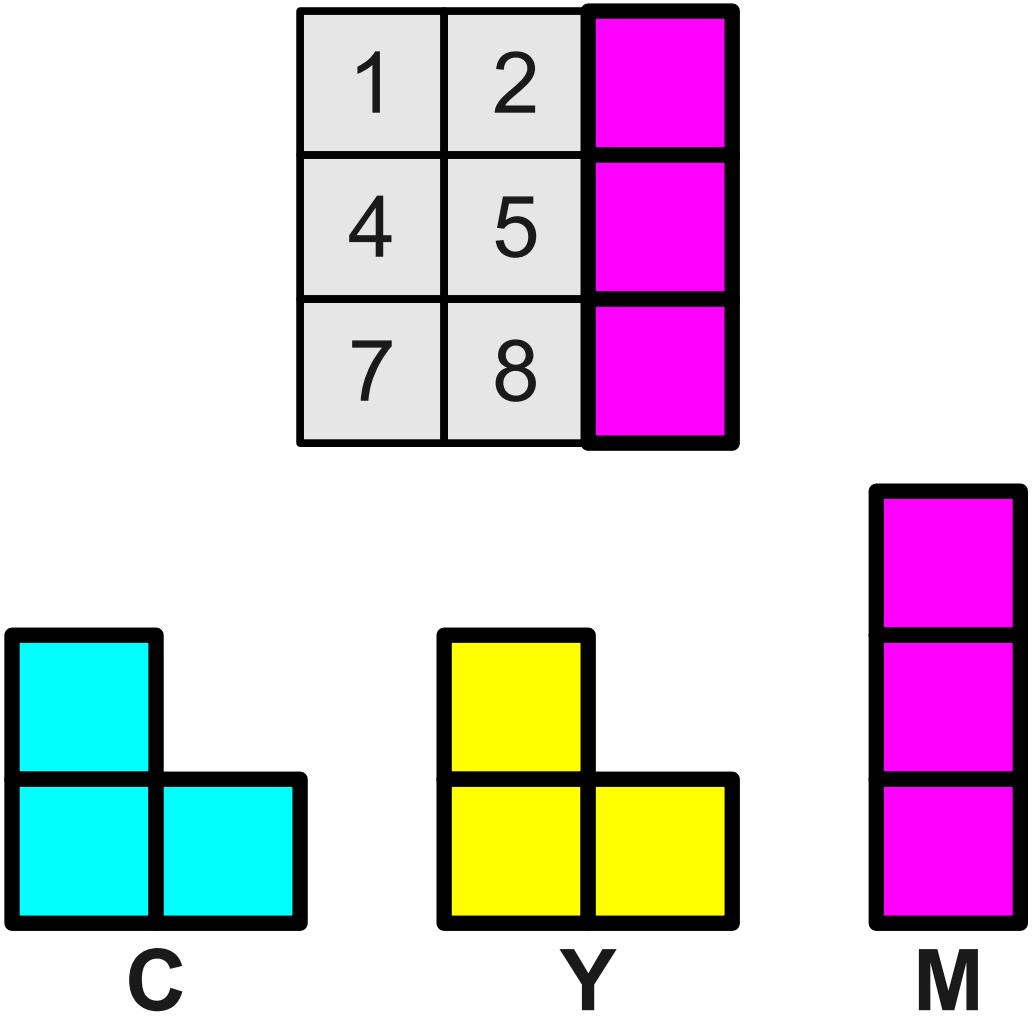
Applications of Exact Covering





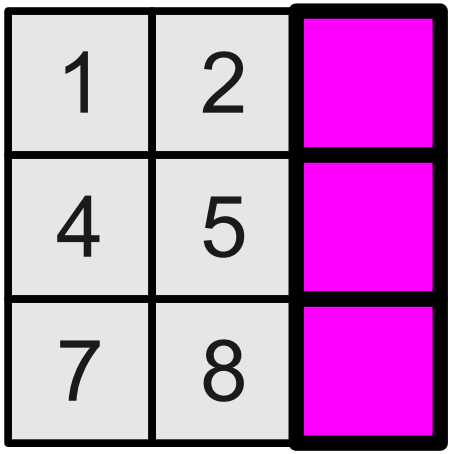
{ C, 1, 4, 5 }
{ C, 1, 2, 4 }
{ C, 1, 2, 5 }
{ C, 2, 4, 5 }
{ M, 1, 4, 7 }
{ M, 2, 5, 8 }

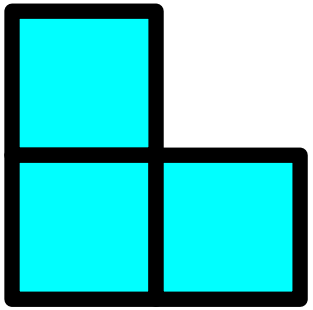
Applications of Exact Covering



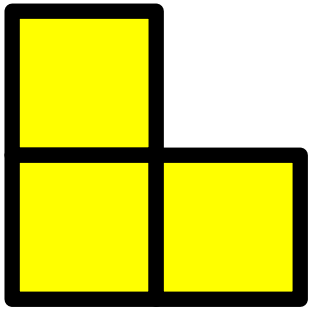
{ C, 1, 4, 5 }
{ C, 1, 2, 4 }
{ C, 1, 2, 5 }
{ C, 2, 4, 5 }
{ M, 1, 4, 7 }
{ M, 2, 5, 8 }

Applications of Exact Covering

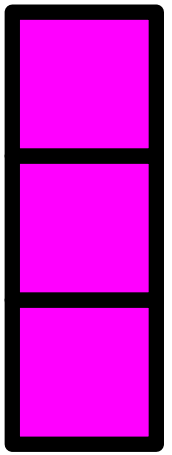




C



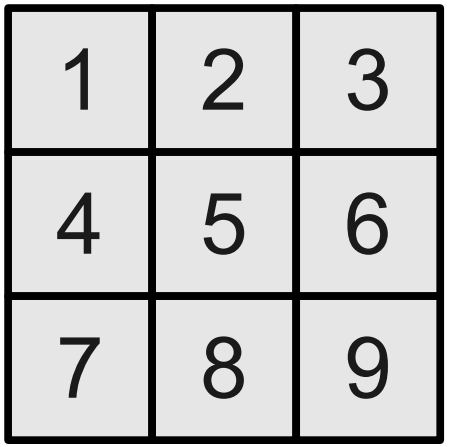
Y

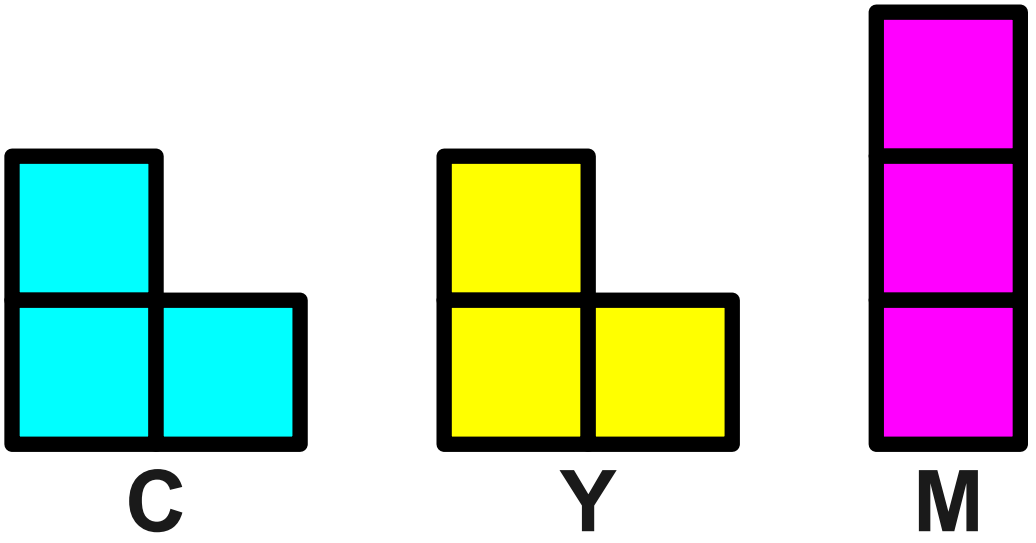


M

{ C, 1, 4, 5 }
{ C, 1, 2, 4 }
{ C, 1, 2, 5 }
{ C, 2, 4, 5 }
{ M, 1, 4, 7 }
{ M, 2, 5, 8 }
{ M, 3, 6, 9 }

Applications of Exact Covering





{ C, 1, 4, 5 }
{ C, 1, 2, 4 }
{ C, 1, 2, 5 }
{ C, 2, 4, 5 }
{ M, 1, 4, 7 }
{ M, 2, 5, 8 }
{ M, 3, 6, 9 }

Exact Covering

- Given a universe U and a set $S \subseteq \wp(U)$, the exact covering problem is

  Does S contain an exact covering of U?

- As a formal language:

  EXACT-COVER =
  $\{ \langle U, S \rangle \mid S \subseteq \wp(U)$ and S contains an exact covering of $U \}$

EXACT-COVER ∈ **NPC**

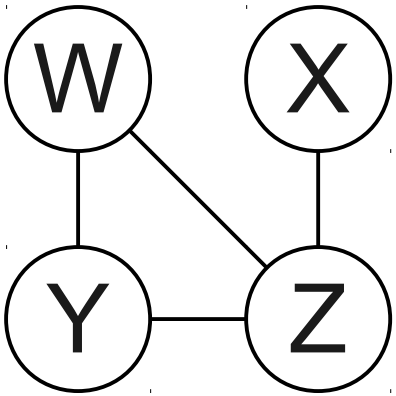
- We will prove that *EXACT-COVER* is **NP**-complete.
- To do this, we will show that
  - *EXACT-COVER* ∈ **NP**, and
  - $3COLOR \leq_P EXACT\text{-}COVER$
- Note that we're using the fact that *3COLOR* is **NP**-complete to establish that *EXACT-COVER* is **NP**-hard.

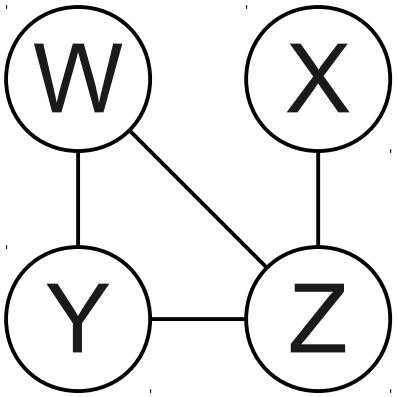
EXACT-COVER ∈ **NP**

- Here is a polynomial-time verifier for *EXACT-COVER*:
- V = "On input $\langle U, S, I \rangle$:
  - Verify that every set in S is a subset of U.
  - Verify that every set in I is an element of S.
  - Verify that every element of U belongs to an element of I.
  - Verify that every element of U belongs to at most one element of I."

$3COLOR \leq_P EXACT\text{-}COVER$

- We now reduce 3-colorability to the exact cover problem.
- A graph is 3-colorable iff
  - Every node is assigned one of three colors, and
  - No two nodes connected by an edge are assigned the same color.
- We will construct our universe U and sets S such that an exact covering
  - Assigns every node in G one of three colors, and
  - Never assigns two adjacent nodes the same color.


W
X
Y
Z

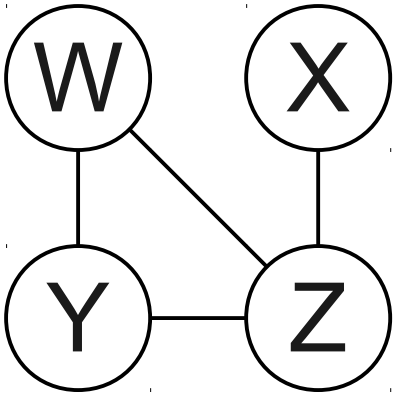


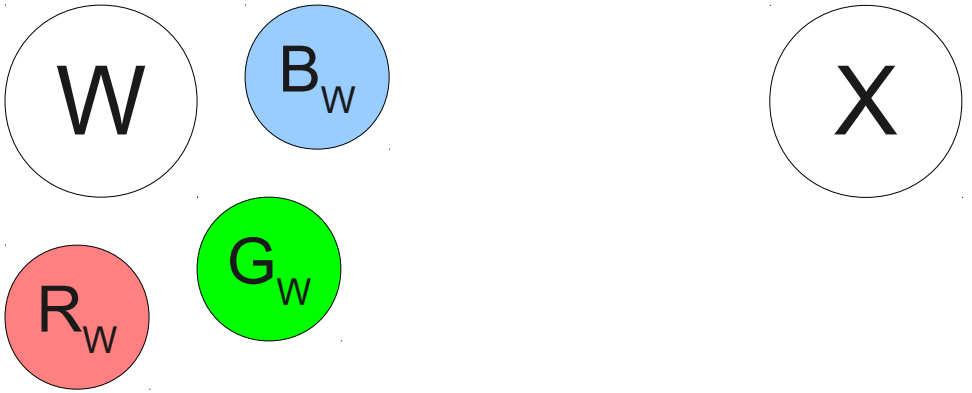
W

X

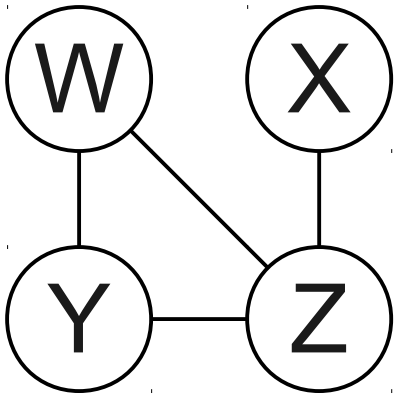
Y

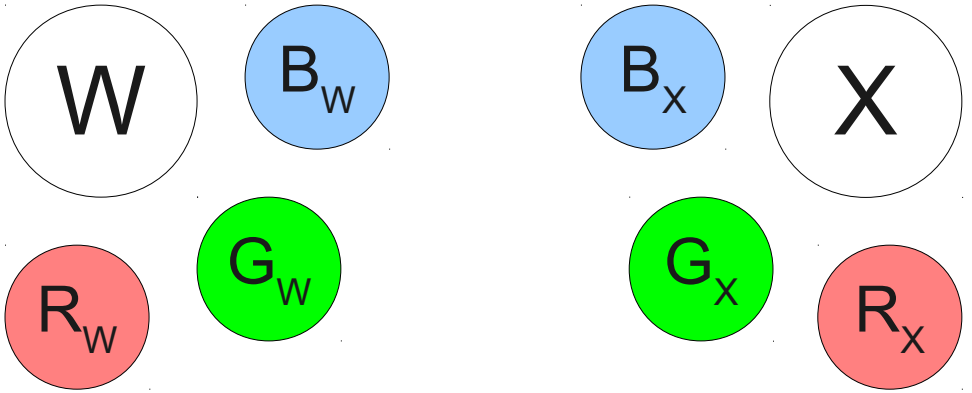
Z

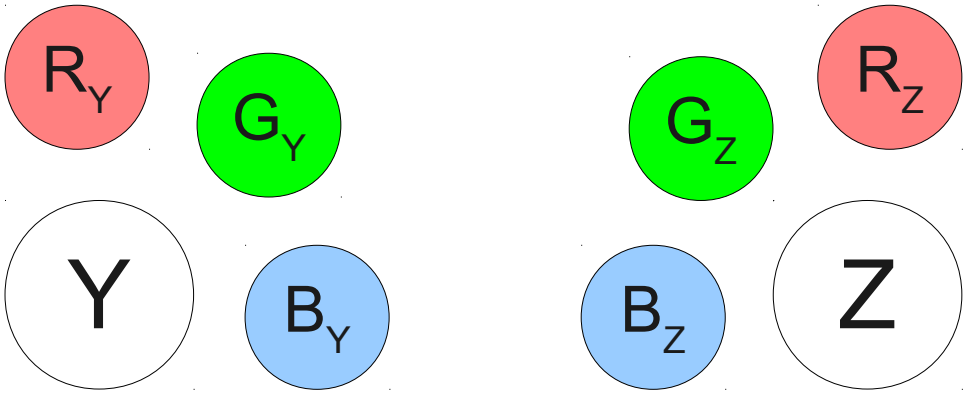

W
X
Y
Z

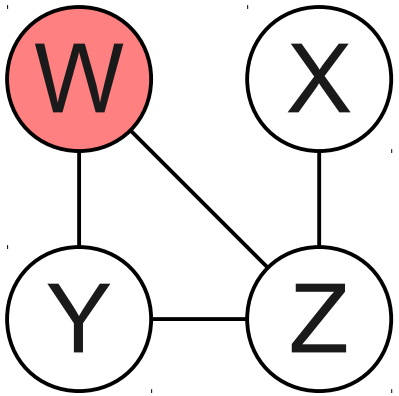

W
B_W
X
G_W
R_W

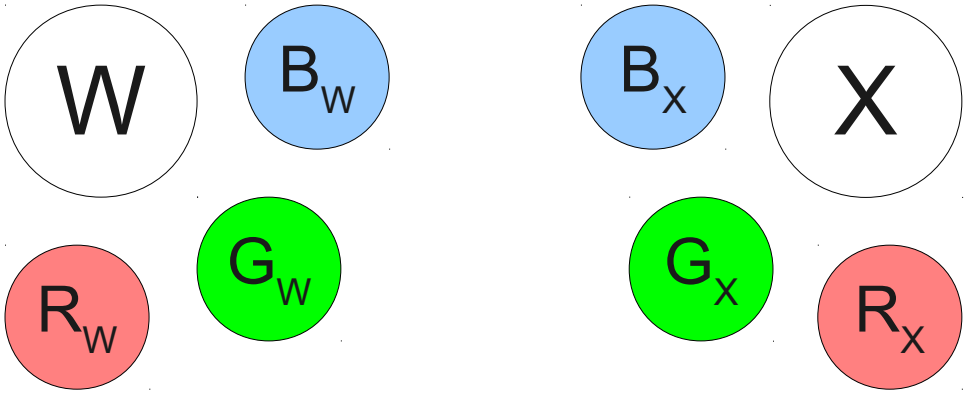

Y
Z

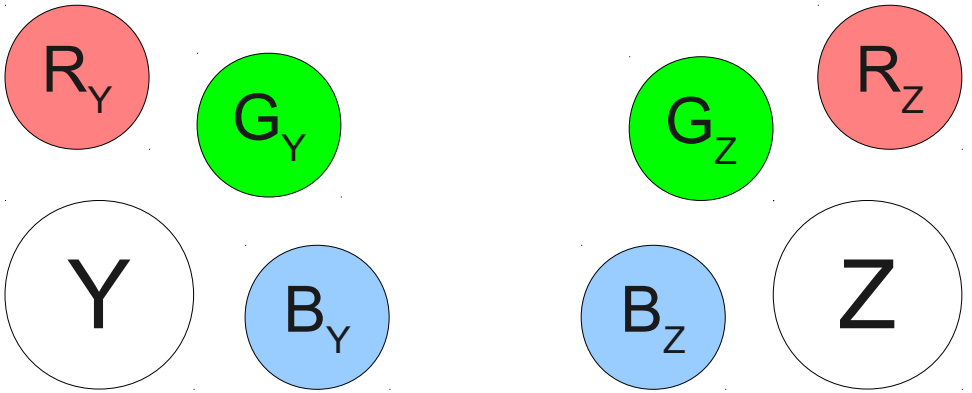

W
X
Y
Z

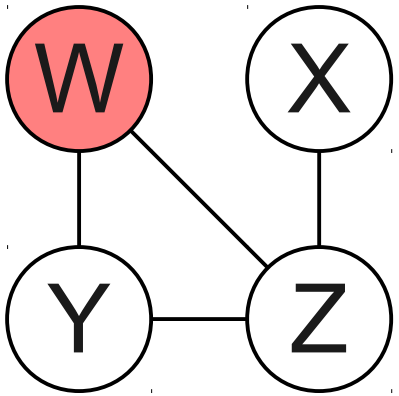

W
B_W
G_W
R_W
B_X
X
G_X
R_X

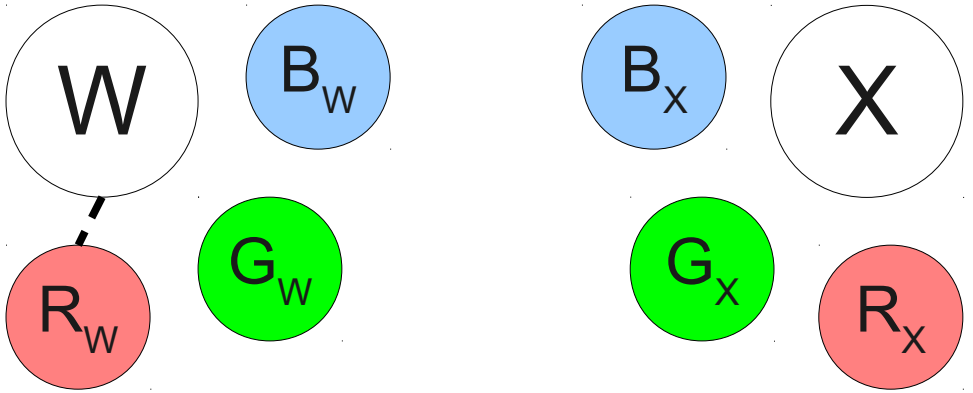

R_Y
G_Y
Y
B_Y
G_Z
R_Z
B_Z
Z

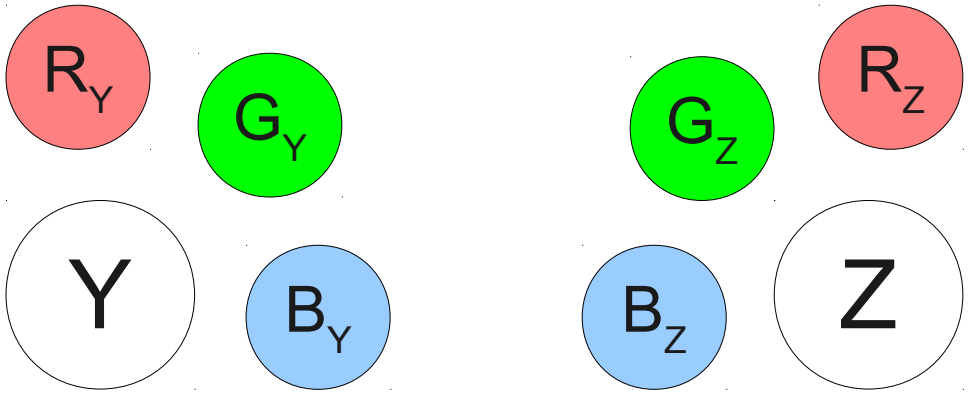

W
X
Y
Z

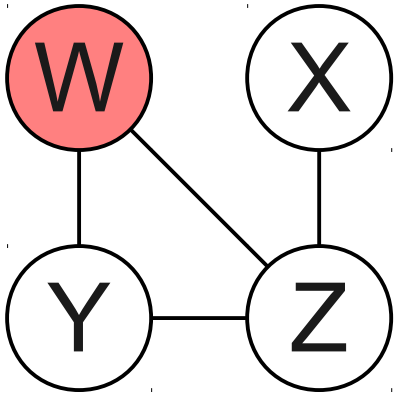

W
B_W
G_W
R_W
B_X
X
G_X
R_X

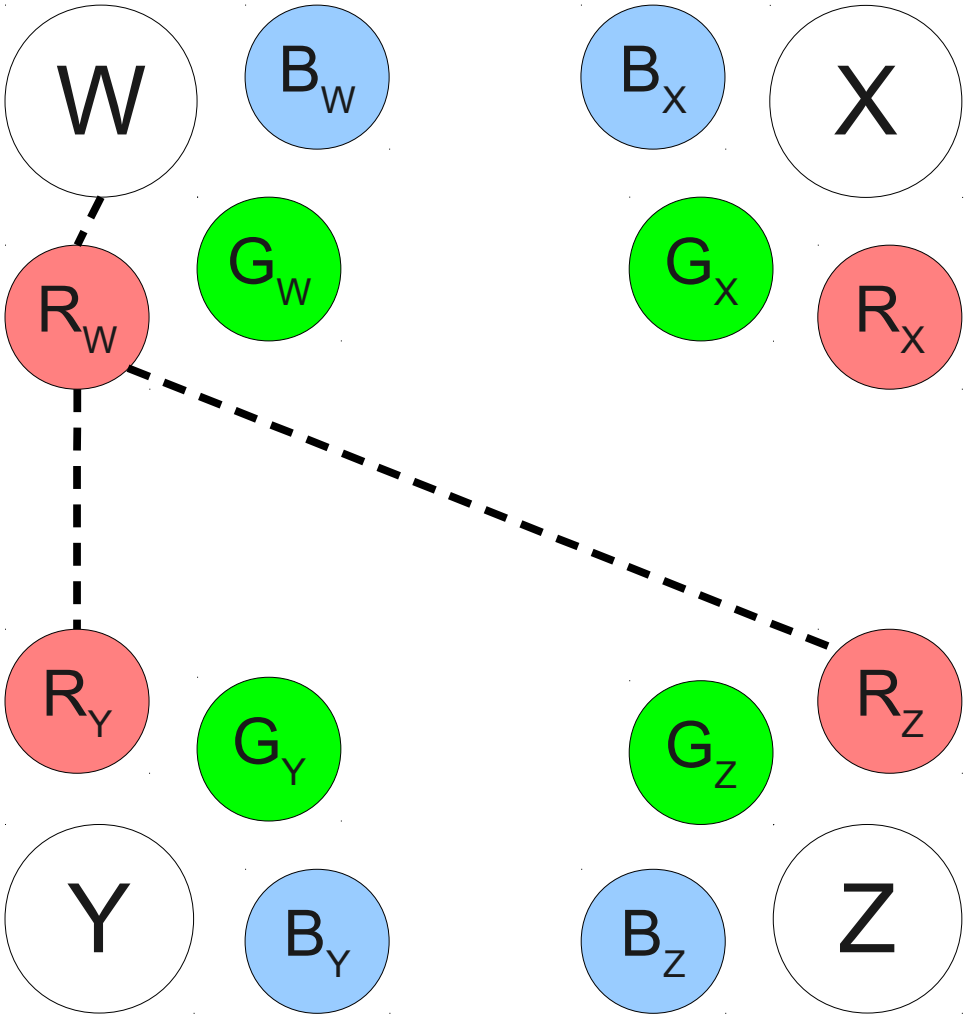

R_Y
G_Y
Y
B_Y
G_Z
R_Z
B_Z
Z

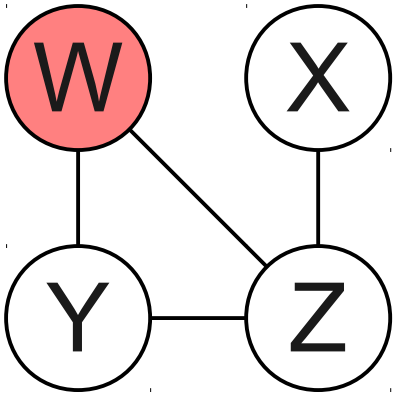

W
X
Y
Z


W
B_W
G_W
R_W
B_X
X
G_X
R_X

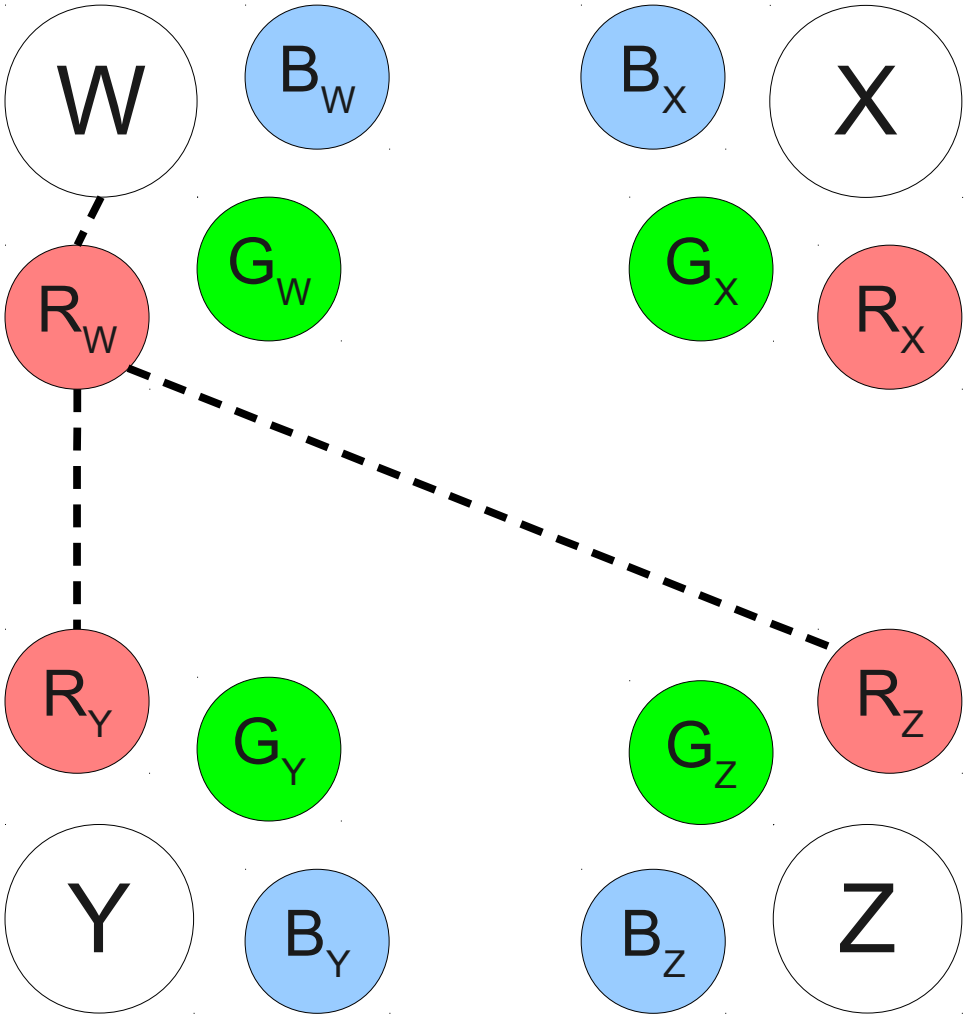

R_Y
G_Y
Y
B_Y
G_Z
R_Z
B_Z
Z

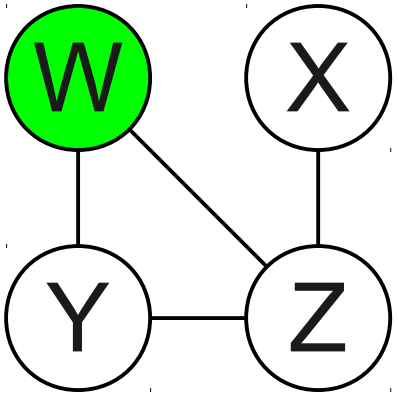

W
X
Y
Z


W
B_W
B_X
X
G_W
G_X
R_W
R_X
R_Y
R_Z
G_Y
G_Z
Y
B_Y
B_Z
Z

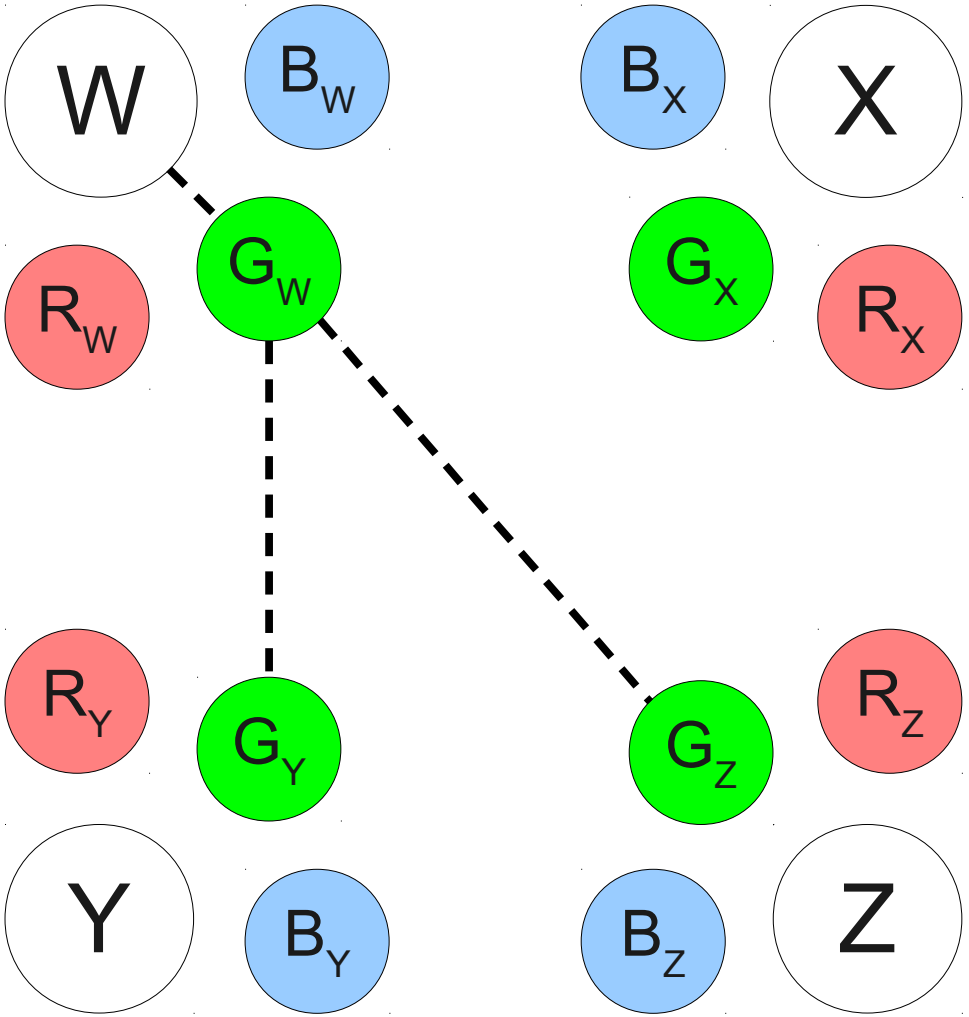


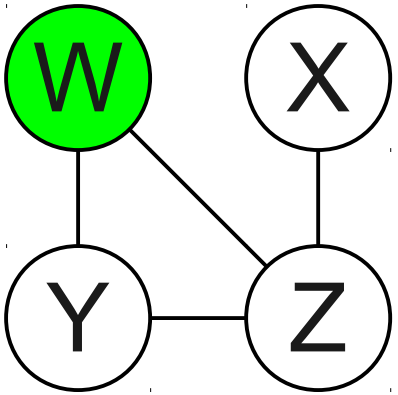
$\{ W, R_W, R_Y, R_Z \}$





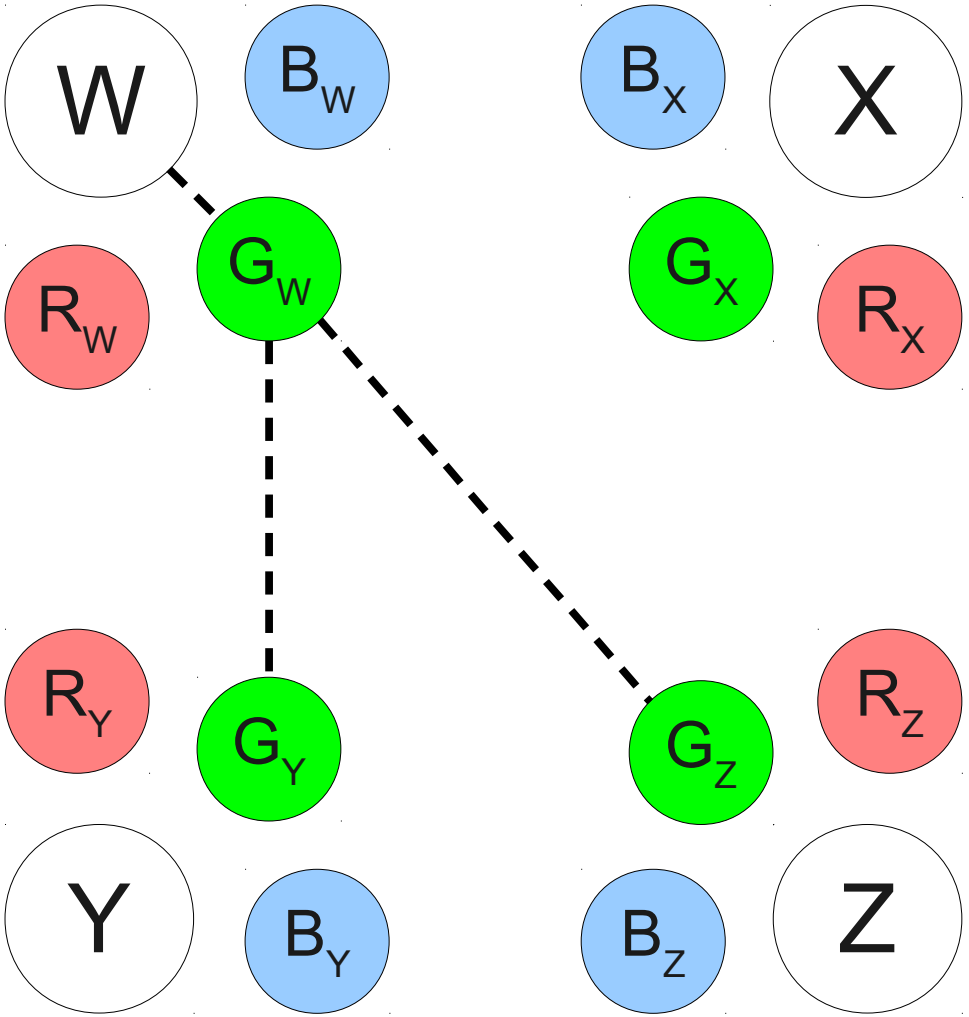
$\{ W, R_W, R_Y, R_Z \}$

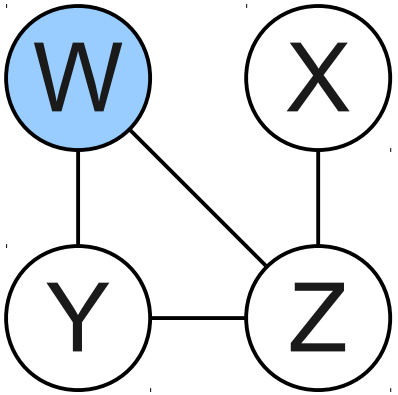




$\{ W, R_W, R_Y, R_Z \}$

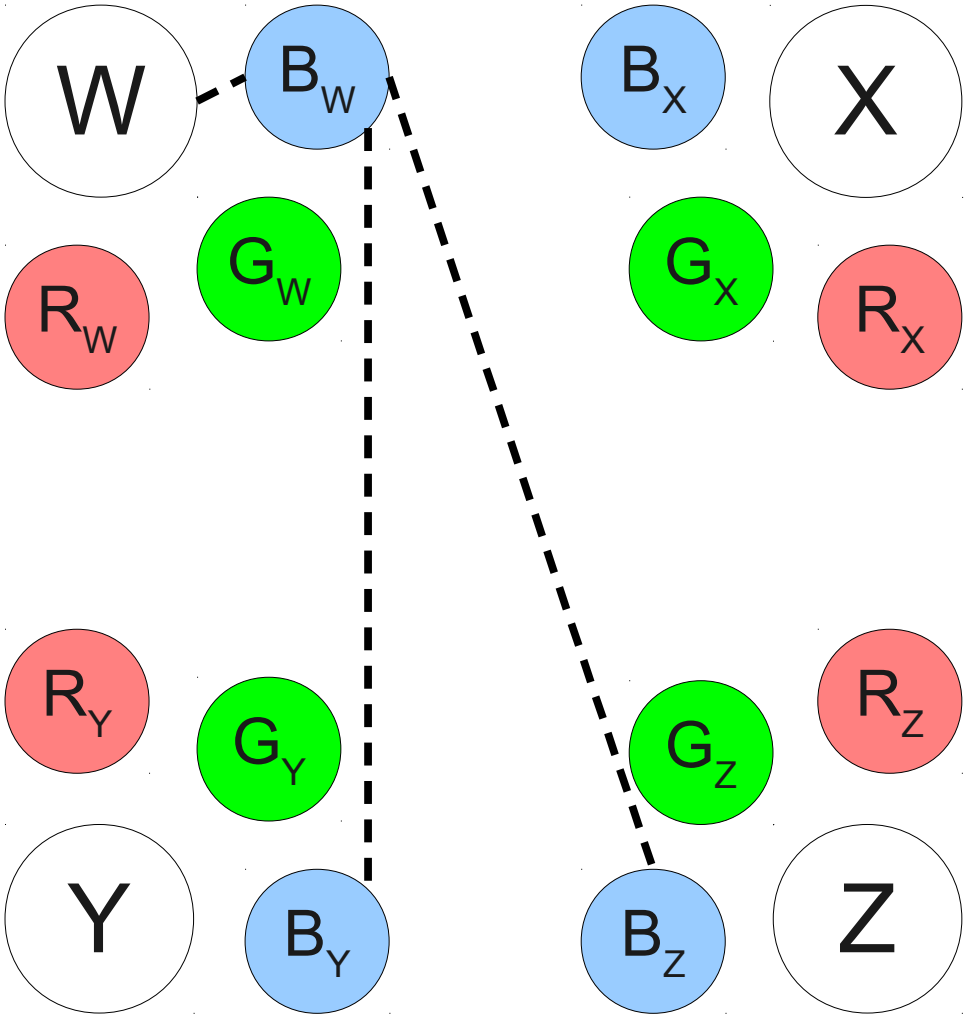
$\{ W, G_W, G_Y, G_Z \}$

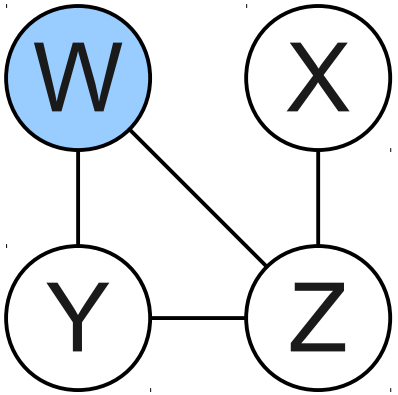




$\{ W, R_W, R_Y, R_Z \}$

$\{ W, G_W, G_Y, G_Z \}$

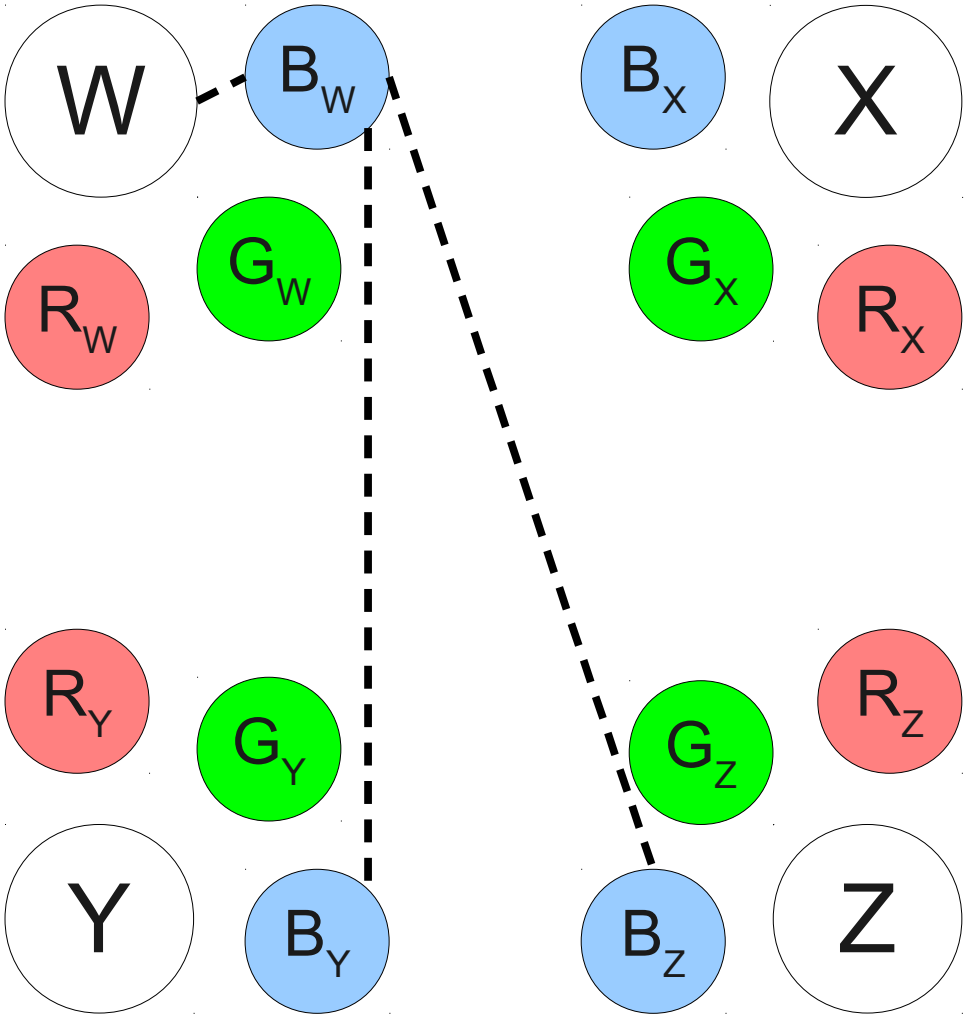


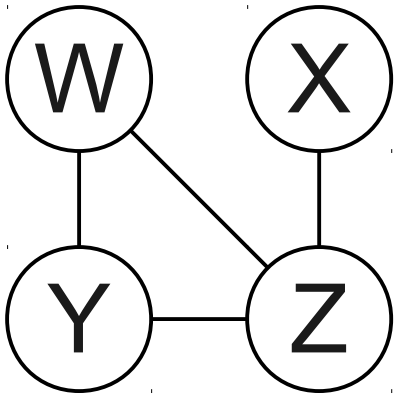


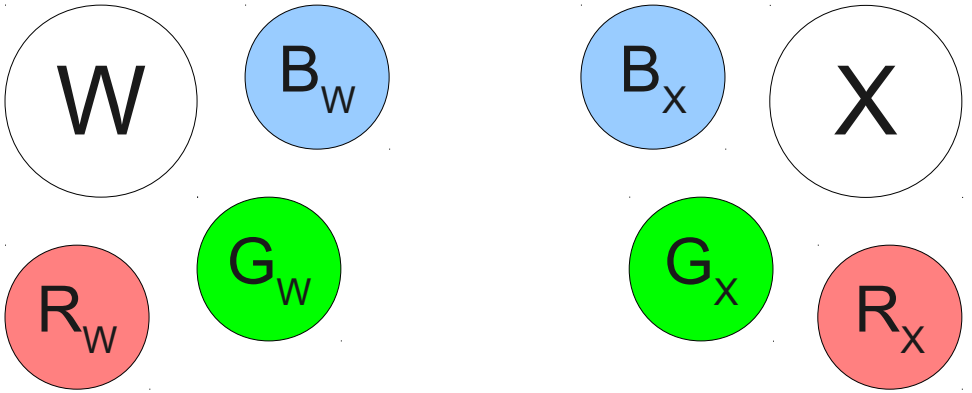
$\{ W, R_W, R_Y, R_Z \}$

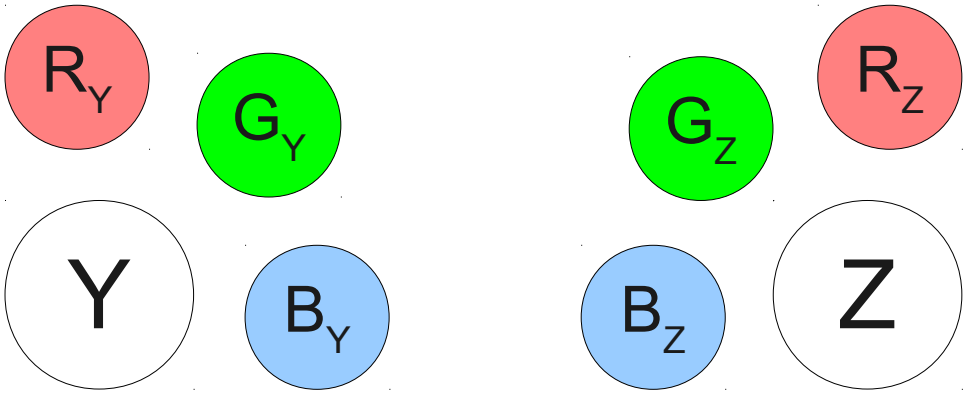
$\{ W, G_W, G_Y, G_Z \}$

$\{ W, B_W, B_Y, B_Z \}$

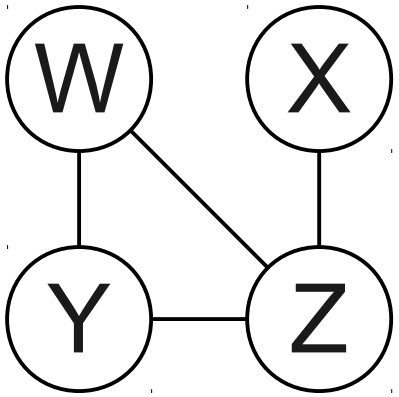


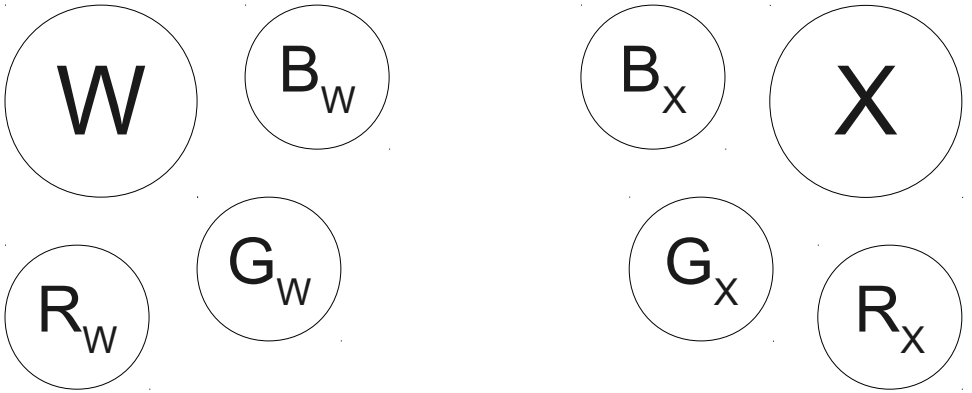


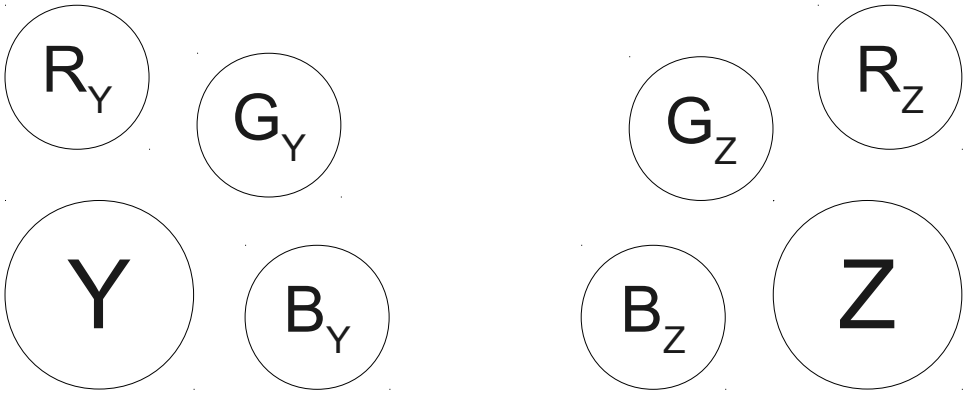




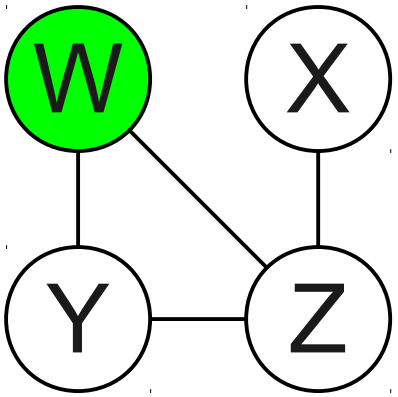
$\{ W, R_W, R_Y, R_Z \}$
$\{ W, G_W, G_Y, G_Z \}$
$\{ W, B_W, B_Y, B_Z \}$
$\{ X, R_X, R_Z \}$
$\{ X, G_X, G_Z \}$
$\{ X, B_X, B_Z \}$
$\{ Y, R_Y, R_W, R_Z \}$
$\{ Y, G_Y, G_W, G_Z \}$
$\{ Y, B_Y, B_W, B_Z \}$
$\{ Z, R_Z, R_W, R_Y \}$
$\{ Z, G_Z, G_W, G_Y \}$
$\{ Z, B_Z, B_W, B_Y \}$

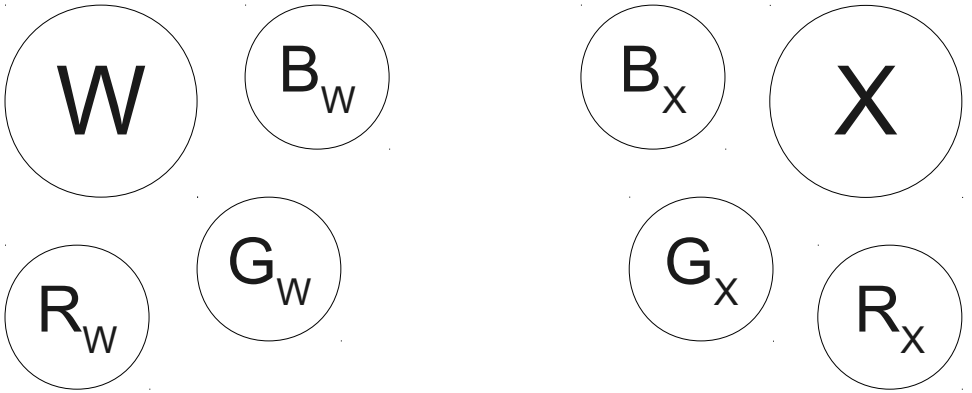


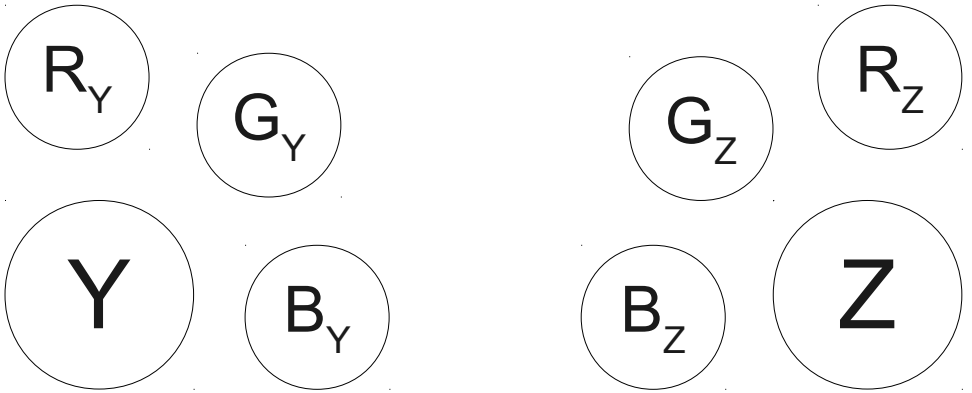




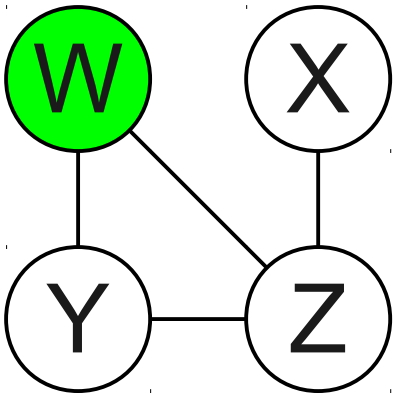
$\{ W, R_W, R_Y, R_Z \}$
$\{ W, G_W, G_Y, G_Z \}$
$\{ W, B_W, B_Y, B_Z \}$
$\{ X, R_X, R_Z \}$
$\{ X, G_X, G_Z \}$
$\{ X, B_X, B_Z \}$
$\{ Y, R_Y, R_W, R_Z \}$
$\{ Y, G_Y, G_W, G_Z \}$
$\{ Y, B_Y, B_W, B_Z \}$
$\{ Z, R_Z, R_W, R_Y \}$
$\{ Z, G_Z, G_W, G_Y \}$
$\{ Z, B_Z, B_W, B_Y \}$

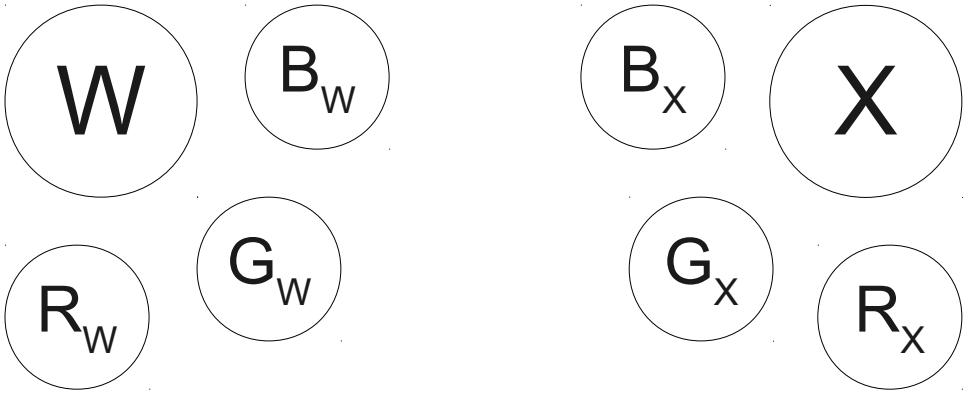


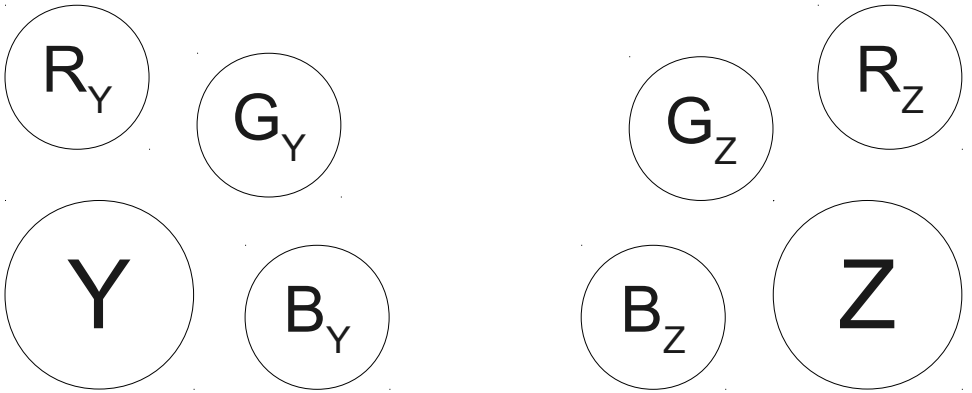




{ W, R_W, R_Y, R_Z }
{ W, G_W, G_Y, G_Z }
{ W, B_W, B_Y, B_Z }
{ X, R_X, R_Z }
{ X, G_X, G_Z }
{ X, B_X, B_Z }
{ Y, R_Y, R_W, R_Z }
{ Y, G_Y, G_W, G_Z }
{ Y, B_Y, B_W, B_Z }
{ Z, R_Z, R_W, R_Y }
{ Z, G_Z, G_W, G_Y }
{ Z, B_Z, B_W, B_Y }







{ W, R_W, R_Y, R_Z }

{ W, G_W, G_Y, G_Z }

{ W, B_W, B_Y, B_Z }

{ X, R_X, R_Z }

{ X, G_X, G_Z }

{ X, B_X, B_Z }

{ Y, R_Y, R_W, R_Z }

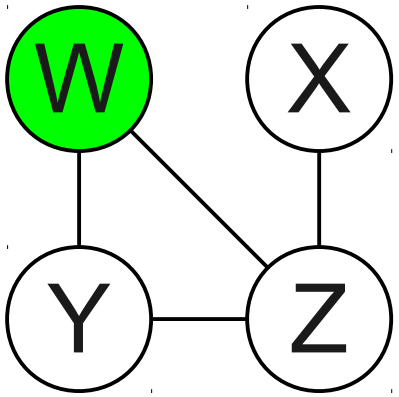
{ Y, G_Y, G_W, G_Z }

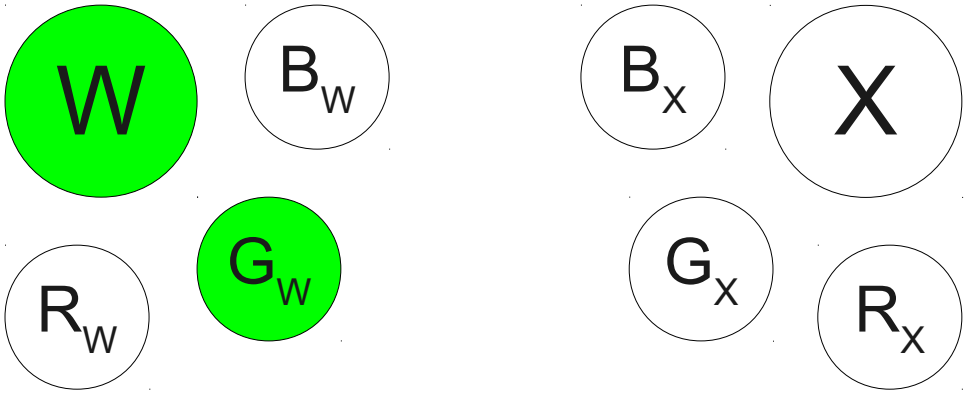
{ Y, B_Y, B_W, B_Z }

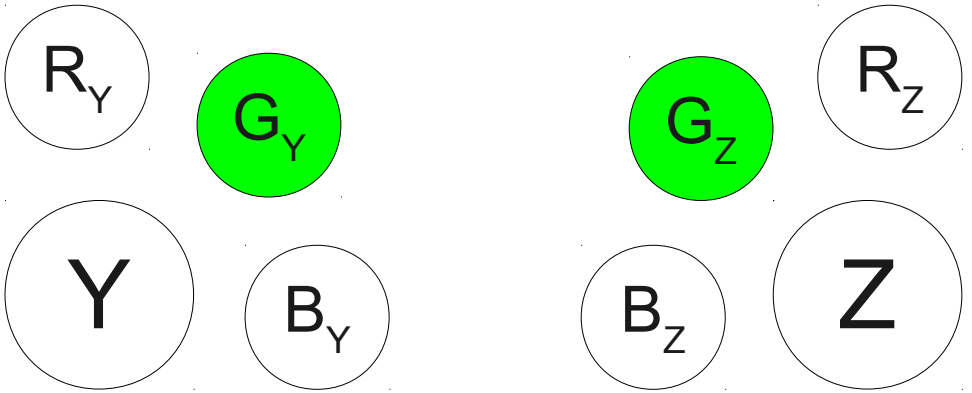
{ Z, R_Z, R_W, R_Y }

{ Z, G_Z, G_W, G_Y }

{ Z, B_Z, B_W, B_Y }







{ W, R_W, R_Y, R_Z }

{ W, G_W, G_Y, G_Z }

{ W, B_W, B_Y, B_Z }

{ X, R_X, R_Z }

{ X, G_X, G_Z }

{ X, B_X, B_Z }

{ Y, R_Y, R_W, R_Z }

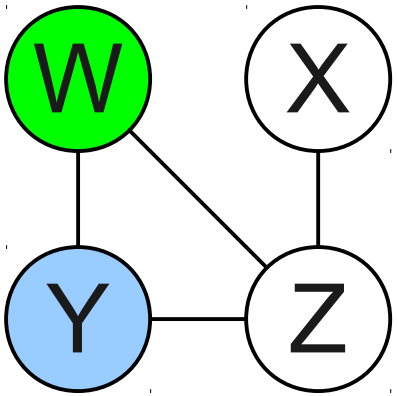
{ Y, G_Y, G_W, G_Z }

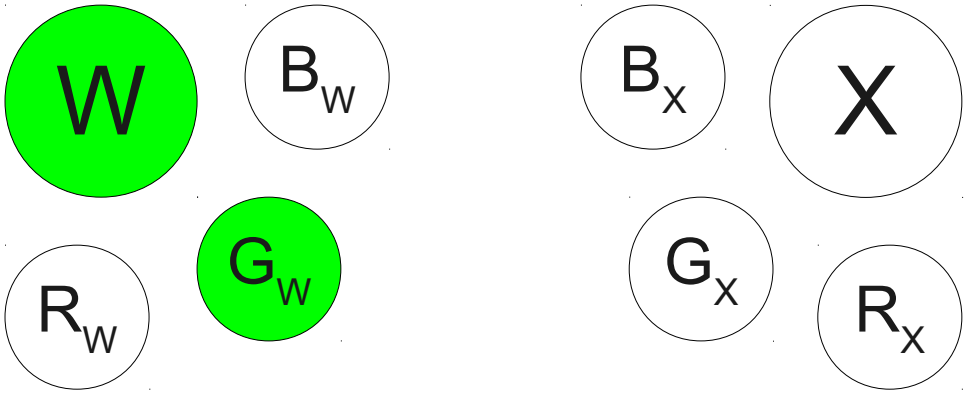
{ Y, B_Y, B_W, B_Z }

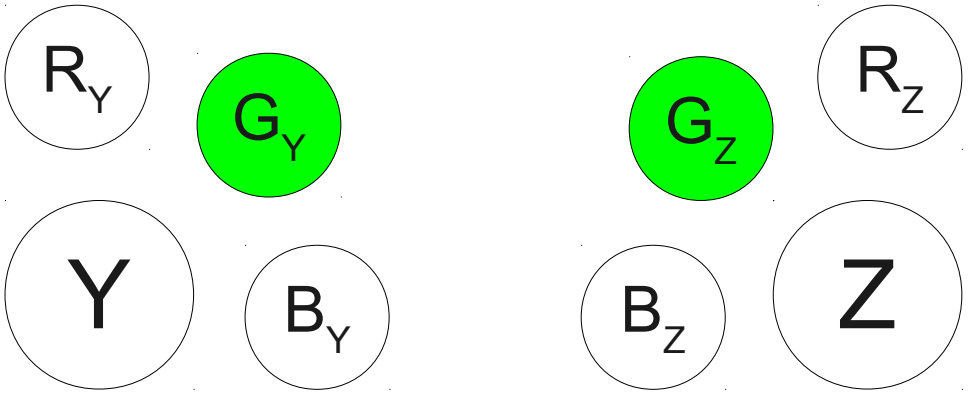
{ Z, R_Z, R_W, R_Y }

{ Z, G_Z, G_W, G_Y }

{ Z, B_Z, B_W, B_Y }







$\{ W, R_W, R_Y, R_Z \}$

$\{ W, G_W, G_Y, G_Z \}$

$\{ W, B_W, B_Y, B_Z \}$

$\{ X, R_X, R_Z \}$

$\{ X, G_X, G_Z \}$

$\{ X, B_X, B_Z \}$

$\{ Y, R_Y, R_W, R_Z \}$

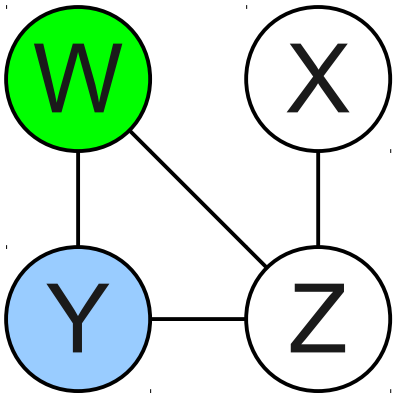
$\{ Y, G_Y, G_W, G_Z \}$

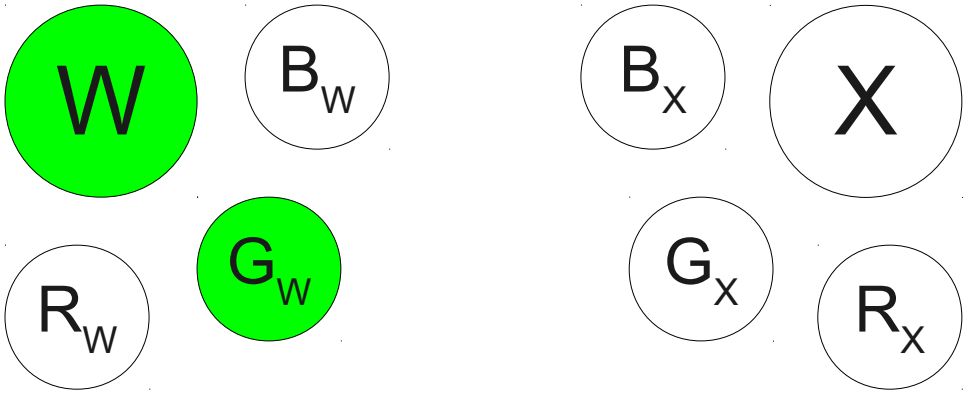
$\{ Y, B_Y, B_W, B_Z \}$

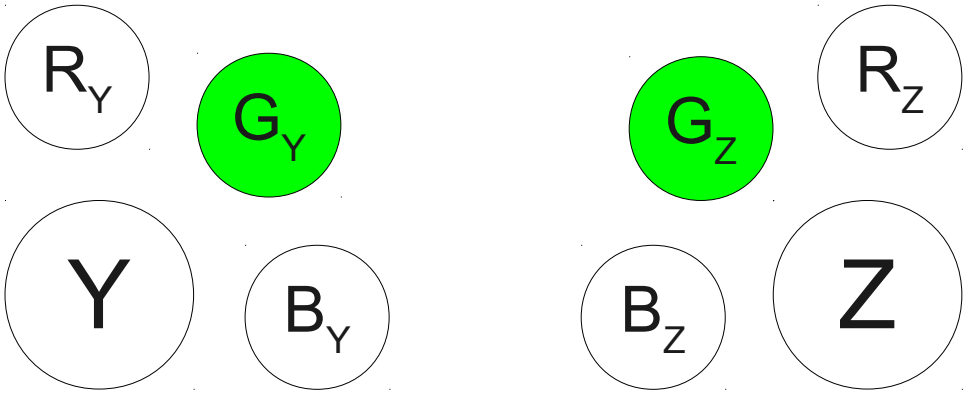
$\{ Z, R_Z, R_W, R_Y \}$

$\{ Z, G_Z, G_W, G_Y \}$

$\{ Z, B_Z, B_W, B_Y \}$







{ W, R_W, R_Y, R_Z }

{ W, G_W, G_Y, G_Z }

{ W, B_W, B_Y, B_Z }

{ X, R_X, R_Z }

{ X, G_X, G_Z }

{ X, B_X, B_Z }

{ Y, R_Y, R_W, R_Z }

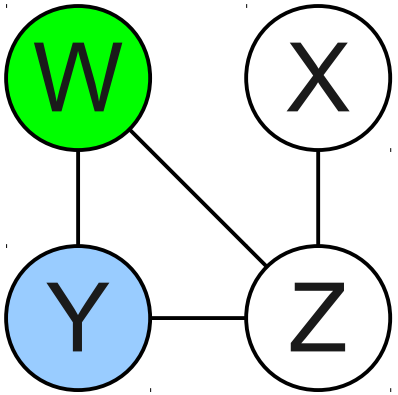
{ Y, G_Y, G_W, G_Z }

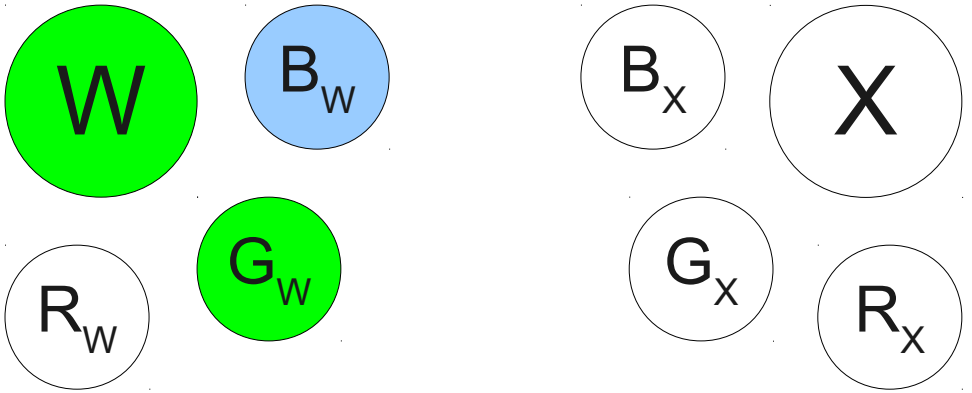
{ Y, B_Y, B_W, B_Z }

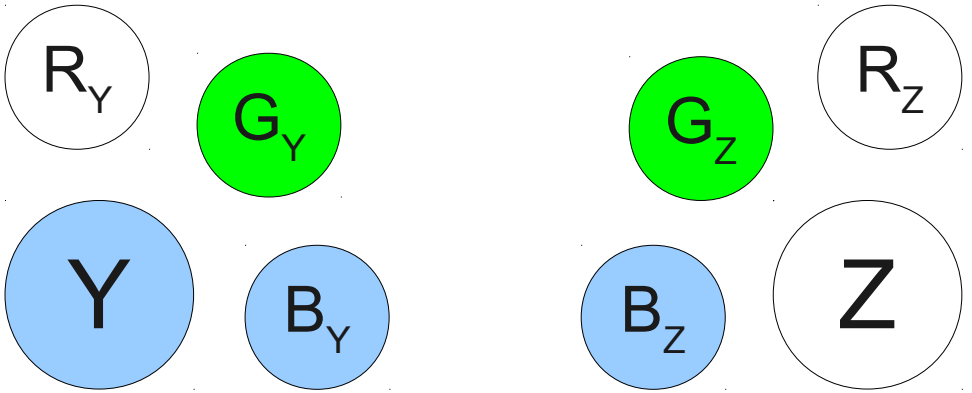
{ Z, R_Z, R_W, R_Y }

{ Z, G_Z, G_W, G_Y }

{ Z, B_Z, B_W, B_Y }







{ W, R_W, R_Y, R_Z }

{ W, G_W, G_Y, G_Z }

{ W, B_W, B_Y, B_Z }

{ X, R_X, R_Z }

{ X, G_X, G_Z }

{ X, B_X, B_Z }

{ Y, R_Y, R_W, R_Z }

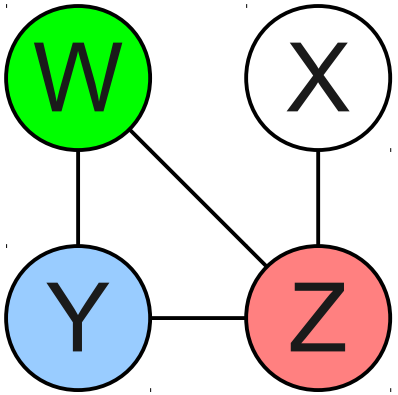
{ Y, G_Y, G_W, G_Z }

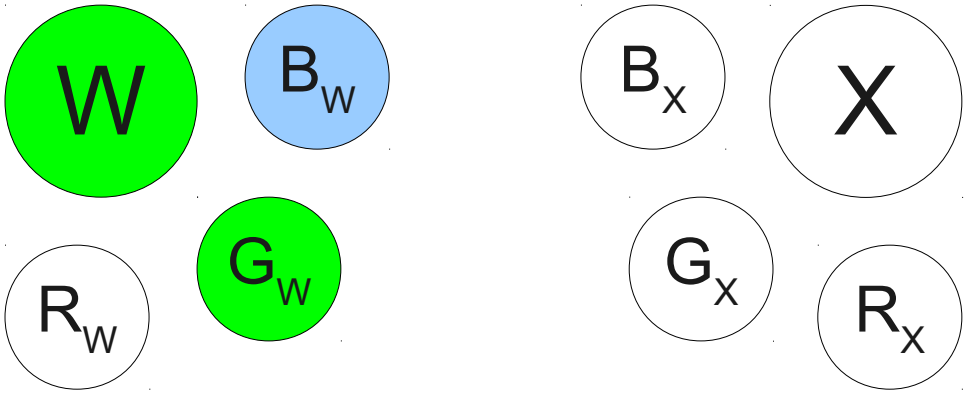
{ Y, B_Y, B_W, B_Z }

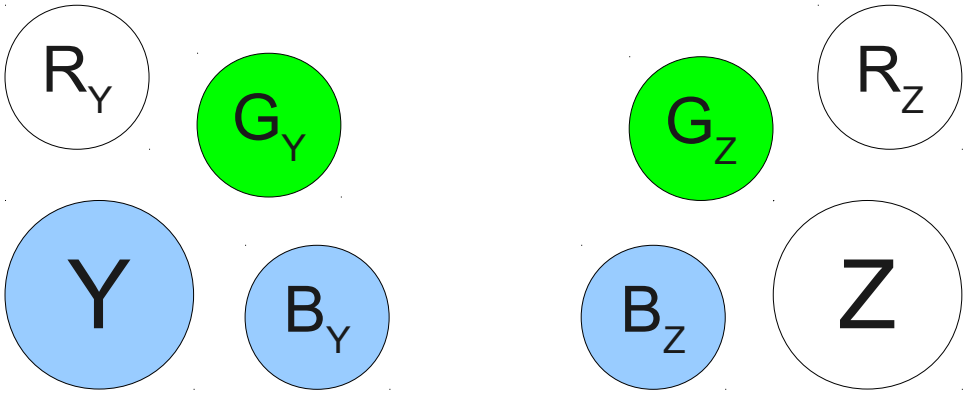
{ Z, R_Z, R_W, R_Y }

{ Z, G_Z, G_W, G_Y }

{ Z, B_Z, B_W, B_Y }







{ W, R_W, R_Y, R_Z }

{ W, G_W, G_Y, G_Z }

{ W, B_W, B_Y, B_Z }

{ X, R_X, R_Z }

{ X, G_X, G_Z }

{ X, B_X, B_Z }

{ Y, R_Y, R_W, R_Z }

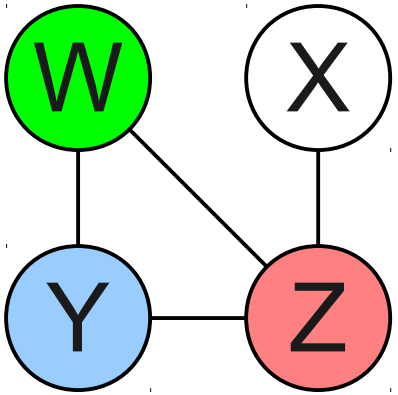
{ Y, G_Y, G_W, G_Z }

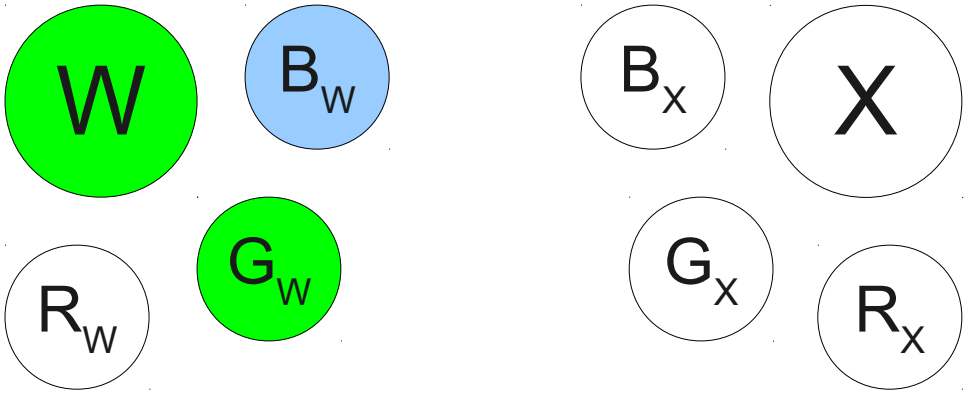
{ Y, B_Y, B_W, B_Z }

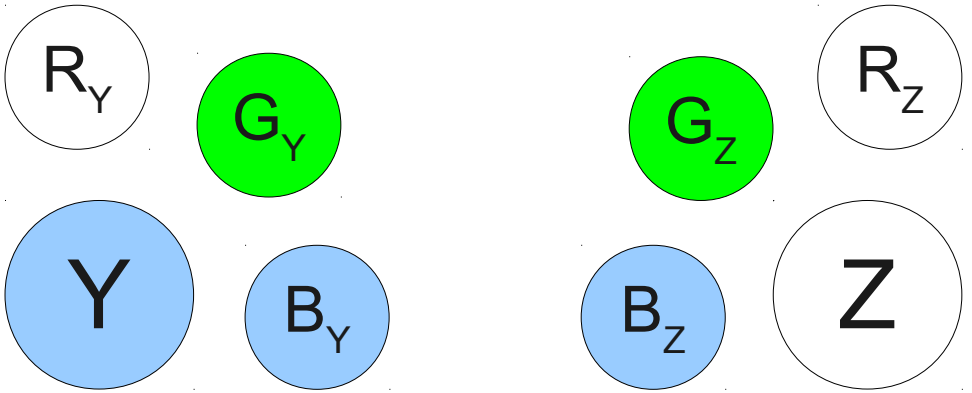
{ Z, R_Z, R_W, R_Y }

{ Z, G_Z, G_W, G_Y }

{ Z, B_Z, B_W, B_Y }







{ W, R_W, R_Y, R_Z }

{ W, G_W, G_Y, G_Z }

{ W, B_W, B_Y, B_Z }

{ X, R_X, R_Z }

{ X, G_X, G_Z }

{ X, B_X, B_Z }

{ Y, R_Y, R_W, R_Z }

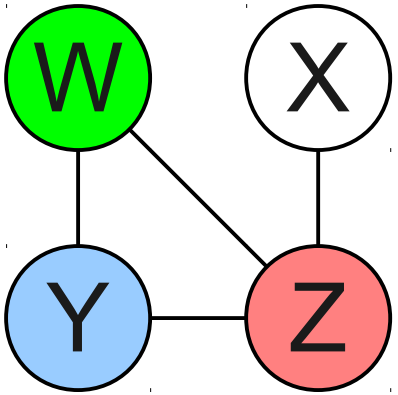
{ Y, G_Y, G_W, G_Z }

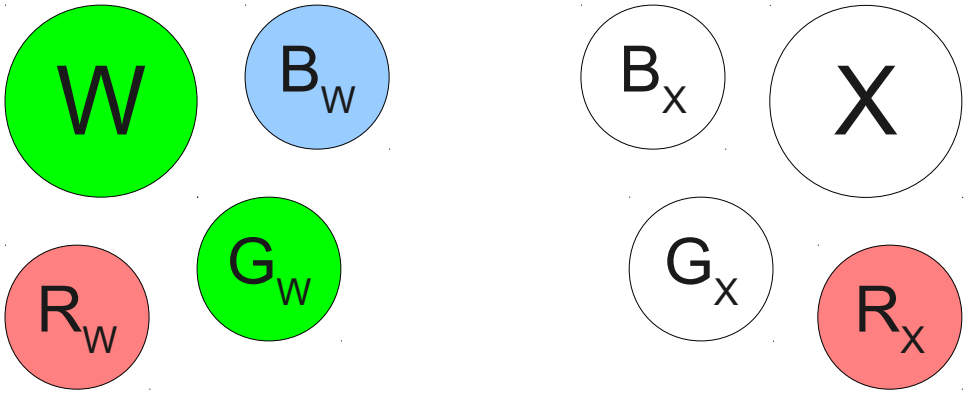
{ Y, B_Y, B_W, B_Z }

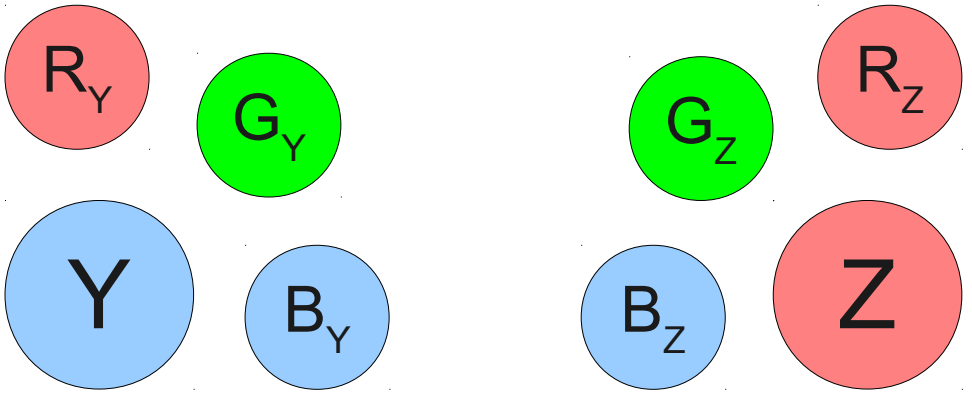
{ Z, R_Z, R_W, R_Y }

{ Z, G_Z, G_W, G_Y }

{ Z, B_Z, B_W, B_Y }







{ W, R_W, R_Y, R_Z }

{ W, G_W, G_Y, G_Z }

{ W, B_W, B_Y, B_Z }

{ X, R_X, R_Z }

{ X, G_X, G_Z }

{ X, B_X, B_Z }

{ Y, R_Y, R_W, R_Z }

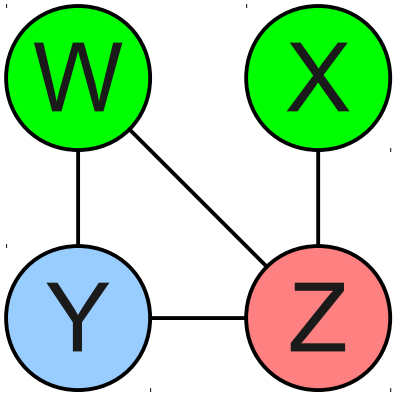
{ Y, G_Y, G_W, G_Z }

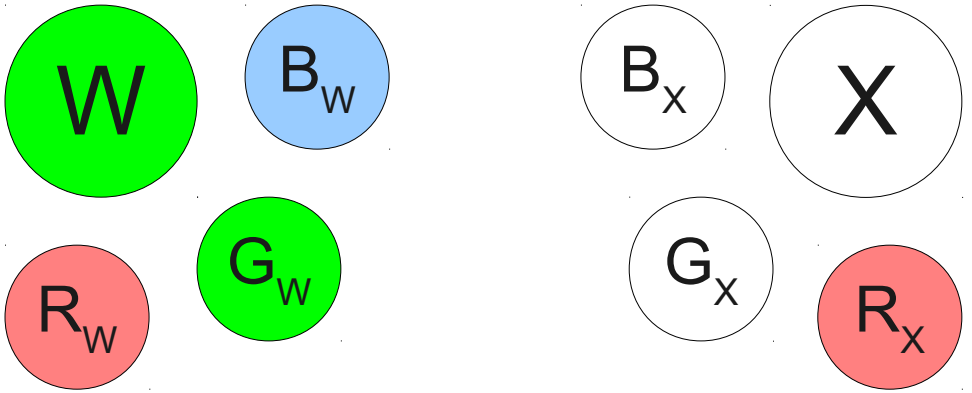
{ Y, B_Y, B_W, B_Z }

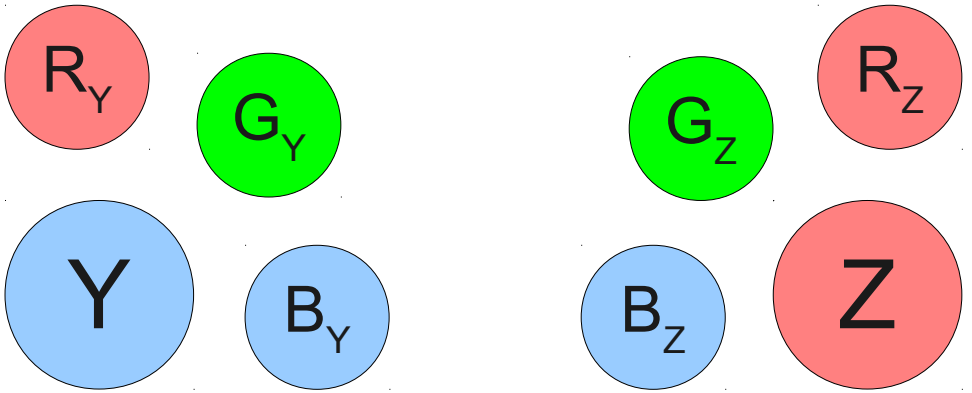
{ Z, R_Z, R_W, R_Y }

{ Z, G_Z, G_W, G_Y }

{ Z, B_Z, B_W, B_Y }







{ W, R_W, R_Y, R_Z }

{ W, G_W, G_Y, G_Z }

{ W, B_W, B_Y, B_Z }

{ X, R_X, R_Z }

{ X, G_X, G_Z }

{ X, B_X, B_Z }

{ Y, R_Y, R_W, R_Z }

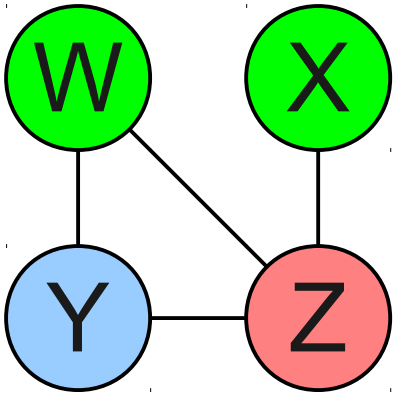
{ Y, G_Y, G_W, G_Z }

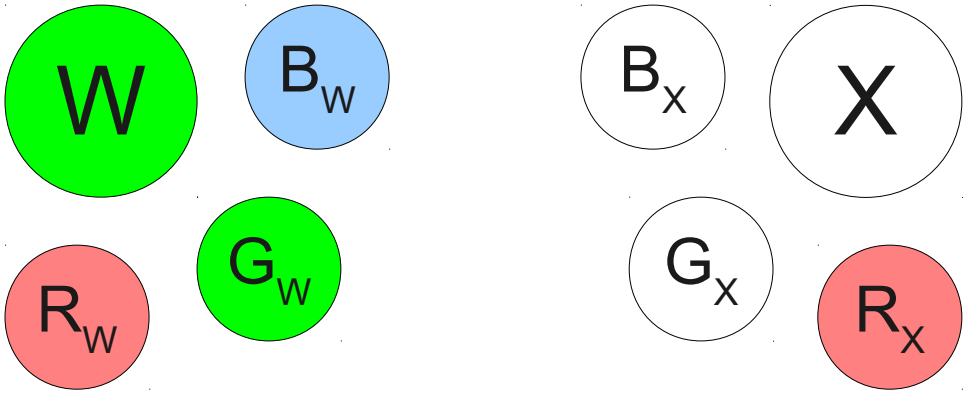
{ Y, B_Y, B_W, B_Z }

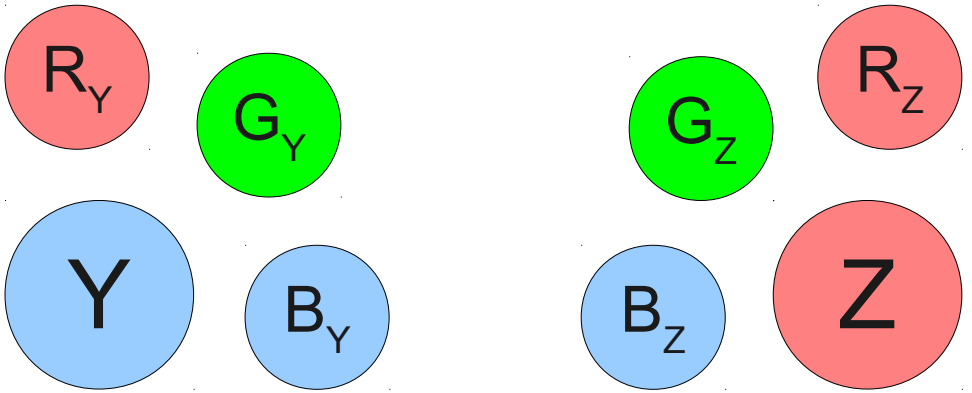
{ Z, R_Z, R_W, R_Y }

{ Z, G_Z, G_W, G_Y }

{ Z, B_Z, B_W, B_Y }







{ W, R_W, R_Y, R_Z }

{ W, G_W, G_Y, G_Z }

{ W, B_W, B_Y, B_Z }

{ X, R_X, R_Z }

{ X, G_X, G_Z }

{ X, B_X, B_Z }

{ Y, R_Y, R_W, R_Z }

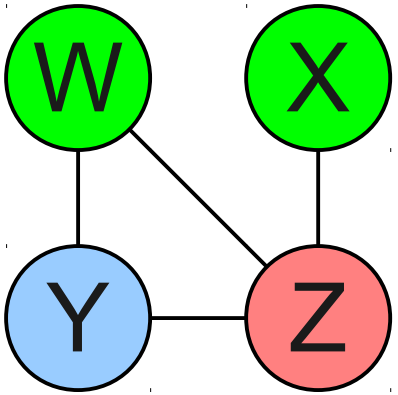
{ Y, G_Y, G_W, G_Z }

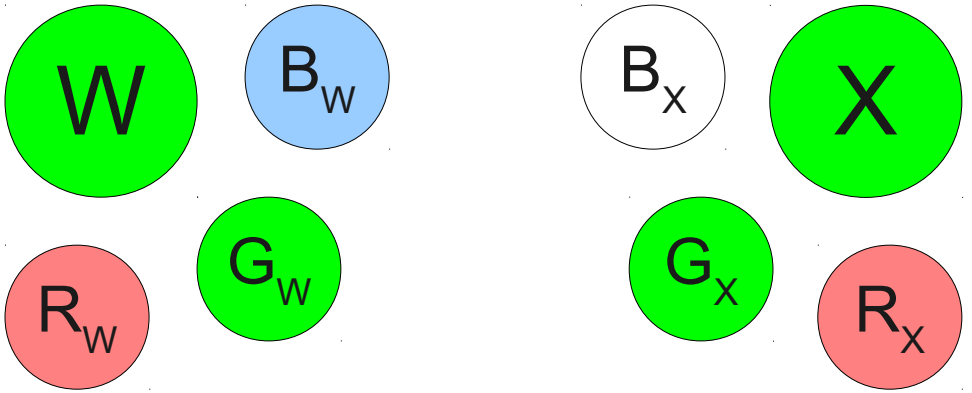
{ Y, B_Y, B_W, B_Z }

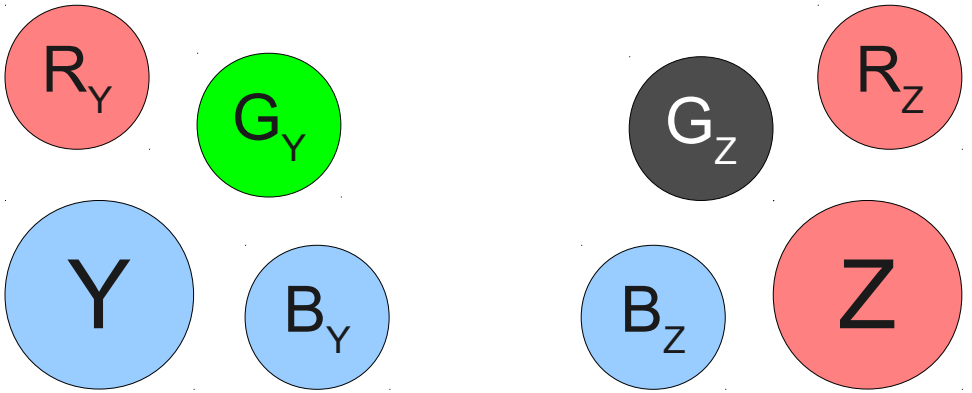
{ Z, R_Z, R_W, R_Y }

{ Z, G_Z, G_W, G_Y }

{ Z, B_Z, B_W, B_Y }







{ W, R_W, R_Y, R_Z }

{ W, G_W, G_Y, G_Z }

{ W, B_W, B_Y, B_Z }

{ X, R_X, R_Z }

{ X, G_X, G_Z }

{ X, B_X, B_Z }

{ Y, R_Y, R_W, R_Z }

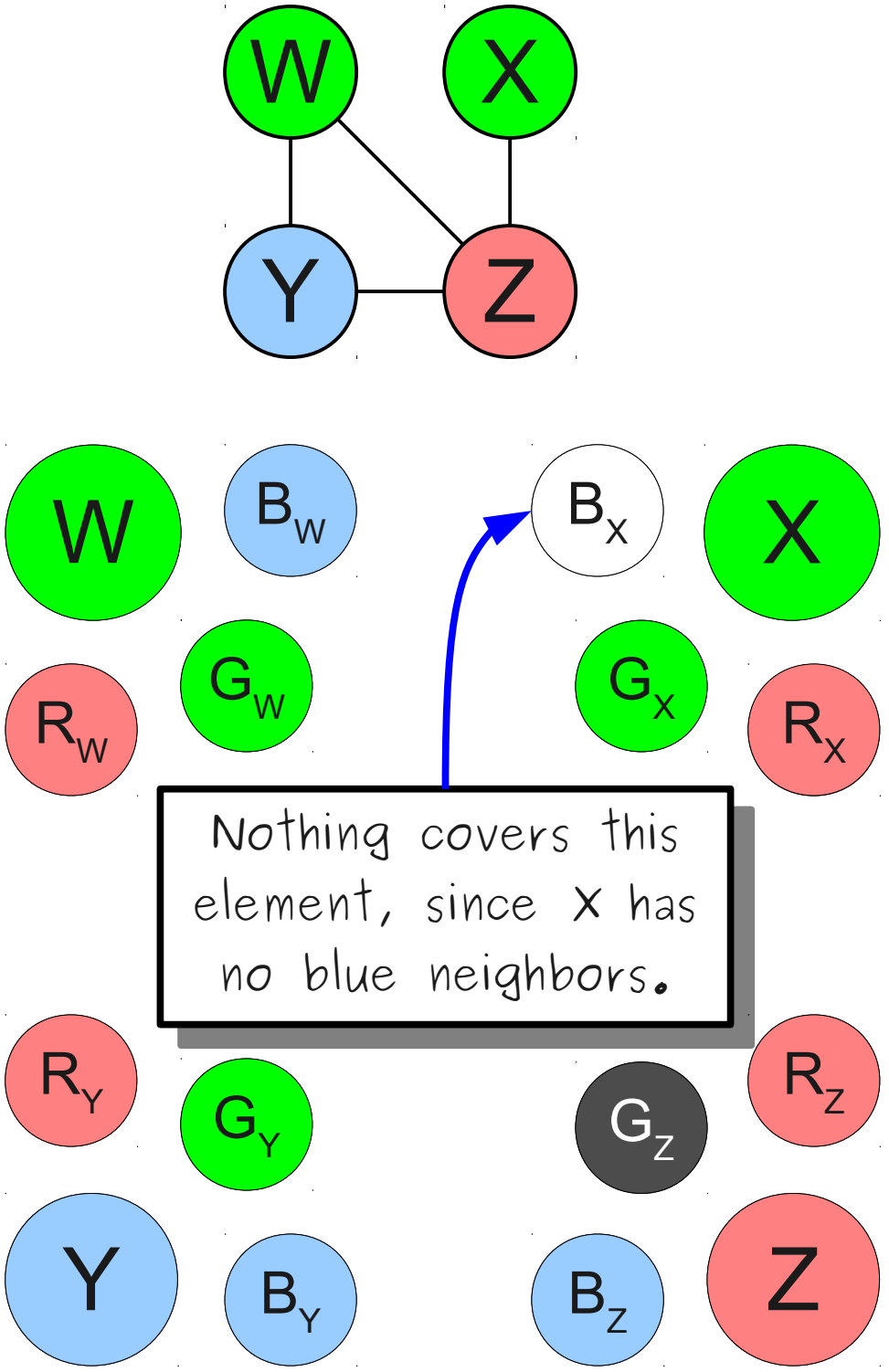
{ Y, G_Y, G_W, G_Z }

{ Y, B_Y, B_W, B_Z }

{ Z, R_Z, R_W, R_Y }

{ Z, G_Z, G_W, G_Y }

{ Z, B_Z, B_W, B_Y }



{ W, R_W, R_Y, R_Z }

{ W, G_W, G_Y, G_Z }

{ W, B_W, B_Y, B_Z }

{ X, R_X, R_Z }

{ X, G_X, G_Z }

{ X, B_X, B_Z }

{ Y, R_Y, R_W, R_Z }

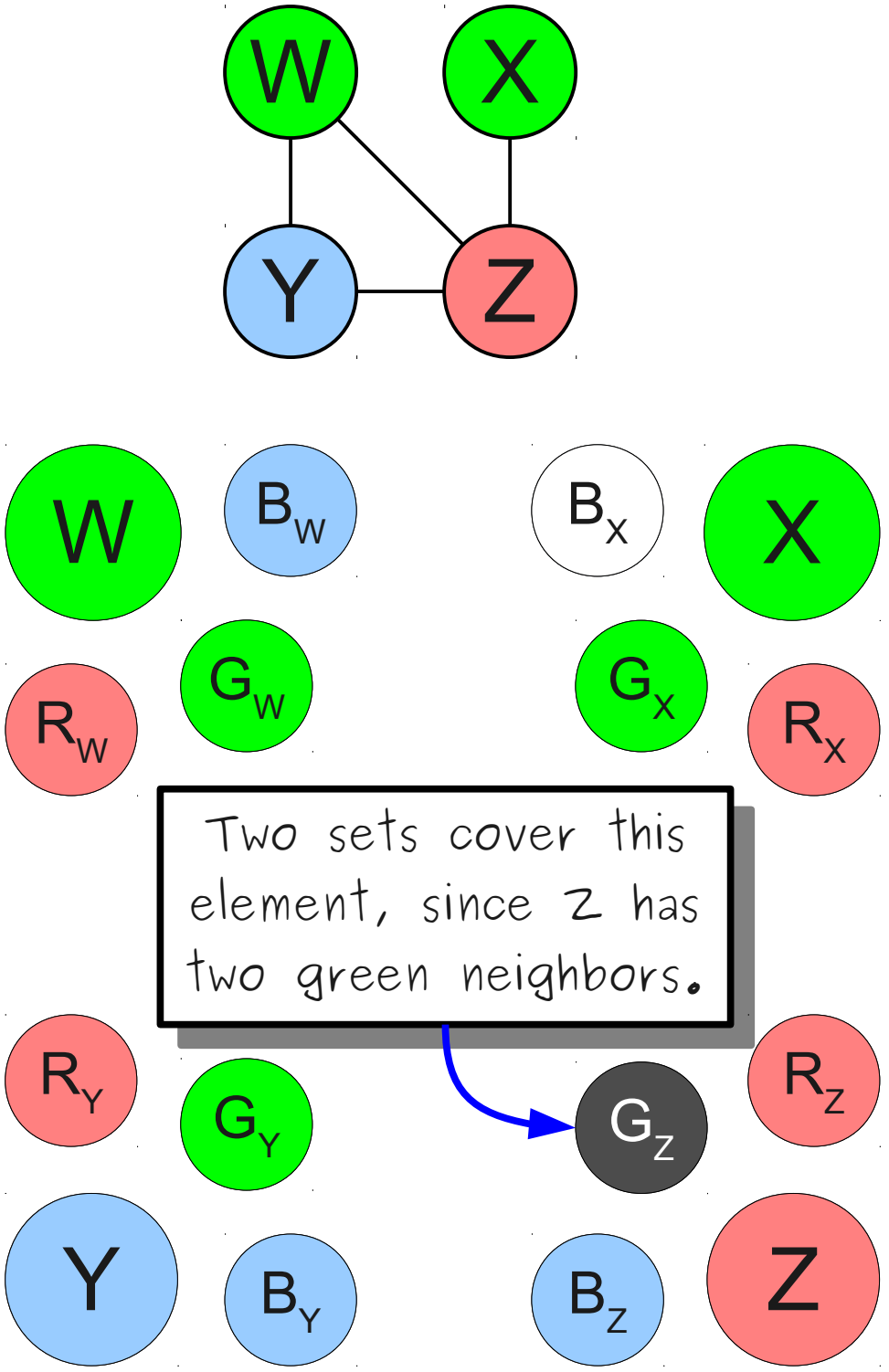
{ Y, G_Y, G_W, G_Z }

{ Y, B_Y, B_W, B_Z }

{ Z, R_Z, R_W, R_Y }

{ Z, G_Z, G_W, G_Y }

{ Z, B_Z, B_W, B_Y }



$\{ W, R_W, R_Y, R_Z \}$

$\{ W, G_W, G_Y, G_Z \}$

$\{ W, B_W, B_Y, B_Z \}$

$\{ X, R_X, R_Z \}$

$\{ X, G_X, G_Z \}$

$\{ X, B_X, B_Z \}$

$\{ Y, R_Y, R_W, R_Z \}$

$\{ Y, G_Y, G_W, G_Z \}$

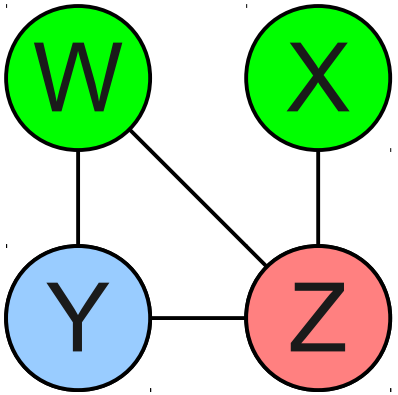
$\{ Y, B_Y, B_W, B_Z \}$

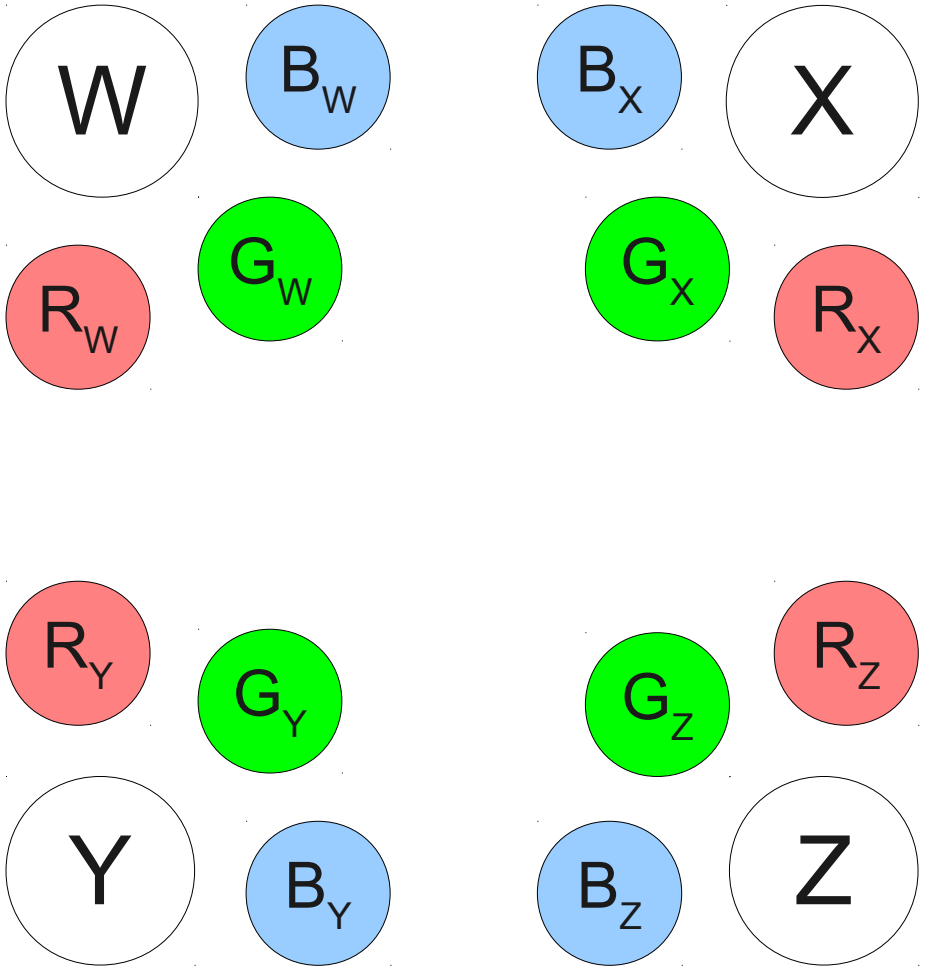
$\{ Z, R_Z, R_W, R_Y \}$

$\{ Z, G_Z, G_W, G_Y \}$

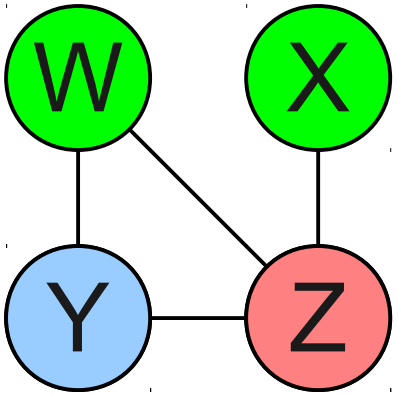
$\{ Z, B_Z, B_W, B_Y \}$

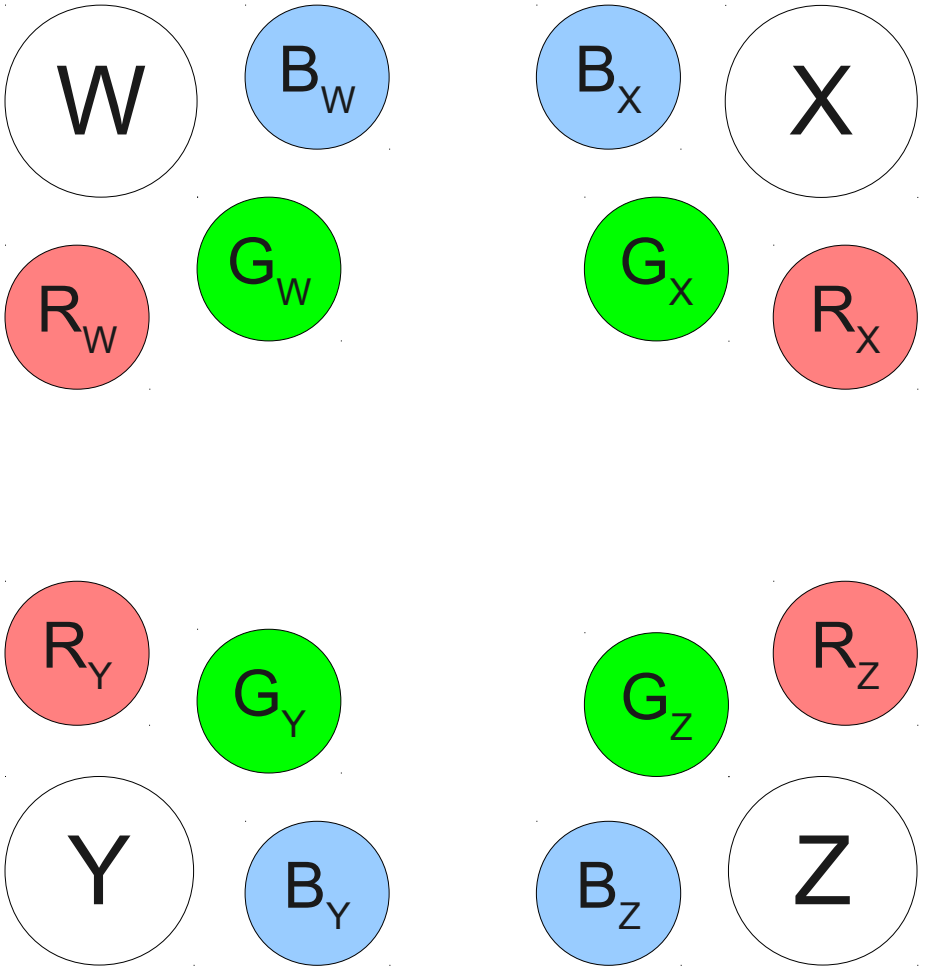
Correction 1: Filling in Gaps



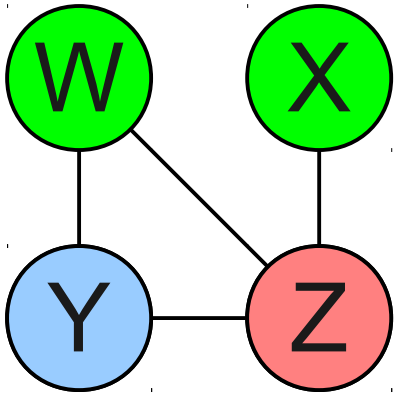


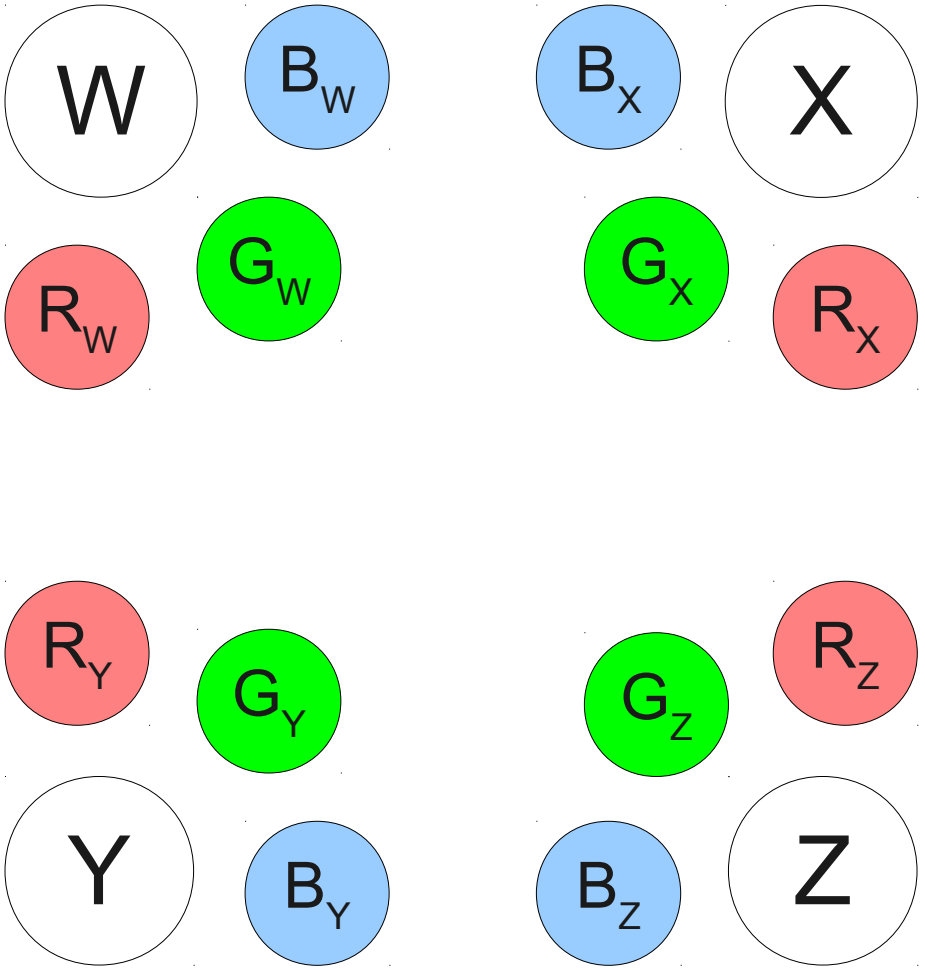
$\{ W, R_W, R_Y, R_Z \}$
$\{ W, G_W, G_Y, G_Z \}$
$\{ W, B_W, B_Y, B_Z \}$
$\{ X, R_X, R_Z \}$
$\{ X, G_X, G_Z \}$
$\{ X, B_X, B_Z \}$
$\{ Y, R_Y, R_W, R_Z \}$
$\{ Y, G_Y, G_W, G_Z \}$
$\{ Y, B_Y, B_W, B_Z \}$
$\{ Z, R_Z, R_W, R_Y \}$
$\{ Z, G_Z, G_W, G_Y \}$
$\{ Z, B_Z, B_W, B_Y \}$





$\{ W, R_W, R_Y, R_Z \}$	$\{ R_W \}$
$\{ W, G_W, G_Y, G_Z \}$	$\{ R_X \}$
$\{ W, B_W, B_Y, B_Z \}$	$\{ R_Y \}$
$\{ X, R_X, R_Z \}$	$\{ R_Z \}$
$\{ X, G_X, G_Z \}$	$\{ G_W \}$
$\{ X, B_X, B_Z \}$	$\{ G_X \}$
$\{ Y, R_Y, R_W, R_Z \}$	$\{ G_Y \}$
$\{ Y, G_Y, G_W, G_Z \}$	$\{ G_Z \}$
$\{ Y, B_Y, B_W, B_Z \}$	$\{ B_W \}$
$\{ Z, R_Z, R_W, R_Y \}$	$\{ B_X \}$
$\{ Z, G_Z, G_W, G_Y \}$	$\{ B_Y \}$
$\{ Z, B_Z, B_W, B_Y \}$	$\{ B_Z \}$





{ W, R_W, R_Y, R_Z } { R_W }

{ W, G_W, G_Y, G_Z } { R_X }

{ W, B_W, B_Y, B_Z } { R_Y }

{ X, R_X, R_Z } { R_Z }

{ X, G_X, G_Z } { G_W }

{ X, B_X, B_Z } { G_X }

{ Y, R_Y, R_W, R_Z } { G_Y }

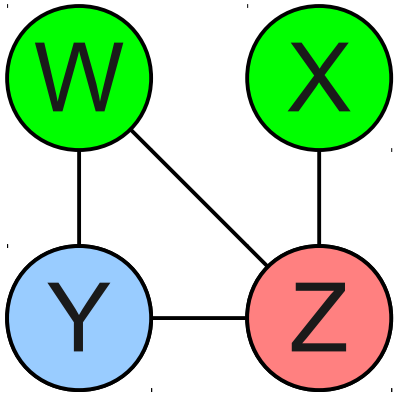
{ Y, G_Y, G_W, G_Z } { G_Z }

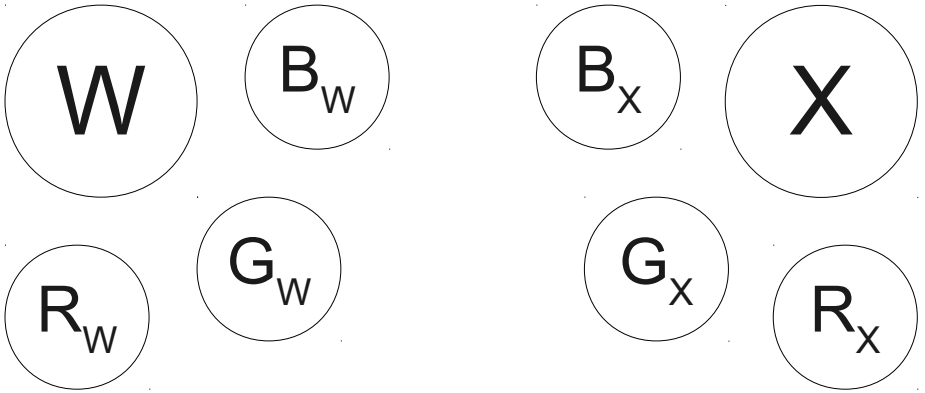
{ Y, B_Y, B_W, B_Z } { B_W }

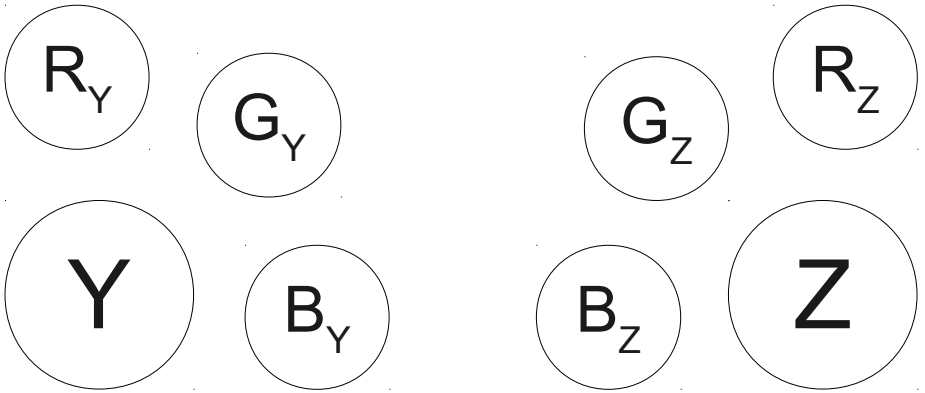
{ Z, R_Z, R_W, R_Y } { B_X }

{ Z, G_Z, G_W, G_Y } { B_Y }

{ Z, B_Z, B_W, B_Y } { B_Z }







{ W, R_W, R_Y, R_Z } { R_W }

{ W, G_W, G_Y, G_Z } { R_X }

{ W, B_W, B_Y, B_Z } { R_Y }

{ X, R_X, R_Z } { R_Z }

{ X, G_X, G_Z } { G_W }

{ X, B_X, B_Z } { G_X }

{ Y, R_Y, R_W, R_Z } { G_Y }

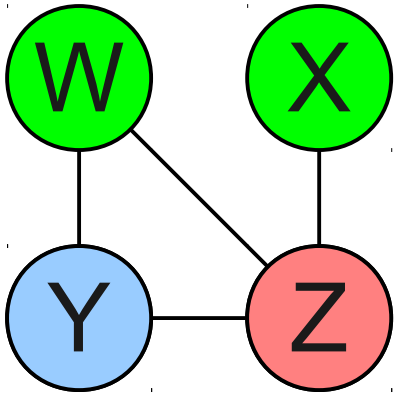
{ Y, G_Y, G_W, G_Z } { G_Z }

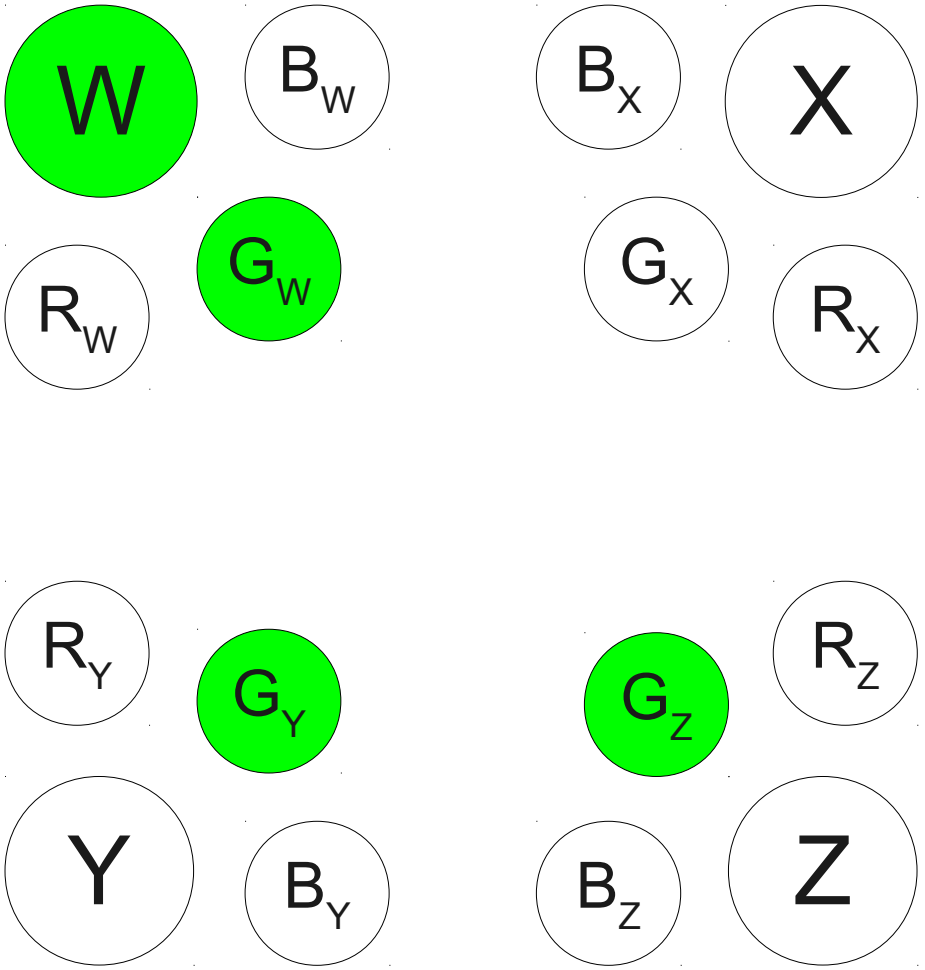
{ Y, B_Y, B_W, B_Z } { B_W }

{ Z, R_Z, R_W, R_Y } { B_X }

{ Z, G_Z, G_W, G_Y } { B_Y }

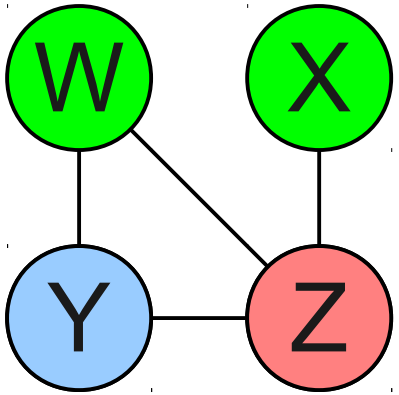
{ Z, B_Z, B_W, B_Y } { B_Z }

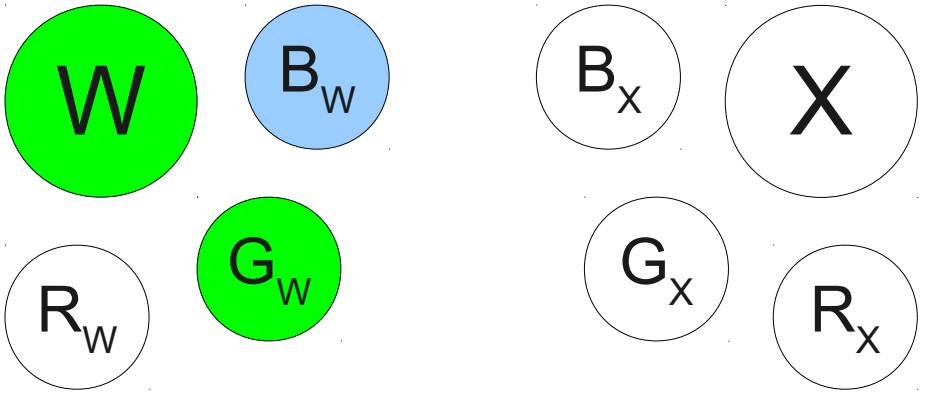


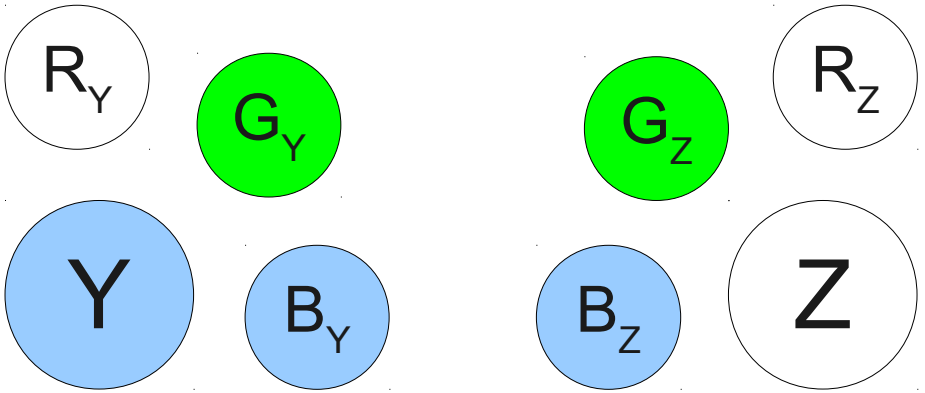


{ W, R_W, R_Y, R_Z }
{ W, G_W, G_Y, G_Z }
{ W, B_W, B_Y, B_Z }
{ X, R_X, R_Z }
{ X, G_X, G_Z }
{ X, B_X, B_Z }
{ Y, R_Y, R_W, R_Z }
{ Y, G_Y, G_W, G_Z }
{ Y, B_Y, B_W, B_Z }
{ Z, R_Z, R_W, R_Y }
{ Z, G_Z, G_W, G_Y }
{ Z, B_Z, B_W, B_Y }

{ R_W }
{ R_X }
{ R_Y }
{ R_Z }
{ G_W }
{ G_X }
{ G_Y }
{ G_Z }
{ B_W }
{ B_X }
{ B_Y }
{ B_Z }







{ W, R_W, R_Y, R_Z } { R_W }

{ W, G_W, G_Y, G_Z } { R_X }

{ W, B_W, B_Y, B_Z } { R_Y }

{ X, R_X, R_Z } { R_Z }

{ X, G_X, G_Z } { G_W }

{ X, B_X, B_Z } { G_X }

{ Y, R_Y, R_W, R_Z } { G_Y }

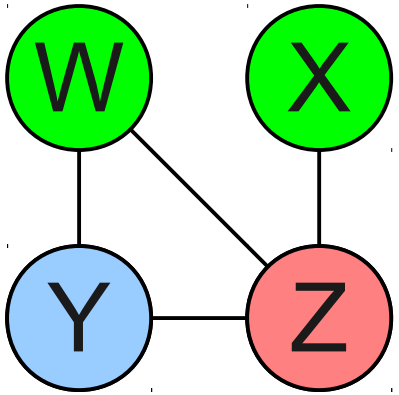
{ Y, G_Y, G_W, G_Z } { G_Z }

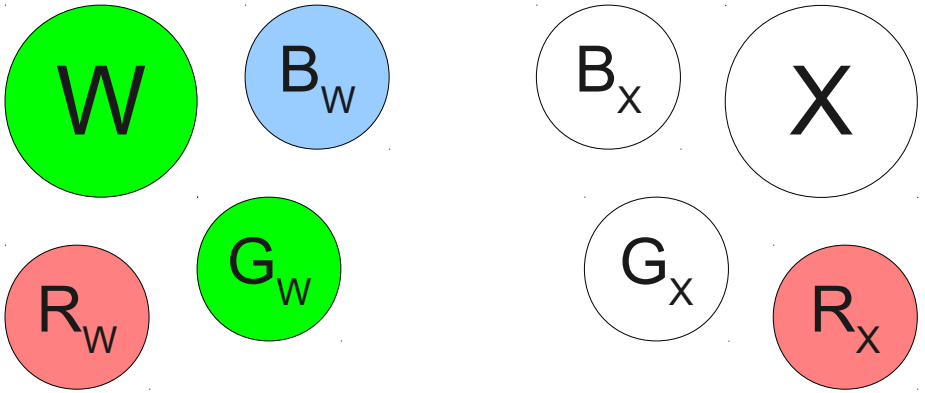
{ Y, B_Y, B_W, B_Z } { B_W }

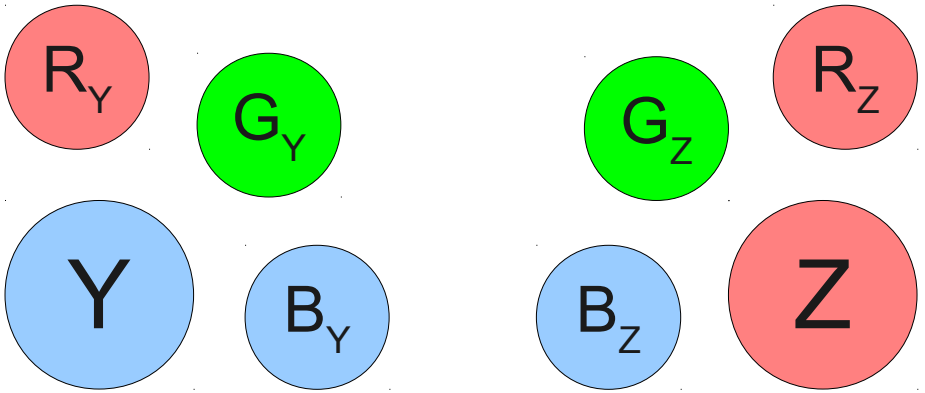
{ Z, R_Z, R_W, R_Y } { B_X }

{ Z, G_Z, G_W, G_Y } { B_Y }

{ Z, B_Z, B_W, B_Y } { B_Z }







{ W, R_W, R_Y, R_Z }

{ W, G_W, G_Y, G_Z }

{ W, B_W, B_Y, B_Z }

{ X, R_X, R_Z }

{ X, G_X, G_Z }

{ X, B_X, B_Z }

{ Y, R_Y, R_W, R_Z }

{ Y, G_Y, G_W, G_Z }

{ Y, B_Y, B_W, B_Z }

{ Z, R_Z, R_W, R_Y }

{ Z, G_Z, G_W, G_Y }

{ Z, B_Z, B_W, B_Y }

{ R_W }

{ R_X }

{ R_Y }

{ R_Z }

{ G_W }

{ G_X }

{ G_Y }

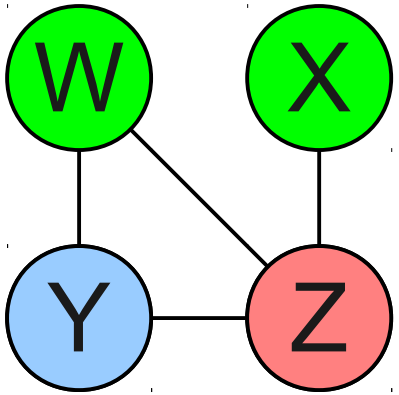
{ G_Z }

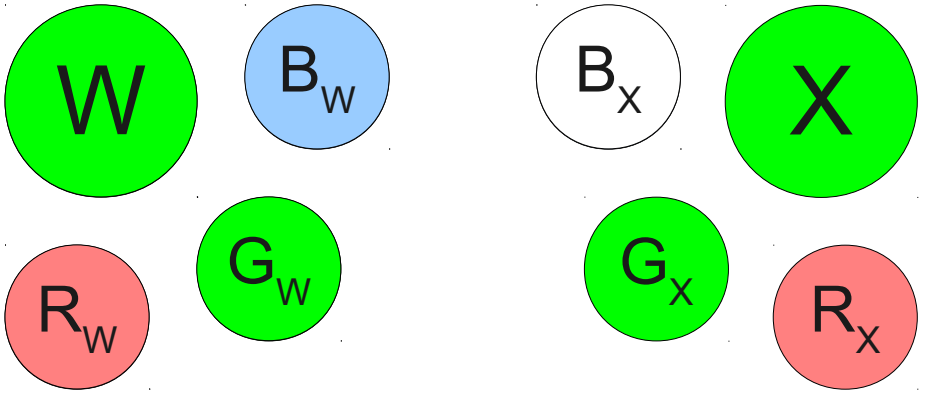
{ B_W }

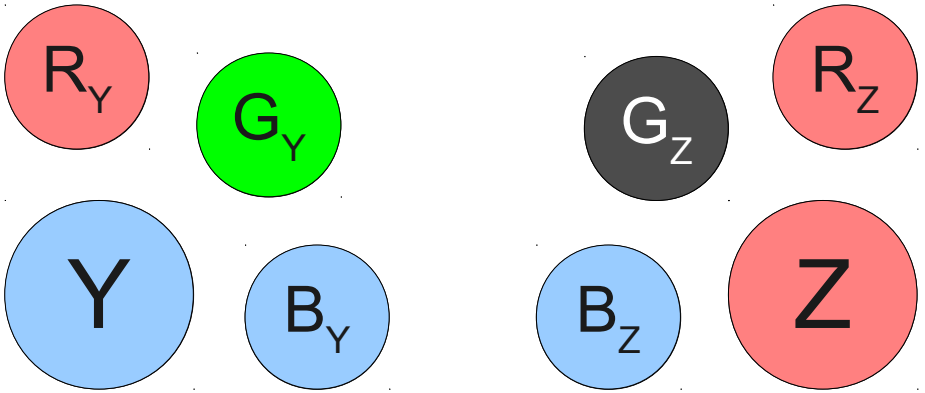
{ B_X }

{ B_Y }

{ B_Z }







{ W, R_W, R_Y, R_Z } { R_W }

{ W, G_W, G_Y, G_Z } { R_X }

{ W, B_W, B_Y, B_Z } { R_Y }

{ X, R_X, R_Z } { R_Z }

{ X, G_X, G_Z } { G_W }

{ X, B_X, B_Z } { G_X }

{ Y, R_Y, R_W, R_Z } { G_Y }

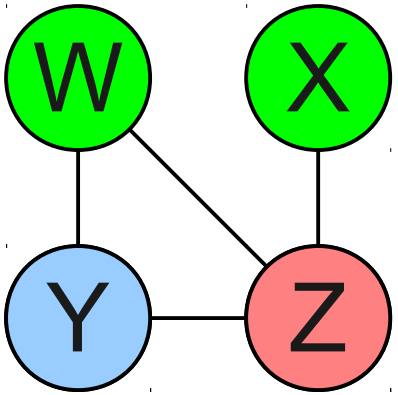
{ Y, G_Y, G_W, G_Z } { G_Z }

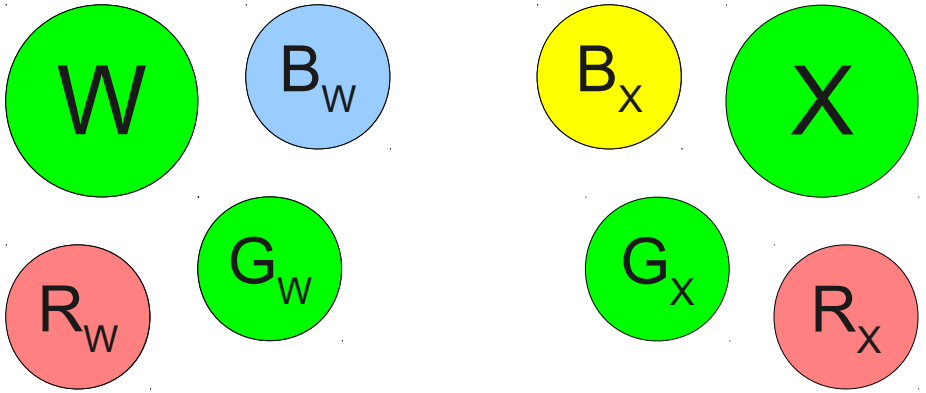
{ Y, B_Y, B_W, B_Z } { B_W }

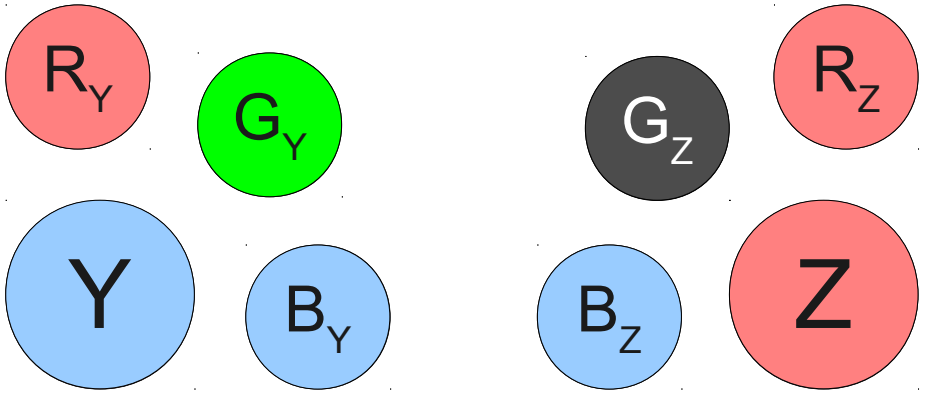
{ Z, R_Z, R_W, R_Y } { B_X }

{ Z, G_Z, G_W, G_Y } { B_Y }

{ Z, B_Z, B_W, B_Y } { B_Z }







{ W, R_W, R_Y, R_Z } { R_W }

{ W, G_W, G_Y, G_Z } { R_X }

{ W, B_W, B_Y, B_Z } { R_Y }

{ X, R_X, R_Z } { R_Z }

{ X, G_X, G_Z } { G_W }

{ X, B_X, B_Z } { G_X }

{ Y, R_Y, R_W, R_Z } { G_Y }

{ Y, G_Y, G_W, G_Z } { G_Z }

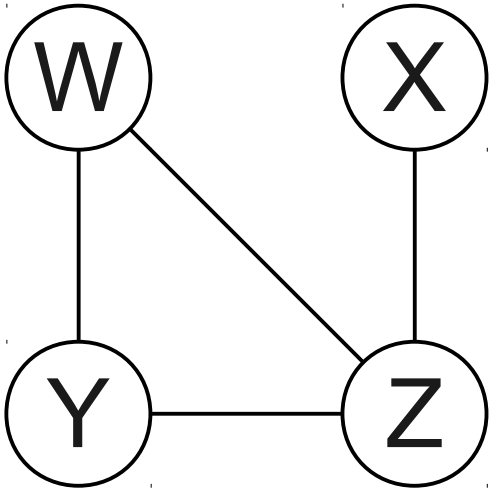
{ Y, B_Y, B_W, B_Z } { B_W }

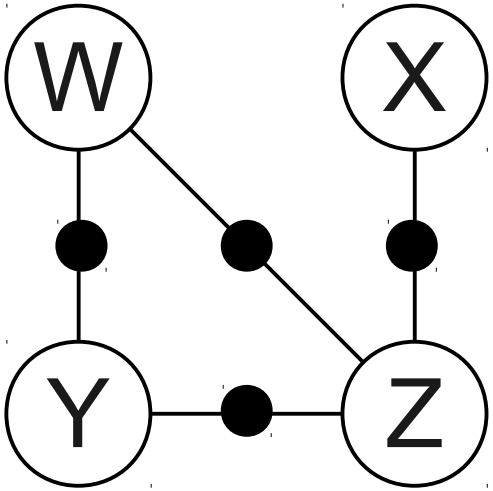
{ Z, R_Z, R_W, R_Y } { B_X }

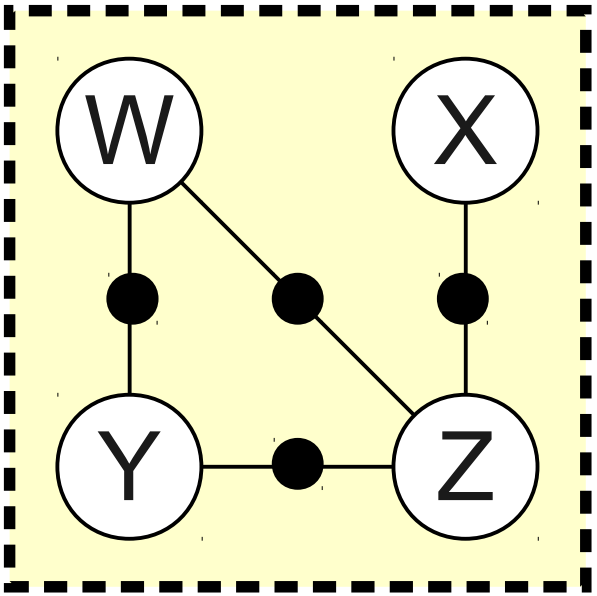
{ Z, G_Z, G_W, G_Y } { B_Y }

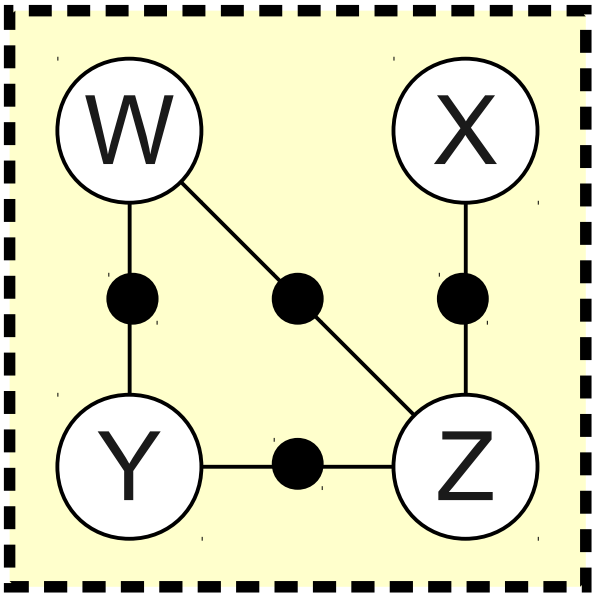
{ Z, B_Z, B_W, B_Y } { B_Z }

Correction 2: Avoiding Duplicates


W
X
Y
Z


W
X
Y
Z


W
X
Y
Z

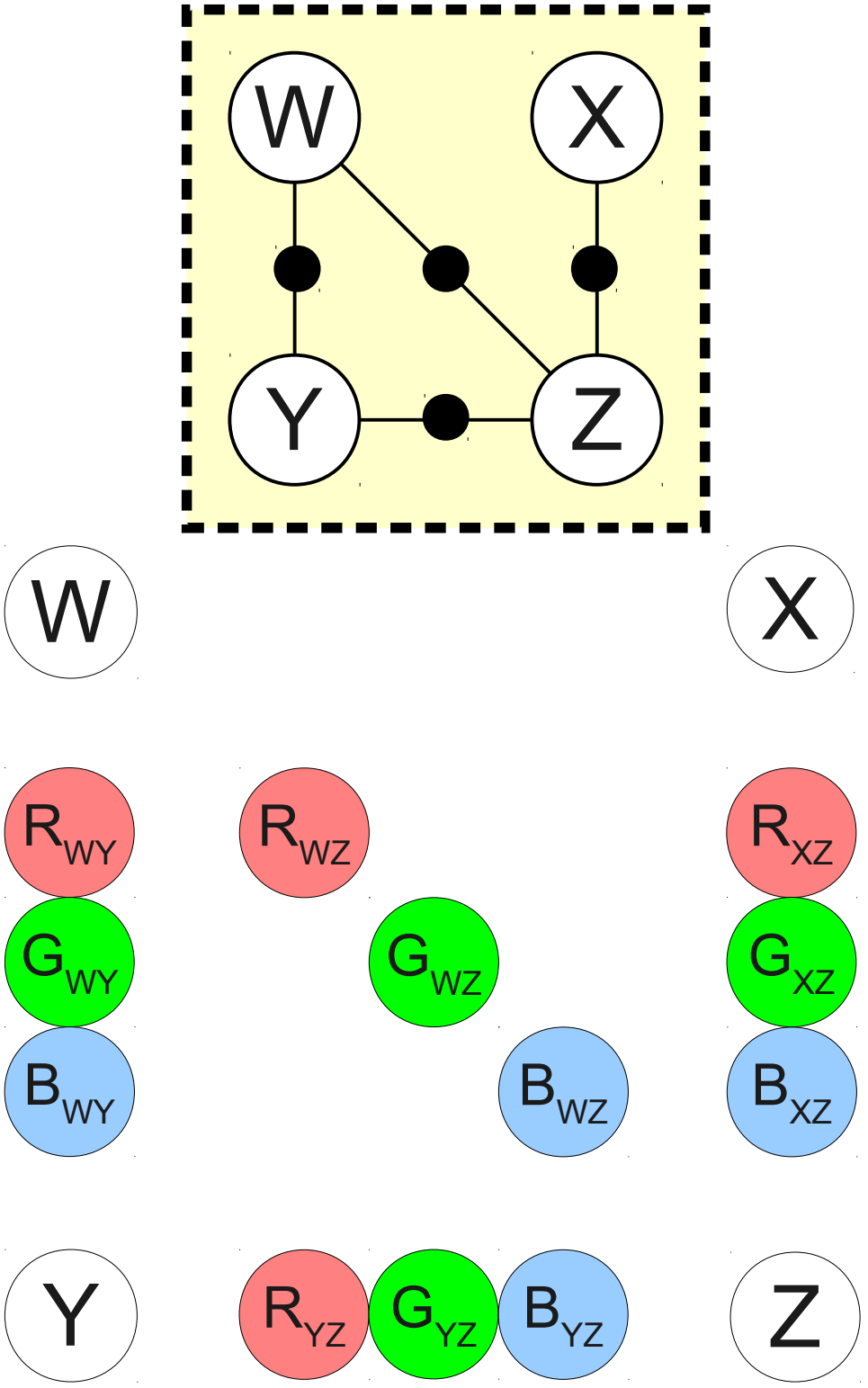


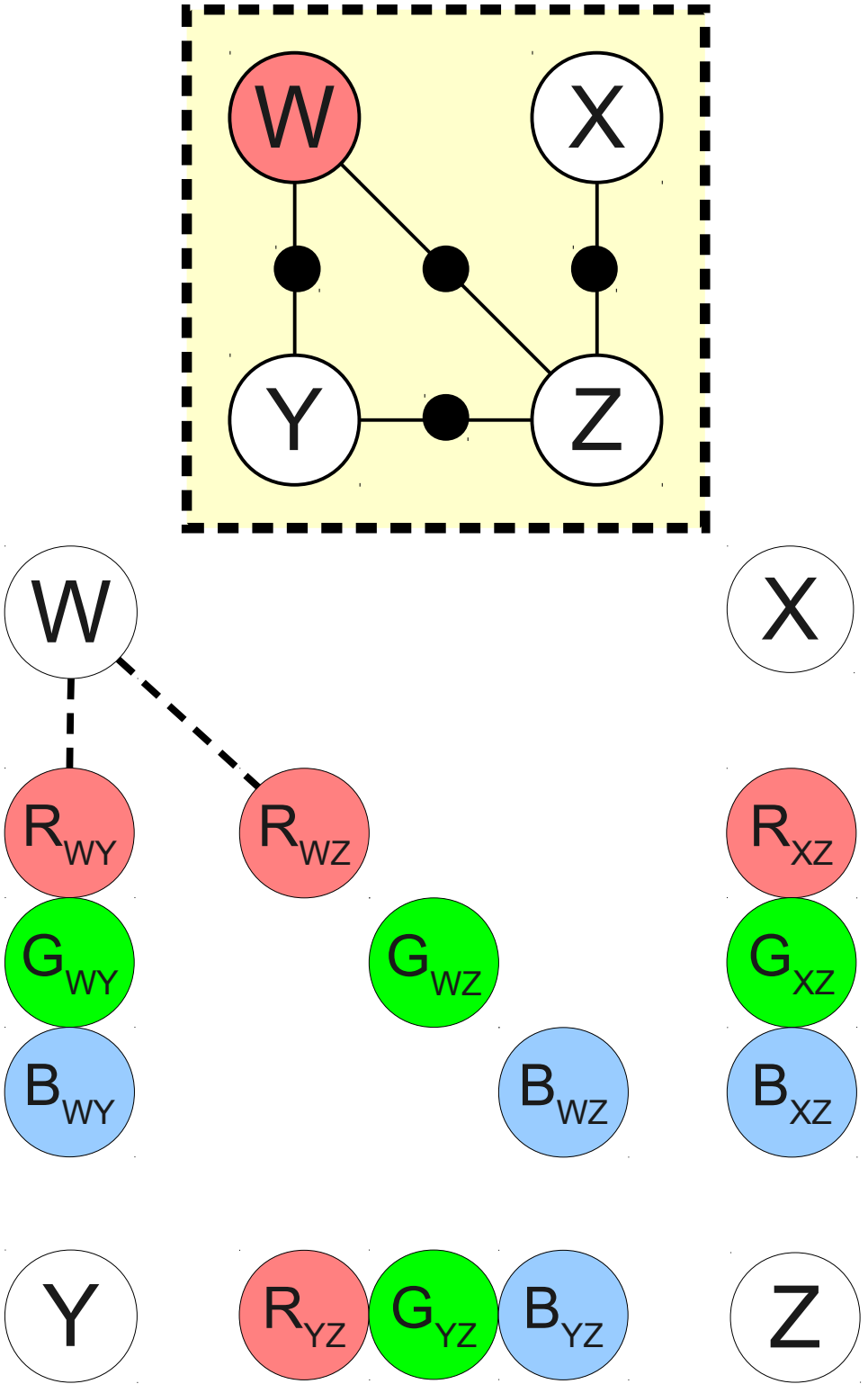
W

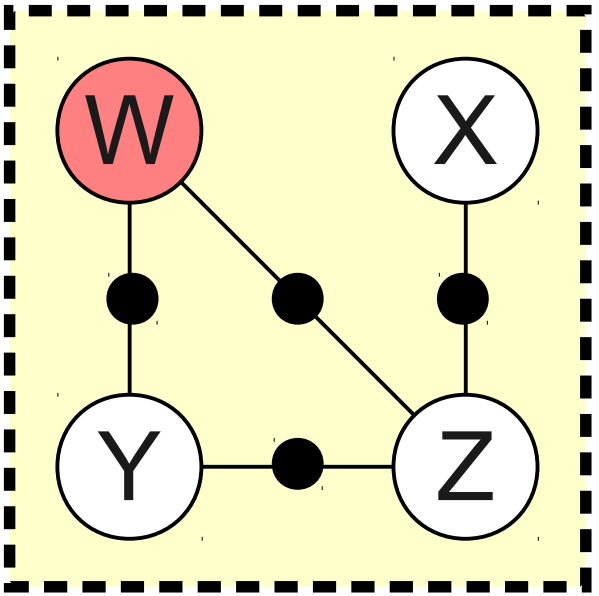
X

Y

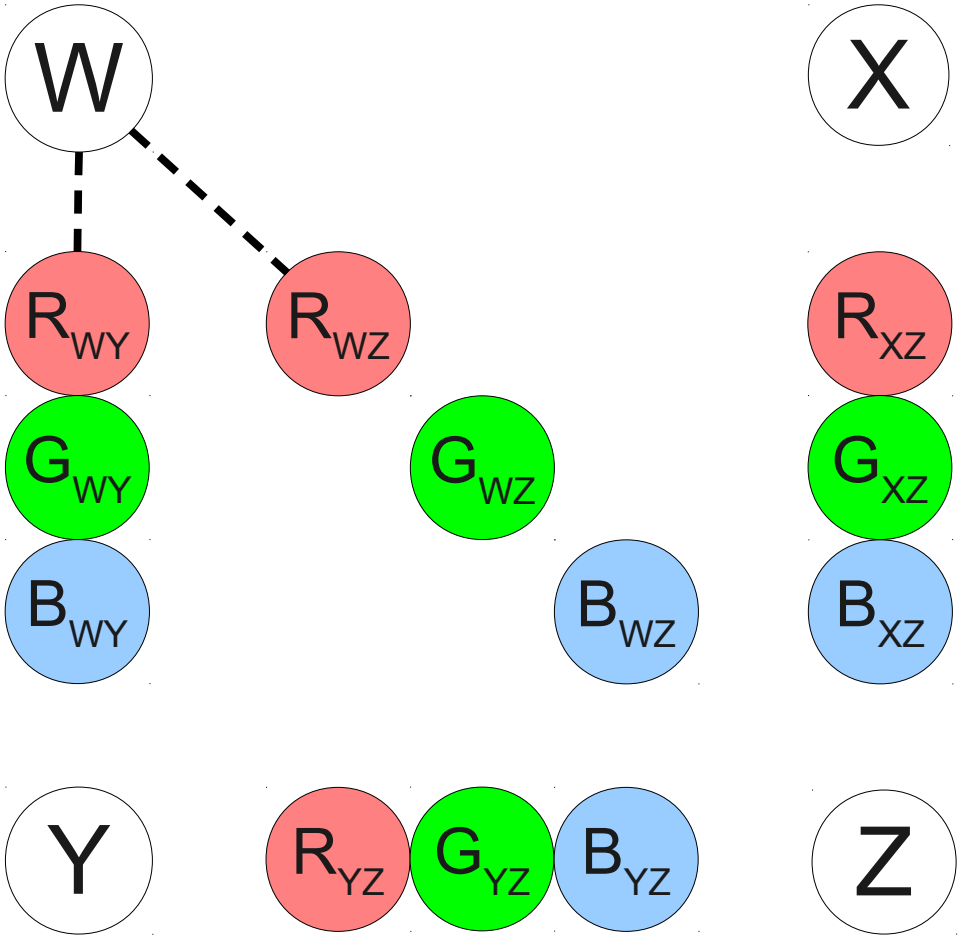
Z

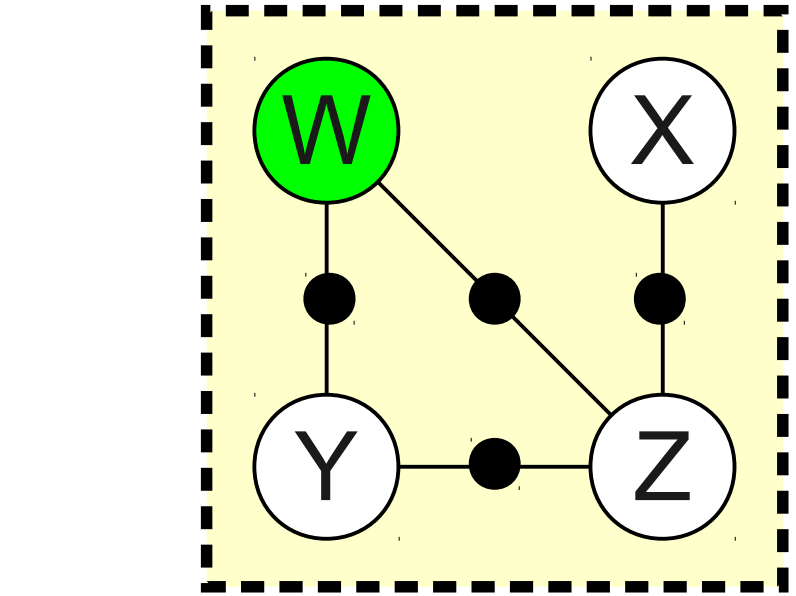

W
X
Y
Z
W
X
R_{WY}
R_{WZ}
R_{XZ}
G_{WY}
G_{WZ}
G_{XZ}
B_{WY}
B_{WZ}
B_{XZ}
Y
R_{YZ}
G_{YZ}
B_{YZ}
Z


W
X
Y
Z
W
X
R_WY
R_WZ
R_XZ
G_WY
G_WZ
G_XZ
B_WY
B_WZ
B_XZ
Y
R_YZ
G_YZ
B_YZ
Z

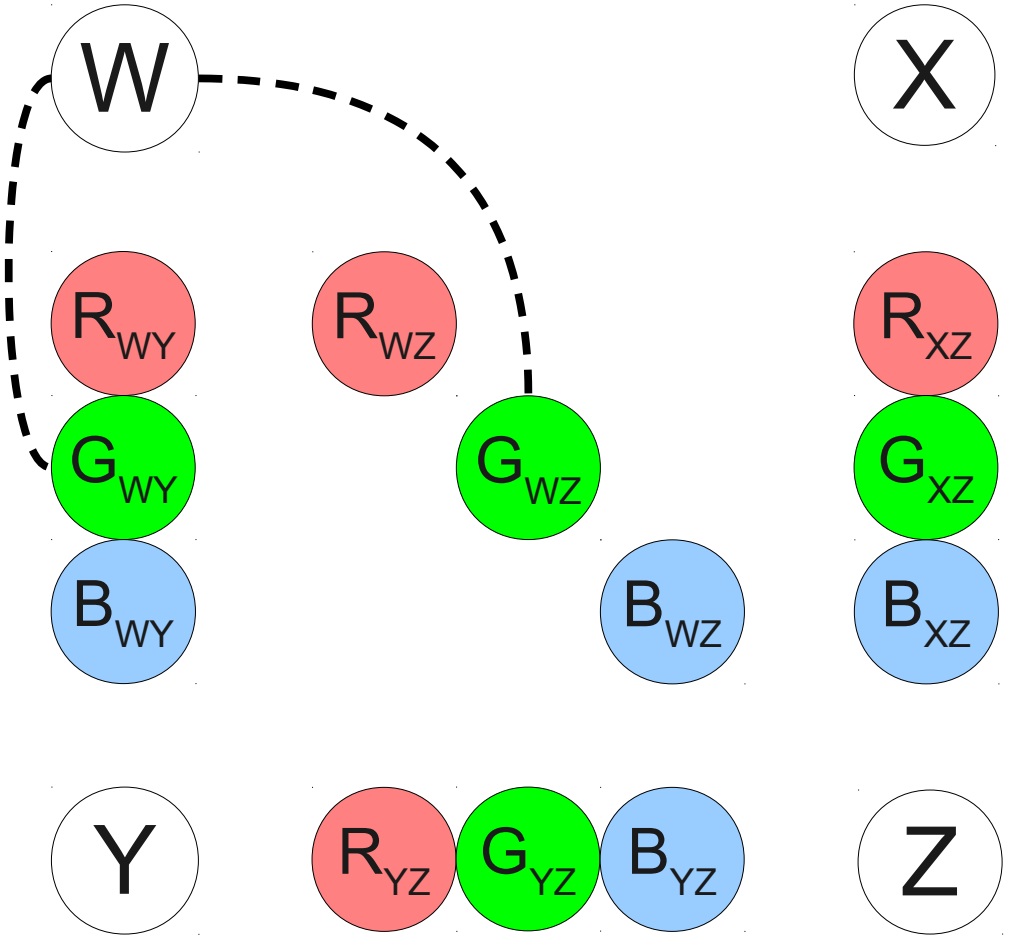


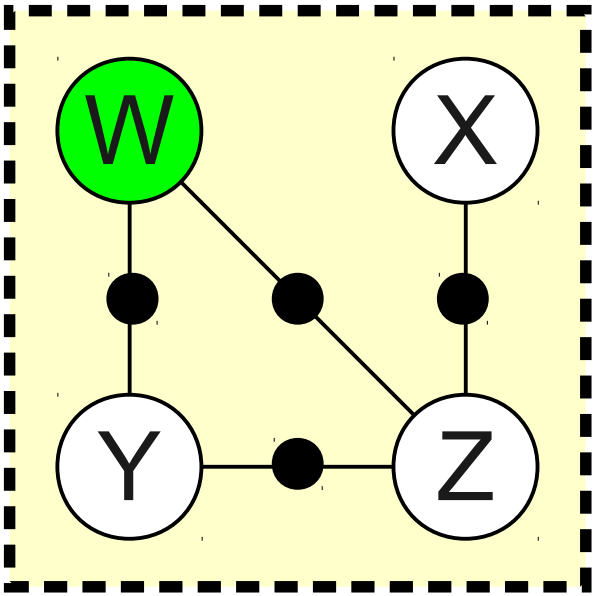
$\{ W, R_{WY}, R_{WZ} \}$

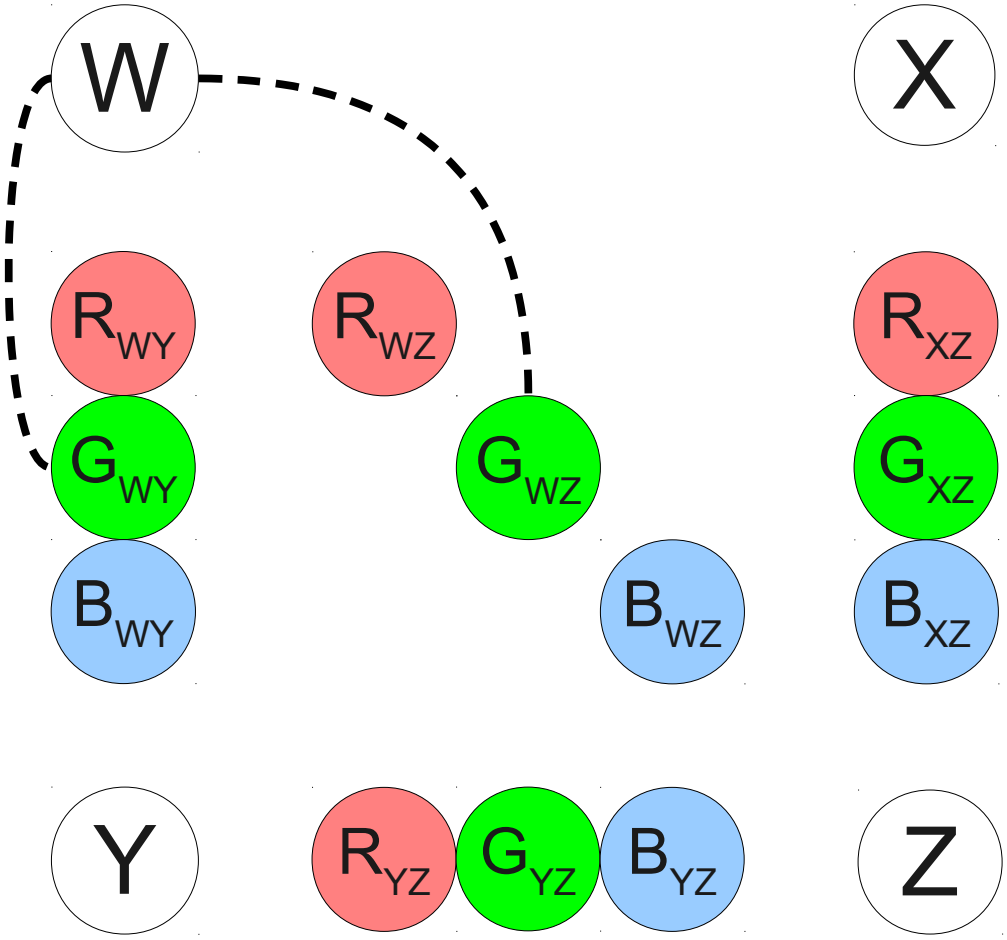




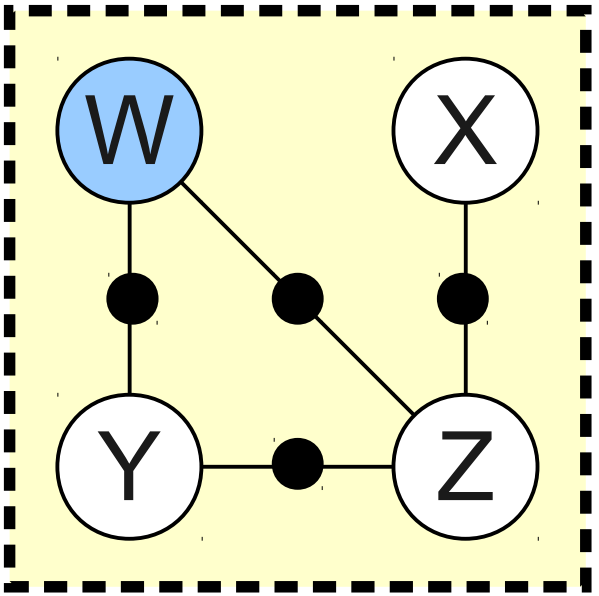
$\{ W, R_{WY}, R_{WZ} \}$

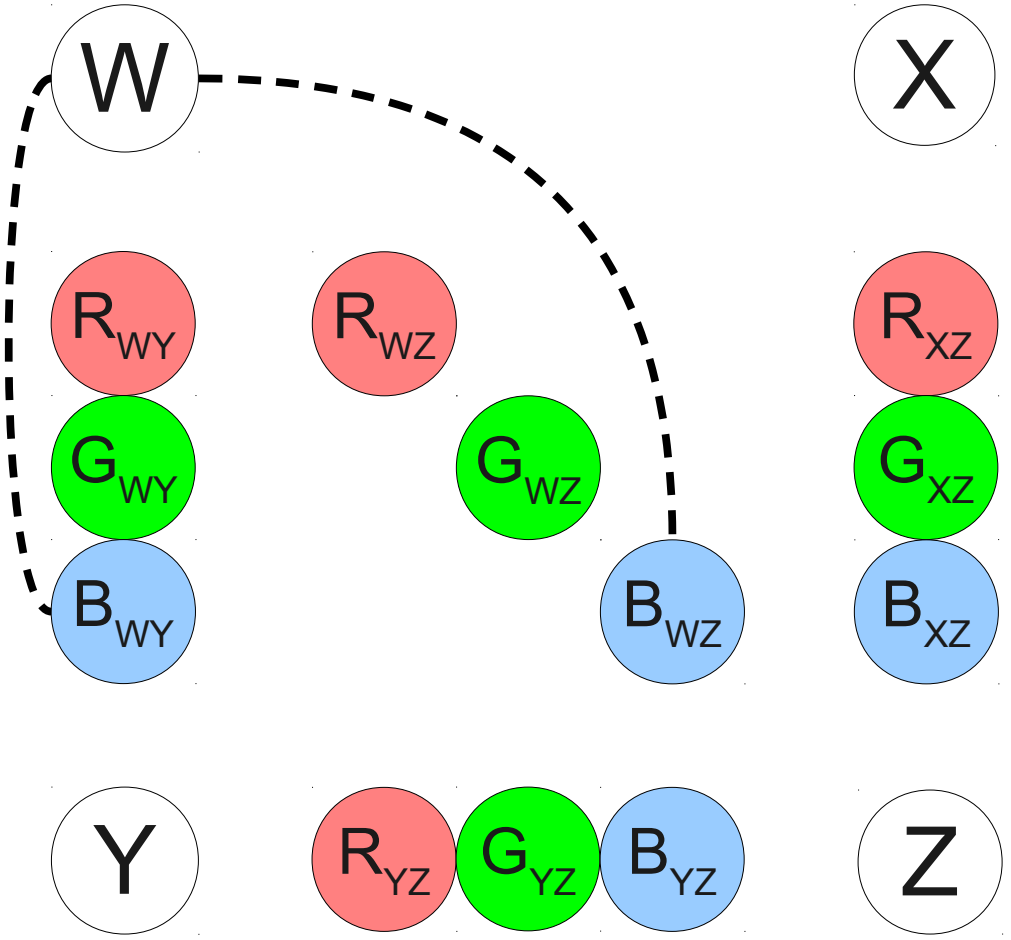






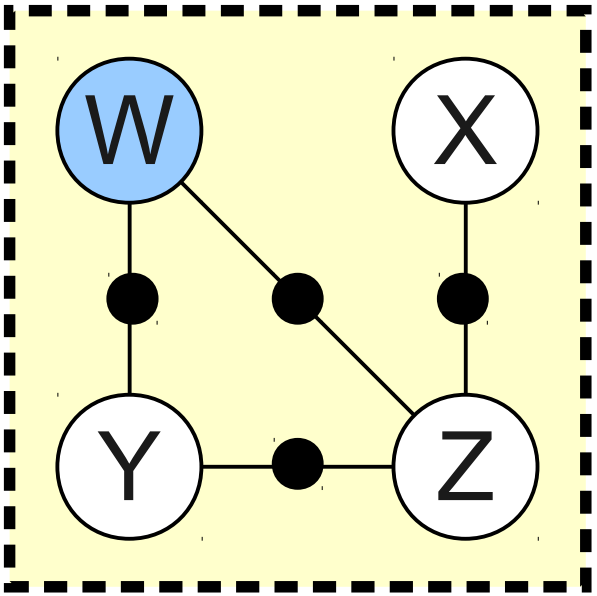
$\{ W, R_{WY}, R_{WZ} \}$
$\{ W, G_{WY}, G_{WZ} \}$

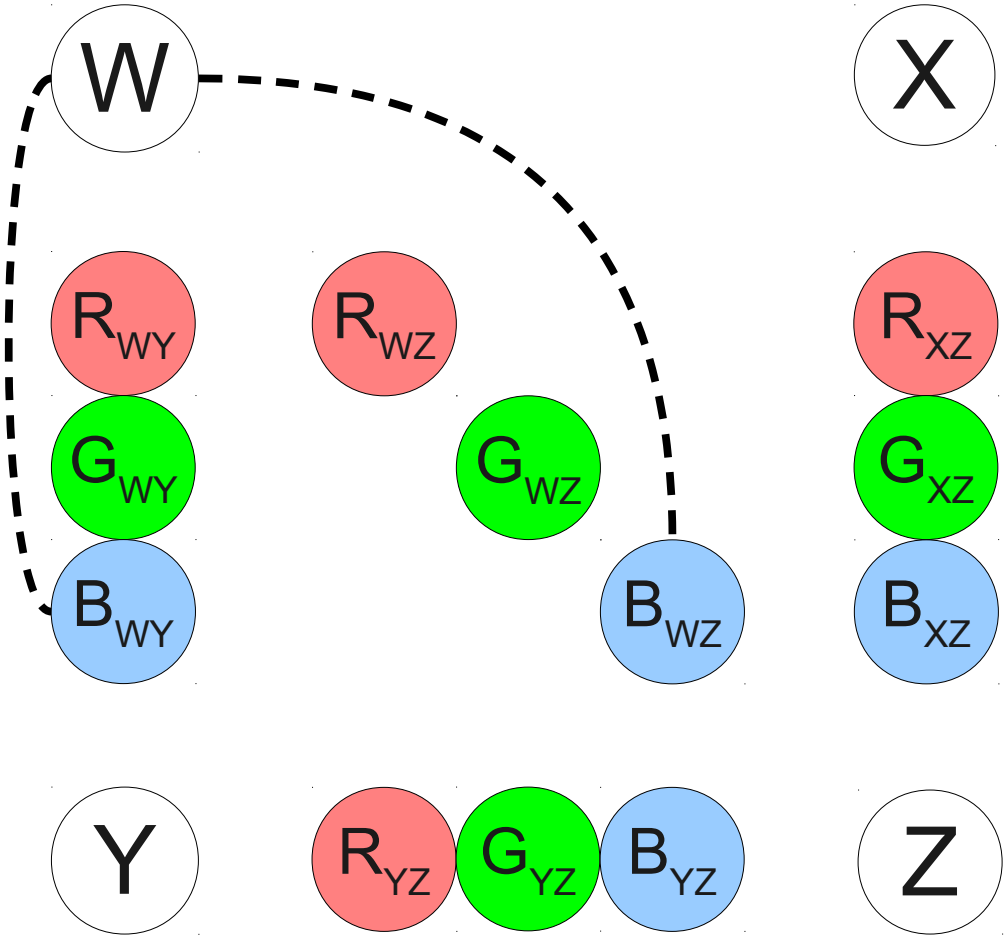




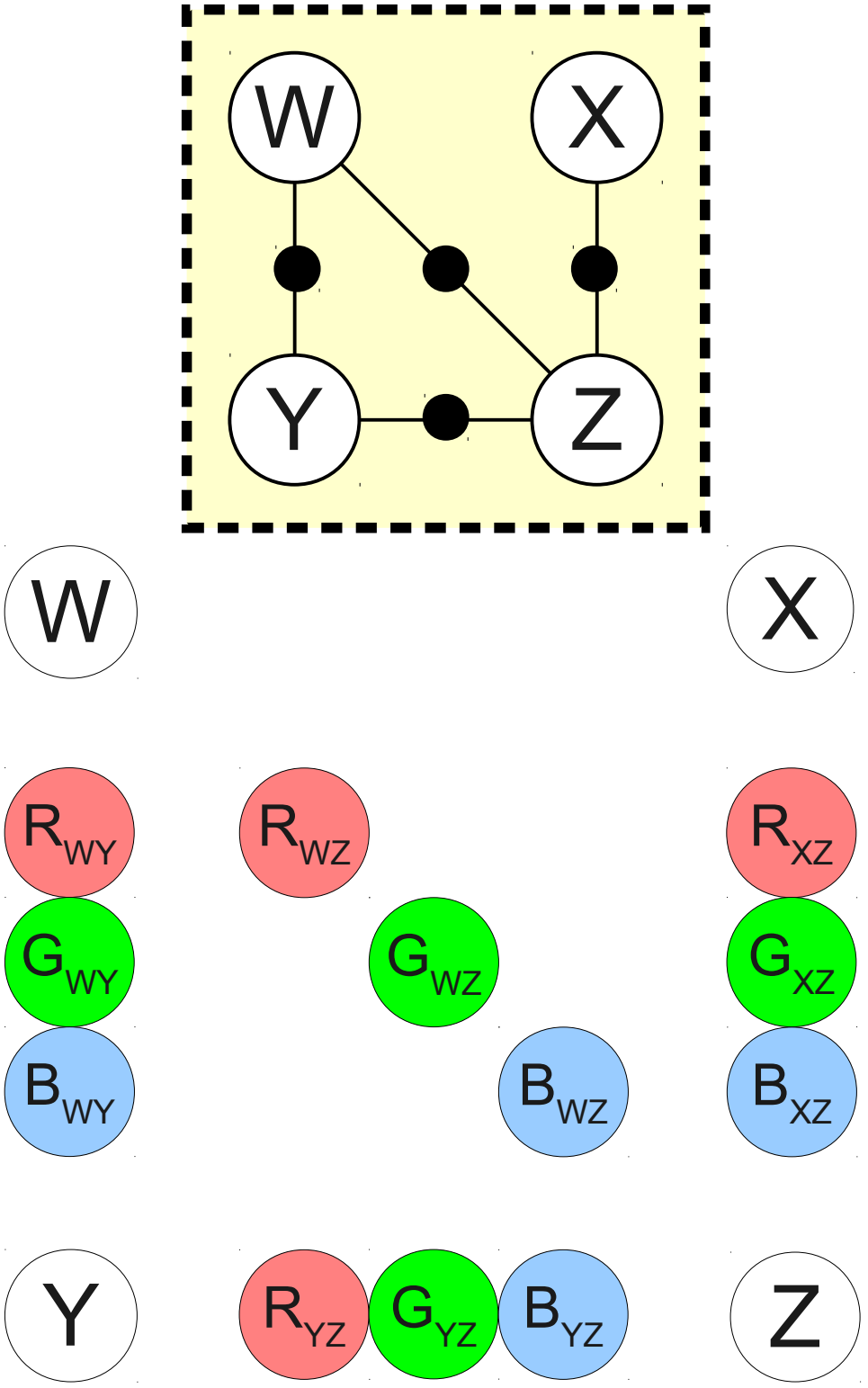
$\{ W, R_{WY}, R_{WZ} \}$

$\{ W, G_{WY}, G_{WZ} \}$

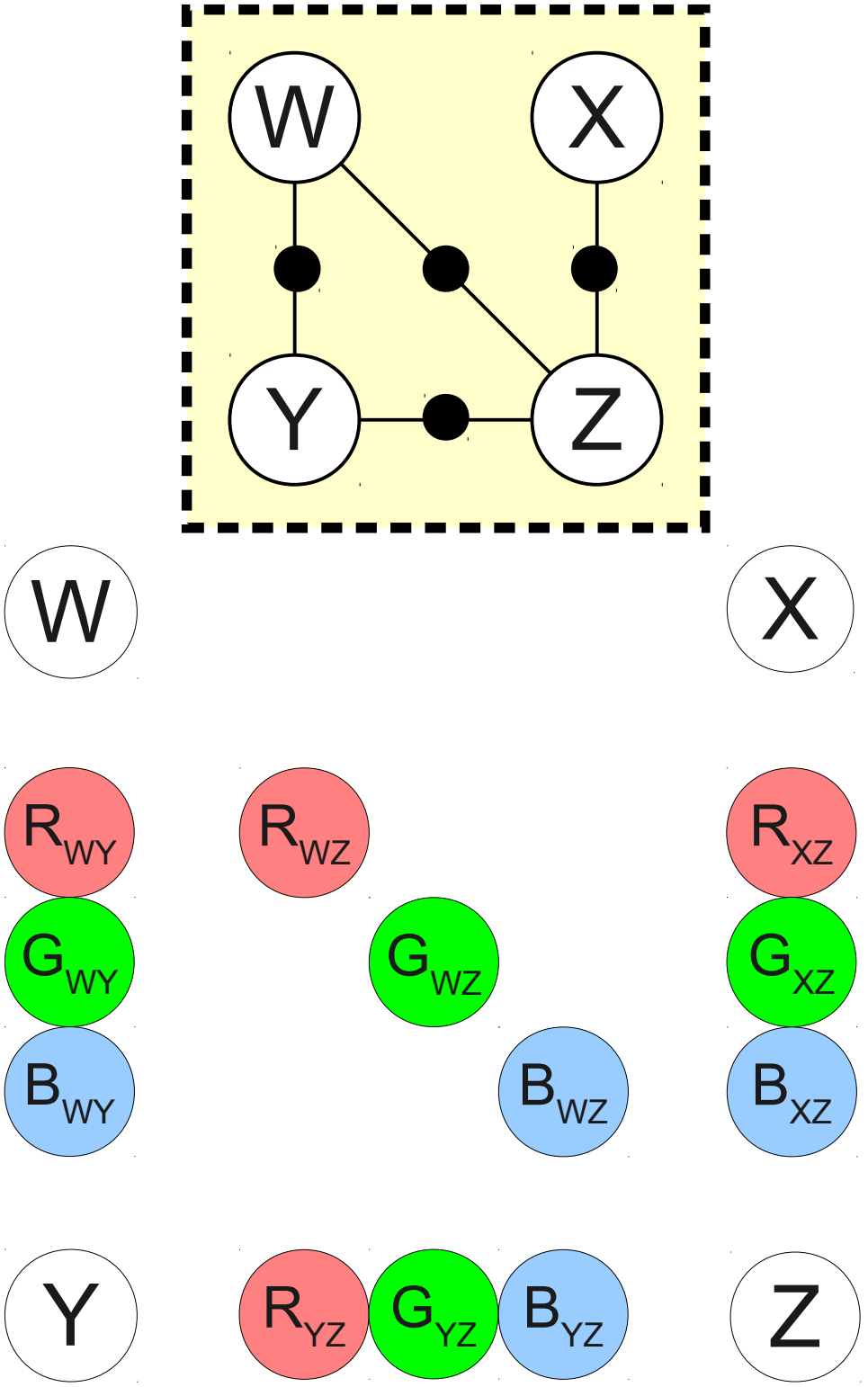




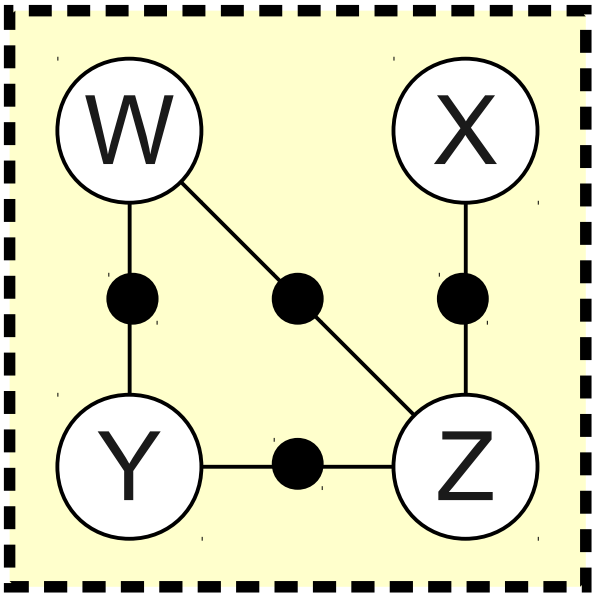
$\{ W, R_{WY}, R_{WZ} \}$
$\{ W, G_{WY}, G_{WZ} \}$
$\{ W, B_{WY}, B_{WZ} \}$

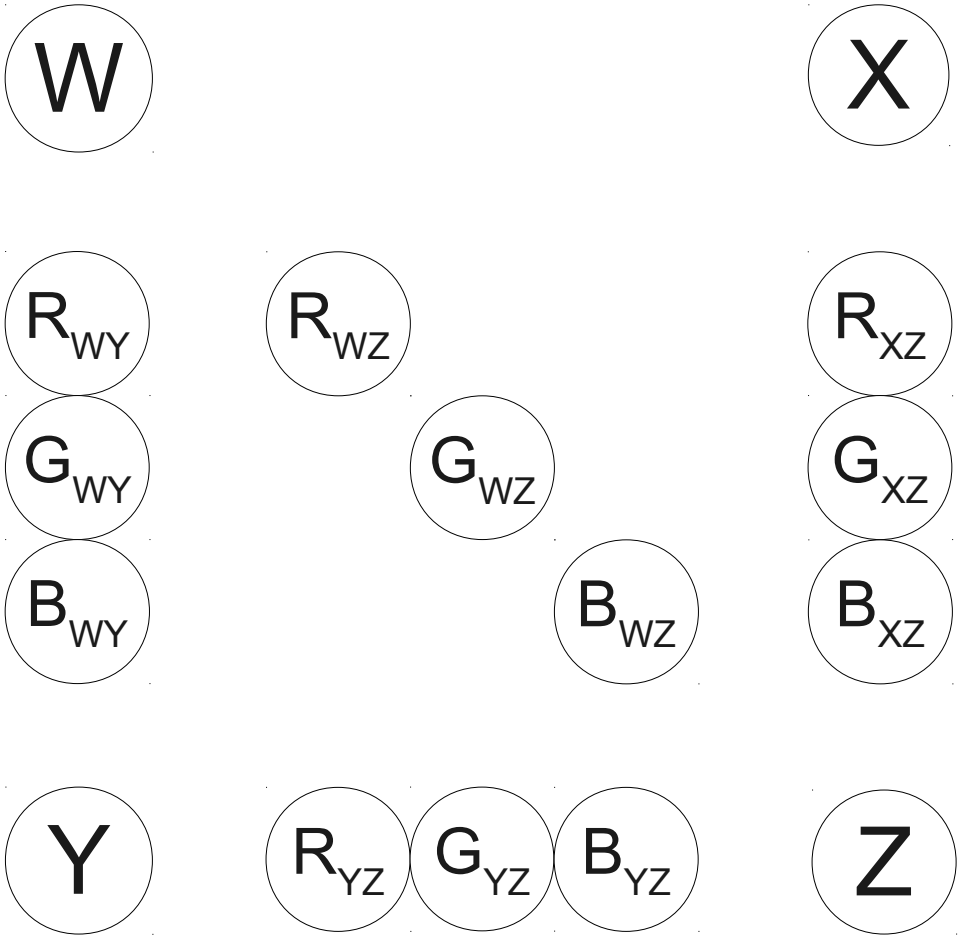


$\{ W, R_{WY}, R_{WZ} \}$
$\{ W, G_{WY}, G_{WZ} \}$
$\{ W, B_{WY}, B_{WZ} \}$
$\{ X, R_{XZ} \}$
$\{ X, G_{XZ} \}$
$\{ X, B_{XZ} \}$
$\{ Y, R_{WY}, R_{YZ} \}$
$\{ Y, G_{WY}, G_{YZ} \}$
$\{ Y, B_{WY}, B_{YZ} \}$
$\{ Z, R_{WZ}, R_{XZ}, R_{YZ} \}$
$\{ Z, G_{WZ}, G_{XZ}, G_{YZ} \}$
$\{ Z, B_{WZ}, B_{XZ}, B_{YZ} \}$

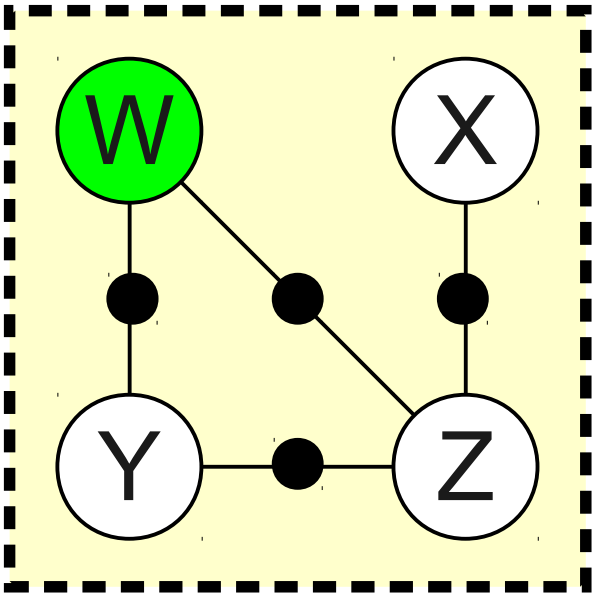


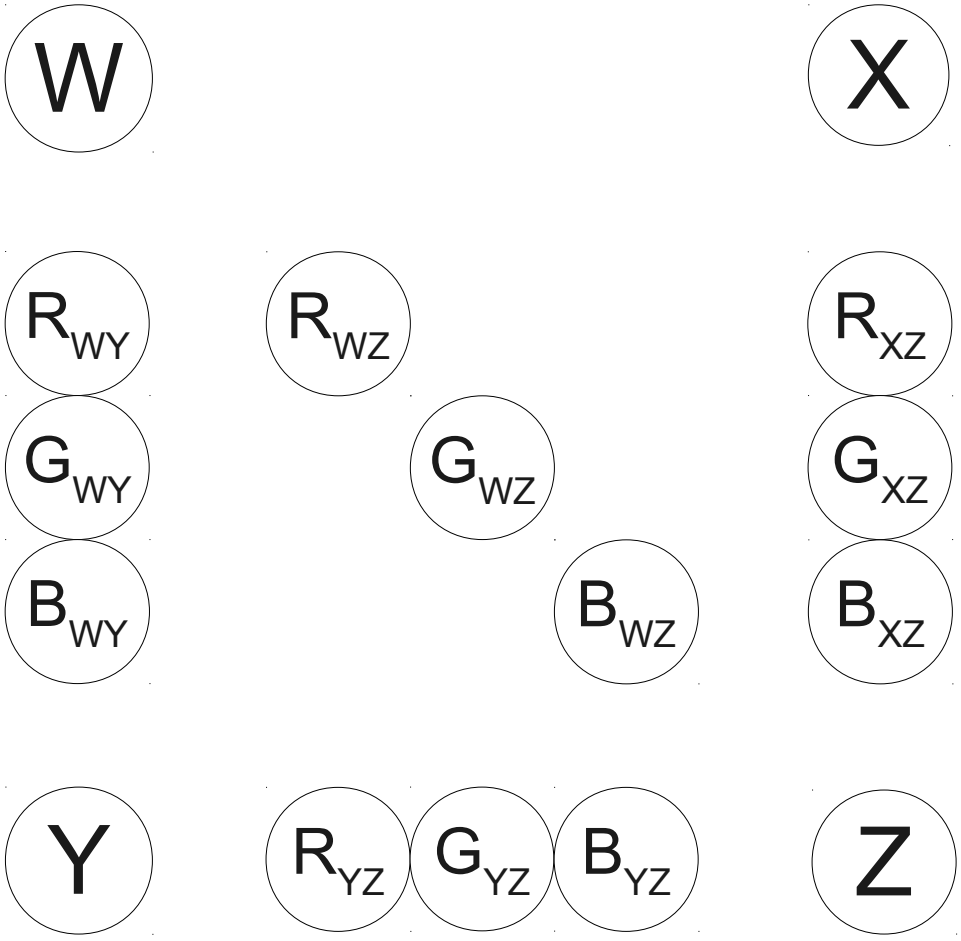
$\{ W, R_{WY}, R_{WZ} \}$	$\{ R_{WY} \}$
$\{ W, G_{WY}, G_{WZ} \}$	$\{ R_{WZ} \}$
$\{ W, B_{WY}, B_{WZ} \}$	$\{ R_{XZ} \}$
$\{ X, R_{XZ} \}$	$\{ R_{YZ} \}$
$\{ X, G_{XZ} \}$	$\{ G_{WY} \}$
$\{ X, B_{XZ} \}$	$\{ G_{WZ} \}$
$\{ Y, R_{WY}, R_{YZ} \}$	$\{ G_{XZ} \}$
$\{ Y, G_{WY}, G_{YZ} \}$	$\{ G_{YZ} \}$
$\{ Y, B_{WY}, B_{YZ} \}$	$\{ B_{WY} \}$
$\{ Z, R_{WZ}, R_{XZ}, R_{YZ} \}$	$\{ B_{WZ} \}$
$\{ Z, G_{WZ}, G_{XZ}, G_{YZ} \}$	$\{ B_{XZ} \}$
$\{ Z, B_{WZ}, B_{XZ}, B_{YZ} \}$	$\{ B_{YZ} \}$



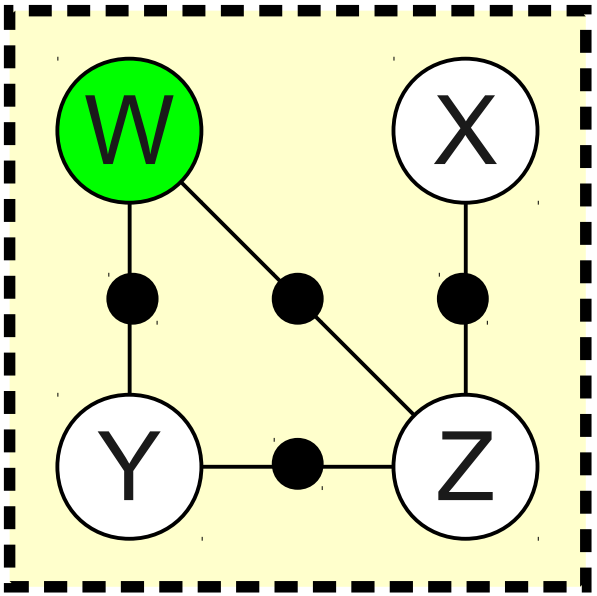


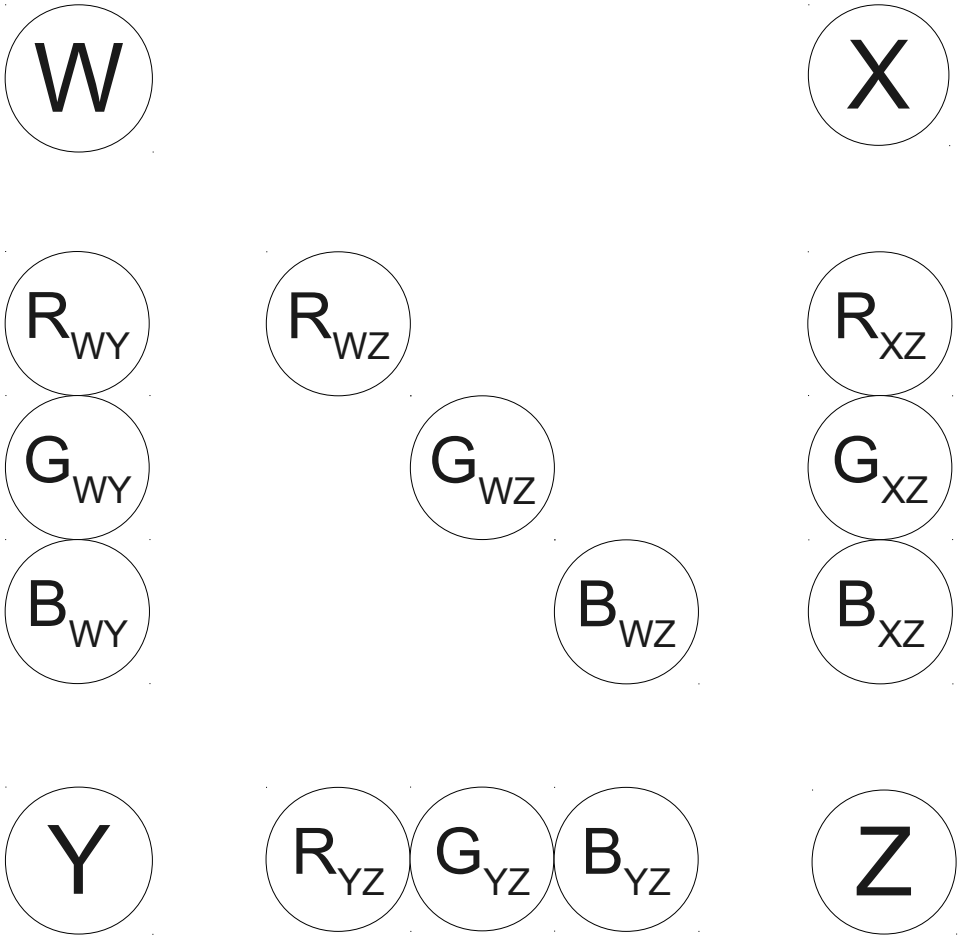
$\{ W, R_{WY}, R_{WZ} \}$ $\{ R_{WY} \}$
$\{ W, G_{WY}, G_{WZ} \}$ $\{ R_{WZ} \}$
$\{ W, B_{WY}, B_{WZ} \}$ $\{ R_{XZ} \}$
$\{ X, R_{XZ} \}$ $\{ R_{YZ} \}$
$\{ X, G_{XZ} \}$ $\{ G_{WY} \}$
$\{ X, B_{XZ} \}$ $\{ G_{WZ} \}$
$\{ Y, R_{WY}, R_{YZ} \}$ $\{ G_{XZ} \}$
$\{ Y, G_{WY}, G_{YZ} \}$ $\{ G_{YZ} \}$
$\{ Y, B_{WY}, B_{YZ} \}$ $\{ B_{WY} \}$
$\{ Z, R_{WZ}, R_{XZ}, R_{YZ} \}$ $\{ B_{WZ} \}$
$\{ Z, G_{WZ}, G_{XZ}, G_{YZ} \}$ $\{ B_{XZ} \}$
$\{ Z, B_{WZ}, B_{XZ}, B_{YZ} \}$ $\{ B_{YZ} \}$





$\{ W, R_{WY}, R_{WZ} \}$ $\{ R_{WY} \}$
$\{ W, G_{WY}, G_{WZ} \}$ $\{ R_{WZ} \}$
$\{ W, B_{WY}, B_{WZ} \}$ $\{ R_{XZ} \}$
$\{ X, R_{XZ} \}$ $\{ R_{YZ} \}$
$\{ X, G_{XZ} \}$ $\{ G_{WY} \}$
$\{ X, B_{XZ} \}$ $\{ G_{WZ} \}$
$\{ Y, R_{WY}, R_{YZ} \}$ $\{ G_{XZ} \}$
$\{ Y, G_{WY}, G_{YZ} \}$ $\{ G_{YZ} \}$
$\{ Y, B_{WY}, B_{YZ} \}$ $\{ B_{WY} \}$
$\{ Z, R_{WZ}, R_{XZ}, R_{YZ} \}$ $\{ B_{WZ} \}$
$\{ Z, G_{WZ}, G_{XZ}, G_{YZ} \}$ $\{ B_{XZ} \}$
$\{ Z, B_{WZ}, B_{XZ}, B_{YZ} \}$ $\{ B_{YZ} \}$





$\{ W, R_{WY}, R_{WZ} \}$ $\{ R_{WY} \}$

$\{ W, G_{WY}, G_{WZ} \}$ $\{ R_{WZ} \}$

$\{ W, B_{WY}, B_{WZ} \}$ $\{ R_{XZ} \}$

$\{ X, R_{XZ} \}$ $\{ R_{YZ} \}$

$\{ X, G_{XZ} \}$ $\{ G_{WY} \}$

$\{ X, B_{XZ} \}$ $\{ G_{WZ} \}$

$\{ Y, R_{WY}, R_{YZ} \}$ $\{ G_{XZ} \}$

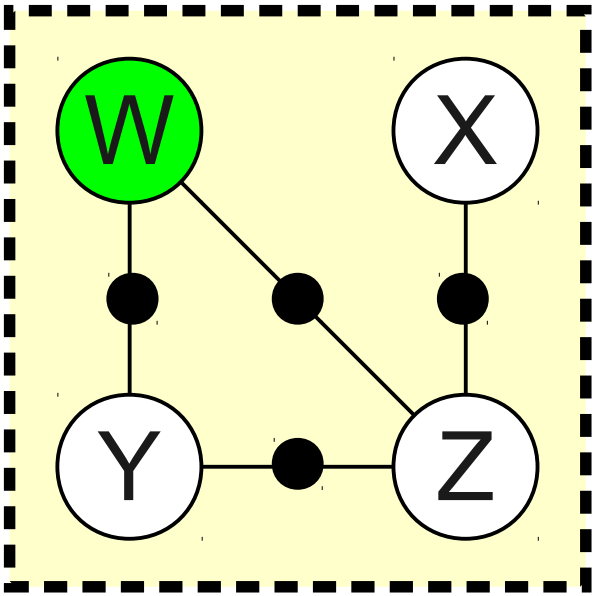
$\{ Y, G_{WY}, G_{YZ} \}$ $\{ G_{YZ} \}$

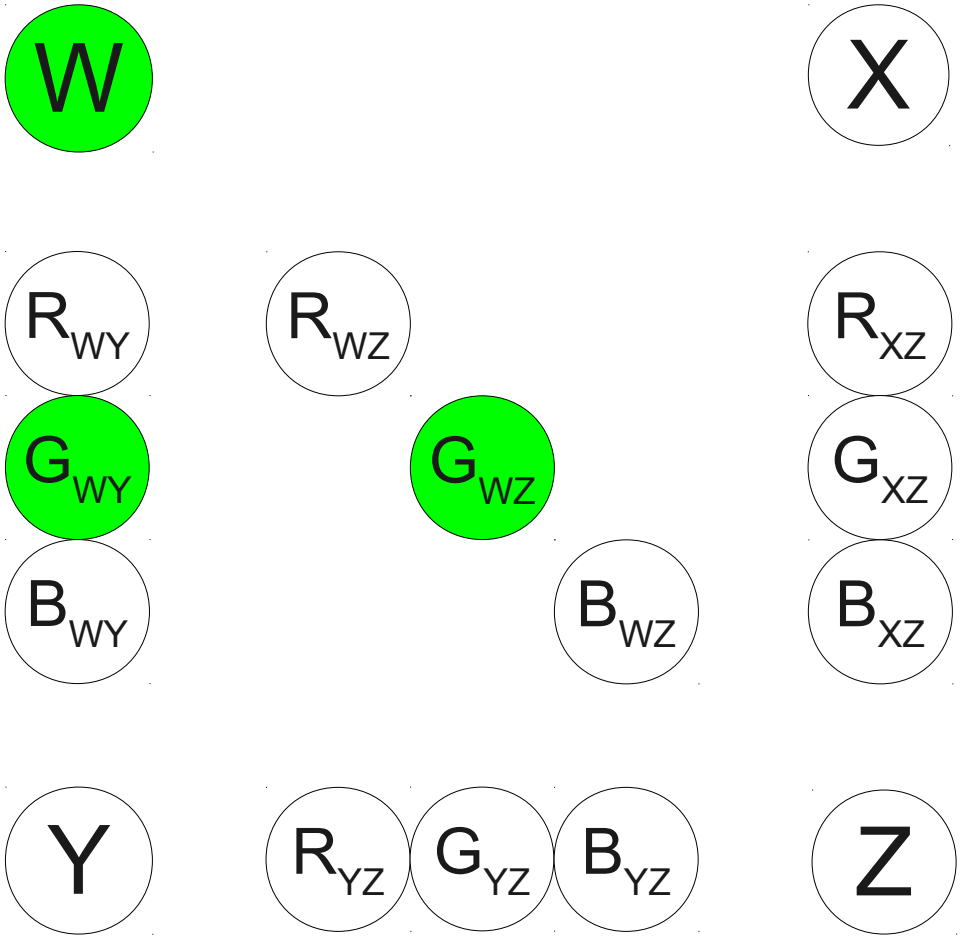
$\{ Y, B_{WY}, B_{YZ} \}$ $\{ B_{WY} \}$

$\{ Z, R_{WZ}, R_{XZ}, R_{YZ} \}$ $\{ B_{WZ} \}$

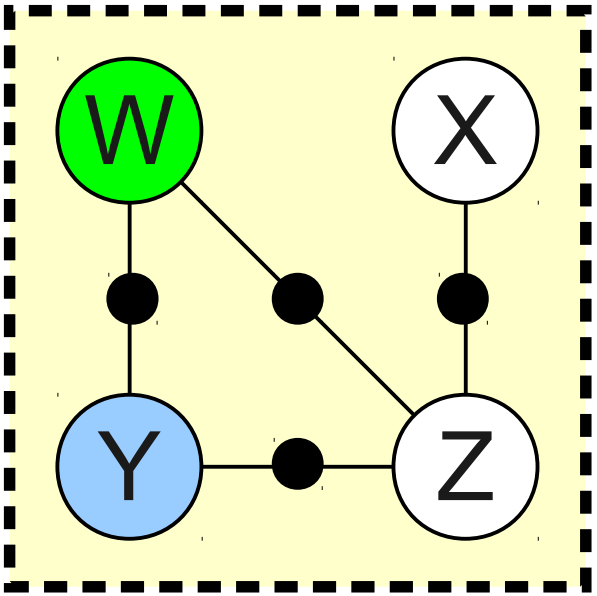
$\{ Z, G_{WZ}, G_{XZ}, G_{YZ} \}$ $\{ B_{XZ} \}$

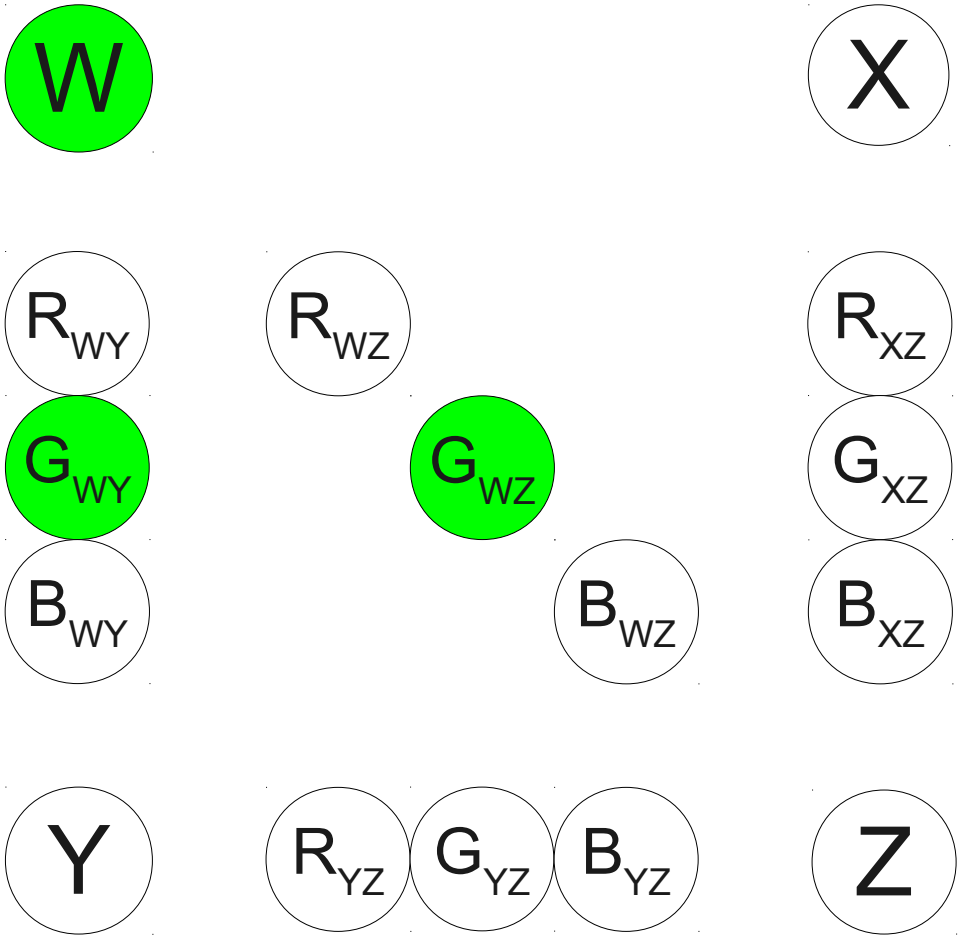
$\{ Z, B_{WZ}, B_{XZ}, B_{YZ} \}$ $\{ B_{YZ} \}$





{ W, R_{WY}, R_{WZ} } { R_{WY} }
{ W, G_{WY}, G_{WZ} } { R_{WZ} }
{ W, B_{WY}, B_{WZ} } { R_{XZ} }
{ X, R_{XZ} } { R_{YZ} }
{ X, G_{XZ} } { G_{WY} }
{ X, B_{XZ} } { G_{WZ} }
{ Y, R_{WY}, R_{YZ} } { G_{XZ} }
{ Y, G_{WY}, G_{YZ} } { G_{YZ} }
{ Y, B_{WY}, B_{YZ} } { B_{WY} }
{ Z, R_{WZ}, R_{XZ}, R_{YZ} } { B_{WZ} }
{ Z, G_{WZ}, G_{XZ}, G_{YZ} } { B_{XZ} }
{ Z, B_{WZ}, B_{XZ}, B_{YZ} } { B_{YZ} }





$\{ W, R_{WY}, R_{WZ} \}$ $\{ R_{WY} \}$

$\{ W, G_{WY}, G_{WZ} \}$ $\{ R_{WZ} \}$

$\{ W, B_{WY}, B_{WZ} \}$ $\{ R_{XZ} \}$

$\{ X, R_{XZ} \}$ $\{ R_{YZ} \}$

$\{ X, G_{XZ} \}$ $\{ G_{WY} \}$

$\{ X, B_{XZ} \}$ $\{ G_{WZ} \}$

$\{ Y, R_{WY}, R_{YZ} \}$ $\{ G_{XZ} \}$

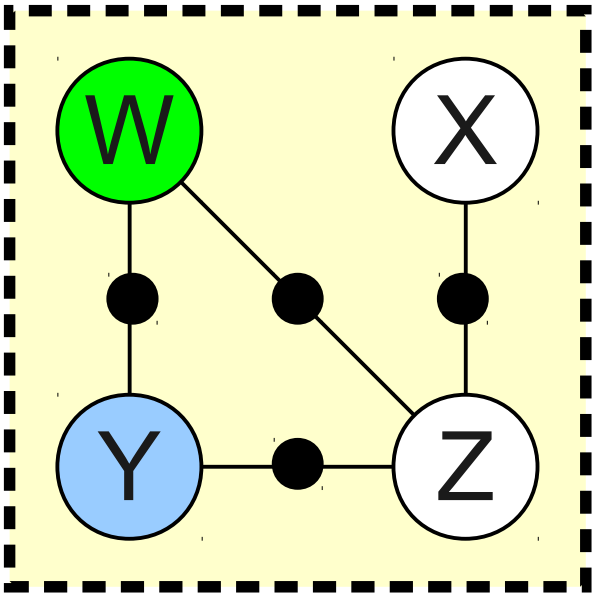
$\{ Y, G_{WY}, G_{YZ} \}$ $\{ G_{YZ} \}$

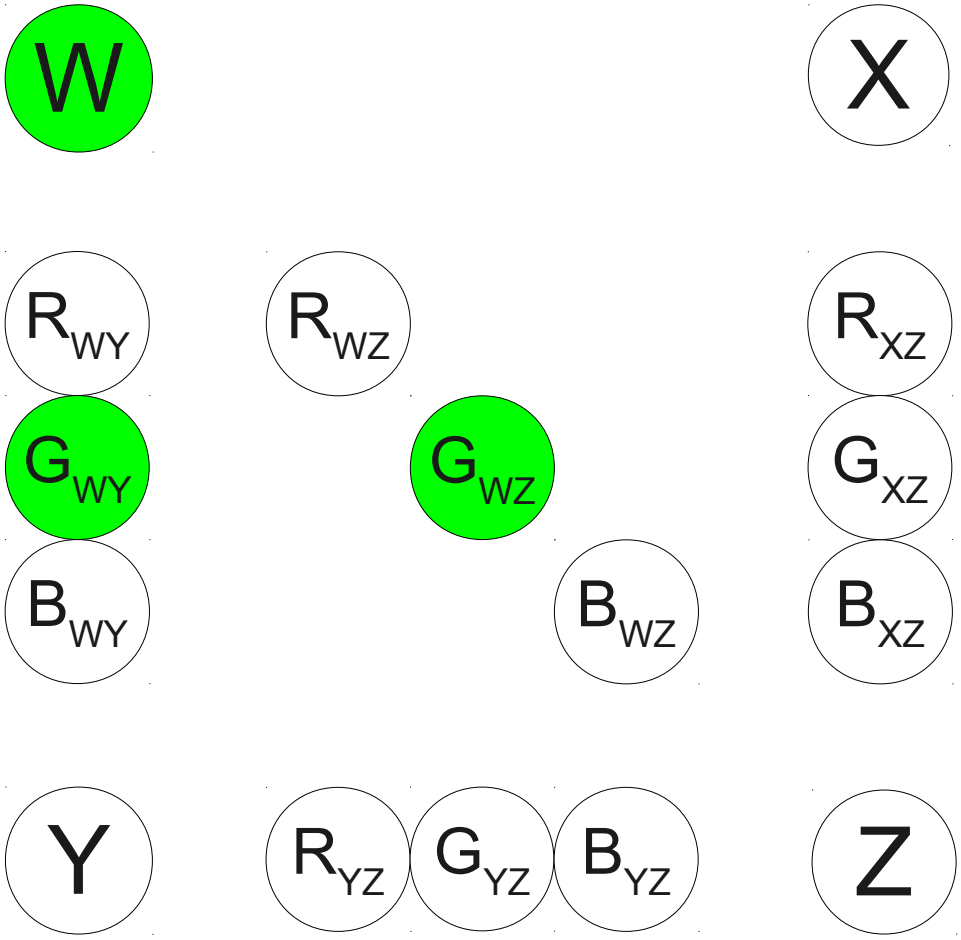
$\{ Y, B_{WY}, B_{YZ} \}$ $\{ B_{WY} \}$

$\{ Z, R_{WZ}, R_{XZ}, R_{YZ} \}$ $\{ B_{WZ} \}$

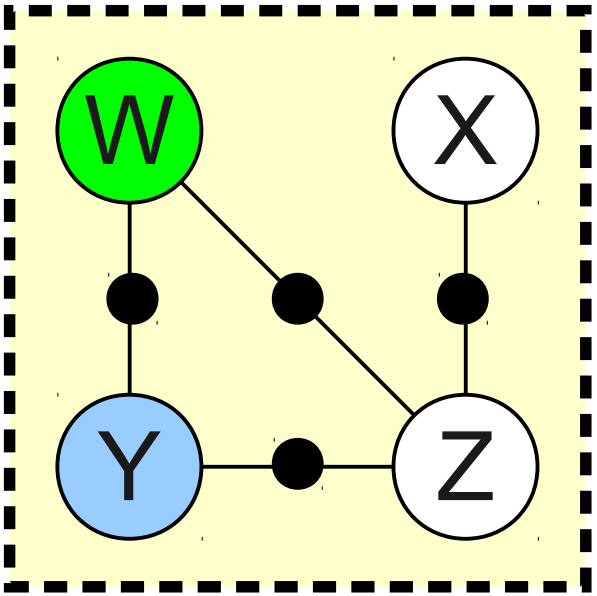
$\{ Z, G_{WZ}, G_{XZ}, G_{YZ} \}$ $\{ B_{XZ} \}$

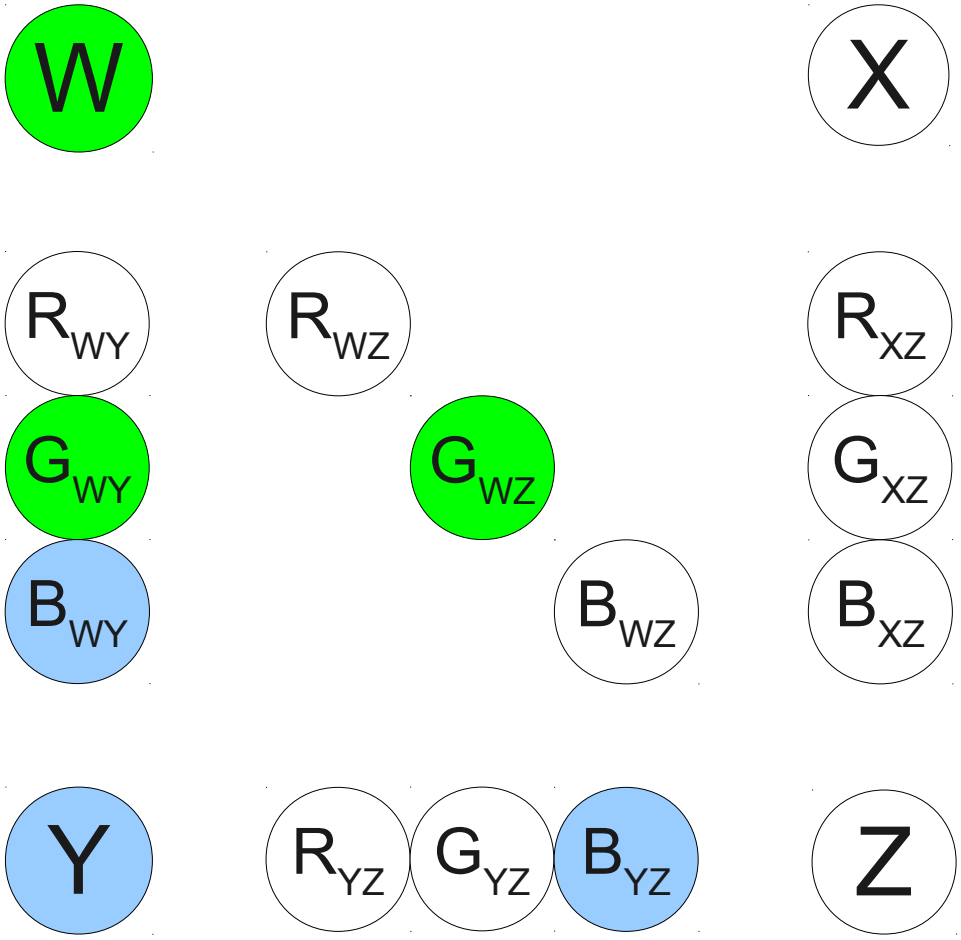
$\{ Z, B_{WZ}, B_{XZ}, B_{YZ} \}$ $\{ B_{YZ} \}$



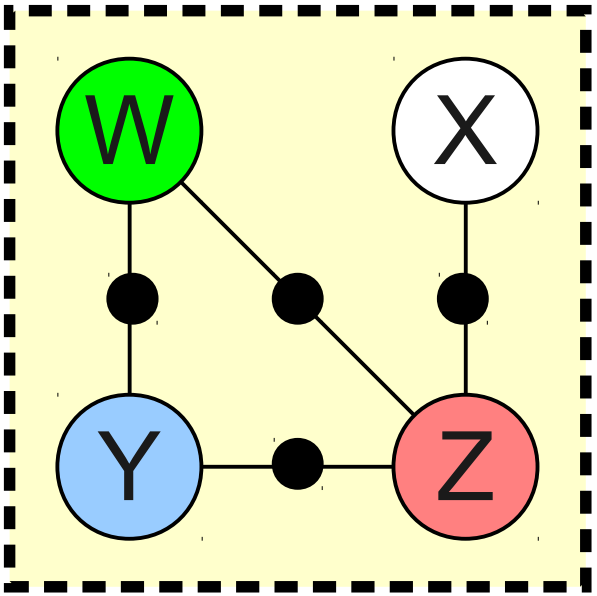


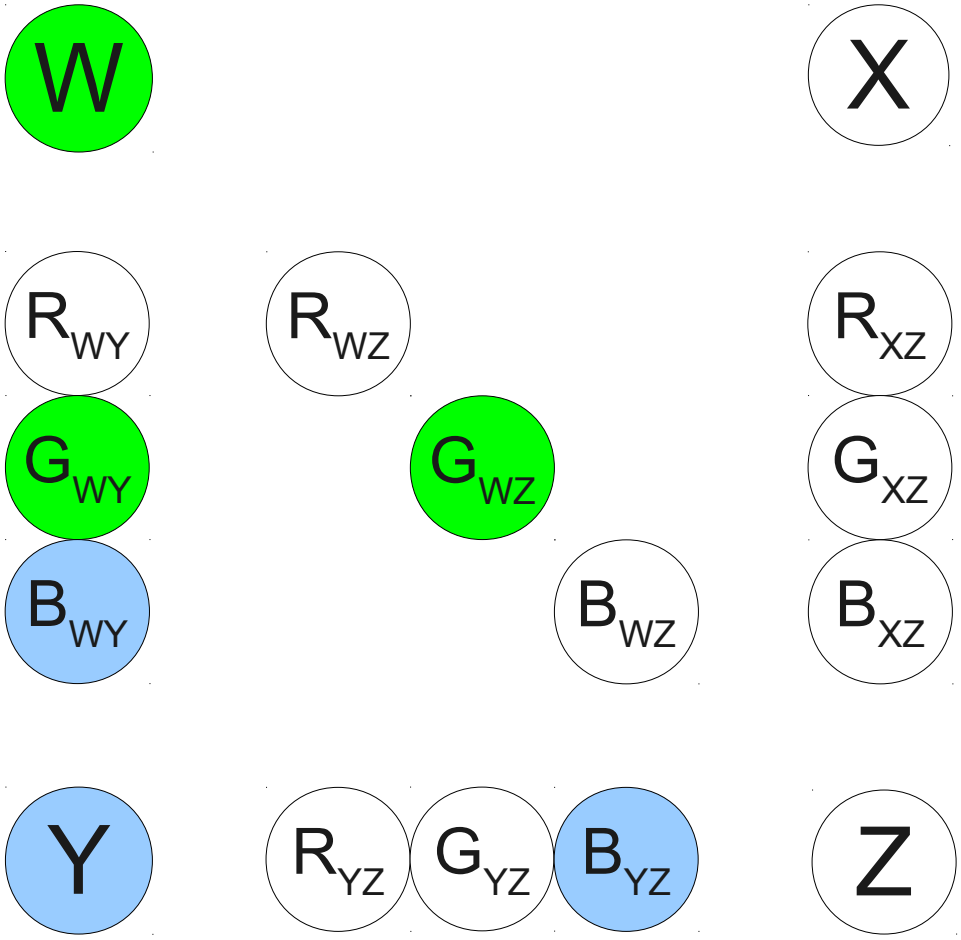
{ W, R_{WY}, R_{WZ} } { R_{WY} }
{ W, G_{WY}, G_{WZ} } { R_{WZ} }
{ W, B_{WY}, B_{WZ} } { R_{XZ} }
{ X, R_{XZ} } { R_{YZ} }
{ X, G_{XZ} } { G_{WY} }
{ X, B_{XZ} } { G_{WZ} }
{ Y, R_{WY}, R_{YZ} } { G_{XZ} }
{ Y, G_{WY}, G_{YZ} } { G_{YZ} }
{ Y, B_{WY}, B_{YZ} } { B_{WY} }
{ Z, R_{WZ}, R_{XZ}, R_{YZ} } { B_{WZ} }
{ Z, G_{WZ}, G_{XZ}, G_{YZ} } { B_{XZ} }
{ Z, B_{WZ}, B_{XZ}, B_{YZ} } { B_{YZ} }



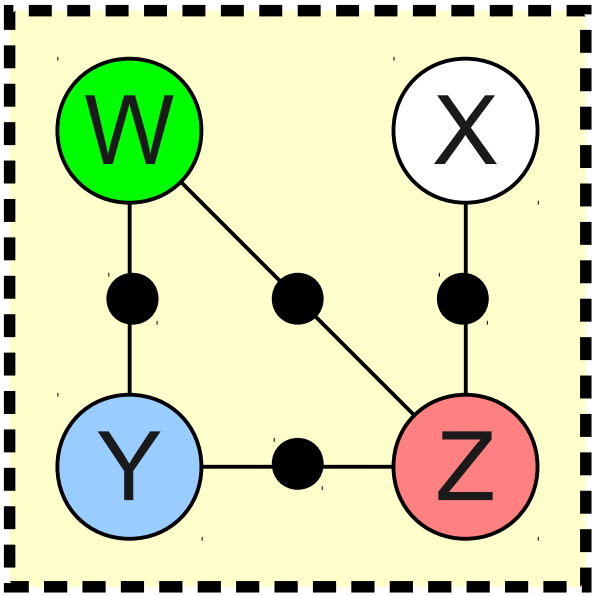


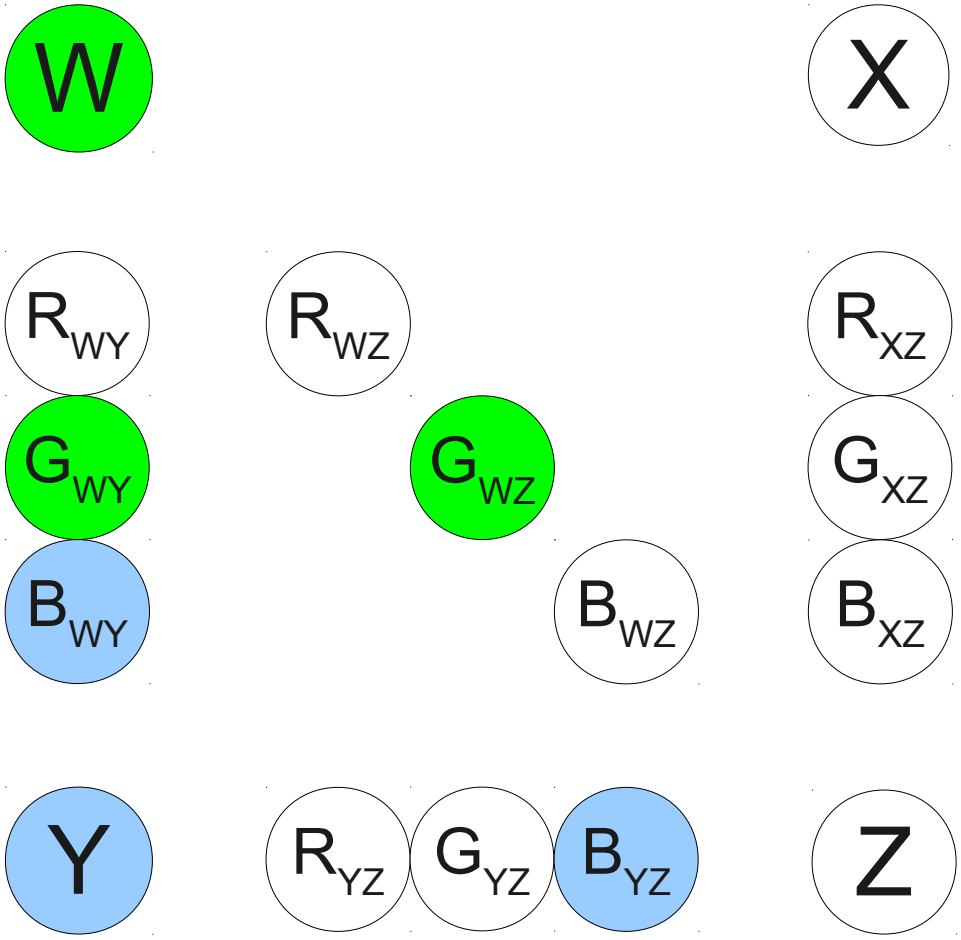
{ W, R_{WY}, R_{WZ} } { R_{WY} }
{ W, G_{WY}, G_{WZ} } { R_{WZ} }
{ W, B_{WY}, B_{WZ} } { R_{XZ} }
{ X, R_{XZ} } { R_{YZ} }
{ X, G_{XZ} } { G_{WY} }
{ X, B_{XZ} } { G_{WZ} }
{ Y, R_{WY}, R_{YZ} } { G_{XZ} }
{ Y, G_{WY}, G_{YZ} } { G_{YZ} }
{ Y, B_{WY}, B_{YZ} } { B_{WY} }
{ Z, R_{WZ}, R_{XZ}, R_{YZ} } { B_{WZ} }
{ Z, G_{WZ}, G_{XZ}, G_{YZ} } { B_{XZ} }
{ Z, B_{WZ}, B_{XZ}, B_{YZ} } { B_{YZ} }



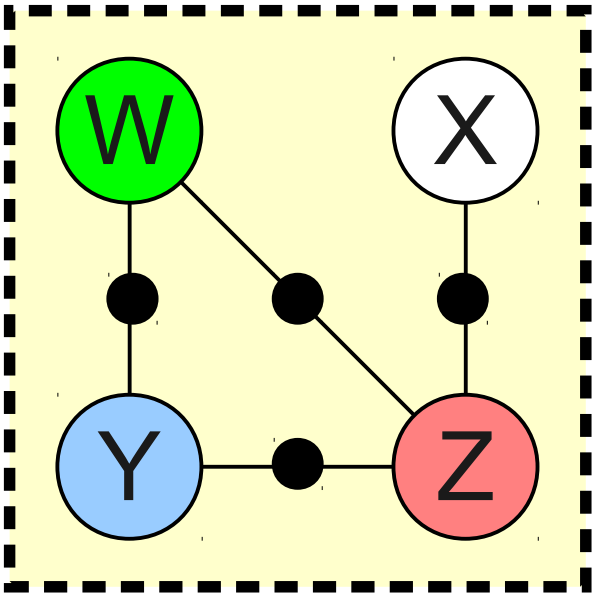


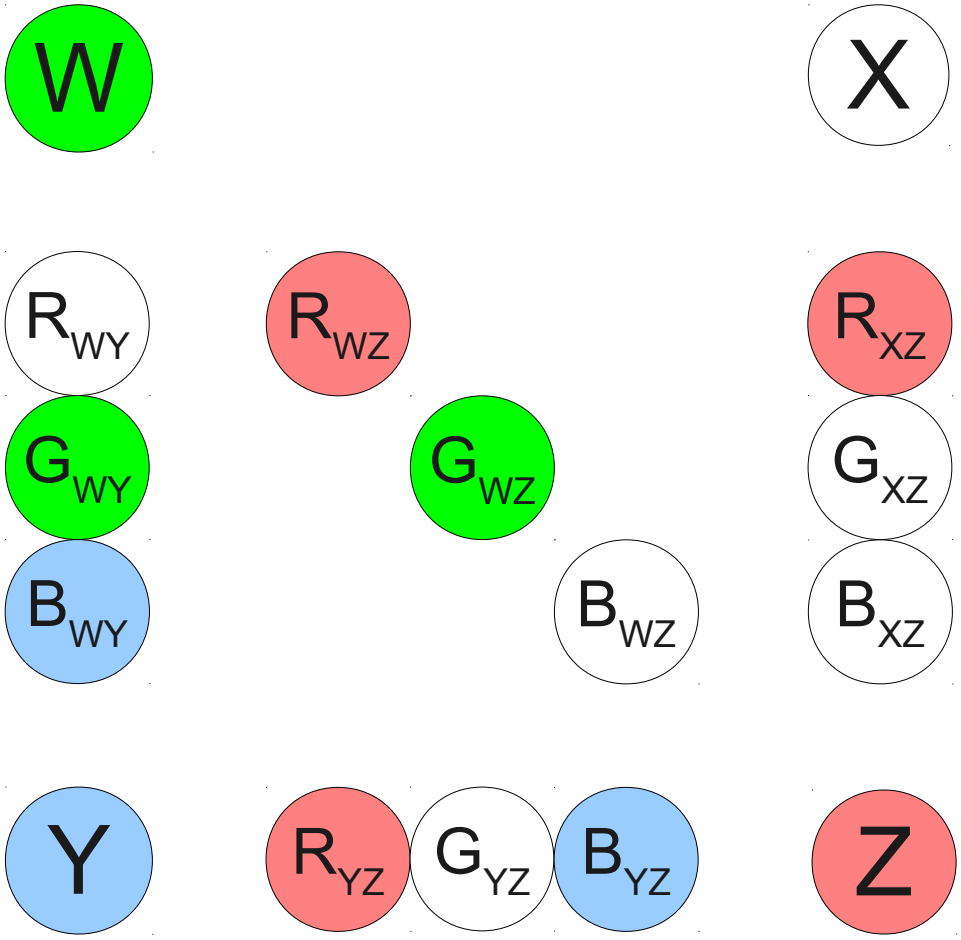
{ W, R_{WY}, R_{WZ} } { R_{WY} }
{ W, G_{WY}, G_{WZ} } { R_{WZ} }
{ W, B_{WY}, B_{WZ} } { R_{XZ} }
{ X, R_{XZ} } { R_{YZ} }
{ X, G_{XZ} } { G_{WY} }
{ X, B_{XZ} } { G_{WZ} }
{ Y, R_{WY}, R_{YZ} } { G_{XZ} }
{ Y, G_{WY}, G_{YZ} } { G_{YZ} }
{ Y, B_{WY}, B_{YZ} } { B_{WY} }
{ Z, R_{WZ}, R_{XZ}, R_{YZ} } { B_{WZ} }
{ Z, G_{WZ}, G_{XZ}, G_{YZ} } { B_{XZ} }
{ Z, B_{WZ}, B_{XZ}, B_{YZ} } { B_{YZ} }



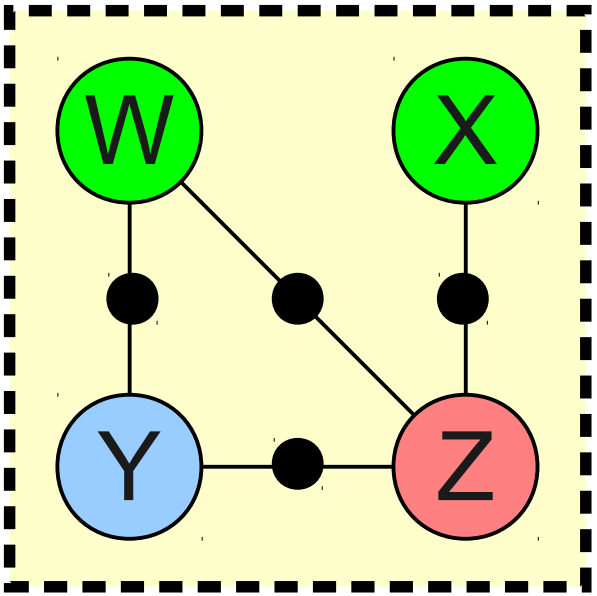


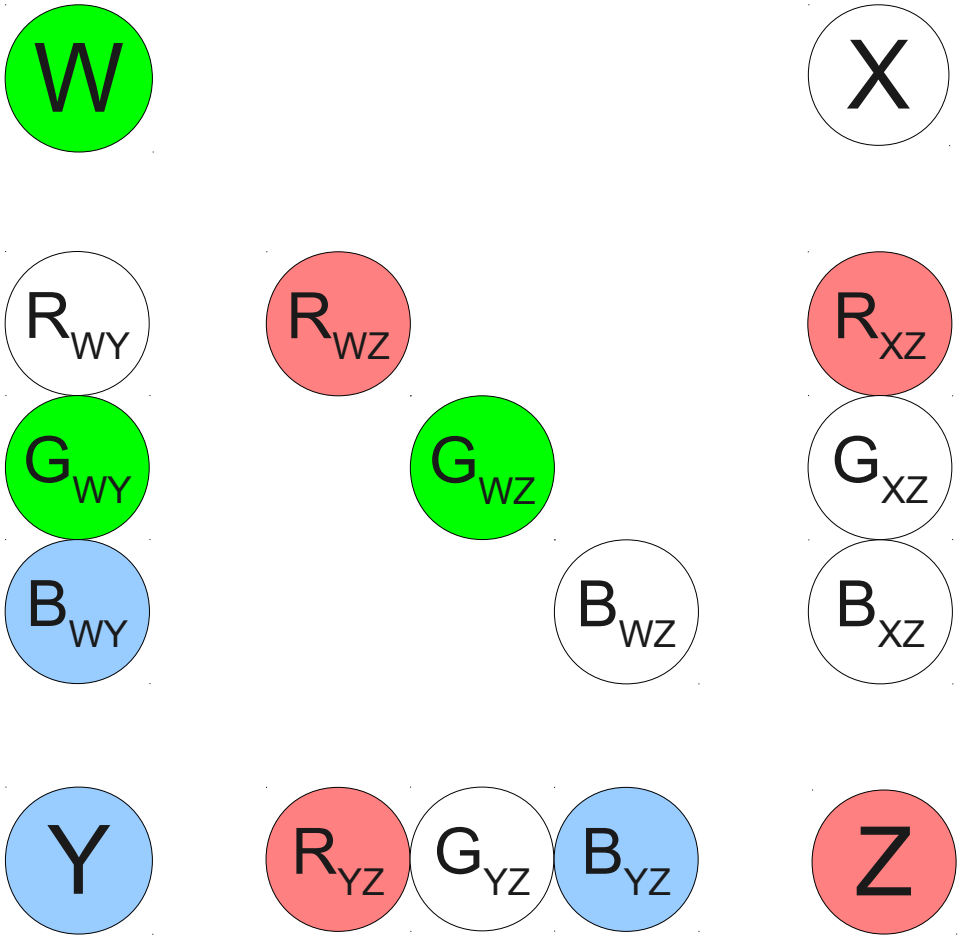
{ W, R_{WY}, R_{WZ} } { R_{WY} }
{ W, G_{WY}, G_{WZ} } { R_{WZ} }
{ W, B_{WY}, B_{WZ} } { R_{XZ} }
{ X, R_{XZ} } { R_{YZ} }
{ X, G_{XZ} } { G_{WY} }
{ X, B_{XZ} } { G_{WZ} }
{ Y, R_{WY}, R_{YZ} } { G_{XZ} }
{ Y, G_{WY}, G_{YZ} } { G_{YZ} }
{ Y, B_{WY}, B_{YZ} } { B_{WY} }
{ Z, R_{WZ}, R_{XZ}, R_{YZ} } { B_{WZ} }
{ Z, G_{WZ}, G_{XZ}, G_{YZ} } { B_{XZ} }
{ Z, B_{WZ}, B_{XZ}, B_{YZ} } { B_{YZ} }



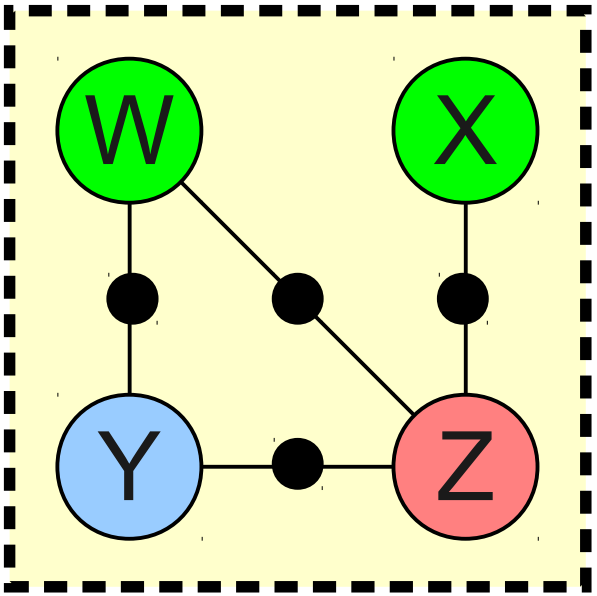


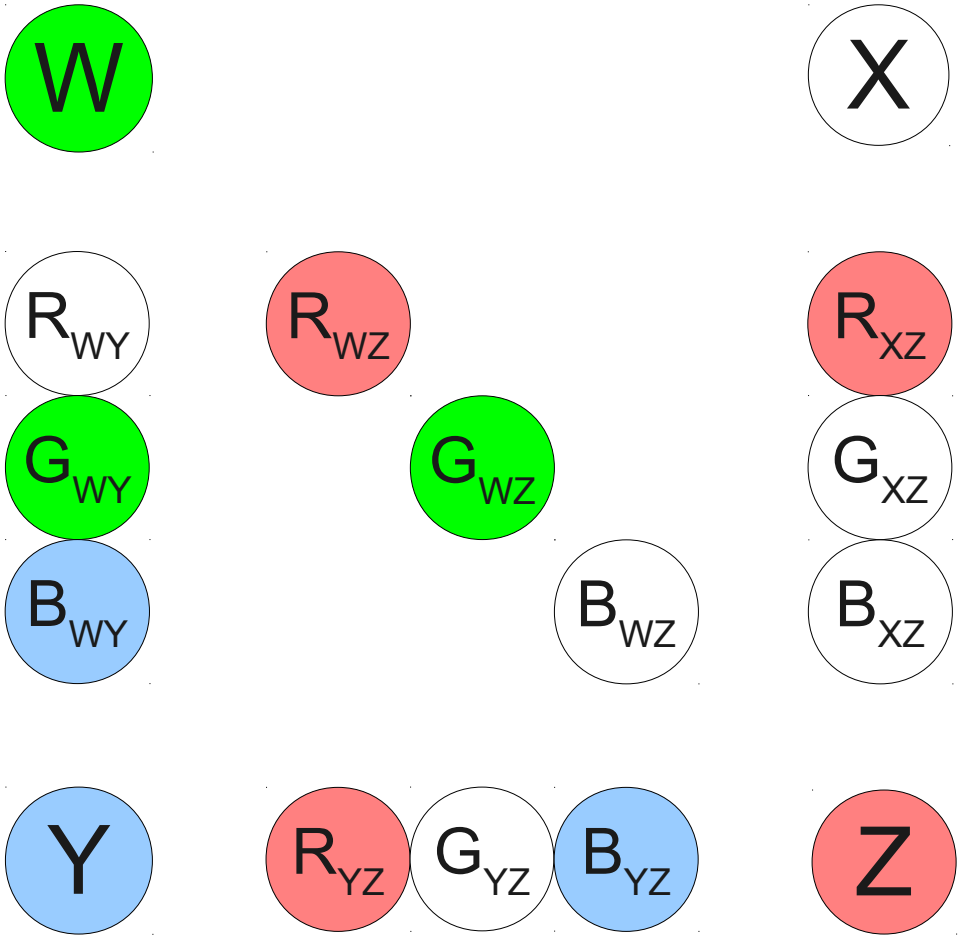
{ W, R_{WY}, R_{WZ} } { R_{WY} }
{ W, G_{WY}, G_{WZ} } { R_{WZ} }
{ W, B_{WY}, B_{WZ} } { R_{XZ} }
{ X, R_{XZ} } { R_{YZ} }
{ X, G_{XZ} } { G_{WY} }
{ X, B_{XZ} } { G_{WZ} }
{ Y, R_{WY}, R_{YZ} } { G_{XZ} }
{ Y, G_{WY}, G_{YZ} } { G_{YZ} }
{ Y, B_{WY}, B_{YZ} } { B_{WY} }
{ Z, R_{WZ}, R_{XZ}, R_{YZ} } { B_{WZ} }
{ Z, G_{WZ}, G_{XZ}, G_{YZ} } { B_{XZ} }
{ Z, B_{WZ}, B_{XZ}, B_{YZ} } { B_{YZ} }





{ W, R_{WY}, R_{WZ} } { R_{WY} }
{ W, G_{WY}, G_{WZ} } { R_{WZ} }
{ W, B_{WY}, B_{WZ} } { R_{XZ} }
{ X, R_{XZ} } { R_{YZ} }
{ X, G_{XZ} } { G_{WY} }
{ X, B_{XZ} } { G_{WZ} }
{ Y, R_{WY}, R_{YZ} } { G_{XZ} }
{ Y, G_{WY}, G_{YZ} } { G_{YZ} }
{ Y, B_{WY}, B_{YZ} } { B_{WY} }
{ Z, R_{WZ}, R_{XZ}, R_{YZ} } { B_{WZ} }
{ Z, G_{WZ}, G_{XZ}, G_{YZ} } { B_{XZ} }
{ Z, B_{WZ}, B_{XZ}, B_{YZ} } { B_{YZ} }





$\{ W, R_{WY}, R_{WZ} \}$ $\{ R_{WY} \}$

$\{ W, G_{WY}, G_{WZ} \}$ $\{ R_{WZ} \}$

$\{ W, B_{WY}, B_{WZ} \}$ $\{ R_{XZ} \}$

$\{ X, R_{XZ} \}$ $\{ R_{YZ} \}$

$\{ X, G_{XZ} \}$ $\{ G_{WY} \}$

$\{ X, B_{XZ} \}$ $\{ G_{WZ} \}$

$\{ Y, R_{WY}, R_{YZ} \}$ $\{ G_{XZ} \}$

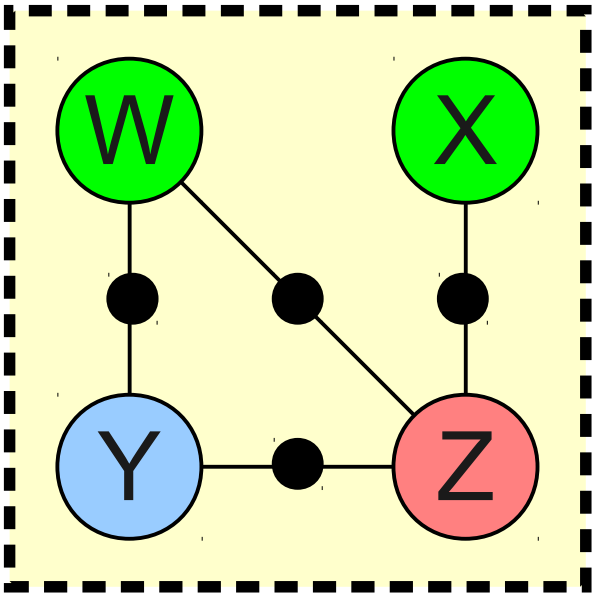
$\{ Y, G_{WY}, G_{YZ} \}$ $\{ G_{YZ} \}$

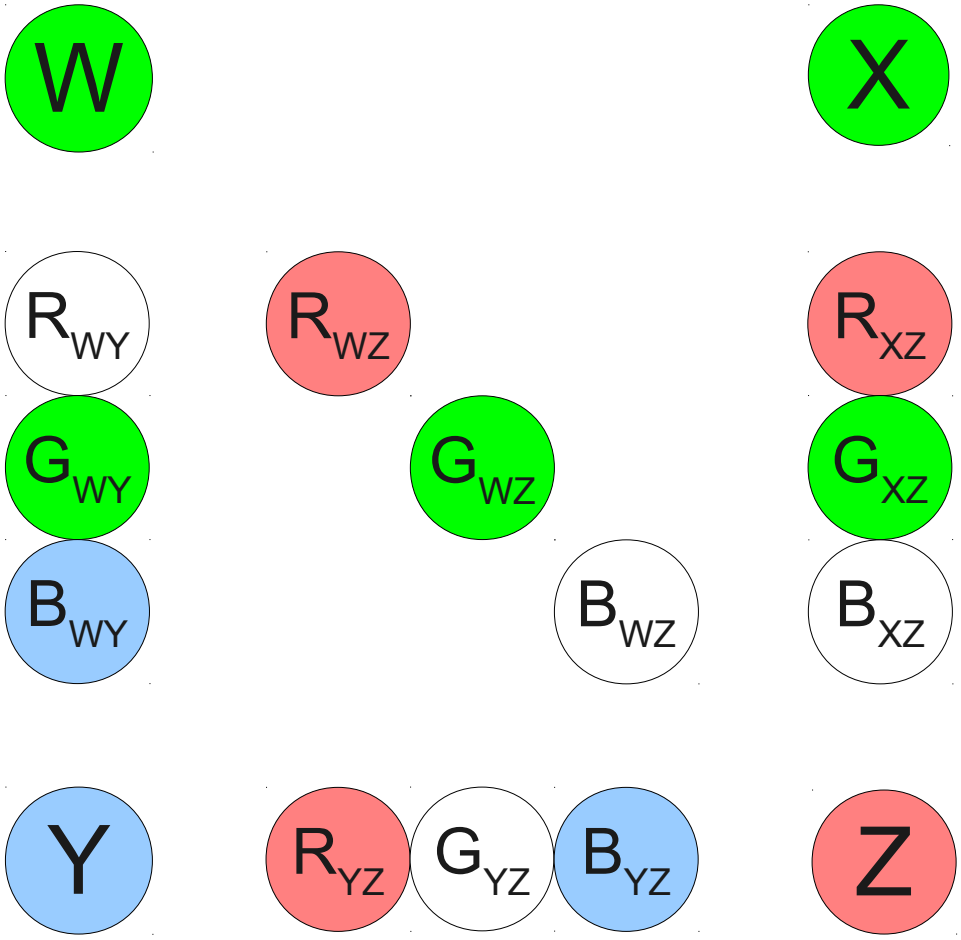
$\{ Y, B_{WY}, B_{YZ} \}$ $\{ B_{WY} \}$

$\{ Z, R_{WZ}, R_{XZ}, R_{YZ} \}$ $\{ B_{WZ} \}$

$\{ Z, G_{WZ}, G_{XZ}, G_{YZ} \}$ $\{ B_{XZ} \}$

$\{ Z, B_{WZ}, B_{XZ}, B_{YZ} \}$ $\{ B_{YZ} \}$





$\{ W, R_{WY}, R_{WZ} \}$ $\{ R_{WY} \}$

$\{ W, G_{WY}, G_{WZ} \}$ $\{ R_{WZ} \}$

$\{ W, B_{WY}, B_{WZ} \}$ $\{ R_{XZ} \}$

$\{ X, R_{XZ} \}$ $\{ R_{YZ} \}$

$\{ X, G_{XZ} \}$ $\{ G_{WY} \}$

$\{ X, B_{XZ} \}$ $\{ G_{WZ} \}$

$\{ Y, R_{WY}, R_{YZ} \}$ $\{ G_{XZ} \}$

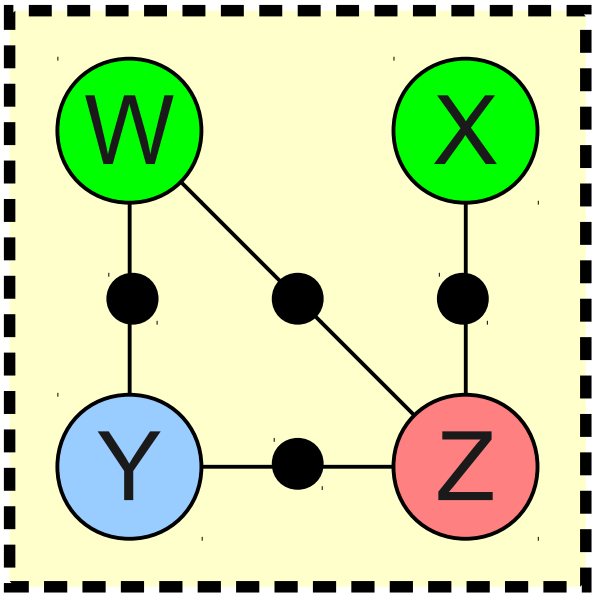
$\{ Y, G_{WY}, G_{YZ} \}$ $\{ G_{YZ} \}$

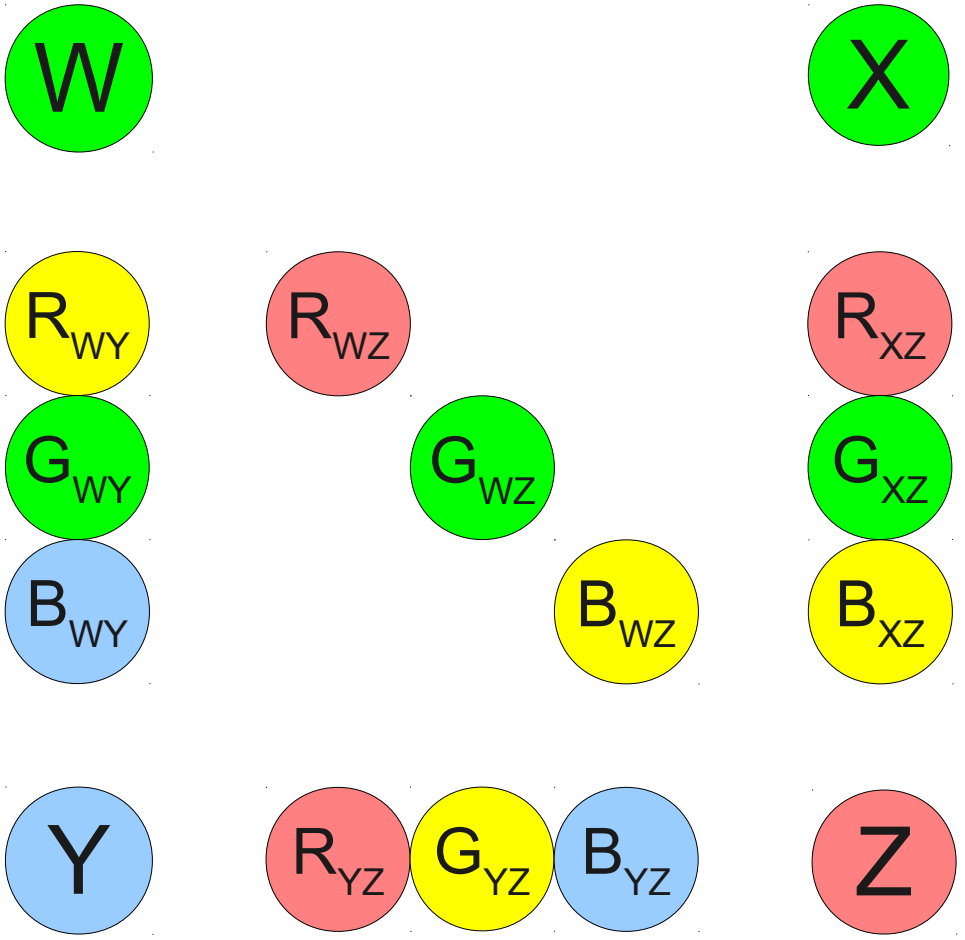
$\{ Y, B_{WY}, B_{YZ} \}$ $\{ B_{WY} \}$

$\{ Z, R_{WZ}, R_{XZ}, R_{YZ} \}$ $\{ B_{WZ} \}$

$\{ Z, G_{WZ}, G_{XZ}, G_{YZ} \}$ $\{ B_{XZ} \}$

$\{ Z, B_{WZ}, B_{XZ}, B_{YZ} \}$ $\{ B_{YZ} \}$





{ W, R_{WY}, R_{WZ} } { R_{WY} }
{ W, G_{WY}, G_{WZ} } { R_{WZ} }
{ W, B_{WY}, B_{WZ} } { R_{XZ} }
{ X, R_{XZ} } { R_{YZ} }
{ X, G_{XZ} } { G_{WY} }
{ X, B_{XZ} } { G_{WZ} }
{ Y, R_{WY}, R_{YZ} } { G_{XZ} }
{ Y, G_{WY}, G_{YZ} } { G_{YZ} }
{ Y, B_{WY}, B_{YZ} } { B_{WY} }
{ Z, R_{WZ}, R_{XZ}, R_{YZ} } { B_{WZ} }
{ Z, G_{WZ}, G_{XZ}, G_{YZ} } { B_{XZ} }
{ Z, B_{WZ}, B_{XZ}, B_{YZ} } { B_{YZ} }

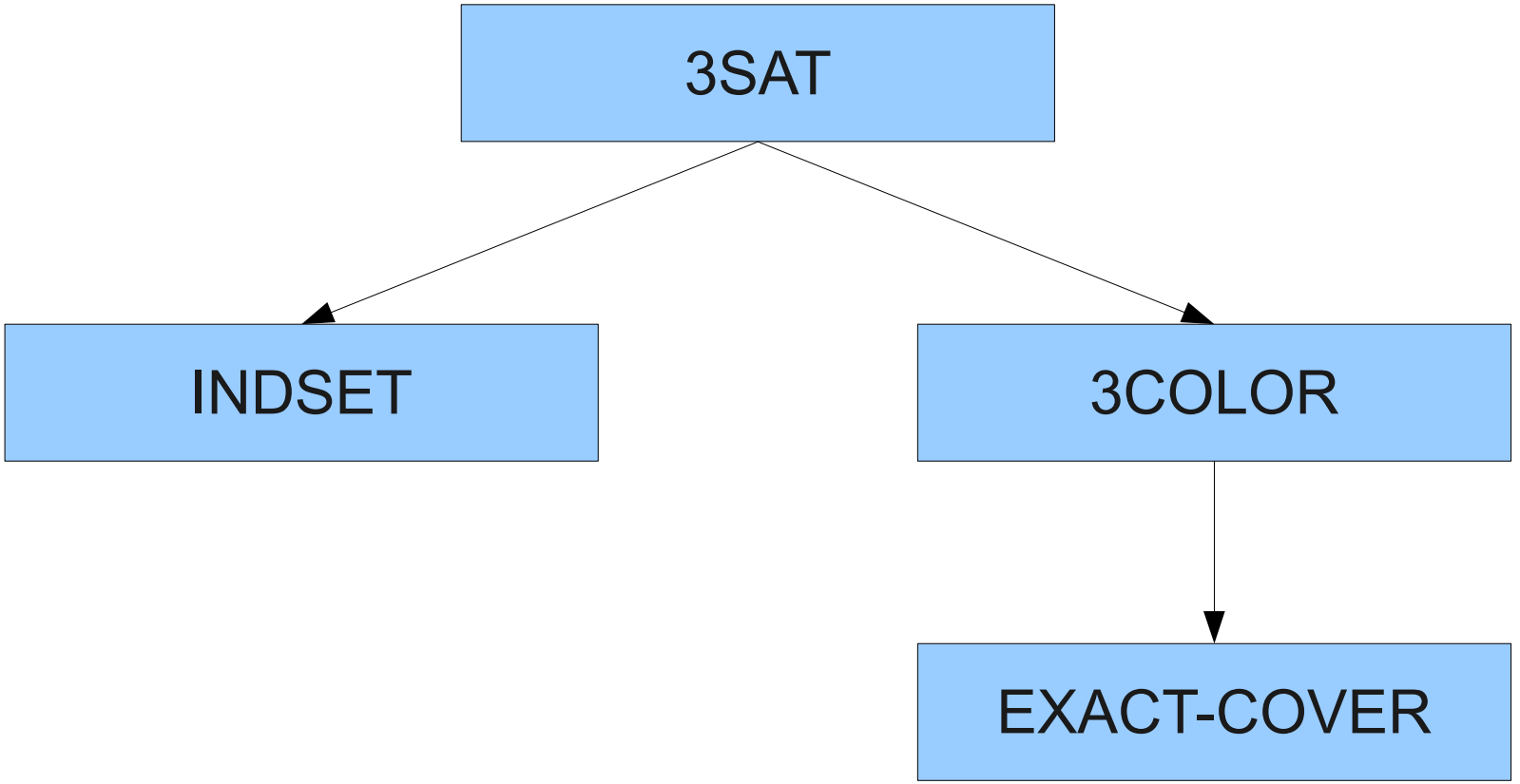
The Construction

- For each node v in graph G, construct four elements in the universe U:
  - An element v.
  - Elements R_v, G_v, and B_v.
- For each edge $\{u, v\}$ in graph G, construct three elements in the universe U:
  - Elements R_{uv}, G_{uv}, B_{uv}
- Total size of the universe U: **$O(|V| + |E|)$**.

The Construction

- For each node v in graph G, construct a set belonging to S containing
  - The element v,
  - Each R_{uv} for each edge $\{u, v\}$ in the graph.
- Repeat the above for colors G and B.
- Add singleton sets containing each individual element except for elements corresponding to nodes.
- Total size of all sets is $\mathbf{O(|V| + |E|)}$
  - Counts each node three times and each edge six times.

The Story So Far



Another **NP**-Complete Problem

Given a set $S \subseteq \mathbb{N}$ and a natural number k, the **subset sum problem** is to find a subset of S whose sum is exactly k.

$$\{\ 137, 42, 271, 103, 154, 16, 3\ \}$$

Given a set $S \subseteq \mathbb{N}$ and a natural number k, the **subset sum problem** is to find a subset of S whose sum is exactly k.

$$\{\ 137, 42, 271, 103, 154, 16, 3\ \}$$

$$k = 452$$

Given a set $S \subseteq \mathbb{N}$ and a natural number k, the **subset sum problem** is to find a subset of S whose sum is exactly k.

$$\{\ 137, 42, 271, 103, 154, 16, 3\ \}$$

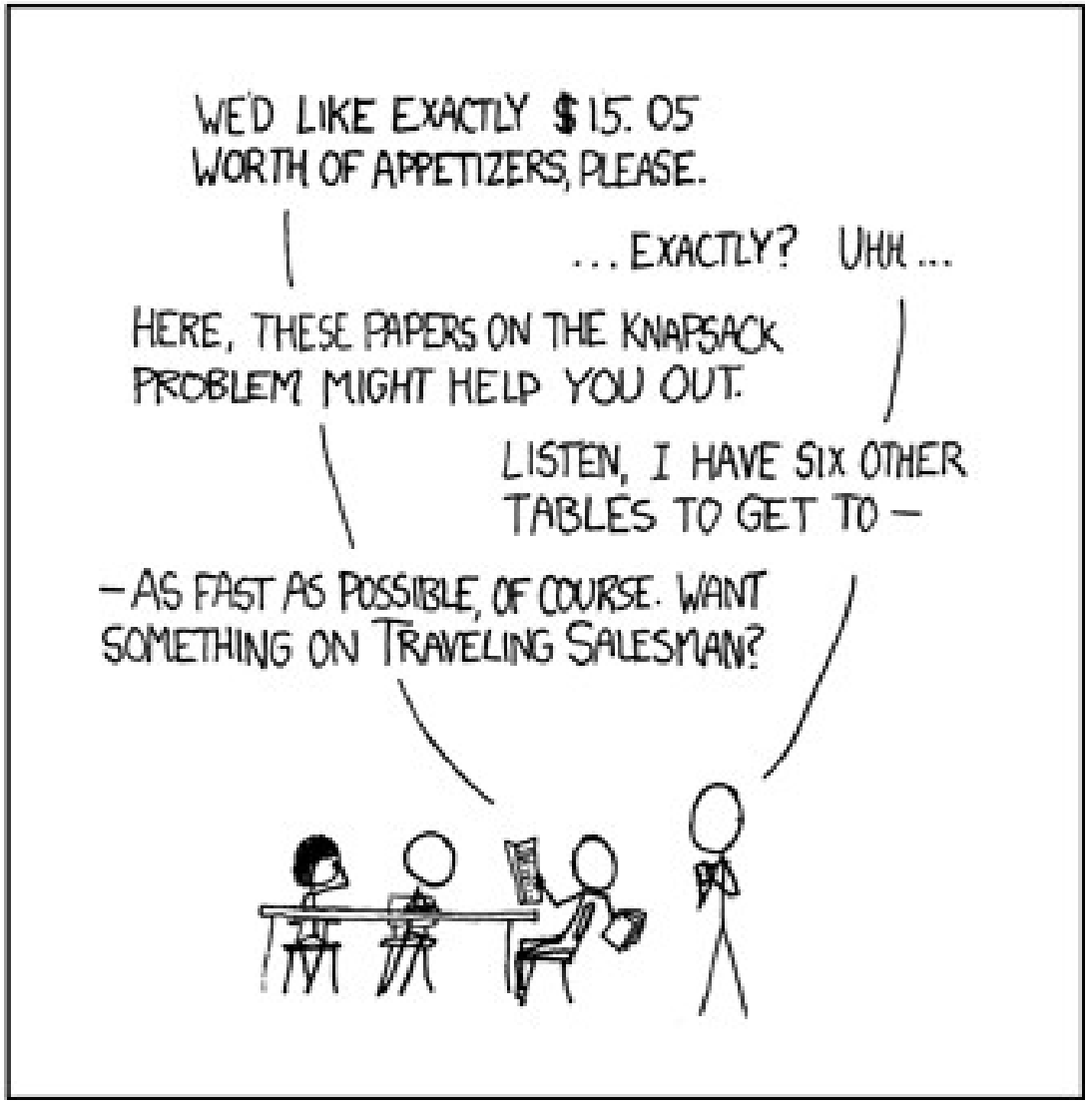
$$k = 452$$

Given a set $S \subseteq \mathbb{N}$ and a natural number k, the **subset sum problem** is to find a subset of S whose sum is exactly k.

My Hobby:

Embedding NP-complete problems in restaurant orders





http://xkcd.com/287/

Subset Sum

- Given a set $S \subseteq \mathbb{N}$ and a natural number k, the subset sum problem is

  Is there a subset of S with sum exactly k?

- As a formal language:

  SUBSET-SUM =
  $\{ \langle S, k \rangle \mid S \subseteq \mathbb{N}, k \in \mathbb{N}$ and there is a subset of S with sum exactly k? $\}$

SUBSET-SUM ∈ **NPC**

- We will prove that *SUBSET-SUM* is **NP**-complete.
- To do this, we will show that
  - *SUBSET-SUM* ∈ **NP**, and
  - *EXACT-COVER* $\leq_P$ *SUBSET-SUM*
- Again, we're using our new **NP**-complete problem to show other languages are **NP**-complete.

SUBSET-SUM ∈ **NP**

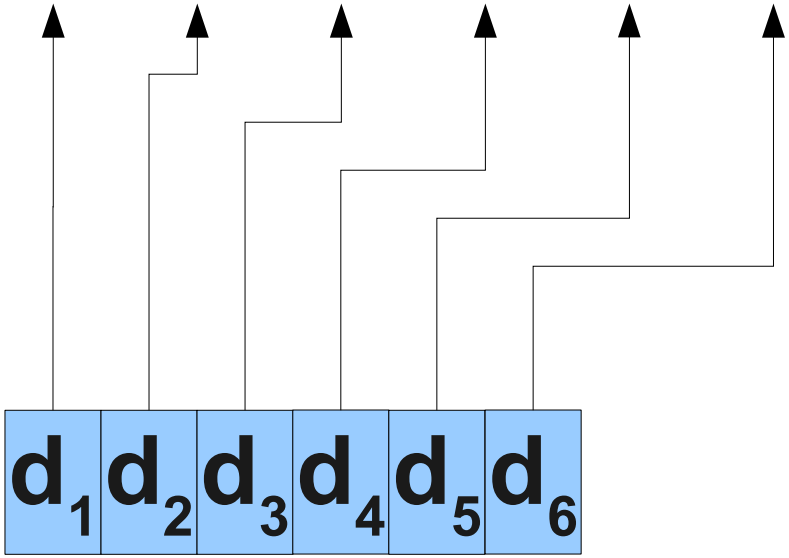
- Here is a nondeterministic polynomial-time algorithm for *SUBSET-SUM*
- N = "On input $\langle S, k \rangle$:
  - **Nondeterministically** guess a subset $I \subseteq S$.
  - **Deterministically** verify whether the sum of the elements of I is equal to k.
  - If so, accept; otherwise reject."

$\mathit{EXACT\text{-}COVER} \leq_{\mathrm{P}} \mathit{SUBSET\text{-}SUM}$

- We now reduce exact cover to subset sum.
- The exact cover problem has a solution iff
  - Every element of the universe belongs to at least one set, and
  - Every element of the universe belongs to at most one set.
- We will construct our set S and number k such that
  - Each number corresponds to a set of elements, and
  - k corresponds to the universe U.

$$S = \left\{ \begin{array}{ccc} \{1, 2, 5\}, & \{2, 5\}, & \{1, 3, 6\}, \\ \{2, 3, 4\}, & \{\ 4\ \}, & \{1, 5, 6\} \end{array} \right\}$$

$$U = \{1, 2, 3, 4, 5, 6\}$$



$$S = \left\{ \begin{array}{ccc} \{1, 2, 5\}, & \{2, 5\}, & \{1, 3, 6\}, \\ \{2, 3, 4\}, & \{\ 4\ \}, & \{1, 5, 6\} \end{array} \right\}$$

$$U = \{1, 2, 3, 4, 5, 6\}$$

$$S' = \left\{ \begin{array}{ccc} 110010 & , 010010 & , 101001 \\ 011100 & , 000100 & , 100011 \end{array} \right\}$$

$$k = 111111$$

$$S = \left\{ \begin{matrix} \{1, 2, 5\}, & \{2, 5\}, & \{1, 3, 6\}, \\ \{2, 3, 4\}, & \{4\}, & \{1, 5, 6\} \end{matrix} \right\}$$

$$U = \{1, 2, 3, 4, 5, 6\}$$

$$S' = \left\{ \begin{matrix} 110010, & 010010, & 101001 \\ 011100, & 000100, & 100011 \end{matrix} \right\}$$

$$k = 111111$$

$$S = \left\{ \begin{matrix} \{1, 2, 5\}, & \{2, 5\}, & \{1, 3, 6\}, \\ \{2, 3, 4\}, & \{4\}, & \{1, 5, 6\} \end{matrix} \right\}$$

$$U = \{1, 2, 3, 4, 5, 6\}$$

$$S' = \left\{ \begin{matrix} 110010, & 010010, & 101001 \\ 011100, & 000100, & 100011 \end{matrix} \right\}$$

$$k = 111111$$

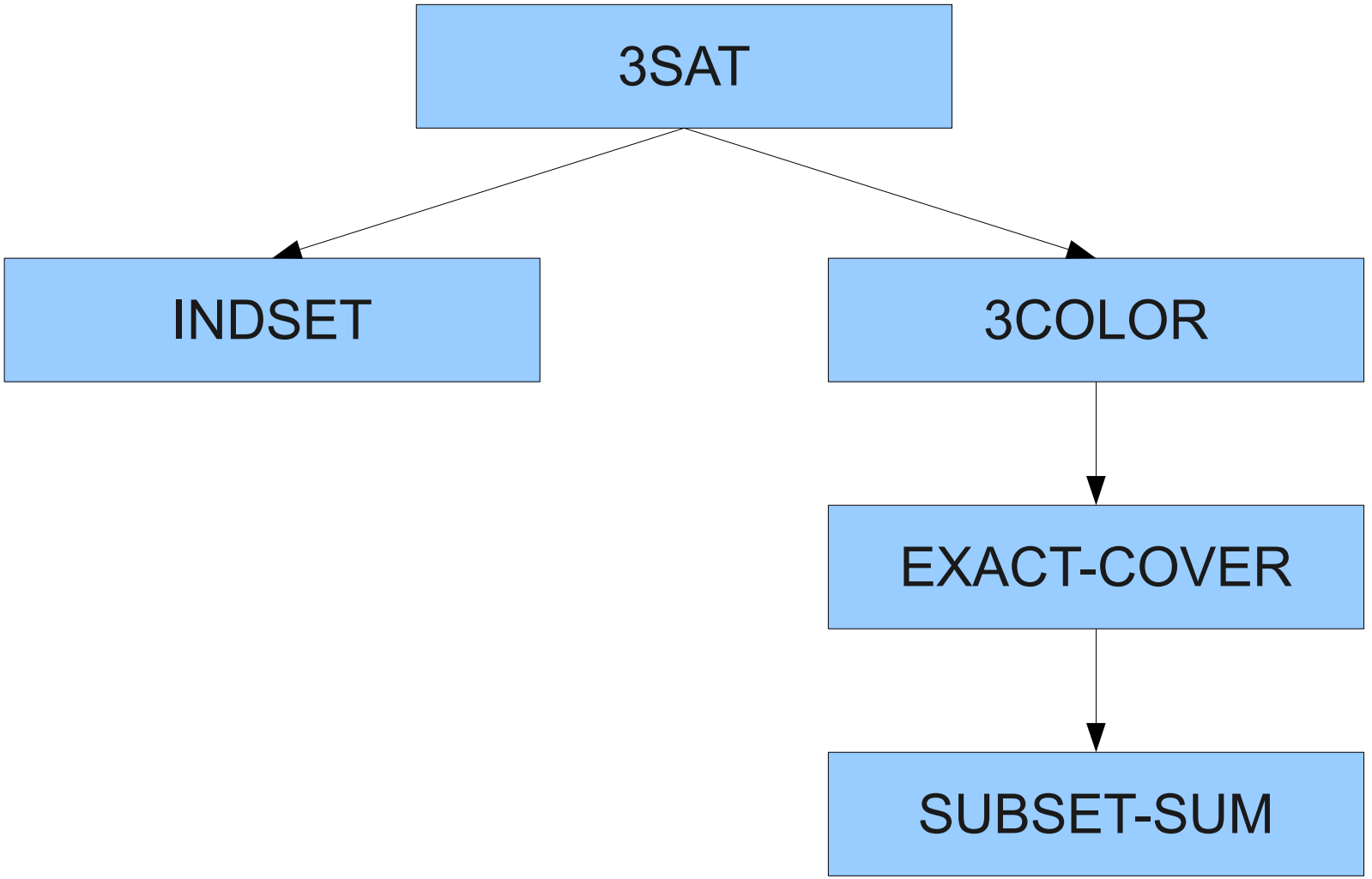
The Basic Intuition

- Suppose there are n elements in the universe and k different sets.
- Replace each set S with a number that is 1 in its ith position if $i \in S$ and has a 0 in its ith position otherwise.
- Set k to a number that is n copies of the number 1.

A Slight Complexity

- To ensure that the columns don't overflow, write the numbers in base $(B + 1)$ where B is the total number of sets.
- That way, the columns can't overflow from one column into the next.

The Story So Far



Yet Another **NP**-Complete Problem

Given a set $S \subseteq \mathbb{N}$, the **partitioning problem** is to find a way to split S into two sets with equal sum.

$$\{\ 13, 137, 56, 42, 103, 58, 271\ \}$$

Given a set $S \subseteq \mathbb{N}$, the **partitioning problem** is to find a way to split S into two sets with equal sum.

$$\{\ 13, 137, 56, 42, 103, 58, 271\ \}$$

Given a set $S \subseteq \mathbb{N}$, the **partitioning problem** is to find a way to split S into two sets with equal sum.

Partitioning

- Given a set $S \subseteq \mathbb{N}$, the partitioning problem is

  Can *S* be split into two sets whose sums are the same?

- As a formal language:

  ***PARTITION* =**
  { ⟨*S*⟩ | *S* ⊆ ℕ, and there is a way to split *S* into two sets with the same sum. }

PARTITION ∈ **NPC**

- We will prove that *PARTITION* is **NP**-complete.
- To do this, we will show that
  - *PARTITION* ∈ **NP**, and
  - *SUBSET-SUM* $\leq_P$ *PARTITION*
- Sense a pattern? ☺

PARTITION ∈ **NP**

- Here is a polynomial-time verifier for *PARTITION*:
- V = "On input $\langle S, S_1, S_2 \rangle$:
  - Check that $S_1 \cup S_2 = S$ and that $S_1 \cap S_2 = \emptyset$.
  - Check that the sum of the elements in S_1 equals the sum of the elements in S_2.
  - If so, accept; otherwise, reject."

$SUBSET\text{-}SUM \leq_P PARTITION$

- We now reduce subset sum to partitioning.
- The subset sum has a solution iff
  - Some subset of the master set S is equal to k.
- We will construct our new set S' such that
  - If a subset of S has total k, we can add in a new element to make up the difference to half the total sum.

$$\{\ 137, 42, 271, 103, 154, 16, 3\ \}$$

$$k = 452$$

$$\{\ 137, 42, 271, 103, 154, 16, 3\ \}$$

$$k = 452$$

$$\{\ 137, 42, 271, 103, 154, 16, 3\ \}$$

$$k = 452$$

Total of all elements in this set: 726

$$\{\ 137, 42, 271, 103, 154, 16, 3\ \}$$

$$k = 452$$

Total of all elements in this set: 726

$$726 - 452 = 274$$

$$\{\ 137, 42, 271, 103, 154, 16, 3\ \}$$

$$k = 452$$

Total of all elements in this set: 726

$$726 - 452 = 274$$

$$452 - 274 = 178$$

$$\{\ 137, 42, 271, 103, 154, 16, 3\ \}$$

$$k = 452$$

Total of all elements in this set: 726

$$726 - 452 = 274$$

$$452 - 274 = 178$$

$$\{\ 137, 42, 271, 103, 154, 16, 3, 178\ \}$$

$$\{\ 137, 42, 271, 103, 154, 16, 3\ \}$$

$$k = 452$$

Total of all elements in this set: 726

$$726 - 452 = 274$$

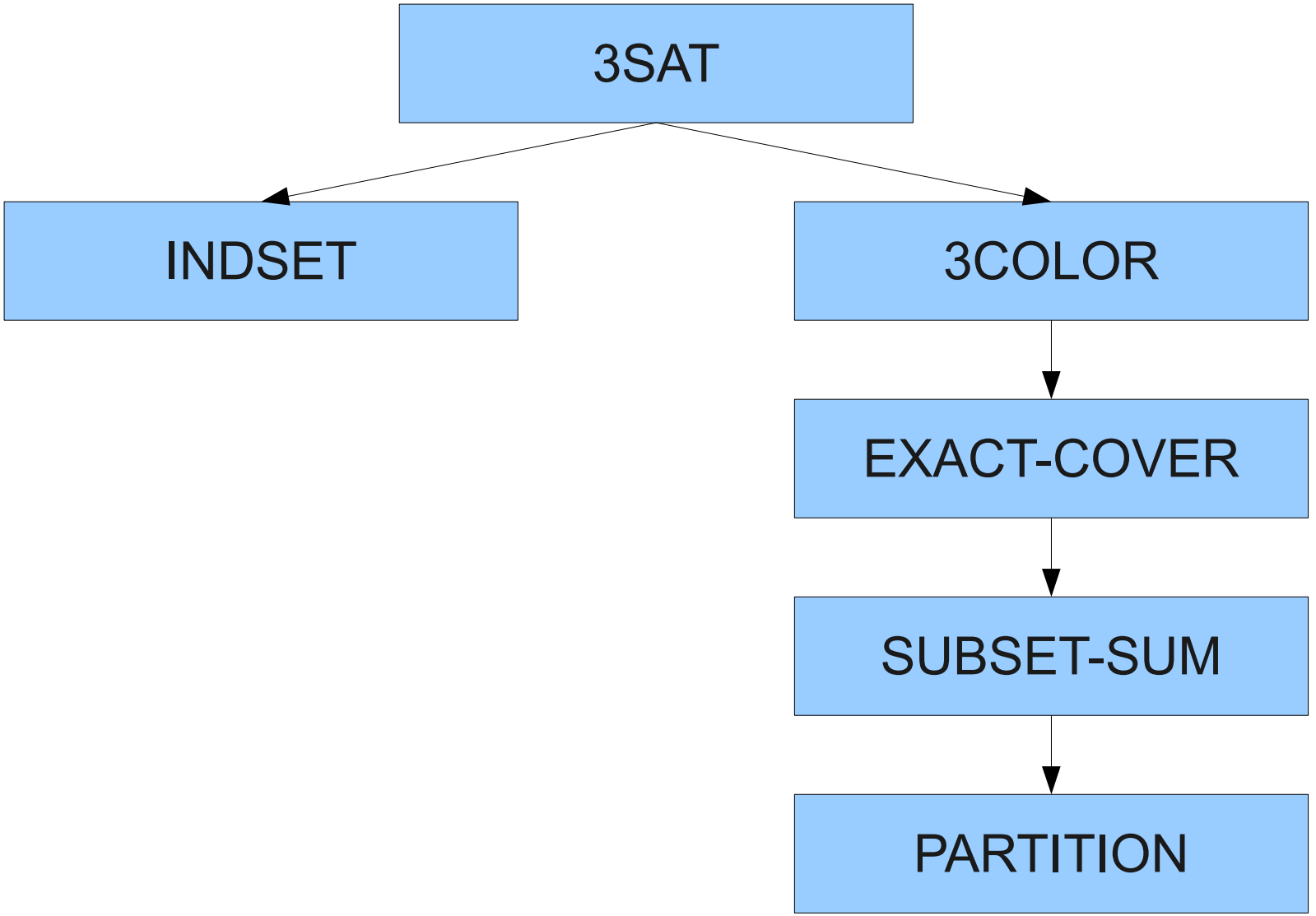
$$452 - 274 = 178$$

$$\{\ 137, 42, 271, 103, 154, 16, 3, 178\ \}$$

The General Idea

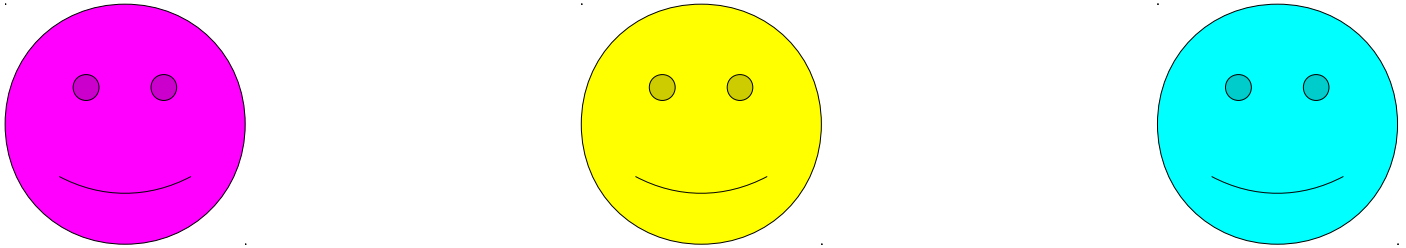
- Add in a new element to the set such that a subset with the appropriate sum also forms a partition.
- The new element added in might need to go in the subset that originally added to k, or it might have to go in the complement of that set.

The Story So Far

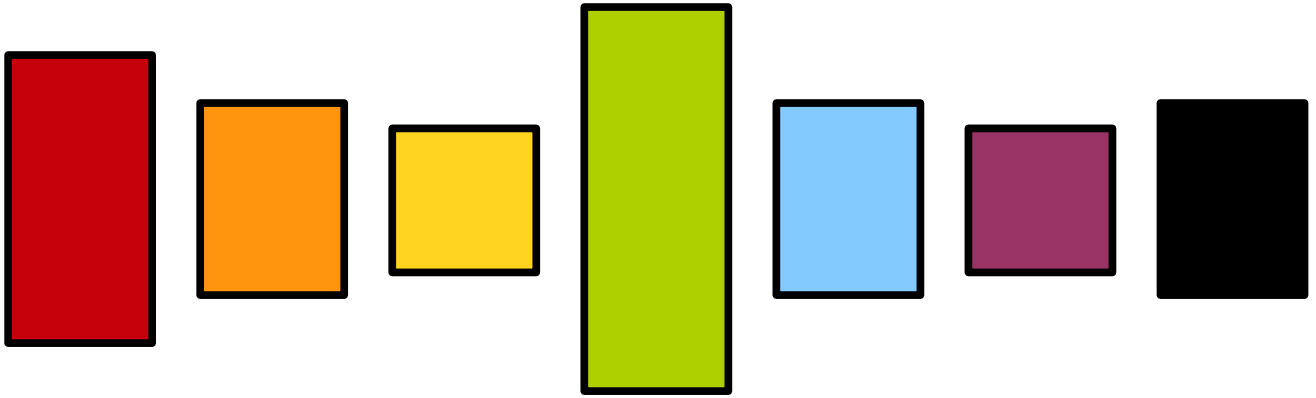


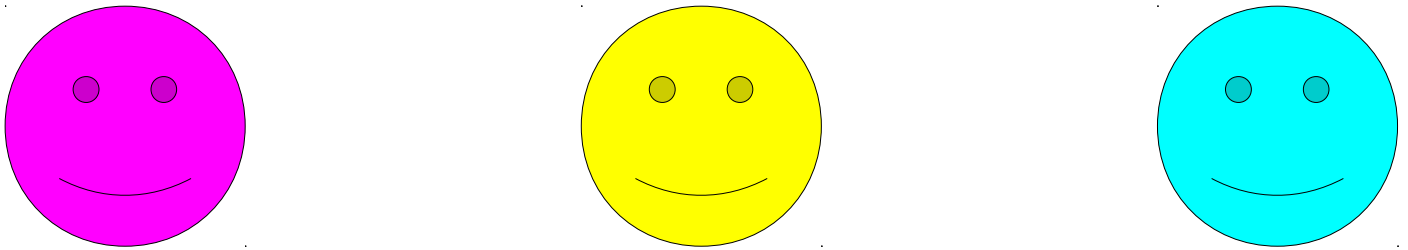
One Final **NP**-Complete Problem

Given a set J of jobs that take some amount of time to complete and k workers, the **job scheduling** problem is to minimize the total time required to complete all jobs (called the **makespan**).

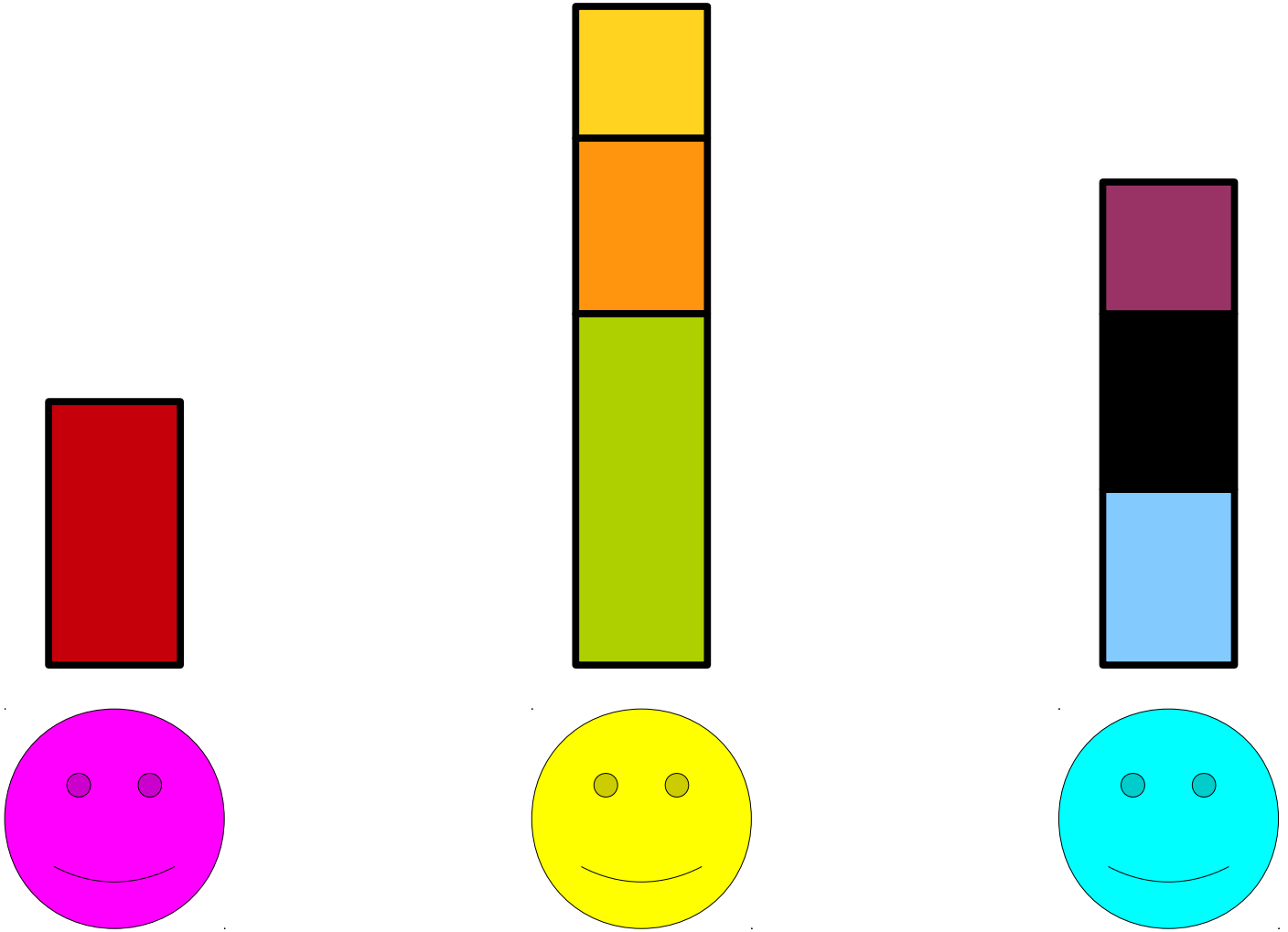


Given a set J of jobs that take some amount of time to complete and k workers, the **job scheduling** problem is to minimize the total time required to complete all jobs (called the **makespan**).

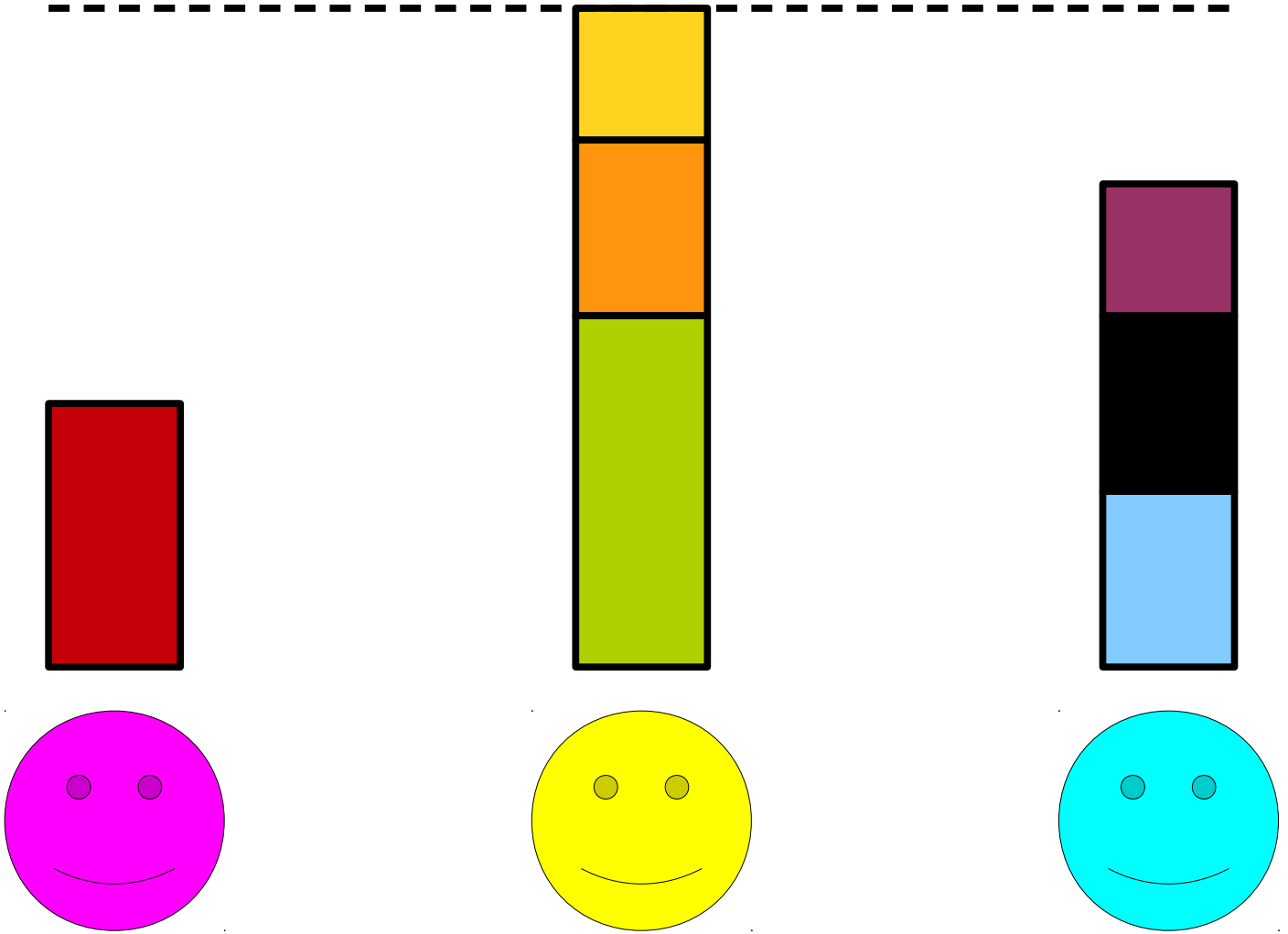




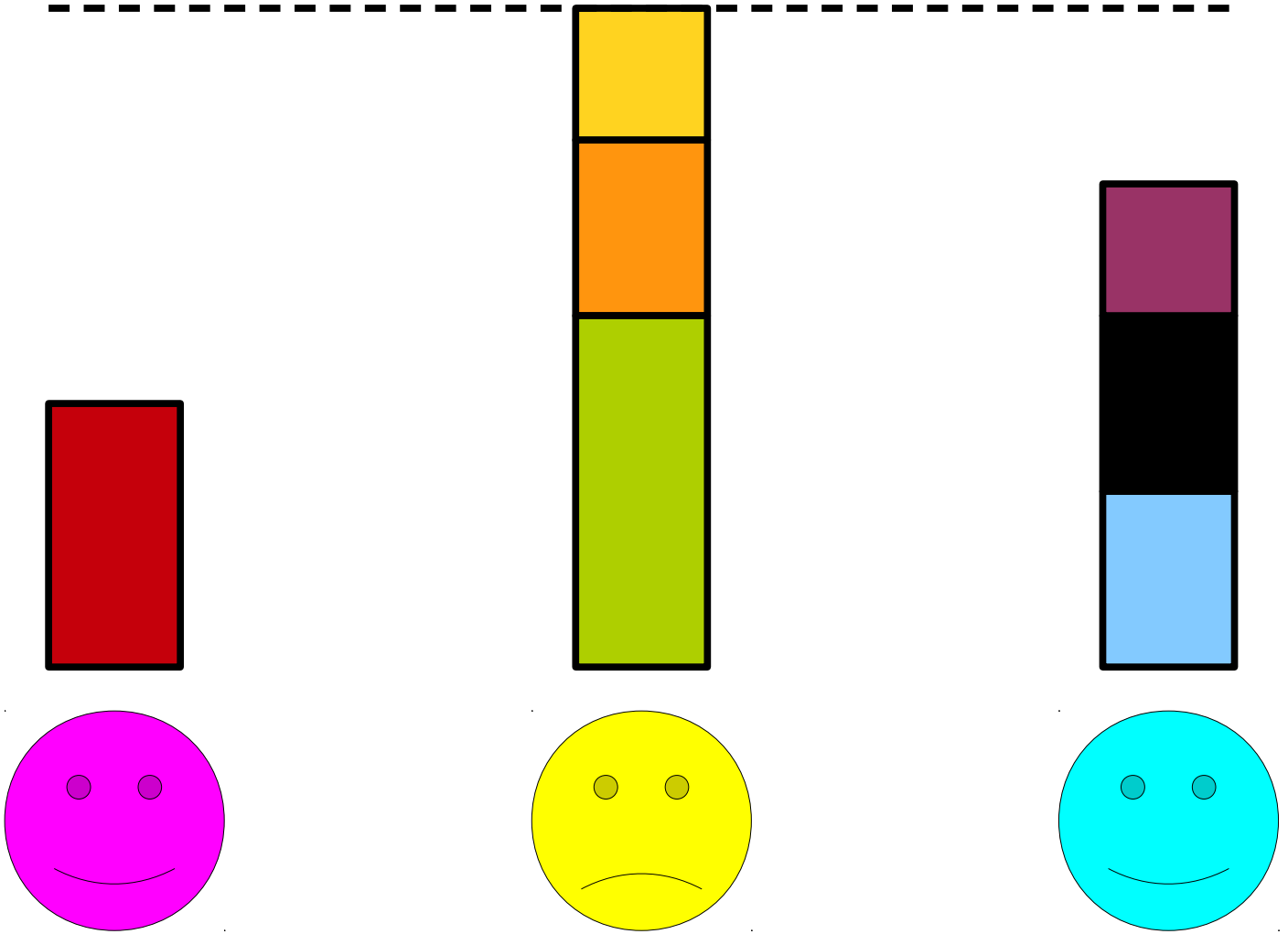
Given a set J of jobs that take some amount of time to complete and k workers, the **job scheduling** problem is to minimize the total time required to complete all jobs (called the **makespan**).



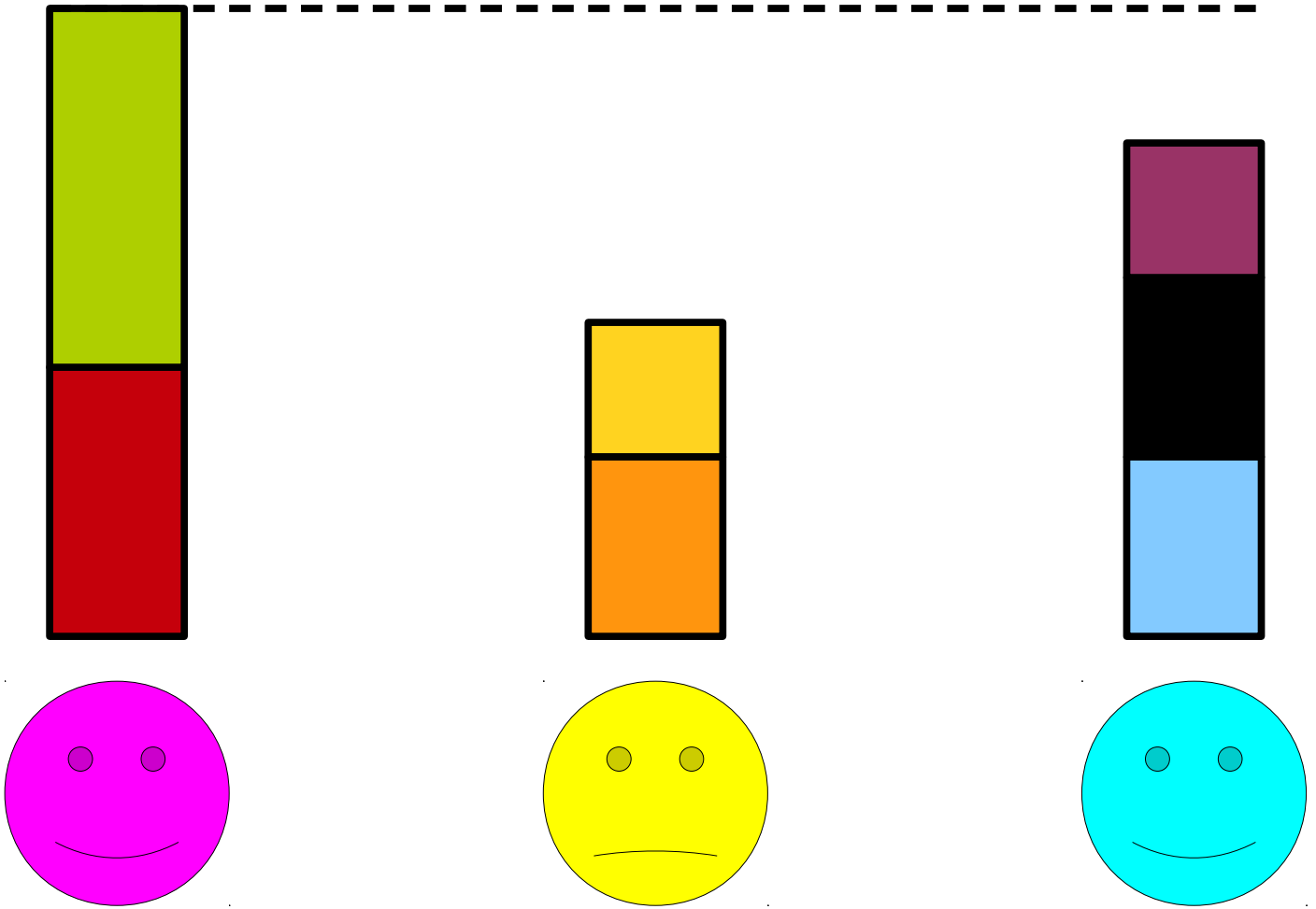
Given a set J of jobs that take some amount of time to complete and k workers, the **job scheduling** problem is to minimize the total time required to complete all jobs (called the **makespan**).



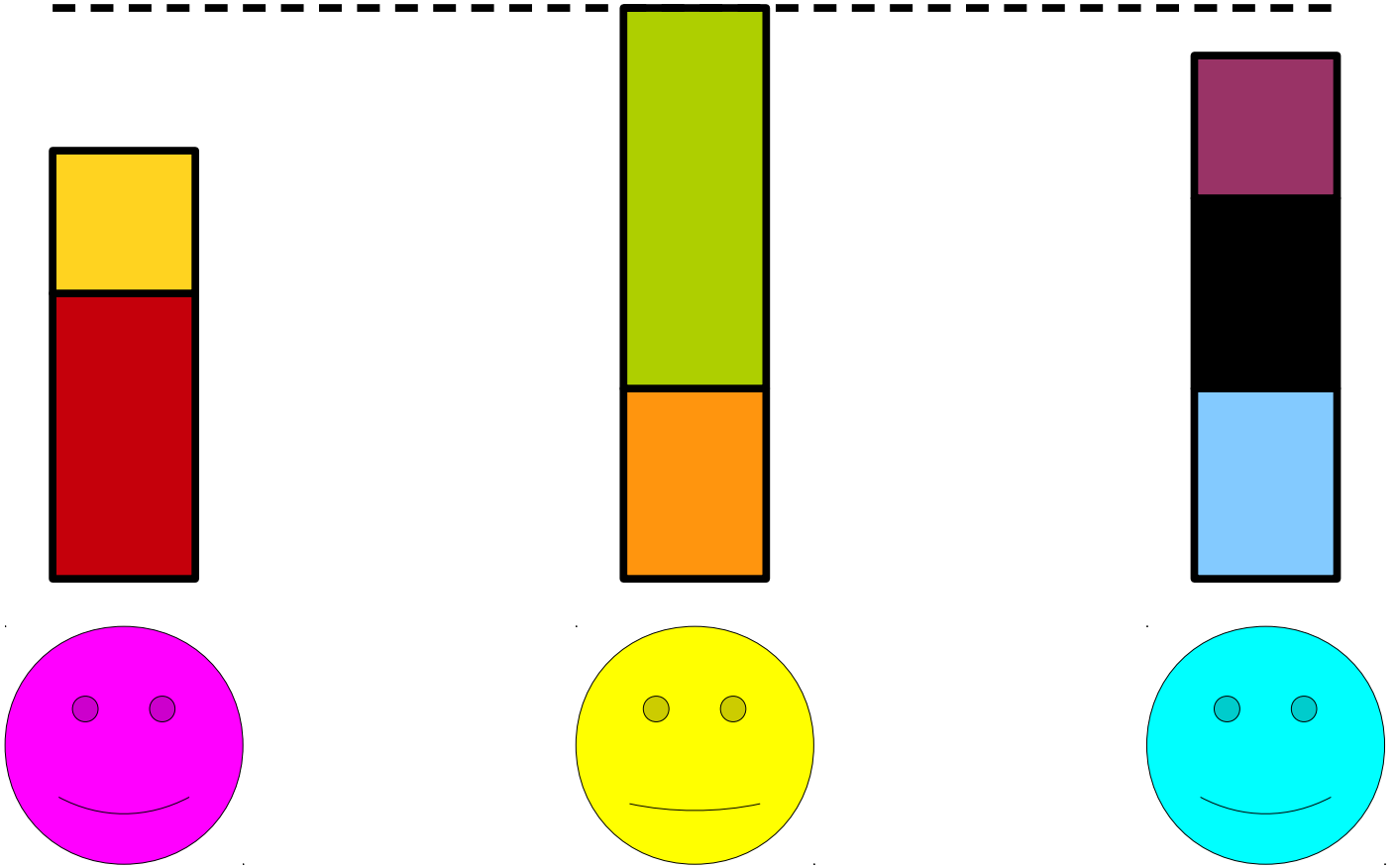
Given a set J of jobs that take some amount of time to complete and k workers, the **job scheduling** problem is to minimize the total time required to complete all jobs (called the **makespan**).



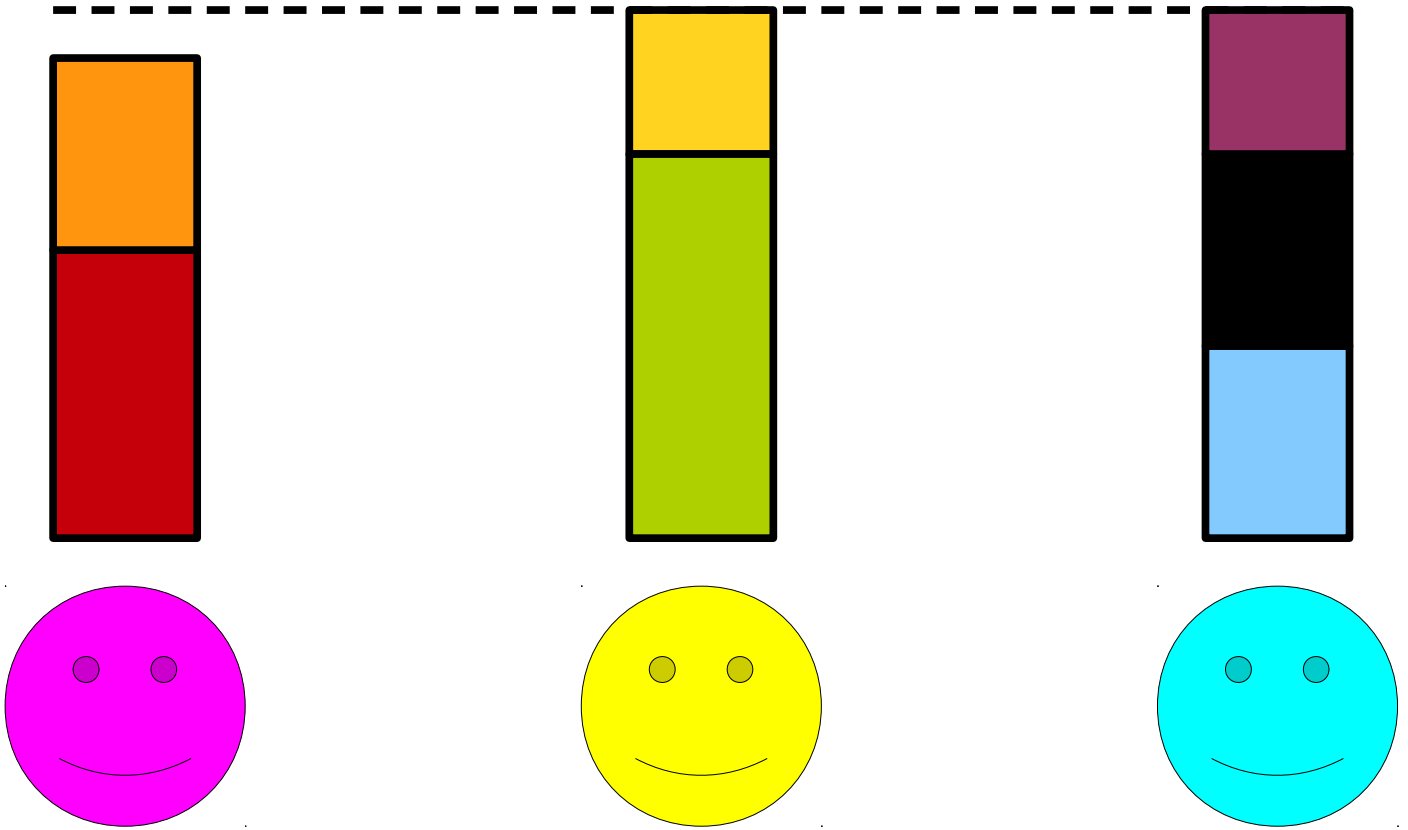
Given a set J of jobs that take some amount of time to complete and k workers, the **job scheduling** problem is to minimize the total time required to complete all jobs (called the **makespan**).



Given a set J of jobs that take some amount of time to complete and k workers, the **job scheduling** problem is to minimize the total time required to complete all jobs (called the **makespan**).



Given a set J of jobs that take some amount of time to complete and k workers, the **job scheduling** problem is to minimize the total time required to complete all jobs (called the **makespan**).



Given a set J of jobs that take some amount of time to complete and k workers, the **job scheduling** problem is to minimize the total time required to complete all jobs (called the **makespan**).

Job Scheduling

- Given a set J of jobs of different lengths, a number of workers k, and a number t, the job scheduling problem is

  Can the jobs in J be assigned to the k workers such that all jobs are finished within t units of time?

- As a formal language:

  ***JOB-SCHEDULING* =**
  { ⟨J, k, t⟩ | The jobs in J can be assigned to the k workers so all jobs are completed within t time }

JOB-SCHEDULING ∈ **NPC**

- We will prove that *JOB-SCHEDULING* is **NP**-complete.
- To do this, we will show that
  - *JOB-SCHEDULING* ∈ **NP**, and
  - *PARTITION* $\leq_P$ *JOB-SCHEDULING*

JOB-SCHEDULING ∈ **NP**

- Here is a polynomial-time NTM for *JOB-SCHEDULING*:
- *N* = "On input ⟨*J*, *k*, *t*⟩:
  - **Nondeterministically** guess an assignment of the jobs in *J* to the *k* workers.
  - **Deterministically** find the maximum amount of time used by any worker.
  - If it is at most *t*, accept; otherwise, reject."

$PARTITION \leq_P JOB\text{-}SCHEDULING$

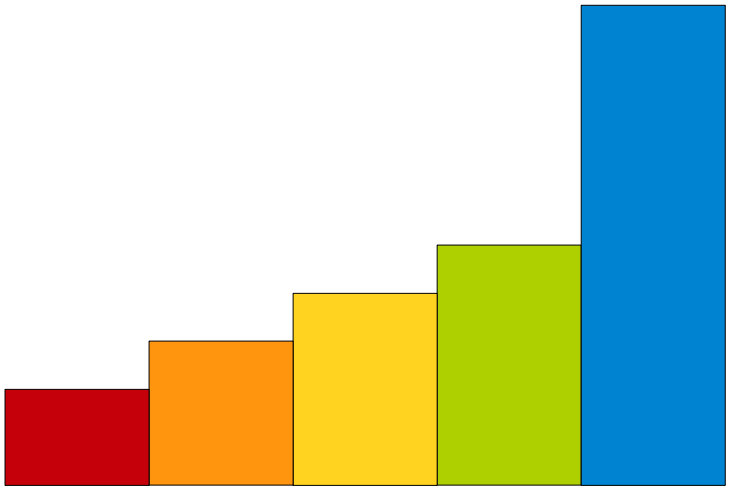
- We now reduce partitioning to job scheduling.
- The reduction is actually straightforward:
  - Given a set of numbers to partition, create one task for each number.
  - Have two workers.
  - See if the workers can complete the tasks in time at most half the total time required to do all jobs.

$$PARTITION \leq_P JOB\text{-}SCHEDULING$$

$$\{\ 2, 3, 4, 5, 10\ \}$$

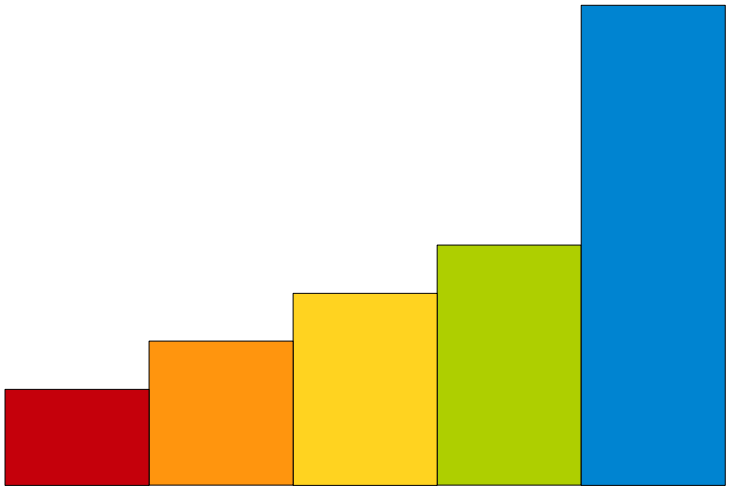
$PARTITION \leq_P JOB\text{-}SCHEDULING$

$\{ 2, 3, 4, 5, 10 \}$



$PARTITION \leq_P JOB\text{-}SCHEDULING$

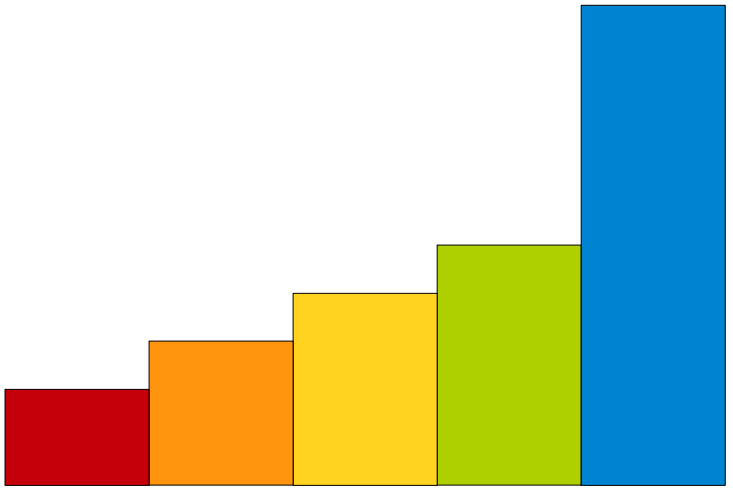
$\{ 2, 3, 4, 5, 10 \}$



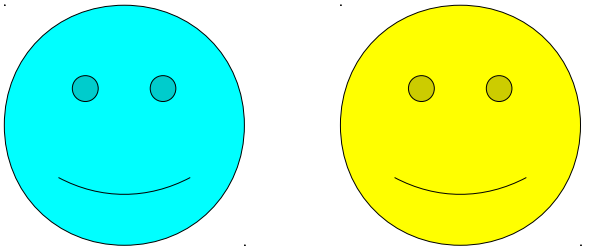
Total time: 24

$PARTITION \leq_P JOB\text{-}SCHEDULING$

$\{ 2, 3, 4, 5, 10 \}$

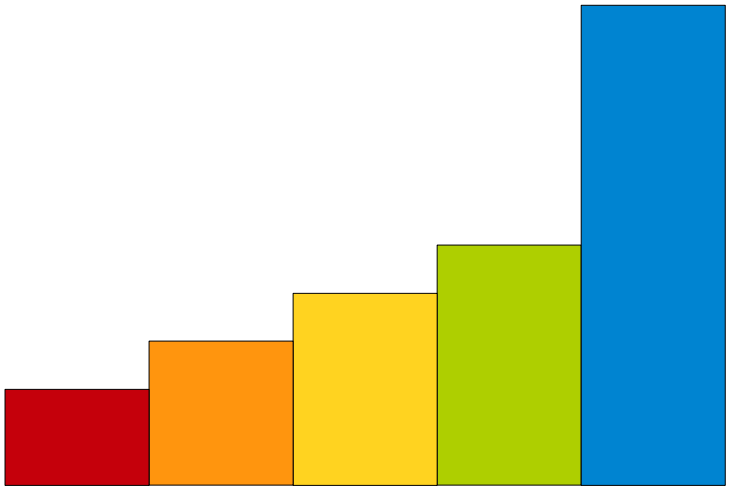


Total time: 24

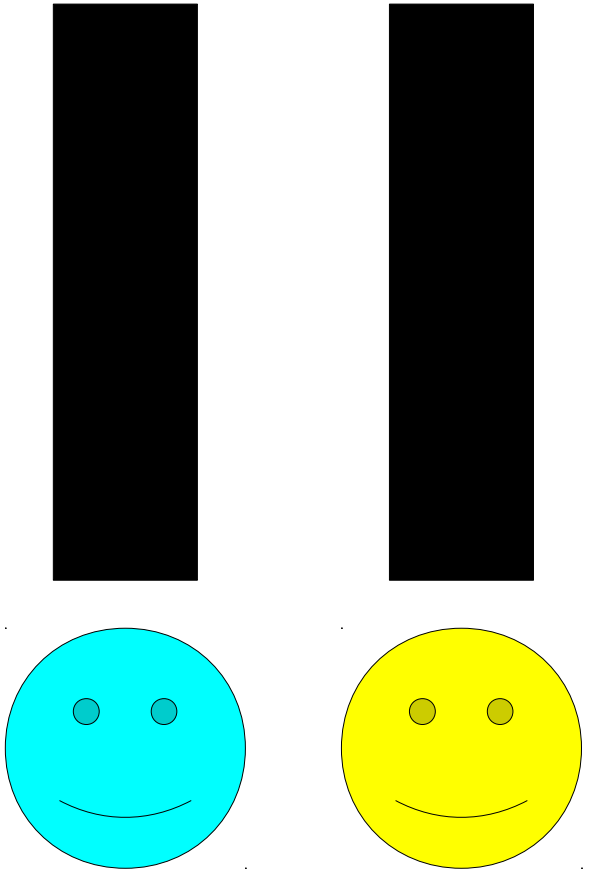


$PARTITION \leq_P JOB\text{-}SCHEDULING$

{ 2, 3, 4, 5, 10 }

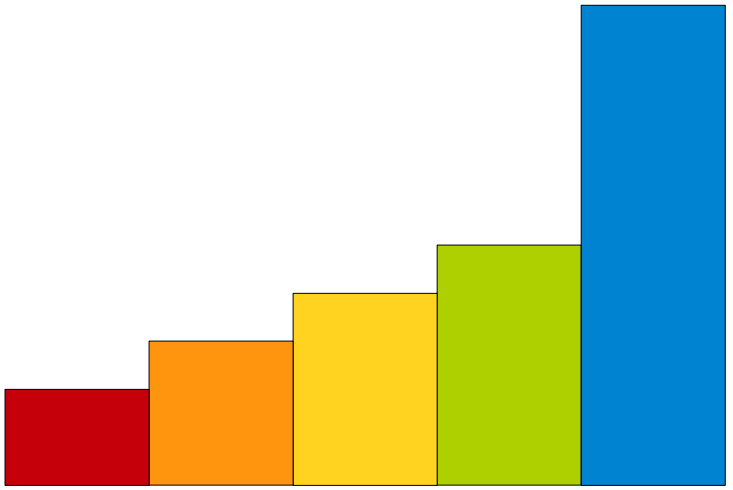


Total time: 24

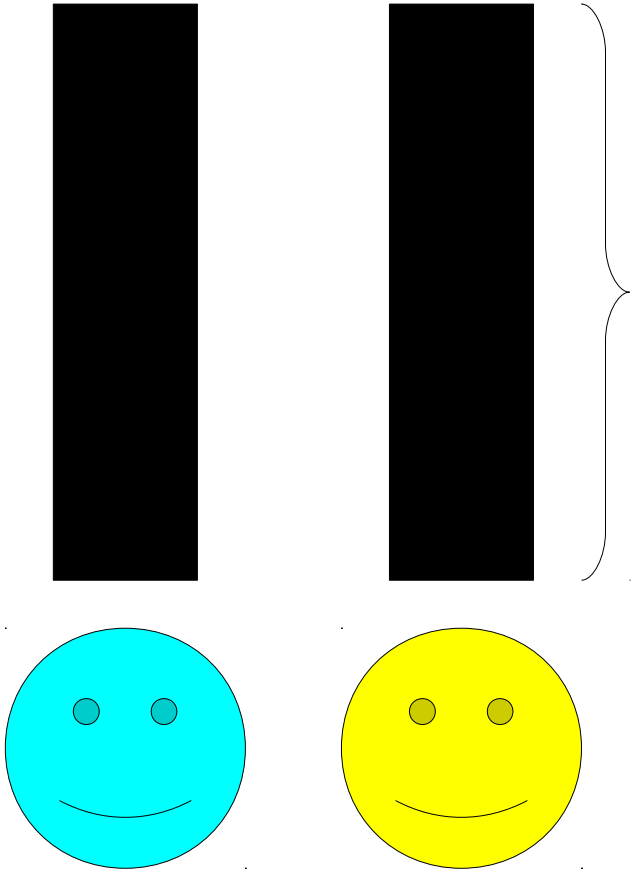


$PARTITION \leq_P JOB\text{-}SCHEDULING$

{ 2, 3, 4, 5, 10 }

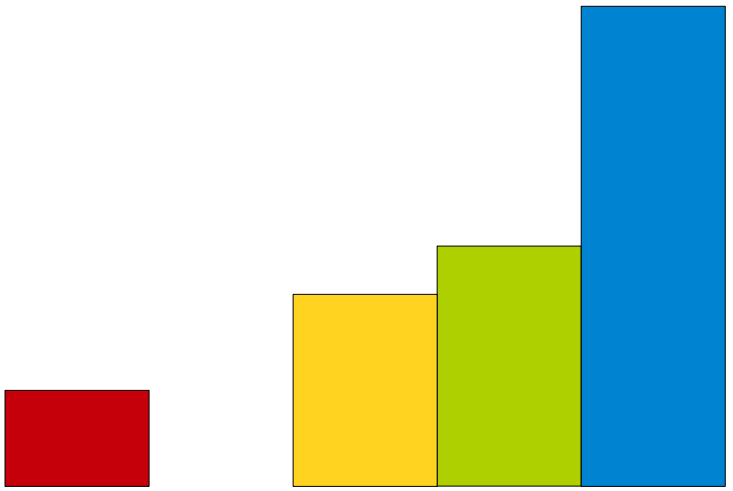


Total time: 24

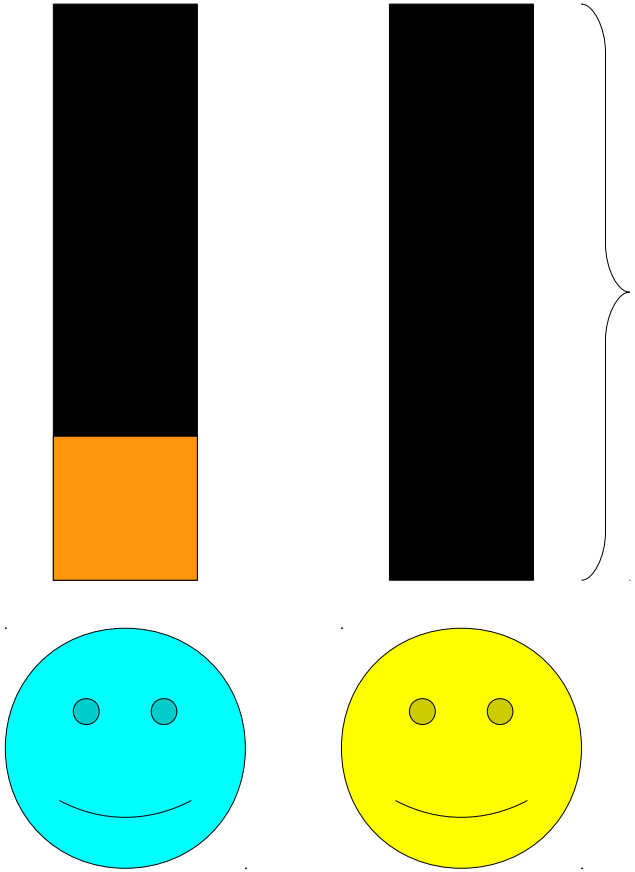


$PARTITION \leq_P JOB\text{-}SCHEDULING$

{ 2, 3, 4, 5, 10 }

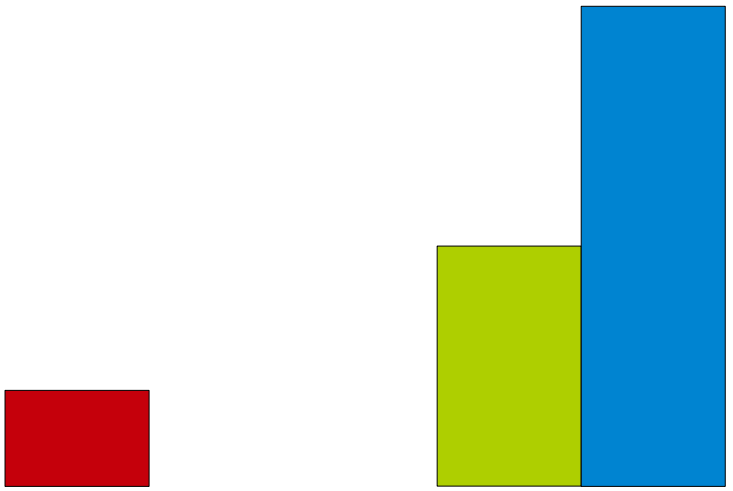


Total time: 24

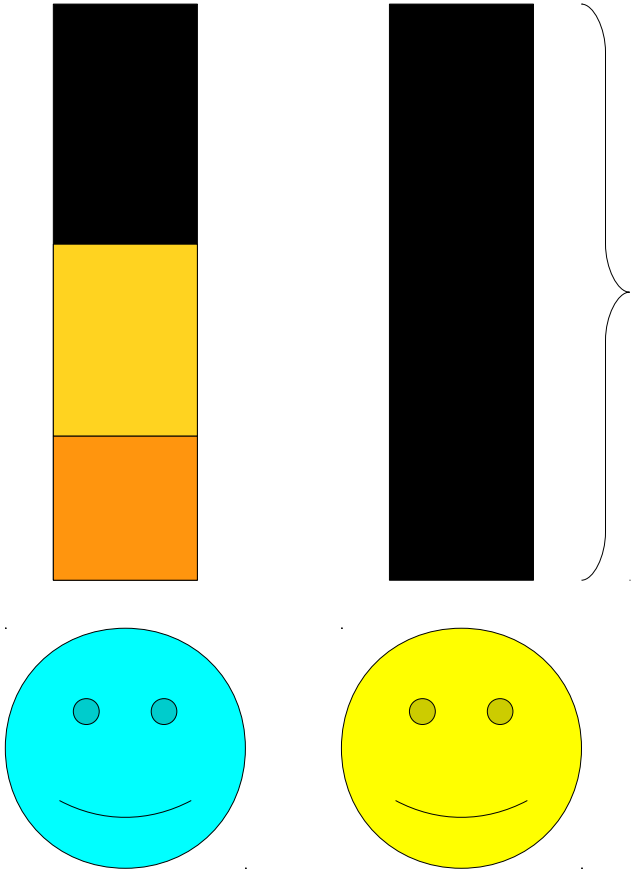


$PARTITION \leq_P JOB\text{-}SCHEDULING$

{ 2, 3, 4, 5, 10 }



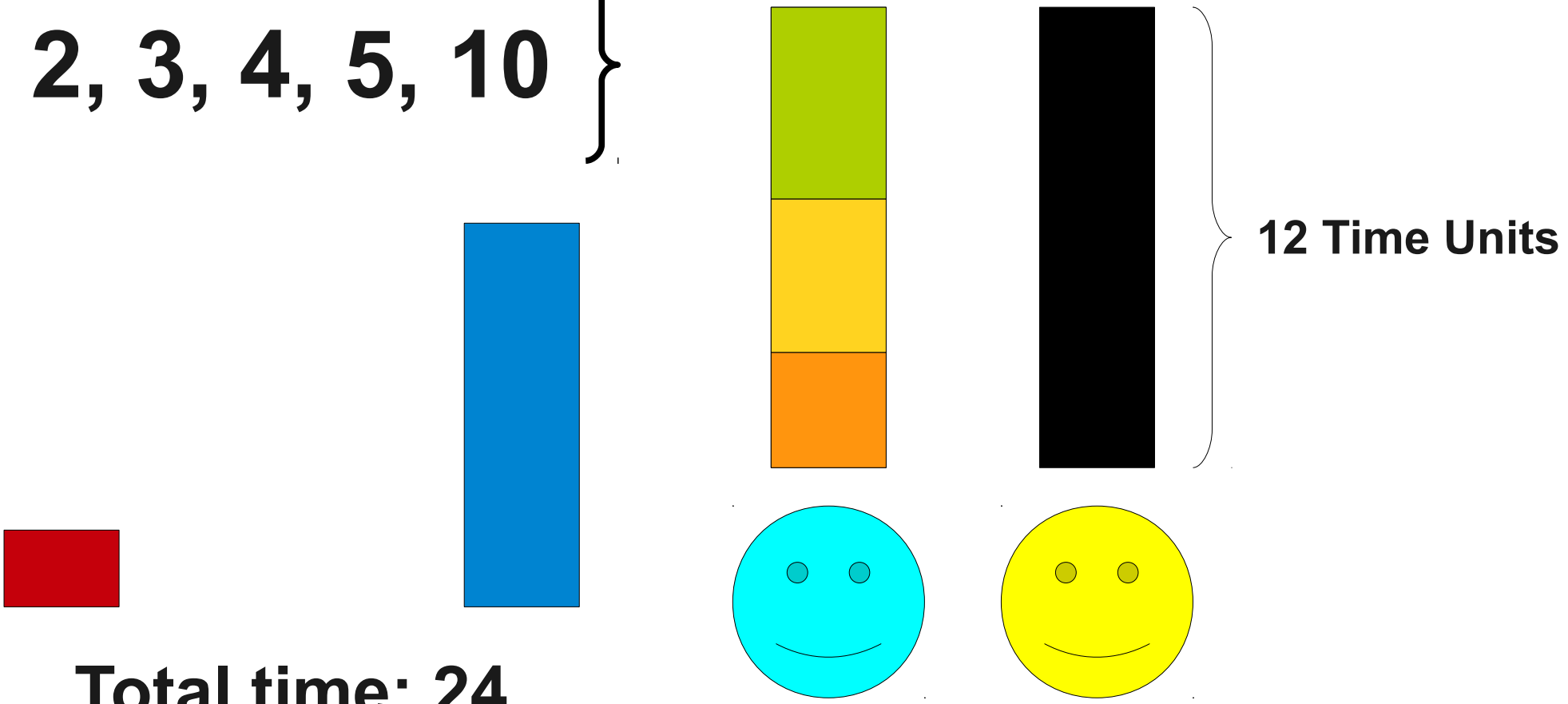
Total time: 24



12 Time Units

$PARTITION \leq_P JOB\text{-}SCHEDULING$

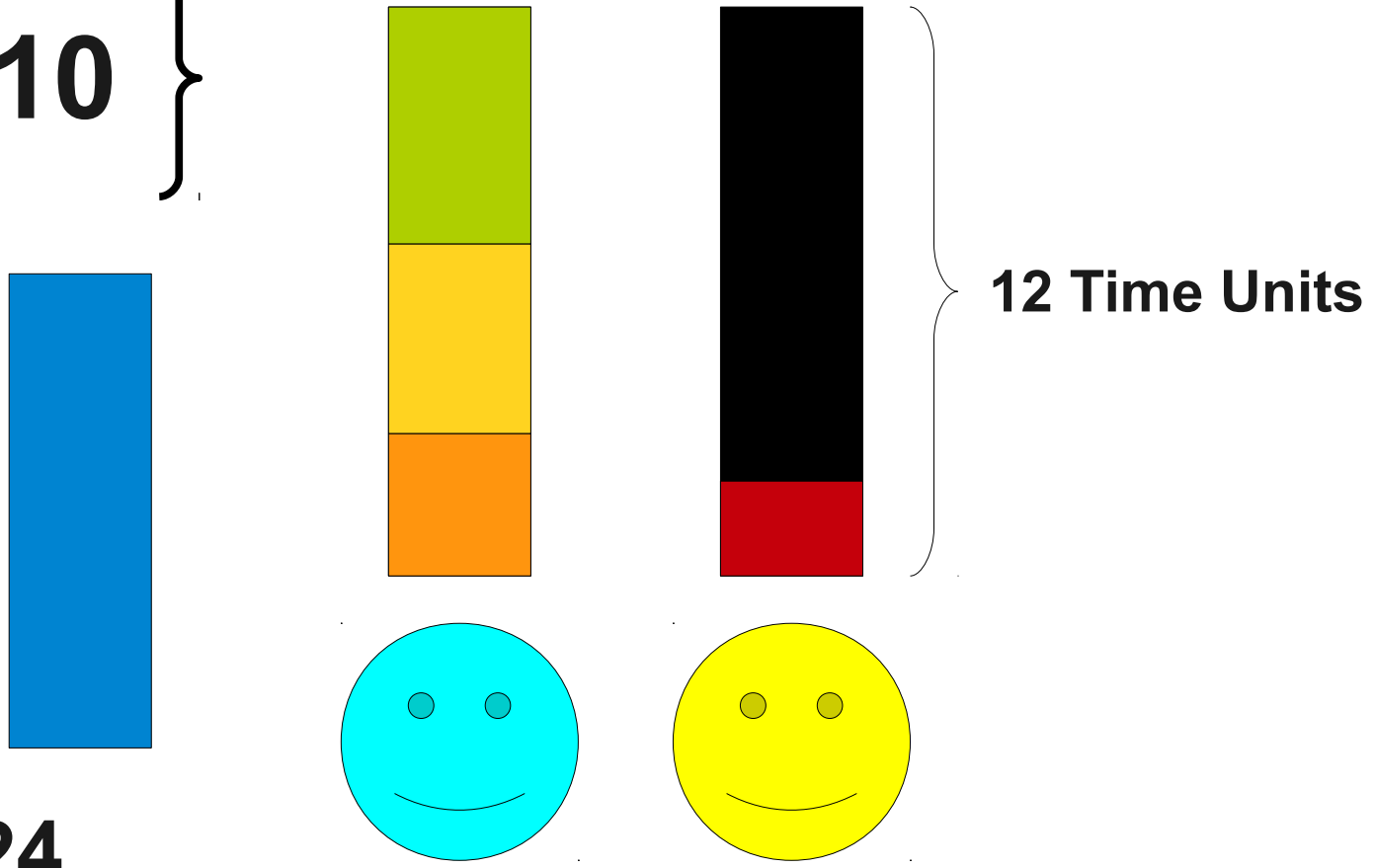
{ 2, 3, 4, 5, 10 }



Total time: 24

$PARTITION \leq_P JOB\text{-}SCHEDULING$

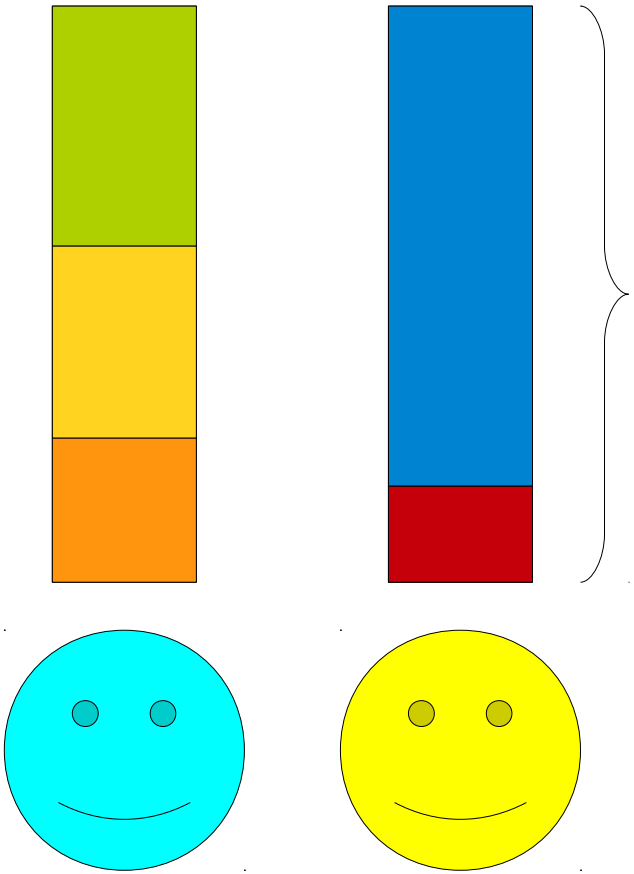
{ 2, 3, 4, 5, 10 }



Total time: 24

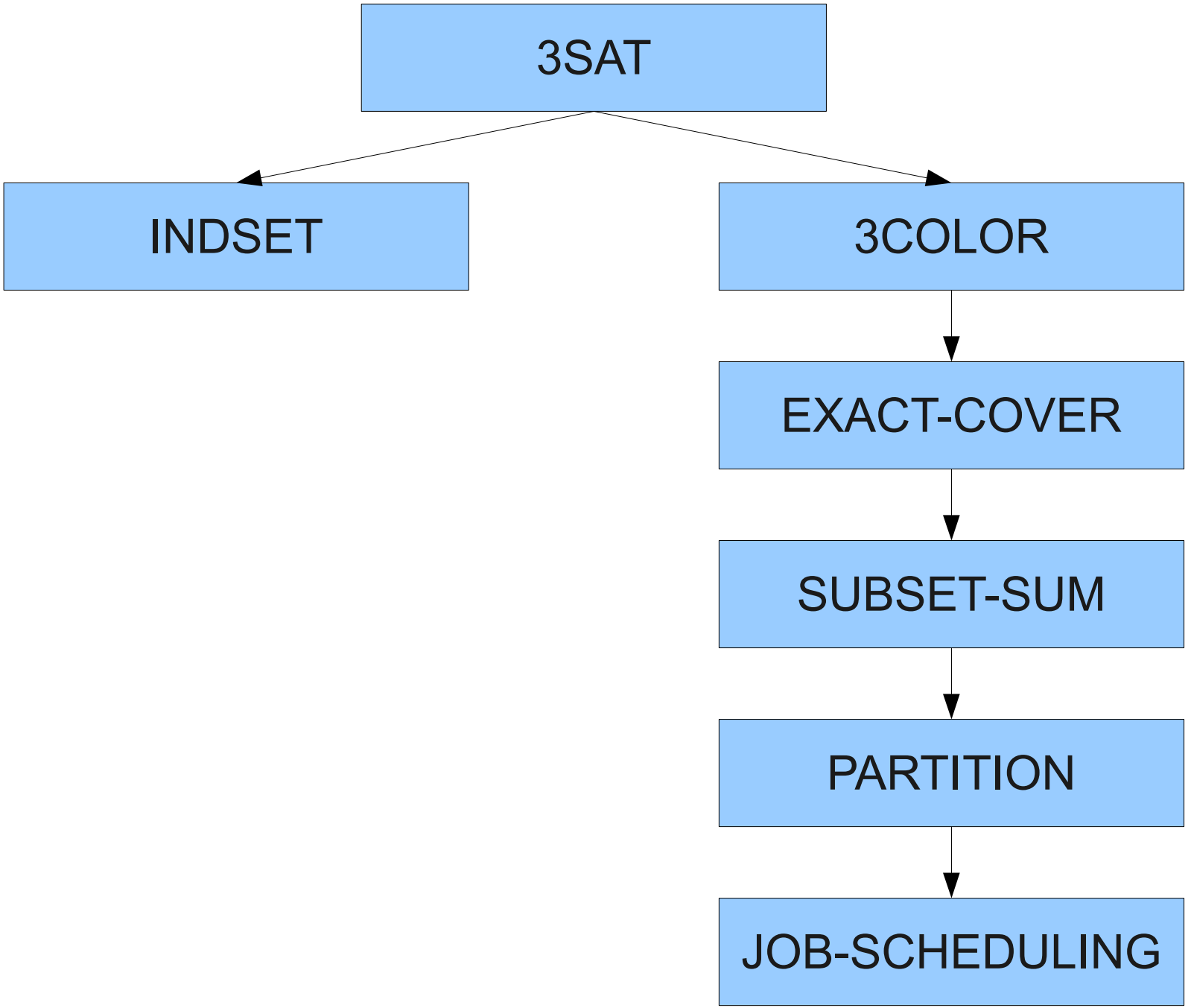
$PARTITION \leq_P JOB\text{-}SCHEDULING$

{ 2, 3, 4, 5, 10 }

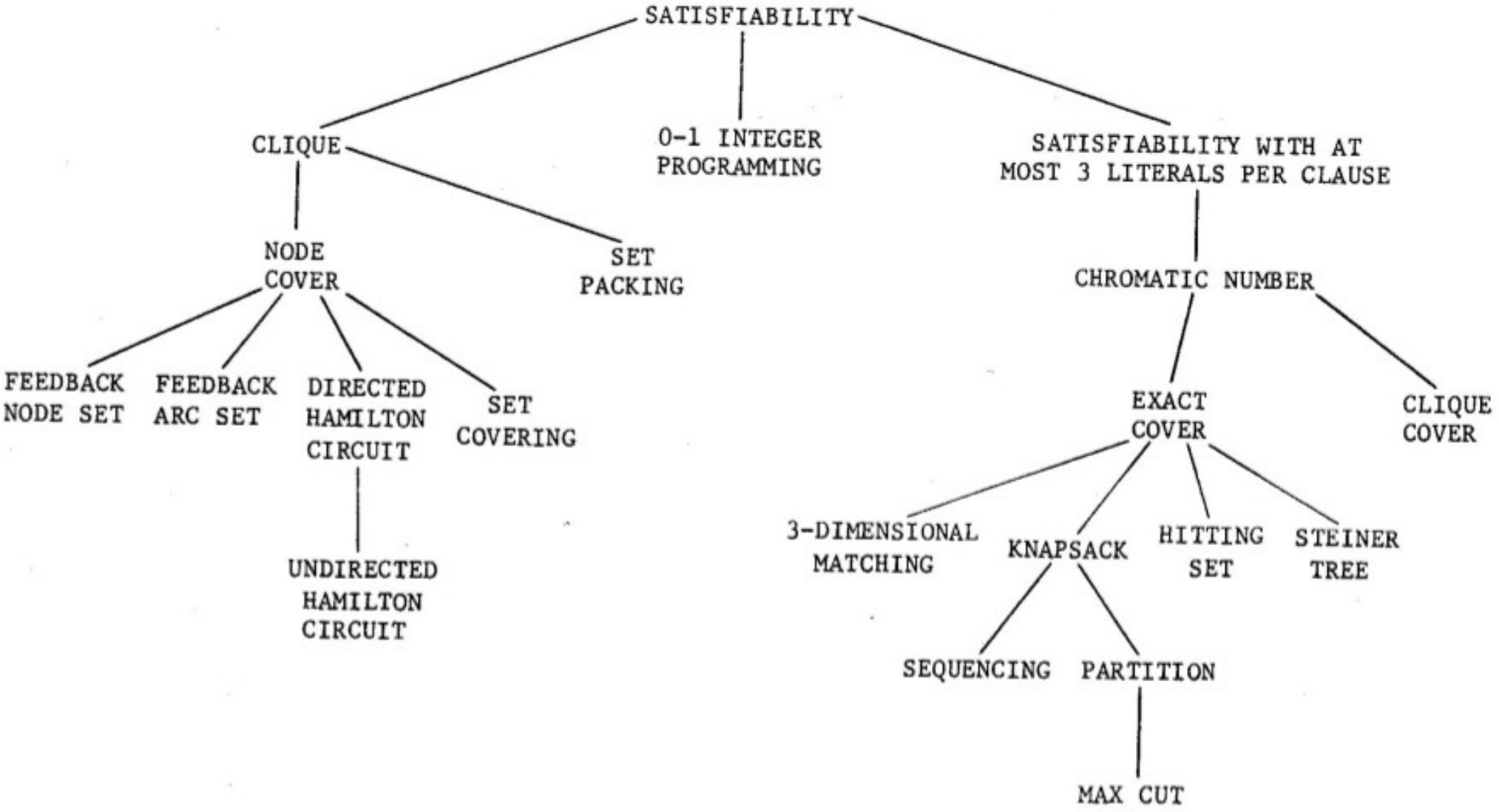


Total time: 24

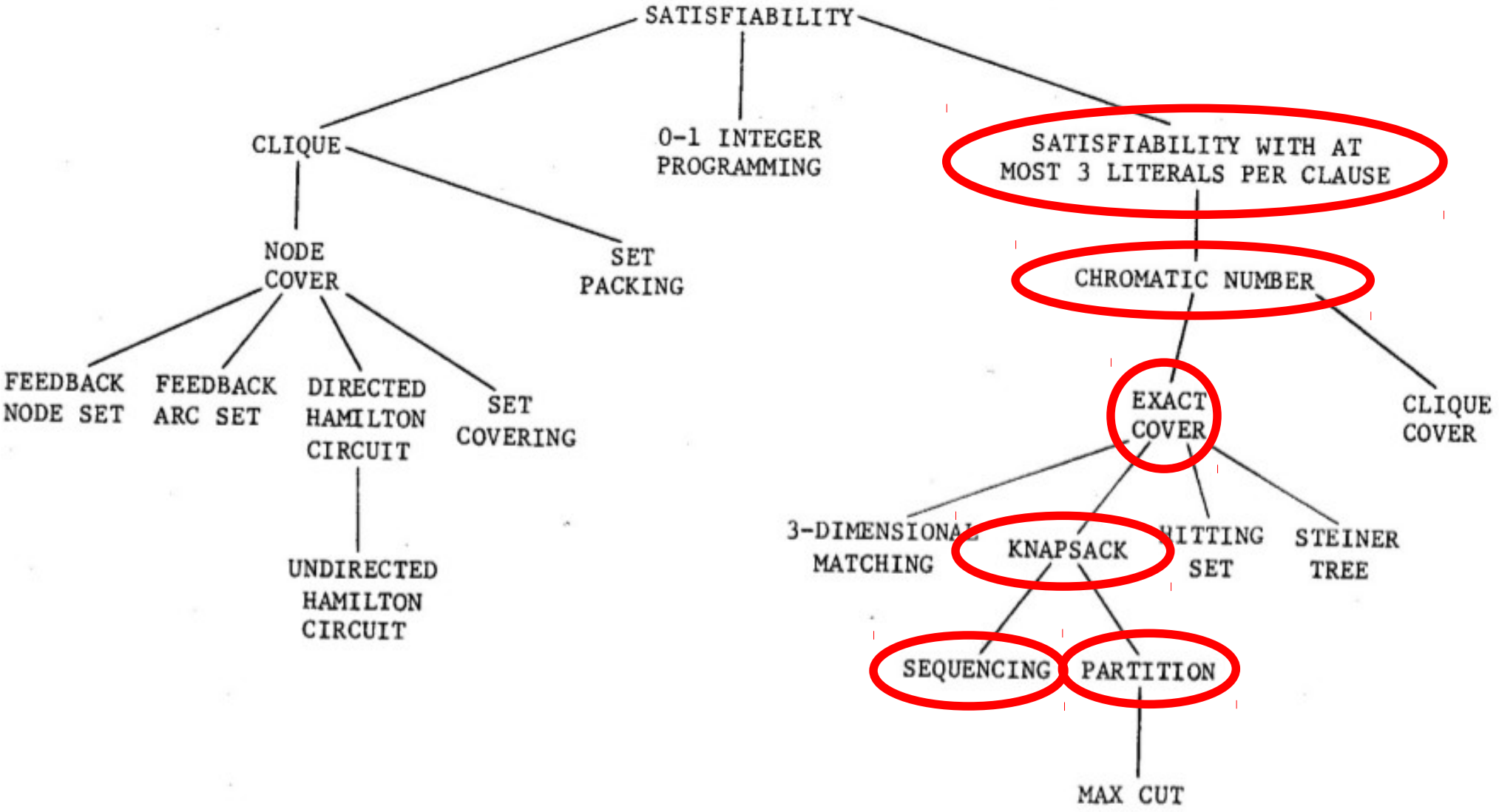
The Story So Far



A Historical Note



Richard Karp. "Reducibility Among Combinatorial Problems." 1972



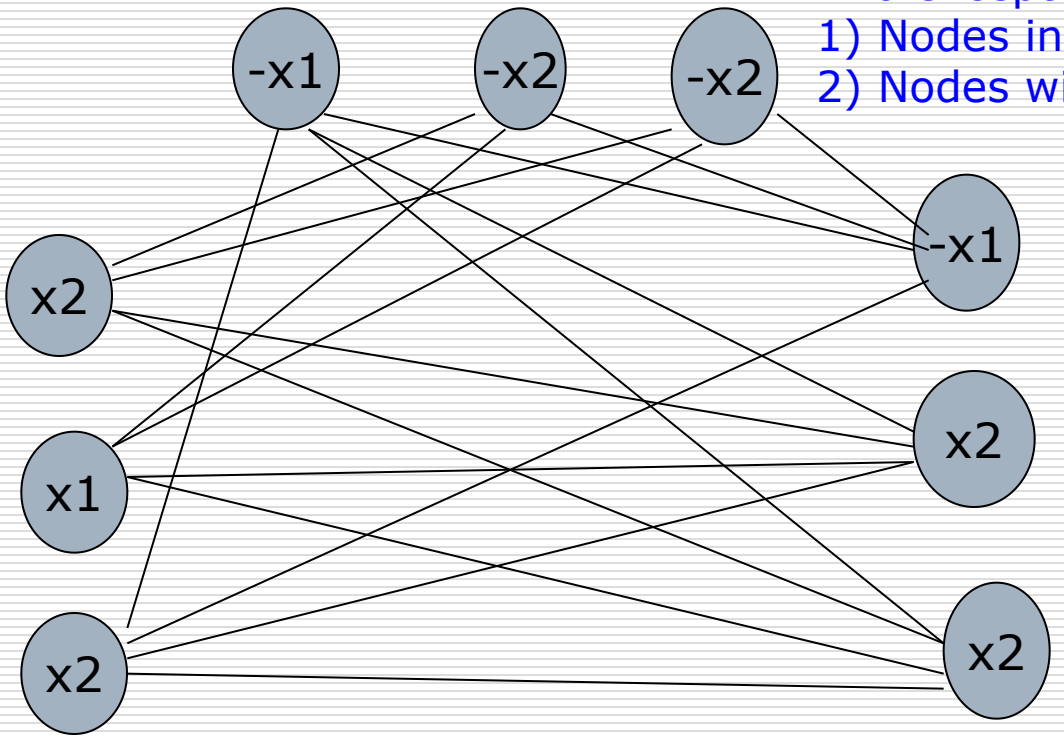
Richard Karp. "Reducibility Among Combinatorial Problems." 1972

A Feel for **NP**-Completeness

- We have just seen **NP**-complete problems from
  - Formal logic (3SAT)
  - Graph theory (3-colorability)
  - Set theory (exact cover)
  - Number theory (subset sum / partition)
  - Operations research (job scheduling)
- **You will encounter NP-complete problems in the real world.**

CLIQUE is NP-complete

Two exceptions:
1) Nodes in the same triples
2) Nodes with contradictory labels



$$\phi = (x_1 \vee x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2 \vee x_2)$$

The formula is satisfiable iff the constructed graph has a k-clique.

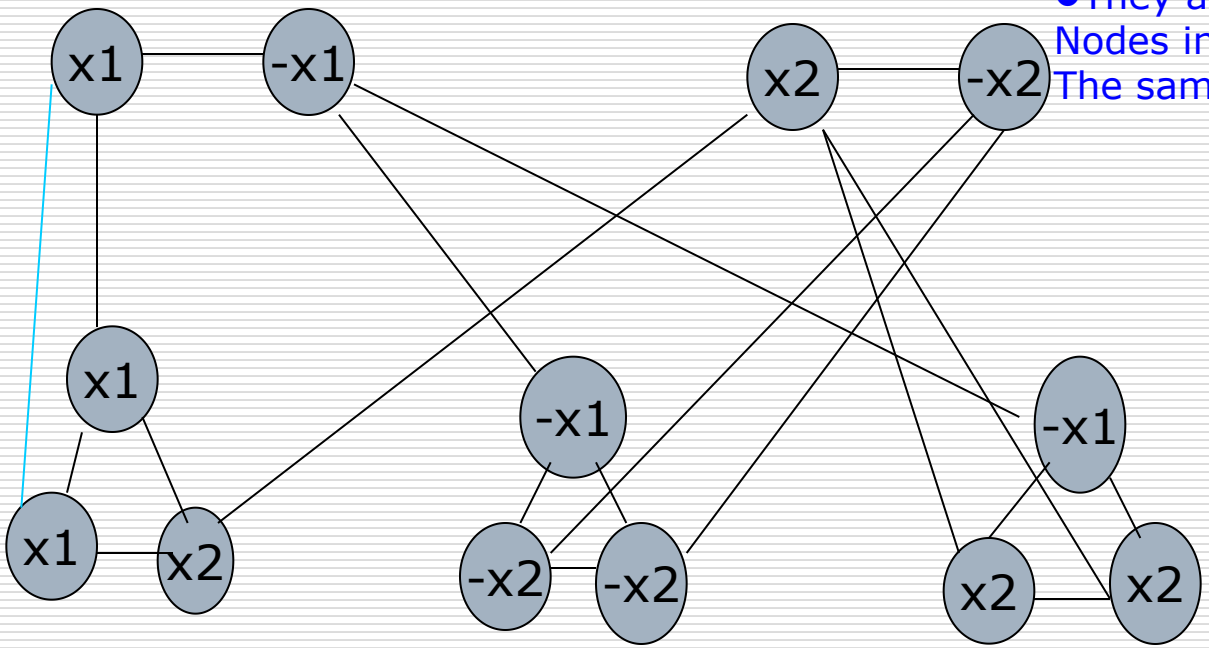
Vertex-cover problem

If G is an undirected graph, a ***vertex cover*** of G is a subset of the nodes where every edge of G touches one of those nodes. The vertex cover problem asks whether a graph contains a vertex cover of a specified size:

$$VERTEX\text{-}COVER = \{\langle G, k\rangle \mid G \text{ is an undirected graph that has a } k\text{-node vertex cover}\}.$$

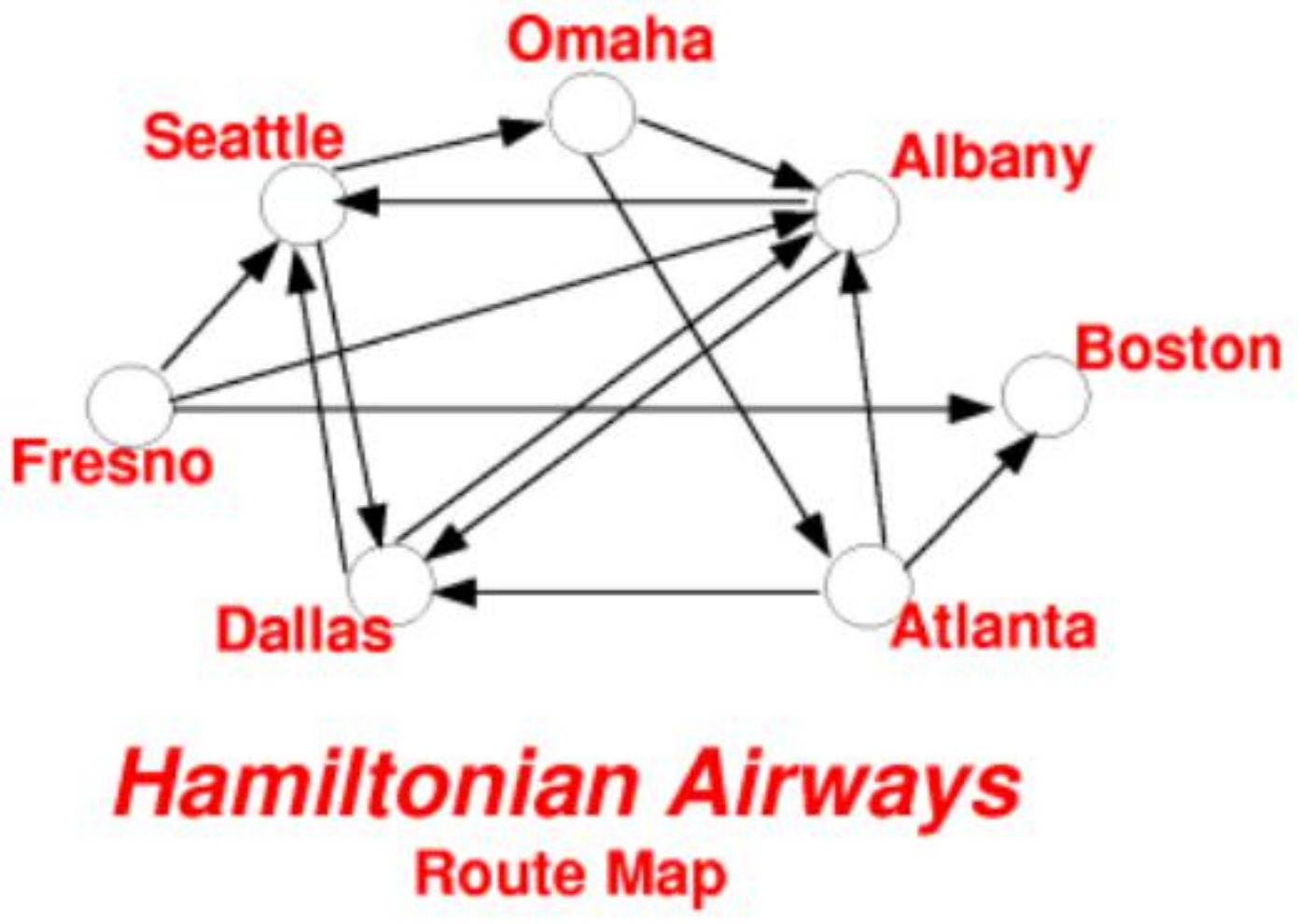
VERTEX-COVER is NP-complete

- The nodes in each gadget Are connected to each other
- They are also connected to the Nodes in the variable gadgets wit The same labels.



$$\phi = (x_1 \vee x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2 \vee x_2)$$

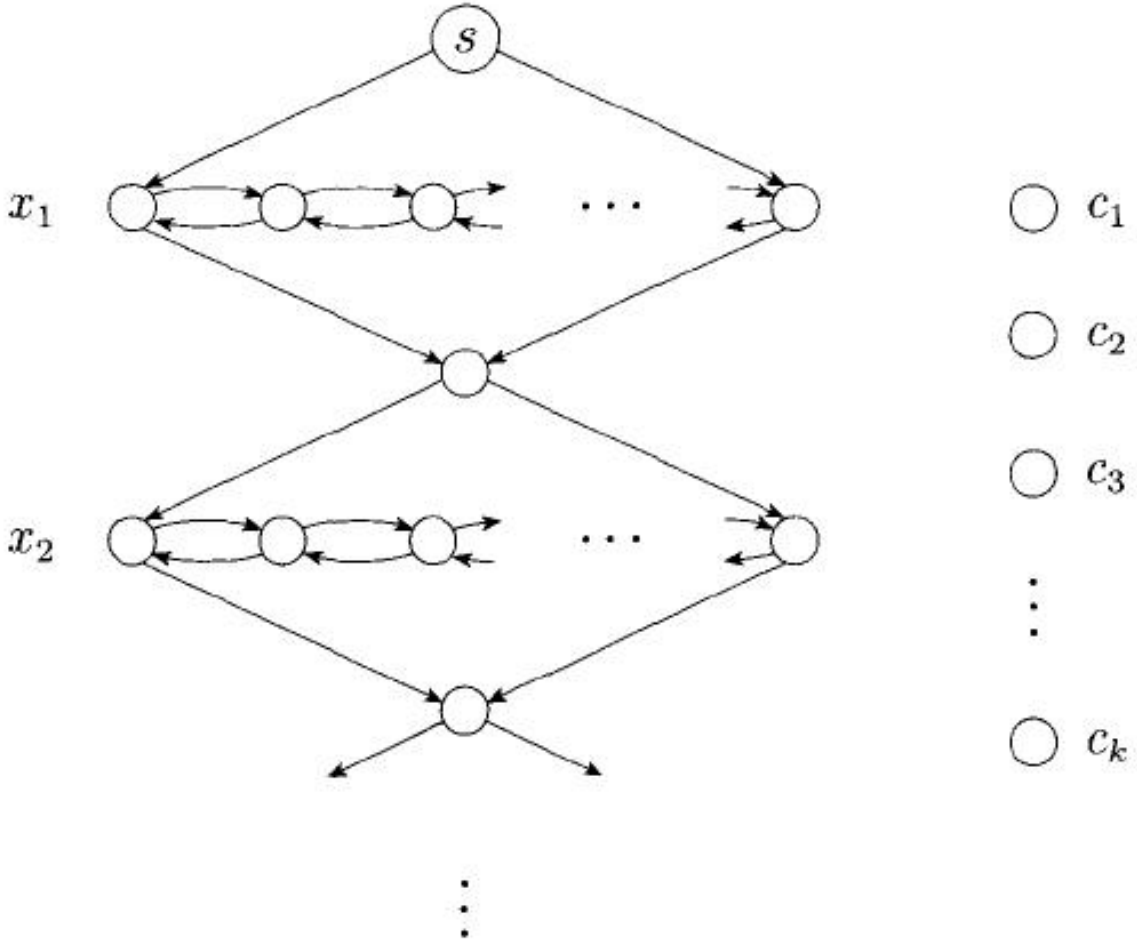
Hamiltonian Path Problem



HAMPATH is NP-complete

- HAMPATH={<G,s,t>: G is a directed graph containing a path from s to t that goes through every node exactly once}
- Proof idea: The variable gadgets are diamond structures that can be traversed in either of 2 ways and the clause gadgets are simple nodes.

Polynomial time reduction



Connecting diamonds to nodes

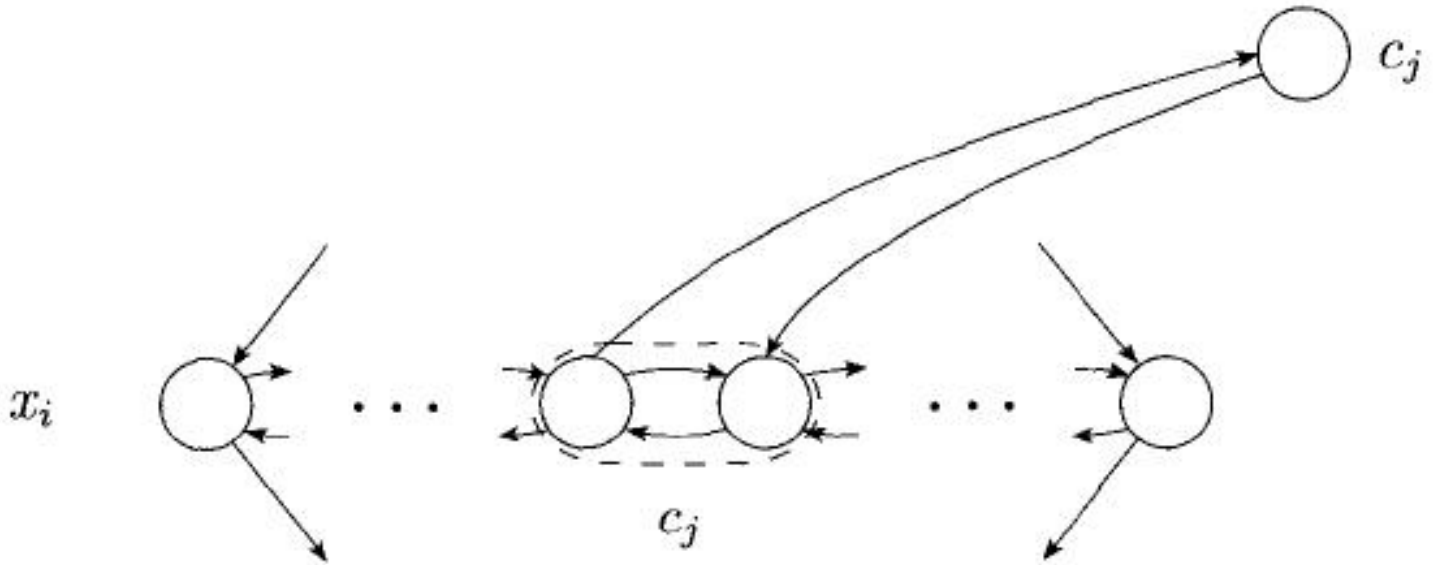


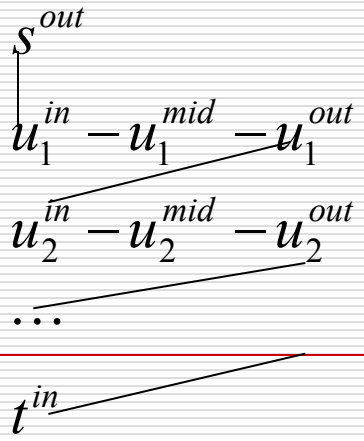
FIGURE **7.51**
The additional edges when clause c_j contains x_i

UHAMPATH is NP-complete

- Proof idea: The reduction takes a directed graph G with nodes s and t, and construct a undirected graph G′ with nodes s′ and t′. A Hamiltonian path P in G

$$s \to u_1 \to u_2 \to \cdots \to u_k \to t$$

has a corresponding Hamiltonian path P′ in G′.



Traveling salesman problem (TSP)

- Input: A graph with integer weights on the edges and a weight limit W.
- Question: does the graph have a Hamilton circuit of total wieght at most W.

Traveling salesman problem (TSP)

- TSP is NP-complete.
- Proof idea: Given a graph, construct a weighted graph G' whose nodes and edges are the same as the nodes and edges of G, with a weight of 1 on each edge, and a limit k that is equal to the number of nodes n of G.
- A Hamiltonian circuit of weight n exists in G' iff there is a Hamilton circuit in G.
  - A Hamilton circuit is a set of edges that connect the nodes into a single cycle, with each node appearing exactly once.

Subset-sum problem

- Input: given a collection of numbers togther with a target number t
- Ask: does the collection contain a subcollection that adds up to t.

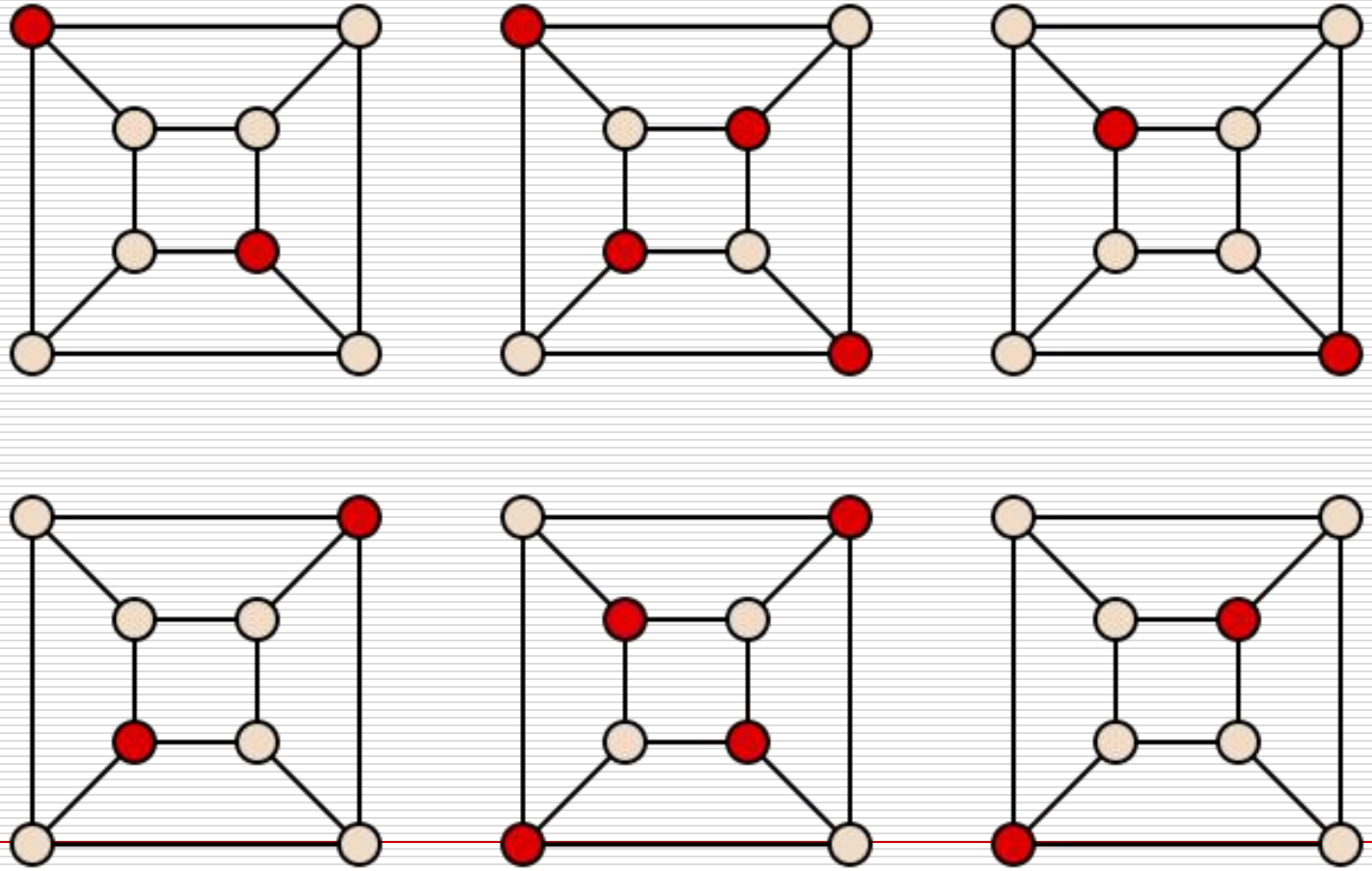
	1	2	3	4	$\cdots$	l	c_1	c_2	$\cdots$	c_k
y_1	1	0	0	0	$\cdots$	0	1	0	$\cdots$	0
z_1	1	0	0	0	$\cdots$	0	0	0	$\cdots$	0
y_2		1	0	0	$\cdots$	0	0	1	$\cdots$	0
z_2		1	0	0	$\cdots$	0	1	0	$\cdots$	0
y_3			1	0	$\cdots$	0	1	1	$\cdots$	0
z_3			1	0	$\cdots$	0	0	0	$\cdots$	1
$\vdots$					$\ddots$	$\vdots$	$\vdots$		$\vdots$	$\vdots$
y_l						1	0	0	$\cdots$	0
z_l						1	0	0	$\cdots$	0
g_1							1	0	$\cdots$	0
h_1							1	0	$\cdots$	0
g_2								1	$\cdots$	0
h_2								1	$\cdots$	0
$\vdots$									$\ddots$	$\vdots$
g_k										1
h_k										1
t	1	1	1	1	$\cdots$	1	3	3	$\cdots$	3

The Problem of Independent sets

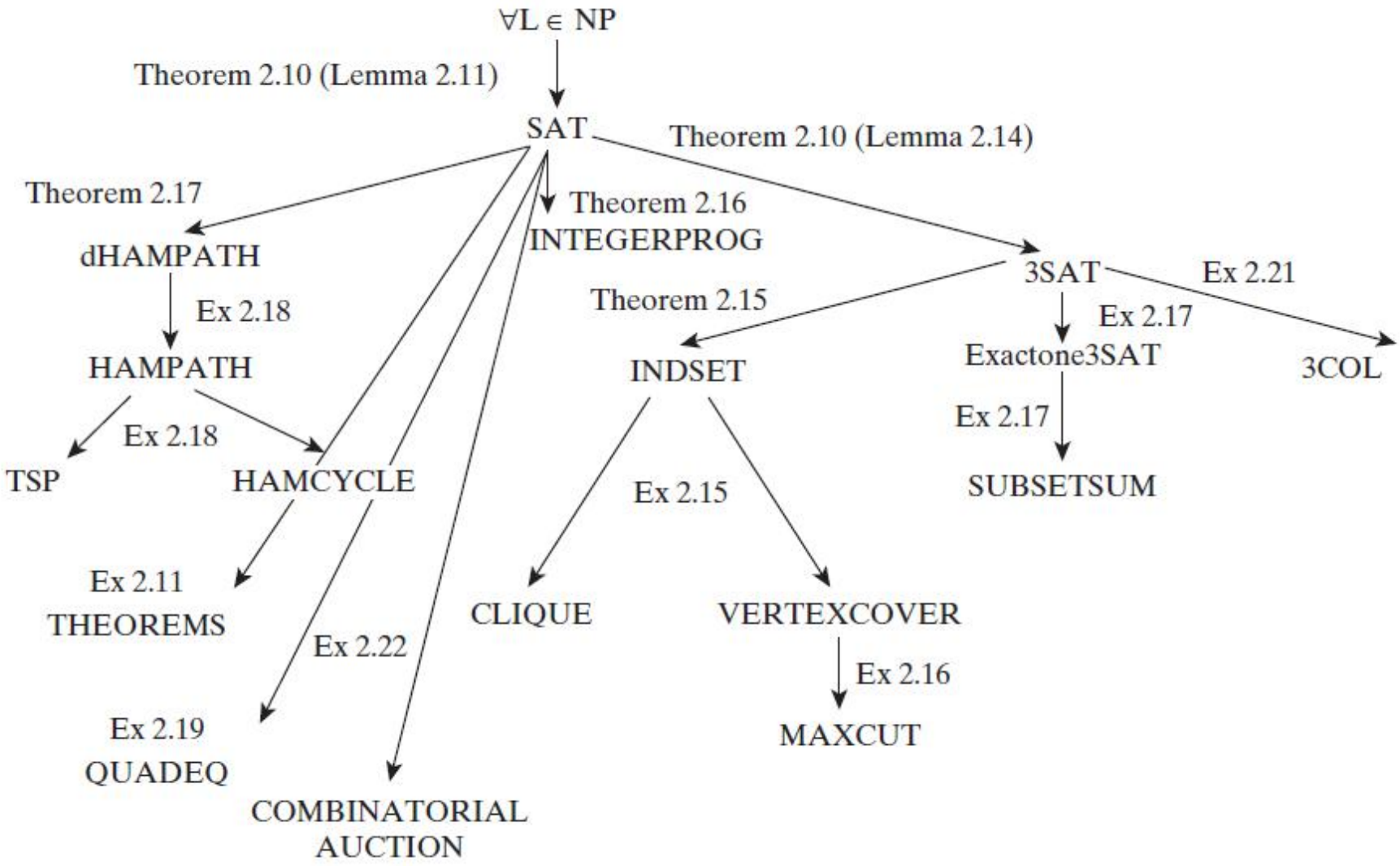
For an undirected graph, a subset I of the nodes of G is an independent set if no two nodes are connected by an edge of G

- **Input**: A graph G and a lower bound k
- **Ask**: does G have an independent set of size k.

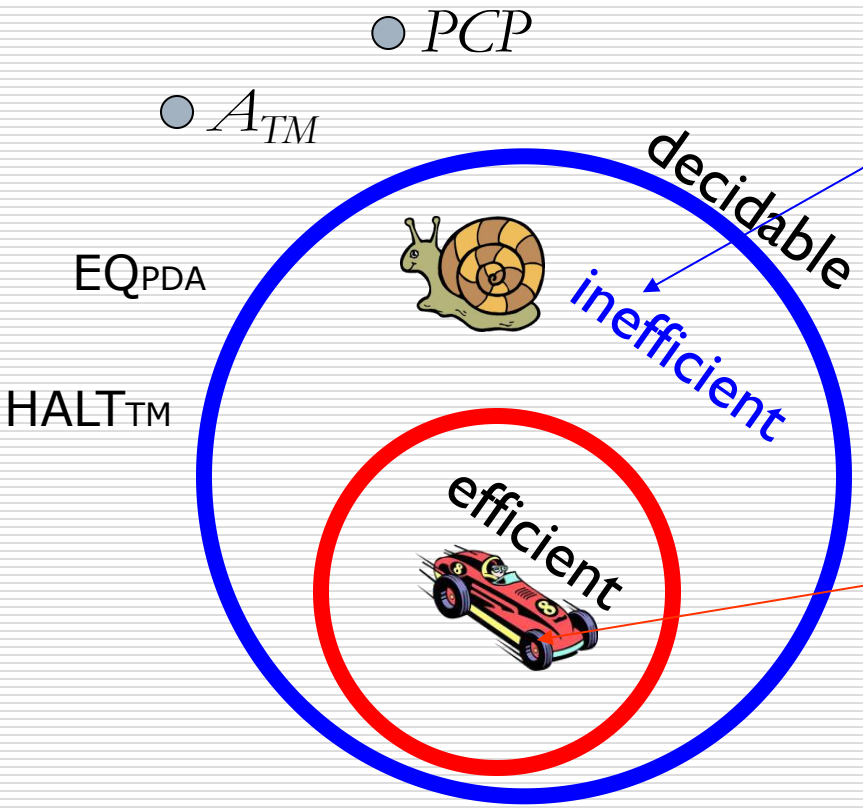
An example



Web of reductions between NP-complete problems

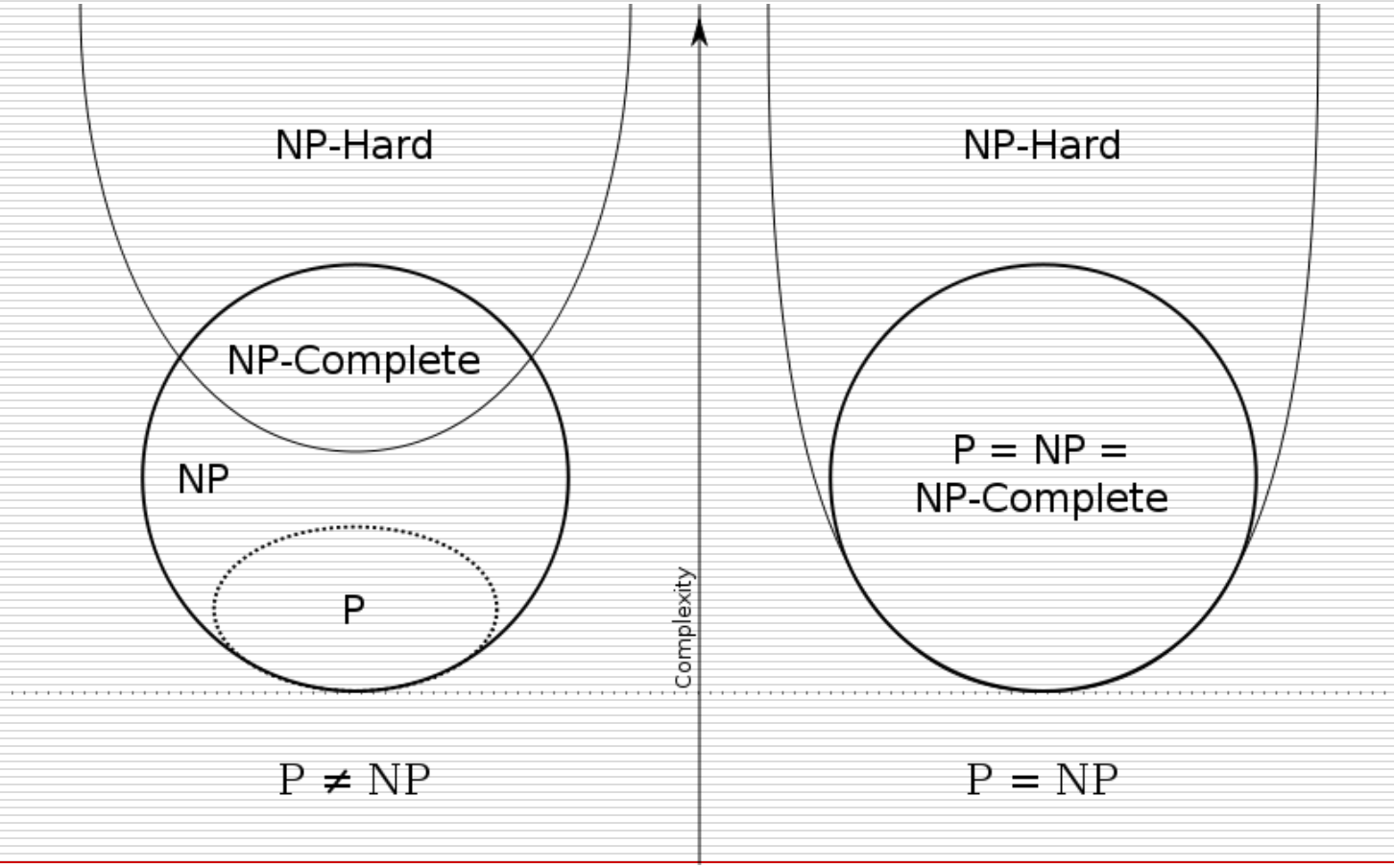


Where are we now?



- SAT
  - 3-SAT
  - CLIQUE
  - Vertex-Cover
  - SUBSET-SUM
  - HAMPATH
- PATH
  - REGULAR (o(nlogn))
  - CFL (O(n^3))
  - Dynamic Programming
  - Greedy algorithm

Location of NP-completeness



coNP

- A language L is in coNP if its complement is in NP.

- Example:

  TAUT = {<p>: p is a tautology which is Boolean formula satisfied by every assignment}